GOS2022

1.1

Generated by Doxygen 1.8.20

1 GOS2022 OS documentation 1
1.1 Revision history . . . 1

2 Module Index 3
2.1 Modules . . . 3

3 Data Structure Index 5
3.1 Data Structures . . . 5

4 File Index 7
4.1 File List . . . 7

5 Module Documentation 9
5.1 Global definitions . . . 9
5.1.1 Detailed Description . . . 9
5.1.2 Macro Definition Documentation . . . 9
5.1.2.1 GLOBAL_STACK . . . 10
5.1.2.2 GOS_ASM . . . 10
5.1.2.3 GOS_CONST . . . 10
5.1.2.4 GOS_DEFAULT_TASK_ID . . . 10
5.1.2.5 GOS_EXTERN . . . 10
5.1.2.6 GOS_INLINE . . . 11
5.1.2.7 GOS_INVALID_TASK_ID . . . 11
5.1.2.8 GOS_NAKED . . . 11
5.1.2.9 GOS_NOP . . . 11
5.1.2.10 GOS_STATIC . . . 11
5.1.2.11 GOS_STATIC_INLINE . . . 12
5.1.2.12 GOS_TASK_IDLE_PRIO . . . 12
5.1.2.13 GOS_TASK_MAX_BLOCK_TIME_MS . . . 12
5.1.2.14 GOS_TASK_MAX_PRIO_LEVELS . . . 12
5.1.2.15 GOS_UNUSED . . . 12
5.1.2.16 GOS_UNUSED_PAR . . . 13
5.1.2.17 MAIN_STACK . . . 13
5.1.2.18 NULL . . . 13
5.1.2.19 RAM_SIZE . . . 13
5.1.2.20 RAM_START . . . 13
5.2 Control macros for scheduling and ISR state setting . . . 14
5.2.1 Detailed Description . . . 14
5.2.2 Macro Definition Documentation . . . 14
5.2.2.1 GOS_ATOMIC_ENTER . . . 14
5.2.2.2 GOS_ATOMIC_EXIT . . . 14
5.2.2.3 GOS_DISABLE_SCHED . . . 15
5.2.2.4 GOS_ENABLE_SCHED . . . 15
5.2.2.5 GOS_ISR_ENTER . . . 15

5.2.2.6 GOS_ISR_EXIT . . . 15
5.3 Privilege bit definitions . . . 16
5.3.1 Detailed Description . . . 16
5.3.2 Macro Definition Documentation . . . 16
5.3.2.1 GOS_PRIV_RESERVED_3 . . . 16
5.3.2.2 GOS_PRIV_RESERVED_4 . . . 16
5.3.2.3 GOS_PRIV_RESERVED_5 . . . 16
5.3.2.4 GOS_PRIV_SIGNALING . . . 17
5.3.2.5 GOS_PRIV_TASK_MANIPULATE . . . 17
5.3.2.6 GOS_PRIV_TASK_PRIO_CHANGE . . . 17
5.3.2.7 GOS_PRIV_TRACE . . . 17
5.4 Basic data type definitions to be used in a GOS system . . . 18
5.4.1 Detailed Description . . . 18
5.5 Task-related type definitions . . . 19
5.5.1 Detailed Description . . . 19
5.6 Hook function type definitions . . . 20
5.6.1 Detailed Description . . . 20
5.7 Time-related definitions . . . 21
5.7.1 Detailed Description . . . 21
5.8 Kernel functions . . . 22
5.8.1 Detailed Description . . . 23
5.8.2 Function Documentation . . . 23
5.8.2.1 gos_kernelCalculateTaskCpuUsages() . . . 23
5.8.2.2 gos_kernelDelayMs() . . . 24
5.8.2.3 gos_kernelDelayUs() . . . 25
5.8.2.4 gos_kernelDump() . . . 25
5.8.2.5 gos_kernelGetCpuUsage() . . . 26
5.8.2.6 gos_kernelGetMaxCpuLoad() . . . 27
5.8.2.7 gos_kernelGetSysTicks() . . . 27
5.8.2.8 gos_kernelInit() . . . 28
5.8.2.9 gos_kernelIsCallerIsr() . . . 29
5.8.2.10 gos_kernelPrivilegedModeSetRequired() . . . 30
5.8.2.11 gos_kernelRegisterIdleHook() . . . 30
5.8.2.12 gos_kernelRegisterPreResetHook() . . . 31
5.8.2.13 gos_kernelRegisterPrivilegedHook() . . . 31
5.8.2.14 gos_kernelRegisterSwapHook() . . . 32
5.8.2.15 gos_kernelRegisterSysTickHook() . . . 32
5.8.2.16 gos_kernelReschedule() . . . 33
5.8.2.17 gos_kernelReset() . . . 34
5.8.2.18 gos_kernelSetMaxCpuLoad() . . . 36
5.8.2.19 gos_kernelStart() . . . 36
5.8.2.20 gos_kernelSubscribeDumpReadySignal() . . . 37

5.8.2.21 gos_taskSubscribeDeleteSignal() . . . 38
5.9 Task-related functions . . . 39
5.9.1 Detailed Description . . . 40
5.9.2 Function Documentation . . . 40
5.9.2.1 gos_taskAddPrivilege() . . . 40
5.9.2.2 gos_taskBlock() . . . 41
5.9.2.3 gos_taskDelete() . . . 42
5.9.2.4 gos_taskGetCurrentId() . . . 43
5.9.2.5 gos_taskGetData() . . . 44
5.9.2.6 gos_taskGetDataByIndex() . . . 45
5.9.2.7 gos_taskGetId() . . . 46
5.9.2.8 gos_taskGetName() . . . 47
5.9.2.9 gos_taskGetNumber() . . . 47
5.9.2.10 gos_taskGetOriginalPriority() . . . 48
5.9.2.11 gos_taskGetPriority() . . . 48
5.9.2.12 gos_taskGetPrivileges() . . . 49
5.9.2.13 gos_taskRegister() . . . 50
5.9.2.14 gos_taskRegisterTasks() . . . 52
5.9.2.15 gos_taskRemovePrivilege() . . . 52
5.9.2.16 gos_taskResume() . . . 53
5.9.2.17 gos_taskSetOriginalPriority() . . . 54
5.9.2.18 gos_taskSetPriority() . . . 55
5.9.2.19 gos_taskSetPrivileges() . . . 57
5.9.2.20 gos_taskSleep() . . . 57
5.9.2.21 gos_taskSuspend() . . . 59
5.9.2.22 gos_taskUnblock() . . . 60
5.9.2.23 gos_taskWakeup() . . . 61
5.9.2.24 gos_taskYield() . . . 62
5.10 Platform initializer weak functions . . . 64
5.10.1 Detailed Description . . . 64
5.10.2 Function Documentation . . . 64
5.10.2.1 gos_platformDriverInit() . . . 64
5.10.2.2 gos_userApplicationInit() . . . 64
5.11 Trace formatter macros . . . 66
5.11.1 Detailed Description . . . 66

6 Data Structure Documentation **67**

6.1 gos_driver_functions_t Struct Reference . . . 67
6.1.1 Detailed Description . . . 67
6.2 gos_gcpChannelFunctions_t Struct Reference . . . 67
6.2.1 Detailed Description . . . 68
6.3 gos_gcpHeaderFrame_t Struct Reference . . . 68

6.3.1 Detailed Description . . . 68
6.4 gos_initStruct_t Struct Reference . . . 68
6.4.1 Detailed Description . . . 69
6.5 gos_message_t Struct Reference . . . 69
6.5.1 Detailed Description . . . 69
6.6 gos_messageWaiterDesc_t Struct Reference . . . 69
6.6.1 Detailed Description . . . 70
6.7 gos_mutex_t Struct Reference . . . 70
6.7.1 Detailed Description . . . 70
6.8 gos_queue_t Struct Reference . . . 71
6.8.1 Detailed Description . . . 71
6.9 gos_queueDescriptor_t Struct Reference . . . 71
6.9.1 Detailed Description . . . 72
6.10 gos_queueElement_t Struct Reference . . . 72
6.10.1 Detailed Description . . . 72
6.11 gos_runtime_t Struct Reference . . . 72
6.11.1 Detailed Description . . . 73
6.12 gos_shellCommand_t Struct Reference . . . 73
6.12.1 Detailed Description . . . 73
6.13 gos_signalDescriptor_t Struct Reference . . . 73
6.13.1 Detailed Description . . . 74
6.14 gos_signalInvokeDescriptor Struct Reference . . . 74
6.14.1 Detailed Description . . . 74
6.15 gos_taskDescriptor_t Struct Reference . . . 74
6.15.1 Detailed Description . . . 76
6.16 gos_time_t Struct Reference . . . 76
6.16.1 Detailed Description . . . 76
6.17 gos_trigger_t Struct Reference . . . 76
6.17.1 Detailed Description . . . 77

7 File Documentation **79**

7.1 GOS2022/OS/driver/inc/gos_crc_driver.h File Reference . . . 79
7.1.1 Detailed Description . . . 80
7.1.2 Function Documentation . . . 80
7.1.2.1 gos_crcDriverGetCrc() . . . 80
7.2 GOS2022/OS/driver/inc/gos_driver.h File Reference . . . 81
7.2.1 Detailed Description . . . 82
7.2.2 Function Documentation . . . 82
7.2.2.1 gos_driverInit() . . . 82
7.3 GOS2022/OS/driver/inc/gos_shell_driver.h File Reference . . . 83
7.3.1 Detailed Description . . . 84
7.3.2 Typedef Documentation . . . 84

7.3.2.1 gos_shellDriverReceiveChar_t . . . 85
7.3.2.2 gos_shellDriverTransmitString_t . . . 85
7.3.3 Function Documentation . . . 85
7.3.3.1 gos_shellDriverReceiveChar() . . . 85
7.3.3.2 gos_shellDriverTransmitString() . . . 86
7.4 GOS2022/OS/driver/inc/gos_timer_driver.h File Reference . . . 87
7.4.1 Detailed Description . . . 88
7.4.2 Typedef Documentation . . . 88
7.4.2.1 gos_timerDriverSysTimerGetVal_t . . . 89
7.4.3 Function Documentation . . . 89
7.4.3.1 gos_timerDriverSysTimerGet() . . . 89
7.5 GOS2022/OS/driver/inc/gos_trace_driver.h File Reference . . . 90
7.5.1 Detailed Description . . . 91
7.5.2 Typedef Documentation . . . 91
7.5.2.1 gos_traceDriverTransmitString_t . . . 91
7.5.2.2 gos_traceDriverTransmitString_Unsafe_t . . . 91
7.5.3 Function Documentation . . . 91
7.5.3.1 gos_traceDriverTransmitString() . . . 91
7.5.3.2 gos_traceDriverTransmitString_Unsafe() . . . 92
7.6 GOS2022/OS/driver/src/gos_crc_driver.c File Reference . . . 93
7.6.1 Detailed Description . . . 94
7.6.2 Macro Definition Documentation . . . 95
7.6.2.1 CRC_INITIAL_VALUE . . . 95
7.6.2.2 CRC_POLYNOMIAL_VALUE . . . 95
7.6.3 Function Documentation . . . 95
7.6.3.1 gos_crcDriverGetCrc() . . . 95
7.7 GOS2022/OS/driver/src/gos_driver.c File Reference . . . 96
7.7.1 Detailed Description . . . 97
7.7.2 Function Documentation . . . 97
7.7.2.1 gos_driverInit() . . . 97
7.8 GOS2022/OS/driver/src/gos_shell_driver.c File Reference . . . 98
7.8.1 Detailed Description . . . 99
7.8.2 Function Documentation . . . 99
7.8.2.1 gos_shellDriverReceiveChar() . . . 99
7.8.2.2 gos_shellDriverTransmitString() . . . 100
7.8.3 Variable Documentation . . . 101
7.8.3.1 formattedBuffer . . . 101
7.9 GOS2022/OS/driver/src/gos_timer_driver.c File Reference . . . 101
7.9.1 Detailed Description . . . 102
7.9.2 Function Documentation . . . 103
7.9.2.1 gos_timerDriverSysTimerGet() . . . 103
7.10 GOS2022/OS/driver/src/gos_trace_driver.c File Reference . . . 103

7.10.1 Detailed Description . . . 104
7.10.2 Function Documentation . . . 105
7.10.2.1 gos_traceDriverTransmitString() . . . 105
7.10.2.2 gos_traceDriverTransmitString_Unsafe() . . . 105
7.11 GOS2022/OS/kernel/inc/gos_bootloader_config.h File Reference . . . 106
7.11.1 Detailed Description . . . 108
7.11.2 Macro Definition Documentation . . . 108
7.11.2.1 ARM_CORTEX_M4 . . . 108
7.11.2.2 CFG_GCP_CHANNELS_MAX_NUMBER . . . 108
7.11.2.3 CFG_IDLE_TASK_STACK_SIZE . . . 109
7.11.2.4 CFG_MESSAGE_MAX_LENGTH . . . 109
7.11.2.5 CFG_MESSAGE_MAX_NUMBER . . . 109
7.11.2.6 CFG_MESSAGE_MAX_WAITER_IDS . . . 109
7.11.2.7 CFG_MESSAGE_MAX_WAITERS . . . 109
7.11.2.8 CFG_QUEUE_MAX_ELEMENTS . . . 110
7.11.2.9 CFG_QUEUE_MAX_LENGTH . . . 110
7.11.2.10 CFG_QUEUE_MAX_NAME_LENGTH . . . 110
7.11.2.11 CFG_QUEUE_MAX_NUMBER . . . 110
7.11.2.12 CFG_QUEUE_USE_NAME . . . 110
7.11.2.13 CFG_RESET_ON_ERROR . . . 111
7.11.2.14 CFG_RESET_ON_ERROR_DELAY_MS . . . 111
7.11.2.15 CFG_SCHED_COOPERATIVE . . . 111
7.11.2.16 CFG_SHELL_COMMAND_BUFFER_SIZE . . . 111
7.11.2.17 CFG_SHELL_MAX_COMMAND_LENGTH . . . 111
7.11.2.18 CFG_SHELL_MAX_COMMAND_NUMBER . . . 112
7.11.2.19 CFG_SHELL_MAX_PARAMS_LENGTH . . . 112
7.11.2.20 CFG_SHELL_STARTUP_DELAY_MS . . . 112
7.11.2.21 CFG_SHELL_USE_SERVICE . . . 112
7.11.2.22 CFG_SIGNAL_MAX_NUMBER . . . 112
7.11.2.23 CFG_SIGNAL_MAX_SUBSCRIBERS . . . 113
7.11.2.24 CFG_SYSTEM_TASK_STACK_SIZE . . . 113
7.11.2.25 CFG_TARGET_CPU . . . 113
7.11.2.26 CFG_TASK_MAX_NAME_LENGTH . . . 113
7.11.2.27 CFG_TASK_MAX_NUMBER . . . 113
7.11.2.28 CFG_TASK_MAX_STACK_SIZE . . . 114
7.11.2.29 CFG_TASK_MESSAGE_DAEMON_PRIO . . . 114
7.11.2.30 CFG_TASK_MESSAGE_DAEMON_STACK . . . 114
7.11.2.31 CFG_TASK_MIN_STACK_SIZE . . . 114
7.11.2.32 CFG_TASK_SHELL_DAEMON_PRIO . . . 114
7.11.2.33 CFG_TASK_SHELL_DAEMON_STACK . . . 115
7.11.2.34 CFG_TASK_SIGNAL_DAEMON_PRIO . . . 115
7.11.2.35 CFG_TASK_SIGNAL_DAEMON_STACK . . . 115

7.11.2.36 CFG_TASK_SYS_PRIO . . . 115
7.11.2.37 CFG_TASK_TIME_DAEMON_PRIO . . . 115
7.11.2.38 CFG_TASK_TIME_DAEMON_STACK . . . 116
7.11.2.39 CFG_TASK_TRACE_DAEMON_PRIO . . . 116
7.11.2.40 CFG_TASK_TRACE_DAEMON_STACK . . . 116
7.11.2.41 CFG_TRACE_MAX_LENGTH . . . 116
7.11.2.42 CFG_USE_PRIO_INHERITANCE . . . 116
7.12 GOS2022/OS/kernel/inc/gos_config.h File Reference . . . 117
7.12.1 Detailed Description . . . 118
7.12.2 Macro Definition Documentation . . . 118
7.12.2.1 ARM_CORTEX_M4 . . . 119
7.12.2.2 CFG_GCP_CHANNELS_MAX_NUMBER . . . 119
7.12.2.3 CFG_IDLE_TASK_STACK_SIZE . . . 119
7.12.2.4 CFG_MESSAGE_MAX_ADDRESSEES . . . 119
7.12.2.5 CFG_MESSAGE_MAX_LENGTH . . . 119
7.12.2.6 CFG_MESSAGE_MAX_NUMBER . . . 120
7.12.2.7 CFG_MESSAGE_MAX_WAITER_IDS . . . 120
7.12.2.8 CFG_MESSAGE_MAX_WAITERS . . . 120
7.12.2.9 CFG_QUEUE_MAX_ELEMENTS . . . 120
7.12.2.10 CFG_QUEUE_MAX_LENGTH . . . 120
7.12.2.11 CFG_QUEUE_MAX_NAME_LENGTH . . . 121
7.12.2.12 CFG_QUEUE_MAX_NUMBER . . . 121
7.12.2.13 CFG_QUEUE_USE_NAME . . . 121
7.12.2.14 CFG_RESET_ON_ERROR . . . 121
7.12.2.15 CFG_RESET_ON_ERROR_DELAY_MS . . . 121
7.12.2.16 CFG_SCHED_COOPERATIVE . . . 122
7.12.2.17 CFG_SHELL_COMMAND_BUFFER_SIZE . . . 122
7.12.2.18 CFG_SHELL_MAX_COMMAND_LENGTH . . . 122
7.12.2.19 CFG_SHELL_MAX_COMMAND_NUMBER . . . 122
7.12.2.20 CFG_SHELL_MAX_PARAMS_LENGTH . . . 122
7.12.2.21 CFG_SHELL_STARTUP_DELAY_MS . . . 123
7.12.2.22 CFG_SHELL_USE_SERVICE . . . 123
7.12.2.23 CFG_SIGNAL_MAX_NUMBER . . . 123
7.12.2.24 CFG_SIGNAL_MAX_SUBSCRIBERS . . . 123
7.12.2.25 CFG_SYSTEM_TASK_STACK_SIZE . . . 123
7.12.2.26 CFG_TARGET_CPU . . . 124
7.12.2.27 CFG_TASK_MAX_NAME_LENGTH . . . 124
7.12.2.28 CFG_TASK_MAX_NUMBER . . . 124
7.12.2.29 CFG_TASK_MAX_STACK_SIZE . . . 124
7.12.2.30 CFG_TASK_MESSAGE_DAEMON_PRIO . . . 124
7.12.2.31 CFG_TASK_MESSAGE_DAEMON_STACK . . . 125
7.12.2.32 CFG_TASK_MIN_STACK_SIZE . . . 125

7.12.2.33 CFG_TASK_SHELL_DAEMON_PRIO . . . 125
7.12.2.34 CFG_TASK_SHELL_DAEMON_STACK . . . 125
7.12.2.35 CFG_TASK_SIGNAL_DAEMON_PRIO . . . 125
7.12.2.36 CFG_TASK_SIGNAL_DAEMON_STACK . . . 126
7.12.2.37 CFG_TASK_SYS_PRIO . . . 126
7.12.2.38 CFG_TASK_TIME_DAEMON_PRIO . . . 126
7.12.2.39 CFG_TASK_TIME_DAEMON_STACK . . . 126
7.12.2.40 CFG_TASK_TRACE_DAEMON_PRIO . . . 126
7.12.2.41 CFG_TASK_TRACE_DAEMON_STACK . . . 127
7.12.2.42 CFG_TRACE_MAX_LENGTH . . . 127
7.12.2.43 CFG_USE_PRIO_INHERITANCE . . . 127
7.13 GOS2022/OS/kernel/inc/gos_kernel.h File Reference . . . 127
7.13.1 Detailed Description . . . 132
7.13.2 Macro Definition Documentation . . . 133
7.13.2.1 GOS_CONCAT_RESULT . . . 133
7.13.2.2 GOS_CONVERT_RESULT . . . 133
7.13.3 Enumeration Type Documentation . . . 134
7.13.3.1 gos_boolValue_t . . . 134
7.13.3.2 gos_kernel_privilege_t . . . 134
7.13.3.3 gos_result_t . . . 134
7.13.3.4 gos_taskPrivilegeLevel_t . . . 135
7.13.3.5 gos_taskState_t . . . 135
7.14 GOS2022/OS/kernel/inc/gos_port.h File Reference . . . 136
7.14.1 Detailed Description . . . 137
7.14.2 Macro Definition Documentation . . . 138
7.14.2.1 gos_ported_doContextSwitch . . . 138
7.14.2.2 gos_ported_enableFaultHandlers . . . 138
7.14.2.3 gos_ported_handleSVC . . . 139
7.14.2.4 gos_ported_handleSVCMain . . . 139
7.14.2.5 gos_ported_kernelStartInit . . . 140
7.14.2.6 gos_ported_pendSVHandler . . . 140
7.14.2.7 gos_ported_procReset . . . 140
7.14.2.8 gos_ported_reschedule . . . 141
7.14.2.9 gos_ported_svcHandler . . . 141
7.14.2.10 gos_ported_svcHandlerMain . . . 141
7.14.2.11 gos_ported_sysTickInterrupt . . . 141
7.15 GOS2022/OS/kernel/src/gos_kernel.c File Reference . . . 142
7.15.1 Detailed Description . . . 144
7.15.2 Macro Definition Documentation . . . 144
7.15.2.1 BINARY_PATTERN . . . 144
7.15.2.2 CONFIG_DUMP_SEPARATOR . . . 145
7.15.2.3 ICSR . . . 145

7.15.2.4 MAX_CPU_DUMP_SEPARATOR . . . 145
7.15.2.5 SHCSR . . . 145
7.15.2.6 STACK_STATS_SEPARATOR . . . 145
7.15.2.7 TASK_DUMP_SEPARATOR . . . 146
7.15.2.8 TO_BINARY . . . 146
7.15.3 Function Documentation . . . 146
7.15.3.1 gos_idleTask() . . . 146
7.15.3.2 gos_kernelCheckTaskStack() . . . 147
7.15.3.3 gos_kernelGetCurrentPsp() . . . 148
7.15.3.4 gos_kernelGetTaskStateString() . . . 148
7.15.3.5 gos_kernelProcessorReset() . . . 149
7.15.3.6 gos_kernelSaveCurrentPsp() . . . 149
7.15.3.7 gos_kernelSelectNextTask() . . . 150
7.15.4 Variable Documentation . . . 150
7.15.4.1 atomicCntr . . . 150
7.15.4.2 cpuUseLimit . . . 151
7.15.4.3 currentTaskIndex . . . 151
7.15.4.4 inIsr . . . 151
7.15.4.5 isKernelRunning . . . 151
7.15.4.6 kernelDumpReadySignal . . . 151
7.15.4.7 kernelDumpSignal . . . 152
7.15.4.8 kernelPreResetHookFunction . . . 152
7.15.4.9 kernelPrivilegedHookFunction . . . 152
7.15.4.10 kernelSwapHookFunction . . . 152
7.15.4.11 kernelSysTickHookFunction . . . 152
7.15.4.12 monitoringTime . . . 153
7.15.4.13 previousTick . . . 153
7.15.4.14 primask . . . 153
7.15.4.15 privilegedModeSetRequired . . . 153
7.15.4.16 resetRequired . . . 153
7.15.4.17 schedDisableCntr . . . 154
7.15.4.18 sysTicks . . . 154
7.15.4.19 sysTimerValue . . . 154
7.16 GOS2022/OS/kernel/src/gos_task.c File Reference . . . 154
7.16.1 Detailed Description . . . 157
7.16.2 Function Documentation . . . 157
7.16.2.1 gos_idleTask() . . . 157
7.16.2.2 gos_taskCheckDescriptor() . . . 158
7.16.3 Variable Documentation . . . 159
7.16.3.1 kernelIdleHookFunction . . . 159
7.16.3.2 kernelTaskDeleteSignal . . . 160
7.16.3.3 taskDescriptors . . . 160

7.17 GOS2022/OS/services/inc/gos.h File Reference . . . 160
7.17.1 Detailed Description . . . 161
7.17.2 Macro Definition Documentation . . . 162
7.17.2.1 GOS_VERSION_MAJOR . . . 162
7.17.2.2 GOS_VERSION_MINOR . . . 162
7.17.3 Function Documentation . . . 162
7.17.3.1 gos_Dump() . . . 162
7.18 GOS2022/OS/services/inc/gos_error.h File Reference . . . 163
7.18.1 Detailed Description . . . 165
7.18.2 Enumeration Type Documentation . . . 165
7.18.2.1 gos_errorLevel_t . . . 165
7.18.3 Function Documentation . . . 166
7.18.3.1 gos_errorHandler() . . . 166
7.18.3.2 gos_errorTraceInit() . . . 167
7.18.3.3 gos_printStartupLogo() . . . 168
7.19 GOS2022/OS/services/inc/gos_gcp.h File Reference . . . 169
7.19.1 Detailed Description . . . 170
7.19.2 Typedef Documentation . . . 170
7.19.2.1 gos_gcpReceiveFunction_t . . . 170
7.19.2.2 gos_gcpTransmitFunction_t . . . 171
7.19.3 Function Documentation . . . 171
7.19.3.1 gos_gcpInit() . . . 171
7.19.3.2 gos_gcpReceiveMessage() . . . 171
7.19.3.3 gos_gcpRegisterPhysicalDriver() . . . 172
7.19.3.4 gos_gcpTransmitMessage() . . . 173
7.20 GOS2022/OS/services/inc/gos_message.h File Reference . . . 174
7.20.1 Detailed Description . . . 176
7.20.2 Macro Definition Documentation . . . 176
7.20.2.1 GOS_MESSAGE_ENDLESS_TMO . . . 176
7.20.2.2 GOS_MESSAGE_INVALID_ID . . . 176
7.20.3 Typedef Documentation . . . 177
7.20.3.1 gos_messageId_t . . . 177
7.20.3.2 gos_messageTimeout_t . . . 177
7.20.4 Function Documentation . . . 177
7.20.4.1 gos_messageInit() . . . 177
7.20.4.2 gos_messageRx() . . . 178
7.20.4.3 gos_messageTx() . . . 179
7.21 GOS2022/OS/services/inc/gos_mutex.h File Reference . . . 180
7.21.1 Detailed Description . . . 181
7.21.2 Macro Definition Documentation . . . 181
7.21.2.1 GOS_MUTEX_ENDLESS_TMO . . . 181
7.21.2.2 GOS_MUTEX_NO_TMO . . . 182

7.21.3 Enumeration Type Documentation . . . 182
7.21.3.1 gos_mutexState_t . . . 182
7.21.4 Function Documentation . . . 182
7.21.4.1 gos_mutexInit() . . . 182
7.21.4.2 gos_mutexLock() . . . 183
7.21.4.3 gos_mutexUnlock() . . . 185
7.22 GOS2022/OS/services/inc/gos_queue.h File Reference . . . 186
7.22.1 Detailed Description . . . 188
7.22.2 Macro Definition Documentation . . . 189
7.22.2.1 GOS_DEFAULT_QUEUE_ID . . . 189
7.22.2.2 GOS_INVALID_QUEUE_ID . . . 189
7.22.3 Typedef Documentation . . . 189
7.22.3.1 gos_queueByte_t . . . 189
7.22.3.2 gos_queueEmptyHook . . . 190
7.22.3.3 gos_queueFullHook . . . 190
7.22.4 Function Documentation . . . 190
7.22.4.1 gos_queueCreate() . . . 190
7.22.4.2 gos_queueDump() . . . 191
7.22.4.3 gos_queueGet() . . . 192
7.22.4.4 gos_queueGetElementNumber() . . . 193
7.22.4.5 gos_queueGetName() . . . 193
7.22.4.6 gos_queueInit() . . . 194
7.22.4.7 gos_queuePeek() . . . 194
7.22.4.8 gos_queuePut() . . . 195
7.22.4.9 gos_queueRegisterEmptyHook() . . . 196
7.22.4.10 gos_queueRegisterFullHook() . . . 197
7.22.4.11 gos_queueReset() . . . 197
7.23 GOS2022/OS/services/inc/gos_shell.h File Reference . . . 199
7.23.1 Detailed Description . . . 201
7.23.2 Typedef Documentation . . . 201
7.23.2.1 gos_shellFunction . . . 202
7.23.3 Function Documentation . . . 202
7.23.3.1 gos_shellEchoOff() . . . 202
7.23.3.2 gos_shellEchoOn() . . . 202
7.23.3.3 gos_shellInit() . . . 203
7.23.3.4 gos_shellRegisterCommand() . . . 203
7.23.3.5 gos_shellRegisterCommands() . . . 204
7.23.3.6 gos_shellResume() . . . 205
7.23.3.7 gos_shellSuspend() . . . 206
7.24 GOS2022/OS/services/inc/gos_signal.h File Reference . . . 207
7.24.1 Detailed Description . . . 208
7.24.2 Function Documentation . . . 208

7.24.2.1 gos_signalCreate() . . . 209
7.24.2.2 gos_signalInit() . . . 210
7.24.2.3 gos_signalInvoke() . . . 211
7.24.2.4 gos_signalSubscribe() . . . 212
7.25 GOS2022/OS/services/inc/gos_time.h File Reference . . . 213
7.25.1 Detailed Description . . . 214
7.25.2 Enumeration Type Documentation . . . 215
7.25.2.1 gos_timeComprareResult_t . . . 215
7.25.2.2 gos_timeElapsedSenderId_t . . . 215
7.25.2.3 gos_timeMonthEnum_t . . . 215
7.25.3 Function Documentation . . . 216
7.25.3.1 gos_runTimeAddMicroseconds() . . . 216
7.25.3.2 gos_runTimeAddMilliseconds() . . . 217
7.25.3.3 gos_runTimeAddSeconds() . . . 218
7.25.3.4 gos_runTimeGet() . . . 218
7.25.3.5 gos_timeAddMilliseconds() . . . 219
7.25.3.6 gos_timeAddSeconds() . . . 220
7.25.3.7 gos_timeCompare() . . . 221
7.25.3.8 gos_timeGet() . . . 221
7.25.3.9 gos_timeIncreaseSystemTime() . . . 222
7.25.3.10 gos_timeInit() . . . 223
7.25.3.11 gos_timeSet() . . . 224
7.26 GOS2022/OS/services/inc/gos_trace.h File Reference . . . 224
7.26.1 Detailed Description . . . 226
7.26.2 Function Documentation . . . 227
7.26.2.1 gos_traceInit() . . . 227
7.26.2.2 gos_traceTrace() . . . 228
7.26.2.3 gos_traceTraceFormatted() . . . 229
7.26.2.4 gos_traceTraceFormattedUnsafe() . . . 229
7.27 GOS2022/OS/services/inc/gos_trigger.h File Reference . . . 231
7.27.1 Detailed Description . . . 233
7.27.2 Macro Definition Documentation . . . 233
7.27.2.1 GOS_TRIGGER_ENDLESS_TMO . . . 233
7.27.2.2 GOS_TRIGGER_NO_TMO . . . 234
7.27.3 Function Documentation . . . 234
7.27.3.1 gos_triggerDecrement() . . . 234
7.27.3.2 gos_triggerIncrement() . . . 235
7.27.3.3 gos_triggerInit() . . . 236
7.27.3.4 gos_triggerReset() . . . 236
7.27.3.5 gos_triggerWait() . . . 237
7.28 GOS2022/OS/services/src/gos.c File Reference . . . 238
7.28.1 Detailed Description . . . 239

7.28.2 Macro Definition Documentation . . . 239
7.28.2.1 GOS_SYS_TASK_SLEEP_TIME . . . 240
7.28.3 Typedef Documentation . . . 240
7.28.3.1 gos_initFunc_t . . . 240
7.28.4 Function Documentation . . . 240
7.28.4.1 gos_Dump() . . . 240
7.28.4.2 gos_Start() . . . 241
7.28.4.3 gos_systemTask() . . . 242
7.28.5 Variable Documentation . . . 242
7.28.5.1 dumpRequired . . . 242
7.28.5.2 initError . . . 243
7.28.5.3 initializers . . . 243
7.28.5.4 systemTaskDesc . . . 243
7.28.5.5 systemTaskId . . . 243
7.29 GOS2022/OS/services/src/gos_error.c File Reference . . . 244
7.29.1 Detailed Description . . . 245
7.29.2 Macro Definition Documentation . . . 245
7.29.2.1 ERROR_BUFFER_SIZE . . . 245
7.29.2.2 RESULT_STRING_ERROR . . . 246
7.29.2.3 RESULT_STRING_SUCCESS . . . 246
7.29.2.4 RESULT_STRING_UNKNOWN . . . 246
7.29.2.5 SEPARATOR_LINE . . . 246
7.29.3 Function Documentation . . . 246
7.29.3.1 gos_errorHandler() . . . 246
7.29.3.2 gos_errorTraceInit() . . . 248
7.29.3.3 gos_printStartupLogo() . . . 249
7.29.3.4 gos_traceResultToString() . . . 250
7.29.4 Variable Documentation . . . 250
7.29.4.1 errorBuffer . . . 251
7.30 GOS2022/OS/services/src/gos_gcp.c File Reference . . . 251
7.30.1 Detailed Description . . . 252
7.30.2 Macro Definition Documentation . . . 253
7.30.2.1 GCP_PROTOCOL_VERSION_MAJOR . . . 253
7.30.2.2 GCP_PROTOCOL_VERSION_MINOR . . . 253
7.30.3 Enumeration Type Documentation . . . 253
7.30.3.1 gos_gcpAck_t . . . 253
7.30.4 Function Documentation . . . 253
7.30.4.1 gos_gcpInit() . . . 254
7.30.4.2 gos_gcpReceiveMessage() . . . 254
7.30.4.3 gos_gcpReceiveMessageInternal() . . . 255
7.30.4.4 gos_gcpRegisterPhysicalDriver() . . . 256
7.30.4.5 gos_gcpTransmitMessage() . . . 257

7.30.4.6 gos_gcpTransmitMessageInternal() . . . 258
7.30.4.7 gos_gcpValidateData() . . . 259
7.30.4.8 gos_gcpValidateHeader() . . . 260
7.30.5 Variable Documentation . . . 261
7.30.5.1 channelFunctions . . . 261
7.30.5.2 gcpRxMutexes . . . 261
7.30.5.3 gcpTxMutexes . . . 262
7.31 GOS2022/OS/services/src/gos_message.c File Reference . . . 262
7.31.1 Detailed Description . . . 263
7.31.2 Macro Definition Documentation . . . 263
7.31.2.1 GOS_MESSAGE_DAEMON_POLL_TIME_MS . . . 263
7.31.3 Function Documentation . . . 264
7.31.3.1 gos_messageDaemonTask() . . . 264
7.31.3.2 gos_messageInit() . . . 264
7.31.3.3 gos_messageRx() . . . 265
7.31.3.4 gos_messageTx() . . . 266
7.31.4 Variable Documentation . . . 267
7.31.4.1 messageArray . . . 267
7.31.4.2 messageDaemonTaskDesc . . . 267
7.31.4.3 messageDaemonTaskId . . . 267
7.31.4.4 messageMutex . . . 267
7.31.4.5 messageWaiterArray . . . 268
7.31.4.6 nextMessageIndex . . . 268
7.31.4.7 nextWaiterIndex . . . 268
7.32 GOS2022/OS/services/src/gos_mutex.c File Reference . . . 268
7.32.1 Detailed Description . . . 269
7.32.2 Macro Definition Documentation . . . 270
7.32.2.1 MUTEX_LOCK_SLEEP_MS . . . 270
7.32.3 Function Documentation . . . 270
7.32.3.1 gos_mutexInit() . . . 270
7.32.3.2 gos_mutexLock() . . . 271
7.32.3.3 gos_mutexUnlock() . . . 272
7.33 GOS2022/OS/services/src/gos_queue.c File Reference . . . 274
7.33.1 Detailed Description . . . 276
7.33.2 Macro Definition Documentation . . . 276
7.33.2.1 DUMP_SEPARATOR . . . 276
7.33.3 Function Documentation . . . 277
7.33.3.1 gos_queueCreate() . . . 277
7.33.3.2 gos_queueDump() . . . 278
7.33.3.3 gos_queueGet() . . . 279
7.33.3.4 gos_queueGetElementNumber() . . . 280
7.33.3.5 gos_queueGetName() . . . 281

7.33.3.6 gos_queueInit() . . . 281
7.33.3.7 gos_queuePeek() . . . 282
7.33.3.8 gos_queuePut() . . . 283
7.33.3.9 gos_queueRegisterEmptyHook() . . . 284
7.33.3.10 gos_queueRegisterFullHook() . . . 285
7.33.3.11 gos_queueReset() . . . 285
7.33.4 Variable Documentation . . . 286
7.33.4.1 queueEmptyHook . . . 286
7.33.4.2 queueFullHook . . . 286
7.33.4.3 queueMutex . . . 286
7.33.4.4 queues . . . 287
7.33.4.5 readCounters . . . 287
7.33.4.6 writeCounters . . . 287
7.34 GOS2022/OS/services/src/gos_shell.c File Reference . . . 287
7.34.1 Detailed Description . . . 288
7.34.2 Macro Definition Documentation . . . 289
7.34.2.1 GOS_SHELL_DAEMON_POLL_TIME_MS . . . 289
7.34.2.2 GOS_SHELL_DISPLAY_TEXT . . . 289
7.34.3 Function Documentation . . . 289
7.34.3.1 gos_shellCommandHandler() . . . 289
7.34.3.2 gos_shellDaemonTask() . . . 290
7.34.3.3 gos_shellEchoOff() . . . 291
7.34.3.4 gos_shellEchoOn() . . . 291
7.34.3.5 gos_shellInit() . . . 292
7.34.3.6 gos_shellRegisterCommand() . . . 292
7.34.3.7 gos_shellRegisterCommands() . . . 293
7.34.3.8 gos_shellResume() . . . 294
7.34.3.9 gos_shellSuspend() . . . 295
7.34.4 Variable Documentation . . . 296
7.34.4.1 actualCommand . . . 296
7.34.4.2 commandBuffer . . . 296
7.34.4.3 commandBufferIndex . . . 296
7.34.4.4 commandParams . . . 296
7.34.4.5 shellCommand . . . 297
7.34.4.6 shellCommands . . . 297
7.34.4.7 shellDaemonTaskDesc . . . 297
7.34.4.8 shellDaemonTaskId . . . 297
7.34.4.9 useEcho . . . 298
7.35 GOS2022/OS/services/src/gos_signal.c File Reference . . . 298
7.35.1 Detailed Description . . . 299
7.35.2 Macro Definition Documentation . . . 299
7.35.2.1 GOS_SIGNAL_DAEMON_POLL_TIME_MS . . . 300

7.35.3 Function Documentation . . . 300
7.35.3.1 gos_signalCreate() . . . 300
7.35.3.2 gos_signalDaemonTask() . . . 301
7.35.3.3 gos_signalInit() . . . 301
7.35.3.4 gos_signalInvoke() . . . 302
7.35.3.5 gos_signalSubscribe() . . . 303
7.35.4 Variable Documentation . . . 303
7.35.4.1 callerTaskDesc . . . 303
7.35.4.2 signalArray . . . 303
7.35.4.3 signalDaemonTaskDescriptor . . . 304
7.35.4.4 signalInvokeTrigger . . . 304
7.36 GOS2022/OS/services/src/gos_time.c File Reference . . . 304
7.36.1 Detailed Description . . . 306
7.36.2 Macro Definition Documentation . . . 306
7.36.2.1 TIME_DEFAULT_DAY . . . 307
7.36.2.2 TIME_DEFAULT_MONTH . . . 307
7.36.2.3 TIME_DEFAULT_YEAR . . . 307
7.36.2.4 TIME_SLEEP_TIME_MS . . . 307
7.36.3 Function Documentation . . . 307
7.36.3.1 gos_runTimeAddMicroseconds() . . . 307
7.36.3.2 gos_runTimeAddMilliseconds() . . . 308
7.36.3.3 gos_runTimeAddSeconds() . . . 309
7.36.3.4 gos_runTimeGet() . . . 310
7.36.3.5 gos_timeAddMilliseconds() . . . 310
7.36.3.6 gos_timeAddSeconds() . . . 311
7.36.3.7 gos_timeCompare() . . . 312
7.36.3.8 gos_timeDaemonTask() . . . 312
7.36.3.9 gos_timeGet() . . . 313
7.36.3.10 gos_timeIncreaseSystemTime() . . . 314
7.36.3.11 gos_timeInit() . . . 315
7.36.3.12 gos_timeSet() . . . 316
7.36.4 Variable Documentation . . . 316
7.36.4.1 dayLookupTable . . . 316
7.36.4.2 systemRunTime . . . 317
7.36.4.3 systemTime . . . 317
7.36.4.4 timeDaemonTaskDesc . . . 317
7.36.4.5 timeDaemonTaskId . . . 317
7.36.4.6 timeSignalId . . . 318
7.37 GOS2022/OS/services/src/gos_trace.c File Reference . . . 318
7.37.1 Detailed Description . . . 319
7.37.2 Macro Definition Documentation . . . 319
7.37.2.1 GOS_TRACE_MUTEX_TMO_MS . . . 319

7.37.2.2 GOS_TRACE_QUEUE_TMO_MS . . . 319
7.37.2.3 GOS_TRACE_TIMESTAMP_FORMAT . . . 320
7.37.2.4 GOS_TRACE_TIMESTAMP_LENGTH . . . 320
7.37.3 Function Documentation . . . 320
7.37.3.1 gos_traceDaemonTask() . . . 320
7.37.3.2 gos_traceInit() . . . 321
7.37.3.3 gos_traceTrace() . . . 321
7.37.3.4 gos_traceTraceFormatted() . . . 322
7.37.3.5 gos_traceTraceFormattedUnsafe() . . . 323
7.37.4 Variable Documentation . . . 324
7.37.4.1 formattedBuffer . . . 325
7.37.4.2 timeStampBuffer . . . 325
7.37.4.3 traceDaemonTaskDesc . . . 325
7.37.4.4 traceLine . . . 325
7.37.4.5 traceMutex . . . 326
7.37.4.6 traceQueue . . . 326
7.38 GOS2022/OS/services/src/gos_trigger.c File Reference . . . 326
7.38.1 Detailed Description . . . 327
7.38.2 Function Documentation . . . 327
7.38.2.1 gos_triggerDecrement() . . . 327
7.38.2.2 gos_triggerIncrement() . . . 328
7.38.2.3 gos_triggerInit() . . . 329
7.38.2.4 gos_triggerReset() . . . 330
7.38.2.5 gos_triggerWait() . . . 331

Index **333**

Chapter 1

GOS2022 OS documentation

Author: Ahmed Gazar
Date: 2025-07-30 **Current Version:** 1.1

1.1 Revision history

Version	Date	Author	Change
1.0	2025-04-16	Ahmed Gazar	Initial version
1.1	2025-07-30	Ahmed Gazar	Updated for OS version 1.1

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

Global definitions 9
Control macros for scheduling and ISR state setting 14
Privilege bit definitions 16
Basic data type definitions to be used in a GOS system 18
Task-related type definitions 19
Hook function type definitions 20
Time-related definitions 21
Kernel functions 22
Task-related functions 39
Platform initializer weak functions 64
Trace formatter macros 66

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

gos_driver_functions_t . . . 67
gos_gcpChannelFunctions_t . . . 67
gos_gcpHeaderFrame_t . . . 68
gos_initStruct_t . . . 68
gos_message_t . . . 69
gos_messageWaiterDesc_t . . . 69
gos_mutex_t . . . 70
gos_queue_t . . . 71
gos_queueDescriptor_t . . . 71
gos_queueElement_t . . . 72
gos_runtime_t . . . 72
gos_shellCommand_t . . . 73
gos_signalDescriptor_t . . . 73
gos_signalInvokeDescriptor . . . 74
gos_taskDescriptor_t . . . 74
gos_time_t . . . 76
gos_trigger_t . . . 76

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

GOS2022/OS/driver/inc/gos_crc_driver.h
GOS Cyclic Redundancy Check driver header . . . 79
GOS2022/OS/driver/inc/gos_driver.h
GOS driver header . . . 81
GOS2022/OS/driver/inc/gos_shell_driver.h
GOS SHELL driver header . . . 83
GOS2022/OS/driver/inc/gos_timer_driver.h
GOS timer driver header . . . 87
GOS2022/OS/driver/inc/gos_trace_driver.h
GOS trace driver header . . . 90
GOS2022/OS/driver/src/gos_crc_driver.c
GOS Cyclic Redundancy Check driver source . . . 93
GOS2022/OS/driver/src/gos_driver.c
GOS driver source . . . 96
GOS2022/OS/driver/src/gos_shell_driver.c
GOS SHELL driver source . . . 98
GOS2022/OS/driver/src/gos_timer_driver.c
GOS timer driver source . . . 101
GOS2022/OS/driver/src/gos_trace_driver.c
GOS trace driver source . . . 103
GOS2022/OS/kernel/inc/gos_bootloader_config.h
GOS bootloader configuration header . . . 106
GOS2022/OS/kernel/inc/gos_config.h
GOS configuration header . . . 117
GOS2022/OS/kernel/inc/gos_kernel.h
GOS kernel header . . . 127
GOS2022/OS/kernel/inc/gos_port.h
GOS port header . . . 136
GOS2022/OS/kernel/src/gos_kernel.c
GOS kernel source . . . 142
GOS2022/OS/kernel/src/gos_task.c
GOS task source . . . 154
GOS2022/OS/services/inc/gos.h
GOS header . . . 160
GOS2022/OS/services/inc/gos_error.h
GOS error handler header . . . 163

GOS2022/OS/services/inc/gos_gcp.h
GOS General Communication Protocol header . . . 169
GOS2022/OS/services/inc/gos_message.h
GOS message service header . . . 174
GOS2022/OS/services/inc/gos_mutex.h
GOS mutex service header . . . 180
GOS2022/OS/services/inc/gos_queue.h
GOS queue service header . . . 186
GOS2022/OS/services/inc/gos_shell.h
GOS shell service header . . . 199
GOS2022/OS/services/inc/gos_signal.h
GOS signal service header . . . 207
GOS2022/OS/services/inc/gos_time.h
GOS time service header . . . 213
GOS2022/OS/services/inc/gos_trace.h
GOS trace service header . . . 224
GOS2022/OS/services/inc/gos_trigger.h
GOS trigger service header . . . 231
GOS2022/OS/services/src/gos.c
GOS source . . . 238
GOS2022/OS/services/src/gos_error.c
GOS error handler service source . . . 244
GOS2022/OS/services/src/gos_gcp.c
GOS General Communication Protocol handler service source . . . 251
GOS2022/OS/services/src/gos_message.c
GOS message service source . . . 262
GOS2022/OS/services/src/gos_mutex.c
GOS mutex service source . . . 268
GOS2022/OS/services/src/gos_queue.c
GOS queue service source . . . 274
GOS2022/OS/services/src/gos_shell.c
GOS shell service source . . . 287
GOS2022/OS/services/src/gos_signal.c
GOS signal service source . . . 298
GOS2022/OS/services/src/gos_time.c
GOS time service source . . . 304
GOS2022/OS/services/src/gos_trace.c
GOS trace service source . . . 318
GOS2022/OS/services/src/gos_trigger.c
GOS trigger service source . . . 326

Chapter 5

Module Documentation

5.1 Global definitions

Macros

- #define NULL ((void *) 0)
- #define RAM_START (0x20000000u)
- #define RAM_SIZE (128 * 1024)
- #define MAIN_STACK (RAM_START + RAM_SIZE)
- #define GLOBAL_STACK (0x1200)
- #define GOS_DEFAULT_TASK_ID ((gos_tid_t)0x8000)
- #define GOS_INVALID_TASK_ID ((gos_tid_t)0x0100)
- #define GOS_TASK_MAX_PRIO_LEVELS (UINT8_MAX)
- #define GOS_TASK_IDLE_PRIO (GOS_TASK_MAX_PRIO_LEVELS)
- #define GOS_TASK_MAX_BLOCK_TIME_MS (0xFFFFFFFFu)
- #define GOS_STATIC static
- #define GOS_CONST const
- #define GOS_INLINE inline __attribute__((always_inline))
- #define GOS_STATIC_INLINE GOS_STATIC GOS_INLINE
- #define GOS_EXTERN extern
- #define GOS_NAKED __attribute__ ((naked))
- #define GOS_UNUSED __attribute__ ((unused))
- #define GOS_NOP __asm volatile ("NOP")
- #define GOS_ASM __asm volatile
- #define GOS_UNUSED_PAR(x) (void_t) x

5.1.1 Detailed Description

5.1.2 Macro Definition Documentation

5.1.2.1 GLOBAL_STACK

```
#define GLOBAL_STACK ( 0x1200 )
```

Global stack.

Definition at line 150 of file gos_kernel.h.

5.1.2.2 GOS_ASM

```
#define GOS_ASM __asm volatile
```

ASM.

Definition at line 220 of file gos_kernel.h.

5.1.2.3 GOS_CONST

```
#define GOS_CONST const
```

Constant macro.

Definition at line 185 of file gos_kernel.h.

5.1.2.4 GOS_DEFAULT_TASK_ID

```
#define GOS_DEFAULT_TASK_ID ( (gos_tid_t)0x8000 )
```

Default task ID.

Definition at line 155 of file gos_kernel.h.

5.1.2.5 GOS_EXTERN

```
#define GOS_EXTERN extern
```

Extern macro.

Definition at line 200 of file gos_kernel.h.

5.1.2.6 GOS_INLINE

```
#define GOS_INLINE inline __attribute__((always_inline))
```

Inline macro.

Definition at line 190 of file gos_kernel.h.

5.1.2.7 GOS_INVALID_TASK_ID

```
#define GOS_INVALID_TASK_ID ( (gos_tid_t)0x0100 )
```

Invalid task ID.

Definition at line 160 of file gos_kernel.h.

5.1.2.8 GOS_NAKED

```
#define GOS_NAKED __attribute__ ((naked))
```

Naked.

Definition at line 205 of file gos_kernel.h.

5.1.2.9 GOS_NOP

```
#define GOS_NOP __asm volatile ("NOP")
```

NOP.

Definition at line 215 of file gos_kernel.h.

5.1.2.10 GOS_STATIC

```
#define GOS_STATIC static
```

Static macro.

Definition at line 180 of file gos_kernel.h.

5.1.2.11 GOS_STATIC_INLINE

```
#define GOS_STATIC_INLINE GOS_STATIC GOS_INLINE
```

Static in-line macro.

Definition at line 195 of file gos_kernel.h.

5.1.2.12 GOS_TASK_IDLE_PRIO

```
#define GOS_TASK_IDLE_PRIO ( GOS_TASK_MAX_PRIO_LEVELS )
```

Idle task priority.

Definition at line 170 of file gos_kernel.h.

5.1.2.13 GOS_TASK_MAX_BLOCK_TIME_MS

```
#define GOS_TASK_MAX_BLOCK_TIME_MS ( 0xFFFFFFFFu )
```

Task maximum block time.

Definition at line 175 of file gos_kernel.h.

5.1.2.14 GOS_TASK_MAX_PRIO_LEVELS

```
#define GOS_TASK_MAX_PRIO_LEVELS ( UINT8_MAX )
```

Maximum task priority levels.

Definition at line 165 of file gos_kernel.h.

5.1.2.15 GOS_UNUSED

```
#define GOS_UNUSED __attribute__ ((unused))
```

Unused.

Definition at line 210 of file gos_kernel.h.

5.1.2.16 GOS_UNUSED_PAR

```
#define GOS_UNUSED_PAR(
              x ) (void_t) x
```

Unused.

Definition at line 225 of file gos_kernel.h.

5.1.2.17 MAIN_STACK

```
#define MAIN_STACK ( RAM_START + RAM_SIZE )
```

Main stack.

Definition at line 145 of file gos_kernel.h.

5.1.2.18 NULL

```
#define NULL ( (void *) 0 )
```

NULL pointer.

Definition at line 129 of file gos_kernel.h.

5.1.2.19 RAM_SIZE

```
#define RAM_SIZE ( 128 * 1024 )
```

RAM size (128kB).

Definition at line 140 of file gos_kernel.h.

5.1.2.20 RAM_START

```
#define RAM_START ( 0x20000000u )
```

RAM start address.

Definition at line 135 of file gos_kernel.h.

5.2 Control macros for scheduling and ISR state setting

Macros

- #define GOS_DISABLE_SCHED
- #define GOS_ENABLE_SCHED
- #define GOS_ISR_ENTER
- #define GOS_ISR_EXIT
- #define GOS_ATOMIC_ENTER
- #define GOS_ATOMIC_EXIT

5.2.1 Detailed Description

5.2.2 Macro Definition Documentation

5.2.2.1 GOS_ATOMIC_ENTER

```
#define GOS_ATOMIC_ENTER
```

Value:

```
    \
                                    {                                                       \
                                    GOS_EXTERN u8_t atomicCntr;                             \
                                    GOS_EXTERN u32_t primask;                               \
                                    if (atomicCntr == 0)                                    \
                                    {                                                       \
                                        GOS_ASM( " cpsid i " :::  "memory" );                \
                                        GOS_ASM( "MRS %0, primask" :  "=r" (primask) ::  "memory"); \
                                        GOS_ASM( "dsb" :::  "memory" );                      \
                                        GOS_ASM( "isb" );                                   \
                                    }                                                       \
                                    atomicCntr++;                                           \
                                    GOS_DISABLE_SCHED                                       \
                                }
```

Atomic operation enter - disable interrupts and kernel rescheduling.

Definition at line 271 of file gos_kernel.h.

5.2.2.2 GOS_ATOMIC_EXIT

```
#define GOS_ATOMIC_EXIT
```

Value:

```
    \
                                    {                                                       \
                                    GOS_EXTERN u8_t atomicCntr;                             \
                                    GOS_EXTERN u32_t primask;                               \
                                    if (atomicCntr > 0)                                     \
                                    {                                                       \
                                        atomicCntr--;                                       \
                                    }                                                       \
                                    if (atomicCntr == 0)                                    \
                                    {                                                       \
                                        GOS_ASM ( "MSR primask, %0" :  :  "r" (primask) :  "memory");
    \
                                        GOS_ASM( " cpsie i " :::  "memory" );                \
                                        GOS_ASM( "dsb" :::  "memory" );                      \
                                        GOS_ASM( "isb" );                                   \
                                    }                                                       \
                                    GOS_ENABLE_SCHED                                        \
                                }
```

Atomic operation exit - enable interrupts kernel rescheduling.

Definition at line 288 of file gos_kernel.h.

5.2.2.3 GOS_DISABLE_SCHED

```
#define GOS_DISABLE_SCHED
```

Value:

```
{                                               \
    GOS_EXTERN u8_t schedDisableCntr;           \
    schedDisableCntr++;                         \
}
```

Disable scheduling.

Definition at line 238 of file gos_kernel.h.

5.2.2.4 GOS_ENABLE_SCHED

```
#define GOS_ENABLE_SCHED
```

Value:

```
{                                               \
    GOS_EXTERN u8_t schedDisableCntr;           \
    if (schedDisableCntr > 0)                   \
    { schedDisableCntr--; }                     \
}
```

Enable scheduling.

Definition at line 245 of file gos_kernel.h.

5.2.2.5 GOS_ISR_ENTER

```
#define GOS_ISR_ENTER
```

Value:

```
{                                               \
    GOS_EXTERN u8_t inIsr;                      \
    if (inIsr == 0) { GOS_DISABLE_SCHED }       \
    inIsr++;                                    \
}
```

Interrupt Service Routine enter.

Definition at line 254 of file gos_kernel.h.

5.2.2.6 GOS_ISR_EXIT

```
#define GOS_ISR_EXIT
```

Value:

```
{                                               \
    GOS_EXTERN u8_t inIsr;                      \
    if (inIsr > 0) { inIsr--; }                 \
    if (inIsr == 0) { GOS_ENABLE_SCHED }        \
}
```

Interrupt service routine exit.

Definition at line 262 of file gos_kernel.h.

5.3 Privilege bit definitions

Macros

- #define GOS_PRIV_TASK_MANIPULATE (1 << 15)
- #define GOS_PRIV_TASK_PRIO_CHANGE (1 << 14)
- #define GOS_PRIV_TRACE (1 << 13)
- #define GOS_PRIV_SIGNALING (1 << 11)
- #define GOS_PRIV_RESERVED_3 (1 << 10)
- #define GOS_PRIV_RESERVED_4 (1 << 9)
- #define GOS_PRIV_RESERVED_5 (1 << 8)

5.3.1 Detailed Description

5.3.2 Macro Definition Documentation

5.3.2.1 GOS_PRIV_RESERVED_3

```
#define GOS_PRIV_RESERVED_3 ( 1 << 10 )
```

Kernel reserved.

Definition at line 335 of file gos_kernel.h.

5.3.2.2 GOS_PRIV_RESERVED_4

```
#define GOS_PRIV_RESERVED_4 ( 1 << 9 )
```

Kernel reserved.

Definition at line 340 of file gos_kernel.h.

5.3.2.3 GOS_PRIV_RESERVED_5

```
#define GOS_PRIV_RESERVED_5 ( 1 << 8 )
```

Kernel reserved.

Definition at line 345 of file gos_kernel.h.

5.3.2.4 GOS_PRIV_SIGNALING

```
#define GOS_PRIV_SIGNALING ( 1 << 11 )
```

Task signal invoking privilege flag.

Definition at line 330 of file gos_kernel.h.

5.3.2.5 GOS_PRIV_TASK_MANIPULATE

```
#define GOS_PRIV_TASK_MANIPULATE ( 1 << 15 )
```

Task manipulation privilege flag.

Definition at line 315 of file gos_kernel.h.

5.3.2.6 GOS_PRIV_TASK_PRIO_CHANGE

```
#define GOS_PRIV_TASK_PRIO_CHANGE ( 1 << 14 )
```

Task priority change privilege flag.

Definition at line 320 of file gos_kernel.h.

5.3.2.7 GOS_PRIV_TRACE

```
#define GOS_PRIV_TRACE ( 1 << 13 )
```

Tracing privilege flag.

Definition at line 325 of file gos_kernel.h.

5.4 Basic data type definitions to be used in a GOS system

Typedefs

- typedef uint8_t bool_t

  Boolean logic type.
- typedef uint8_t u8_t

  8-bit unsigned type.
- typedef uint16_t u16_t

  16-bit unsigned type.
- typedef uint32_t u32_t

  32-bit unsigned type.
- typedef uint64_t u64_t

  64-bit unsigned type.
- typedef int8_t s8_t

  8-bit signed type.
- typedef int16_t s16_t

  16-bit signed type.
- typedef int32_t s32_t

  32-bit signed type.
- typedef int64_t s64_t

  64-bit signed type.
- typedef char char_t

  8-bit character type.
- typedef float float_t

  Single precision float type.
- typedef double double_t

  Double precision float type.
- typedef void void_t

  Void type.

5.4.1 Detailed Description

5.5 Task-related type definitions

Typedefs

- typedef u16_t gos_tid_t

  Task ID type.
- typedef char_t gos_taskName_t[CFG_TASK_MAX_NAME_LENGTH]

  Task name type.
- typedef void_t(∗ gos_task_t) (void_t)

  Task function type.
- typedef u8_t gos_taskPrio_t

  Task priority type.
- typedef u32_t gos_taskSleepTick_t

  Sleep tick type.
- typedef u32_t gos_blockMaxTick_t

  Block max. tick type.
- typedef u32_t gos_taskAddress_t

  Memory address type.
- typedef u32_t gos_taskRunCounter_t

  Run counter type.
- typedef u64_t gos_taskRunTime_t

  Run-time type.
- typedef u32_t gos_taskCSCounter_t

  Context-switch counter type.
- typedef u16_t gos_taskStackSize_t

  Task stack size type.

5.5.1 Detailed Description

5.6 Hook function type definitions

Typedefs

- typedef void_t(∗ gos_taskSwapHook_t) (gos_tid_t, gos_tid_t)

  Task swap hook type.
- typedef void_t(∗ gos_taskIdleHook_t) (void_t)

  Task idle hook type.
- typedef void_t(∗ gos_taskSleepHook_t) (gos_tid_t)

  Task sleep hook type.
- typedef void_t(∗ gos_taskWakeupHook_t) (gos_tid_t)

  Task wake-up hook type.
- typedef void_t(∗ gos_taskSuspendHook_t) (gos_tid_t)

  Task suspend hook type.
- typedef void_t(∗ gos_taskResumeHook_t) (gos_tid_t)

  Task resume hook type.
- typedef void_t(∗ gos_taskBlockHook_t) (gos_tid_t)

  Task block hook type.
- typedef void_t(∗ gos_taskUnblockHook_t) (gos_tid_t)

  Task unblock hook type.
- typedef void_t(∗ gos_taskDeleteHook_t) (gos_tid_t)

  Task delete hook type.
- typedef void_t(∗ gos_sysTickHook_t) (void_t)

  System tick hook type.
- typedef void_t(∗ gos_privilegedHook_t) (void_t)

  Privileged mode hook type.
- typedef void_t(∗ gos_preResetHook_t) (void_t)

  Kernel pre-reset hook type.

5.6.1 Detailed Description

5.7 Time-related definitions

Data Structures

- struct gos_runtime_t

Typedefs

- typedef u16_t gos_microsecond_t

  Microsecond type.
- typedef u16_t gos_millisecond_t

  Millisecond type.
- typedef u8_t gos_second_t

  Second type.
- typedef u8_t gos_minute_t

  Minute type.
- typedef u8_t gos_hour_t

  Hour type.
- typedef u16_t gos_day_t

  Day type.
- typedef u8_t gos_month_t

  Month type.
- typedef u16_t gos_year_t

  Year type.

5.7.1 Detailed Description

5.8 Kernel functions

Functions

- gos_result_t gos_kernelInit (void_t)

  This function initializes the kernel.
- gos_result_t gos_kernelRegisterSwapHook (gos_taskSwapHook_t swapHookFunction)

  Registers a task swap hook function.
- gos_result_t gos_kernelRegisterIdleHook (gos_taskIdleHook_t idleHookFunction)

  Registers an idle hook function.
- gos_result_t gos_kernelRegisterSysTickHook (gos_sysTickHook_t sysTickHookFunction)

  Registers a system tick hook function.
- gos_result_t gos_kernelRegisterPrivilegedHook (gos_privilegedHook_t privilegedHookFunction)

  Registers a privileged hook function.
- gos_result_t gos_kernelRegisterPreResetHook (gos_preResetHook_t preResetHookFunction)

  Registers a pre-reset hook function.
- gos_result_t gos_taskSubscribeDeleteSignal (void_t(∗deleteSignalHandler)(u16_t))

  Subscribes the given handler to the task delete signal.
- gos_result_t gos_kernelSubscribeDumpReadySignal (void_t(∗dumpReadySignalHandler)(u16_t))

  Subscribes the given handler to the dump ready signal.
- u32_t gos_kernelGetSysTicks (void_t)

  Returns the system ticks.
- u16_t gos_kernelGetCpuUsage (void_t)

  Returns the CPU usage.
- gos_result_t gos_kernelStart (void_t)

  Starts the kernel.
- void_t gos_kernelReset (void_t)

  Resets the microcontroller.
- void_t gos_kernelPrivilegedModeSetRequired (void_t)

  Requests a privileged mode setting.
- void_t gos_kernelDelayUs (u16_t microseconds)

  Blocking delay in microsecond range.
- void_t gos_kernelDelayMs (u16_t milliseconds)

  Blocking delay in millisecond range.
- void_t gos_kernelCalculateTaskCpuUsages (bool_t isResetRequired)

  Calculates the CPU usage for the tasks.
- void_t gos_kernelDump (void_t)

  Kernel dump.
- gos_result_t gos_kernelSetMaxCpuLoad (u16_t maxCpuLoad)

  Sets the maximum (global) CPU load.
- gos_result_t gos_kernelGetMaxCpuLoad (u16_t ∗pMaxCpuLoad)

  Gets the maximum (global) CPU load.
- bool_t gos_kernelIsCallerIsr (void_t)

  Returns if the current task is ISR.
- void_t gos_kernelReschedule (gos_kernel_privilege_t privilege)

  Reschedules the kernel.

5.8.1 Detailed Description

5.8.2 Function Documentation

5.8.2.1 gos_kernelCalculateTaskCpuUsages()

```
void_t gos_kernelCalculateTaskCpuUsages (
            bool_t isResetRequired )
```

Calculates the CPU usage for the tasks.

Based on the total system time range, it refreshes the CPU-usage statistics of tasks.

Parameters

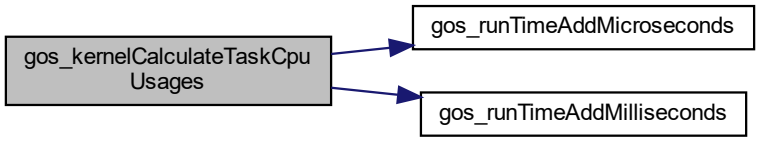
in	*isResetRequired*	Flag to indicate whether the measurement should be reset.

Returns

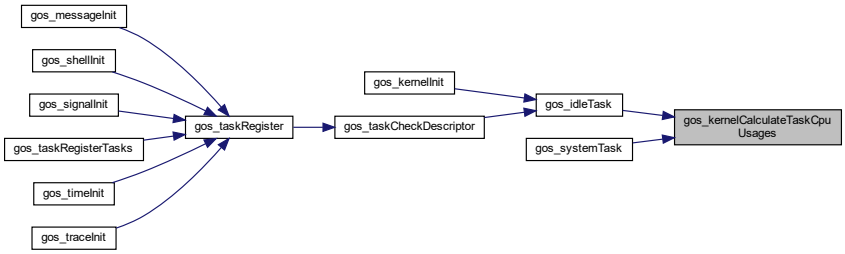
-

Definition at line 658 of file gos_kernel.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.8.2.2 gos_kernelDelayMs()

```
void_t gos_kernelDelayMs (
            u16_t milliseconds )
```

Blocking delay in millisecond range.

This function waits in a while loop until the given number of system ticks have elapsed.

Parameters

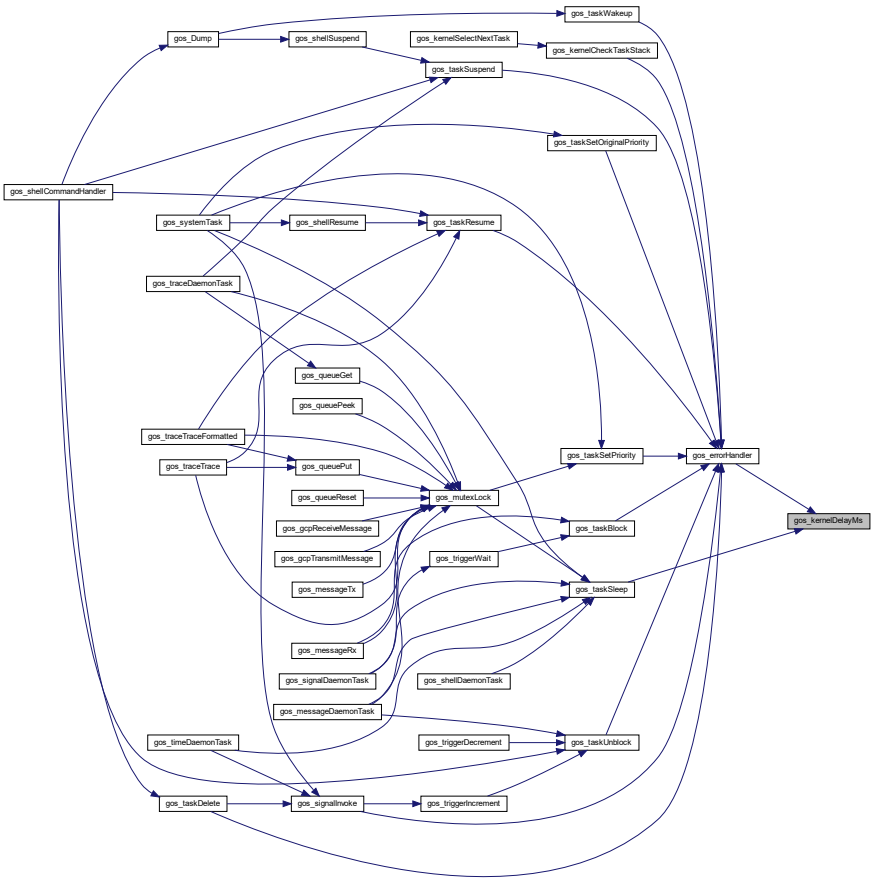
in	*milliseconds*	Milliseconds to wait.

Returns

-

Definition at line 642 of file gos_kernel.c.

Here is the caller graph for this function:



5.8.2.3 gos_kernelDelayUs()

```
void_t gos_kernelDelayUs (
            u16_t microseconds )
```

Blocking delay in microsecond range.

This function waits in a while loop until the given number of milliseconds have elapsed based on the system timer.

Parameters

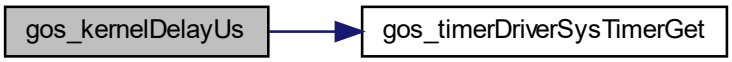
in	*microseconds*	Microseconds to wait.

Returns

-

Definition at line 620 of file gos_kernel.c.

Here is the call graph for this function:



5.8.2.4 gos_kernelDump()

```
void_t gos_kernelDump (
            void_t  )
```

Kernel dump.

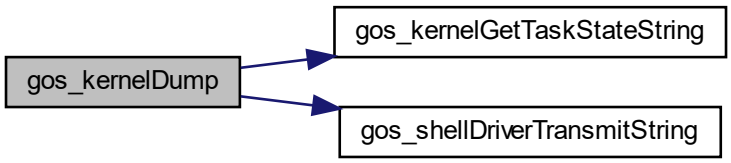
This function prints the kernel configuration and task data to the trace output.

Returns

-

Definition at line 760 of file gos_kernel.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.8.2.5 gos_kernelGetCpuUsage()

```
u16_t gos_kernelGetCpuUsage (
            void_t  )
```

Returns the CPU usage.

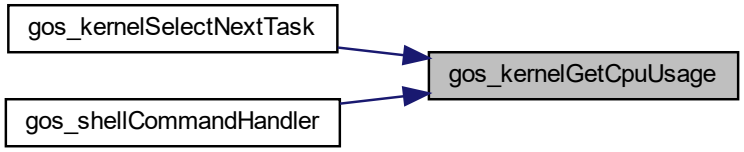
Refreshes the CPU statistics and returns the current CPU usage in [CPU%] $*$ 100 format that results in a range of 0...10000 where the last two digits represent two decimals. The usage is 100% - idle task%.

Returns

Overall CPU usage.

Definition at line 576 of file gos_kernel.c.

Here is the caller graph for this function:



5.8.2.6 gos_kernelGetMaxCpuLoad()

```
gos_result_t gos_kernelGetMaxCpuLoad (
            u16_t * pMaxCpuLoad )
```

Gets the maximum (global) CPU load.

Returns the value of the maximum CPU load (limit).

Parameters

out	*pMaxCpuLoad*	Target variable to store the maximum CPU load.

Returns

Result of maximum CPU load getting.

Return values

GOS_SUCCESS	Maximum CPU load getting successful.
GOS_ERROR	Target variable is NULL.

Definition at line 898 of file gos_kernel.c.

5.8.2.7 gos_kernelGetSysTicks()

```
u32_t gos_kernelGetSysTicks (
```

```
            void_t  )
```

Returns the system ticks.

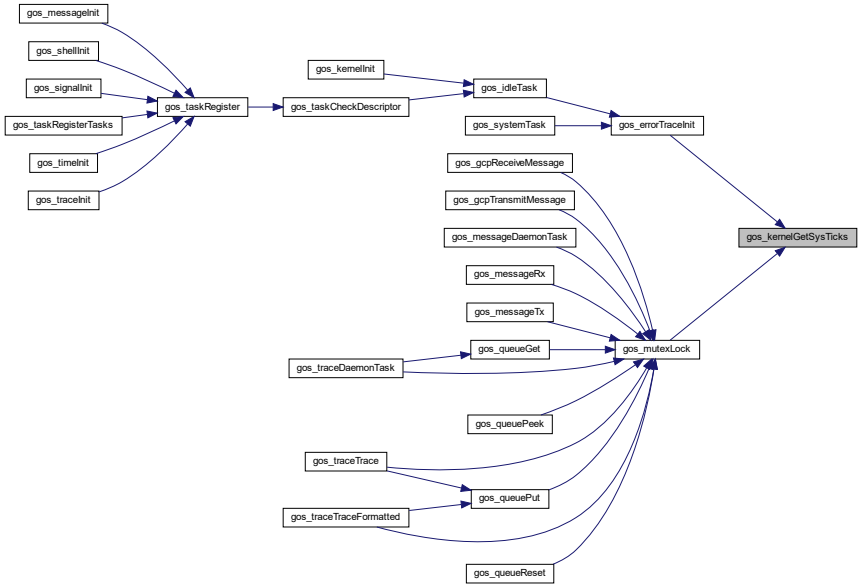
Returns the internal system tick counter value.

Returns

System ticks.

Definition at line 565 of file gos_kernel.c.

Here is the caller graph for this function:



5.8.2.8 gos_kernelInit()

```
gos_result_t gos_kernelInit (
            void_t  )
```

This function initializes the kernel.

Initializes the internal task array, fills out the PSP for the idle task, and registers the kernel dump task and suspends it.

Returns

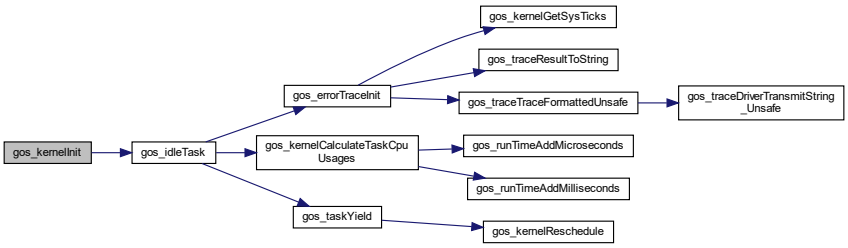
Result of initialization.

Return values

GOS_SUCCESS	Kernel initialization successful.
GOS_ERROR	Kernel task suspension unsuccessful.

Definition at line 293 of file gos_kernel.c.

Here is the call graph for this function:



5.8.2.9 gos_kernelIsCallerIsr()

```
bool_t gos_kernelIsCallerIsr (
            void_t  )
```

Returns if the current task is ISR.

Returns if the inIsr flag is greater than zero.

Returns

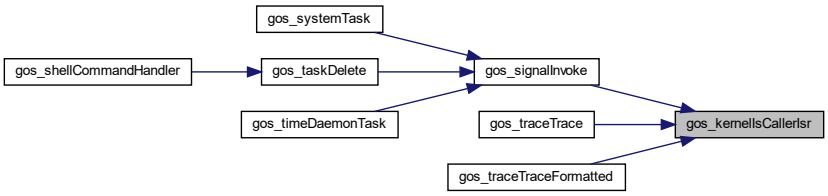
Whether the caller is ISR (Interrupt Service Routine).

Return values

GOS_TRUE	Caller is ISR.
GOS_FALSE	Caller is not ISR.

Definition at line 924 of file gos_kernel.c.

Here is the caller graph for this function:



5.8.2.10 gos_kernelPrivilegedModeSetRequired()

```
void_t gos_kernelPrivilegedModeSetRequired (
            void_t  )
```

Requests a privileged mode setting.

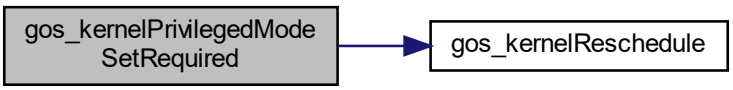
Initiates an SVC call while setting the corresponding flag to set the execution mode to privileged (permanently). After this, it calls the corresponding hook function, so the caller can access special registers immediately.

Returns

-

Definition at line 607 of file gos_kernel.c.

Here is the call graph for this function:



5.8.2.11 gos_kernelRegisterIdleHook()

```
gos_result_t gos_kernelRegisterIdleHook (
            gos_taskIdleHook_t idleHookFunction )
```

Registers an idle hook function.

Checks whether the param is NULL pointer and a hook function is already registered, and both conditions are false, it registers the hook function.

Parameters

in	*idleHookFunction*	Idle hook function.

Returns

Result of registration.

Return values

GOS_SUCCESS	Registration successful.
GOS_ERROR	Registration failed (hook function already exists or parameter is NULL).

5.8.2.12 gos_kernelRegisterPreResetHook()

```
gos_result_t gos_kernelRegisterPreResetHook (
            gos_preResetHook_t preResetHookFunction )
```

Registers a pre-reset hook function.

Checks if a hook function has already been registered, and, if not, it registers the new hook function.

Parameters

in	*preResetHookFunction*	Pre-reset hook function.

Returns

Result of registration.

Return values

GOS_SUCCESS	Registration successful.
GOS_ERROR	Registration failed (hook function already exists or parameter is NULL).

Definition at line 470 of file gos_kernel.c.

5.8.2.13 gos_kernelRegisterPrivilegedHook()

```
gos_result_t gos_kernelRegisterPrivilegedHook (
            gos_privilegedHook_t privilegedHookFunction )
```

Registers a privileged hook function.

Checks if a hook function has already been registered, and, if not, it registers the new hook function.

Parameters

in	*privilegedHookFunction*	Privileged hook function.

Returns

Result of registration.

Return values

GOS_SUCCESS	Registration successful.
GOS_ERROR	Registration failed (hook function already exists or parameter is NULL).

Definition at line 444 of file gos_kernel.c.

5.8.2.14 gos_kernelRegisterSwapHook()

```
gos_result_t gos_kernelRegisterSwapHook (
            gos_taskSwapHook_t swapHookFunction )
```

Registers a task swap hook function.

Checks whether the param is NULL pointer and a hook function is already registered, and both conditions are false, it registers the hook function.

Parameters

in	*swapHookFunction*	Swap hook function.

Returns

Result of registration.

Return values

GOS_SUCCESS	Registration successful.
GOS_ERROR	Registration failed (hook function already exists or parameter is NULL).

Definition at line 392 of file gos_kernel.c.

5.8.2.15 gos_kernelRegisterSysTickHook()

```
gos_result_t gos_kernelRegisterSysTickHook (
            gos_sysTickHook_t sysTickHookFunction )
```

Registers a system tick hook function.

Checks whether the param is NULL pointer and a hook function is already registered, and both conditions are false, it registers the hook function.

Parameters

in	*sysTickHookFunction*	System tick hook function.

Returns

Result of registration.

Return values

GOS_SUCCESS	Registration successful.
GOS_ERROR	Registration failed (hook function already exists or parameter is NULL).

Definition at line 418 of file gos_kernel.c.

5.8.2.16 gos_kernelReschedule()

```
void_t gos_kernelReschedule (
            gos_kernel_privilege_t privilege )
```

Reschedules the kernel.

Based on the privilege, it invokes a kernel reschedule event.

Parameters

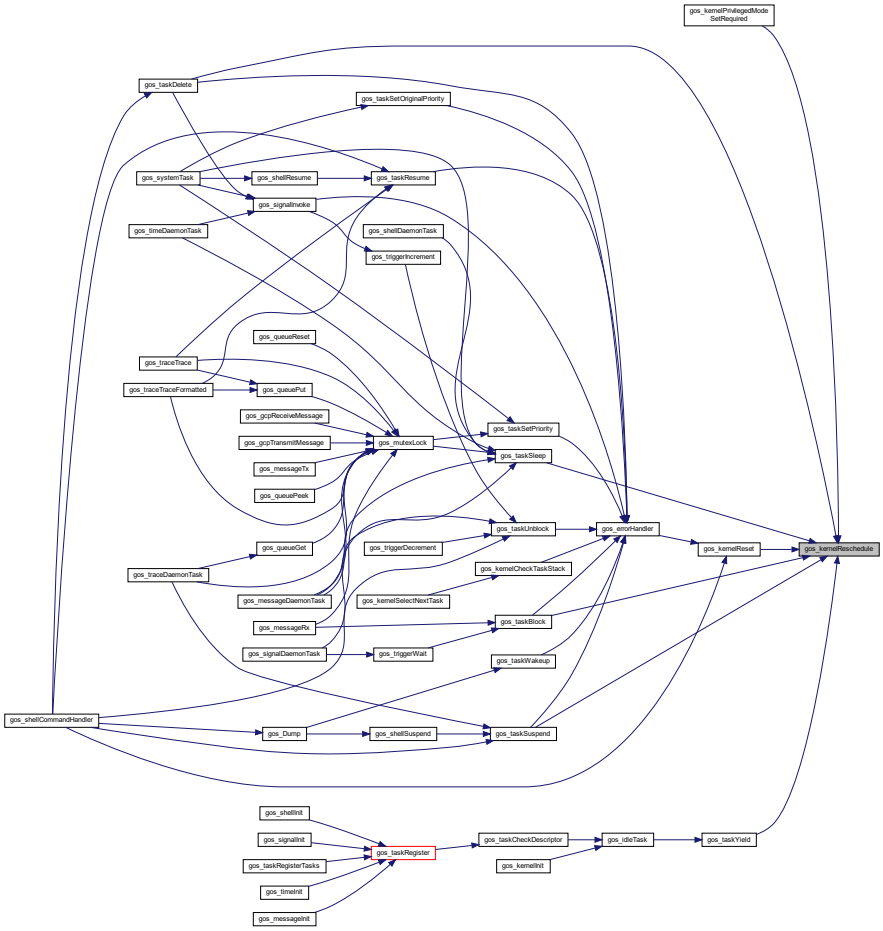
in	*privilege*	Privilege level.

Returns

-

Definition at line 983 of file gos_kernel.c.

Here is the caller graph for this function:



5.8.2.17 gos_kernelReset()

```
void_t gos_kernelReset (
            void_t  )
```

Resets the microcontroller.

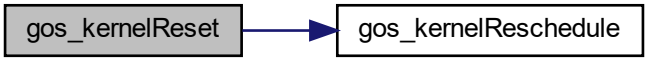
Sets the reset required flag and gets privileged access to reset the controller.

Returns

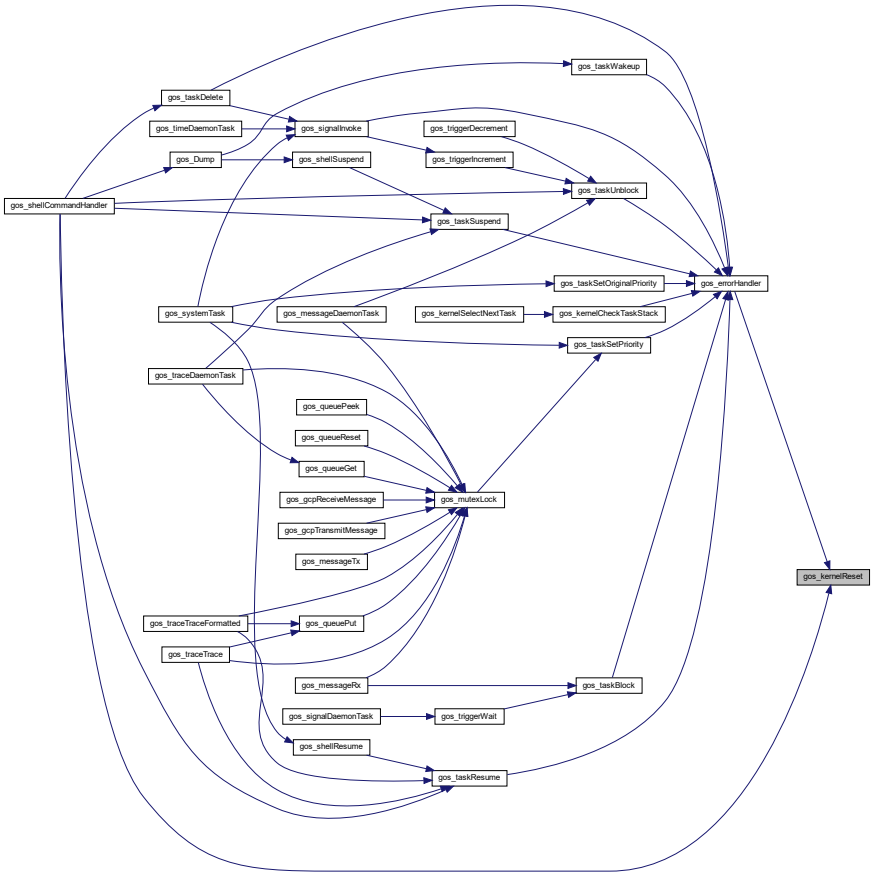
-

Definition at line 587 of file gos_kernel.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.8.2.18 gos_kernelSetMaxCpuLoad()

```
gos_result_t gos_kernelSetMaxCpuLoad (
            u16_t maxCpuLoad )
```

Sets the maximum (global) CPU load.

Sets the value of the global CPU load above which the scheduler will start running the idle task until the CPU load falls below the limit value.

Parameters

in	*maxCpuLoad*	Desired maximum CPU load (0...10000 where 100 % = 10000).

Returns

Result of maximum CPU load setting.

Return values

GOS_SUCCESS	Maximum CPU load setting successful.
GOS_ERROR	Desired value is out of range.

Definition at line 872 of file gos_kernel.c.

5.8.2.19 gos_kernelStart()

```
gos_result_t gos_kernelStart (
            void_t  )
```

Starts the kernel.

Prepares the PSP for the first task, changes to unprivileged level, initializes the system timer value, and starts executing the first task.

Returns

Result of kernel start.

Return values

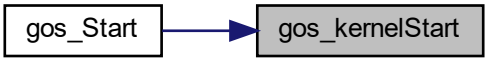
GOS_ERROR	First task terminated.

Definition at line 354 of file gos_kernel.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.8.2.20 gos_kernelSubscribeDumpReadySignal()

```
gos_result_t gos_kernelSubscribeDumpReadySignal (
             void_t(*)(u16_t) dumpReadySignalHandler )
```

Subscribes the given handler to the dump ready signal.

Subscribes the given handler to the dump ready signal.

Parameters

in	*dumpReadySignalHandler*	Dump ready signal handler function.

Returns

Result of subscription.

Return values

GOS_SUCCESS	Subscription successful.
GOS_ERROR	Subscription failed or signal handler is NULL.

5.8.2.21 gos_taskSubscribeDeleteSignal()

```
gos_result_t gos_taskSubscribeDeleteSignal (
            void_t(*)(u16_t) deleteSignalHandler )
```

Subscribes the given handler to the task delete signal.

Subscribes the given handler to the task delete signal.

Parameters

in	*deleteSignalHandler*	Delete signal handler function.

Returns

Result of subscription.

Return values

GOS_SUCCESS	Subscription successful.
GOS_ERROR	Subscription failed or signal handler is NULL.

5.9 Task-related functions

Functions

- gos_result_t gos_taskRegisterTasks (gos_taskDescriptor_t *taskDescriptors, u16_t arraySize)
  This function registers an array of tasks for scheduling.
- gos_result_t gos_taskRegister (gos_taskDescriptor_t *taskDescriptor, gos_tid_t *taskId)
  This function registers a task for scheduling.
- gos_result_t gos_taskSleep (gos_taskSleepTick_t sleepTicks)
  Sends the current task to sleeping state.
- gos_result_t gos_taskWakeup (gos_tid_t taskId)
  Wakes up the given task.
- gos_result_t gos_taskSuspend (gos_tid_t taskId)
  Sends the given task to suspended state.
- gos_result_t gos_taskResume (gos_tid_t taskId)
  Resumes the given task.
- gos_result_t gos_taskBlock (gos_tid_t taskId, gos_blockMaxTick_t blockTicks)
  Sends the given task to blocked state.
- gos_result_t gos_taskUnblock (gos_tid_t taskId)
  Unblocks the given task.
- gos_result_t gos_taskDelete (gos_tid_t taskId)
  Deletes the given task from the scheduling array.
- gos_result_t gos_taskSetPriority (gos_tid_t taskId, gos_taskPrio_t taskPriority)
  Sets the current priority of the given task to the given value (for temporary change).
- gos_result_t gos_taskSetOriginalPriority (gos_tid_t taskId, gos_taskPrio_t taskPriority)
  Sets the original priority of the given task to the given value (for permanent change).
- gos_result_t gos_taskGetPriority (gos_tid_t taskId, gos_taskPrio_t *taskPriority)
  Gets the current priority of the given task.
- gos_result_t gos_taskGetOriginalPriority (gos_tid_t taskId, gos_taskPrio_t *taskPriority)
  Gets the original priority of the given task.
- gos_result_t gos_taskAddPrivilege (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)
  Adds the given privileges to the given task.
- gos_result_t gos_taskRemovePrivilege (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)
  Removes the given privileges from the given task.
- gos_result_t gos_taskSetPrivileges (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)
  Sets the given privileges for the given task.
- gos_result_t gos_taskGetPrivileges (gos_tid_t taskId, gos_taskPrivilegeLevel_t *pPrivileges)
  Gets the privileges of the given task.
- gos_result_t gos_taskGetName (gos_tid_t taskId, gos_taskName_t taskName)
  Gets the task name of the task with the given ID.
- gos_result_t gos_taskGetId (gos_taskName_t taskName, gos_tid_t *pTaskId)
  Gets the task ID of the task with the given name.
- gos_result_t gos_taskGetCurrentId (gos_tid_t *pTaskId)
  Returns the ID of the currently running task.
- gos_result_t gos_taskGetData (gos_tid_t taskId, gos_taskDescriptor_t *pTaskData)
  Returns the task data of the given task.
- gos_result_t gos_taskGetDataByIndex (u16_t taskIndex, gos_taskDescriptor_t *pTaskData)
  Returns the task data of the given task.
- gos_result_t gos_taskGetNumber (u16_t *pTaskNum)
  Returns the number of registered tasks.
- gos_result_t gos_taskYield (void_t)
  Yields the current task.

5.9.1 Detailed Description

5.9.2 Function Documentation

5.9.2.1 gos_taskAddPrivilege()

```
gos_result_t gos_taskAddPrivilege (
            gos_tid_t taskId,
            gos_taskPrivilegeLevel_t privileges )
```

Adds the given privileges to the given task.

Checks the caller task if it has the privilege to modify task privileges and if so, it adds the given privileges to the given task.

Parameters

in	*taskId*	ID of the task to give the privileges to.
in	*privileges*	Privileges to be added.

Returns

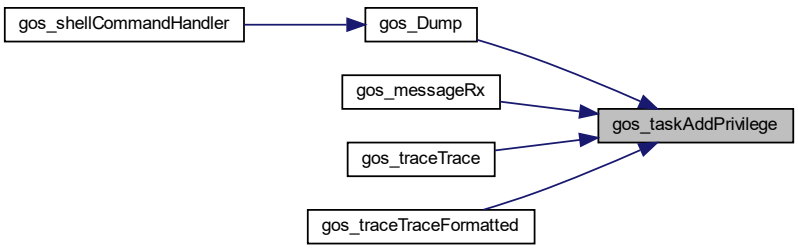
Result of privilege adding.

Return values

GOS_SUCCESS	Privileges added successfully.
GOS_ERROR	Invalid task ID or caller does not have the privilege to modify task privileges.

Definition at line 885 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.2 gos_taskBlock()

```
gos_result_t gos_taskBlock (
            gos_tid_t taskId,
            gos_blockMaxTick_t blockTicks )
```

Sends the given task to blocked state.

Checks the given task ID and its state, modified it to blocked, and if there is a block hook function registered, it calls it. If the blocked function is the currently running one, it invokes a rescheduling.

Parameters

in	*taskId*	ID of the task to be blocked.
in	*blockTicks*	Maximum number of ticks for blocking the task (timeout).

Returns

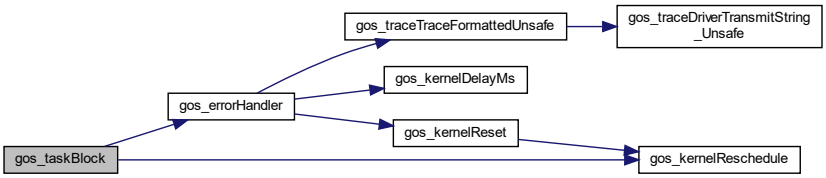
Result of task blocking.

Return values

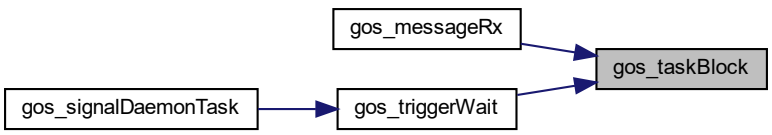
GOS_SUCCESS	Task blocked successfully.
GOS_ERROR	Task ID is invalid, or task state is not ready.

Definition at line 519 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.3 gos_taskDelete()

```
gos_result_t gos_taskDelete (
            gos_tid_t taskId )
```

Deletes the given task from the scheduling array.

Checks the given task ID and its state, modifies it to zombie, and if there is a delete hook function registered, it calls it.

Parameters

in	*taskId*	ID of the task to be deleted.

Returns

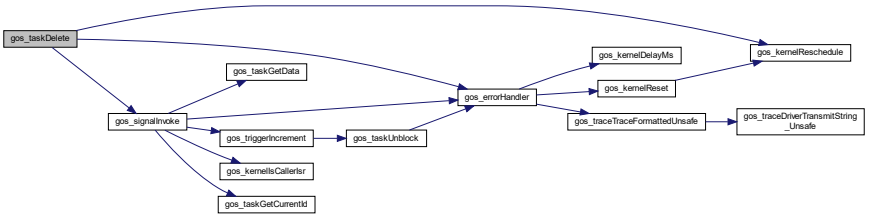
Result of deletion.

Return values

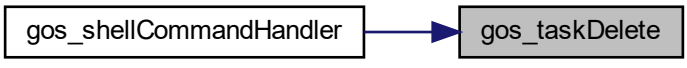
GOS_SUCCESS	Task deleted successfully.
GOS_ERROR	Task is already a zombie.

Definition at line 651 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.4 gos_taskGetCurrentId()

```
gos_result_t gos_taskGetCurrentId (
            gos_tid_t * pTaskId )
```

Returns the ID of the currently running task.

Returns the ID of the currently running task.

Parameters

out	*p↩ TaskId*	Pointer to a task ID variable to store the current task ID.

Returns

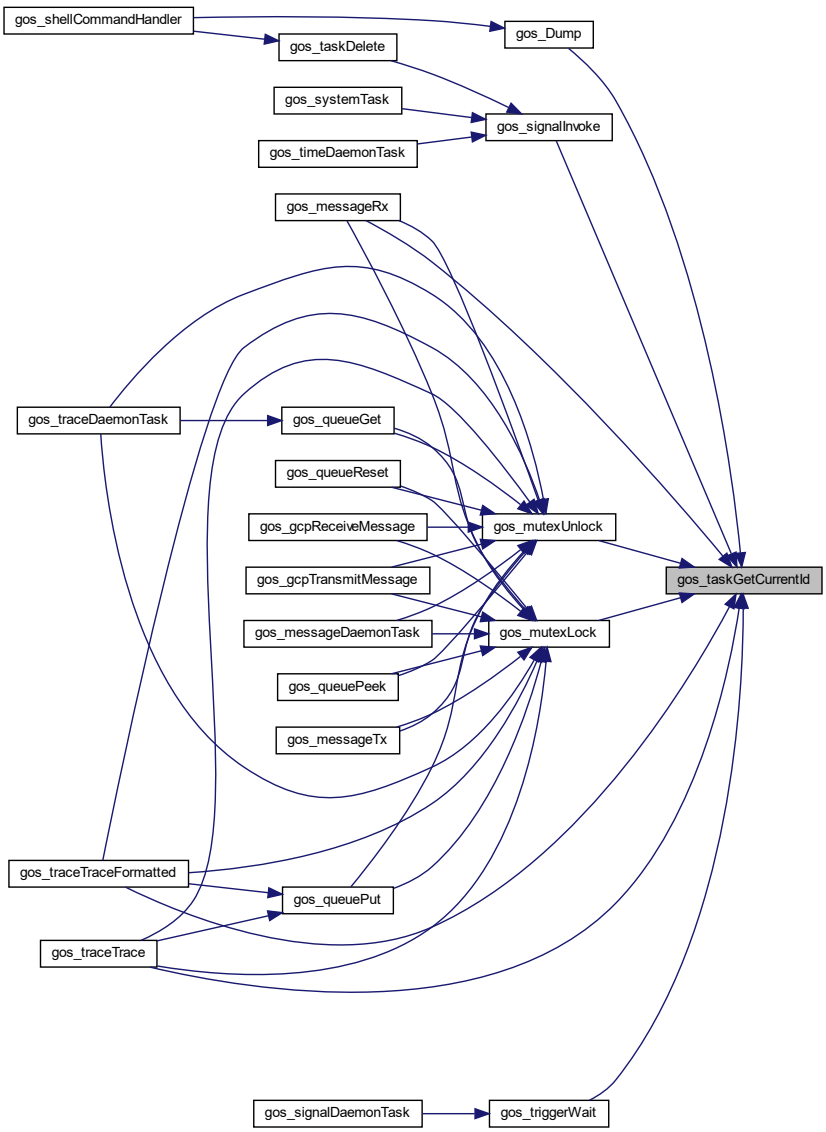
Result of current task ID get.

Return values

GOS_SUCCESS	Current task ID returned successfully.
GOS_ERROR	Task ID pointer is NULL.

Definition at line 1086 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.5 gos_taskGetData()

```
gos_result_t gos_taskGetData (
            gos_tid_t taskId,
            gos_taskDescriptor_t * pTaskData )
```

Returns the task data of the given task.

Based on the task ID, it copies the content of the internal task descriptor array element to the given task descriptor.

Parameters

in	*taskId*	ID of the task to get the data of.
out	*pTaskData*	Pointer to the task descriptor to save the task data in.

Returns

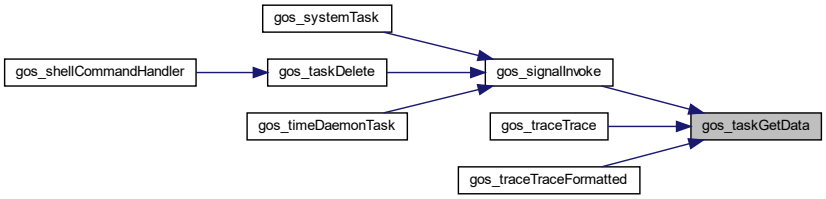
Result of task data get.

Return values

GOS_SUCCESS	Task data copied successfully.
GOS_ERROR	Invalid task ID or task data pointer is NULL.

Definition at line 1114 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.6 gos_taskGetDataByIndex()

```
gos_result_t gos_taskGetDataByIndex (
            u16_t taskIndex,
            gos_taskDescriptor_t * pTaskData )
```

Returns the task data of the given task.

Based on the task index, it copies the content of the internal task descriptor array element to the given task descriptor.

Parameters

in	*taskIndex*	Index of the task to get the data of.
out	*pTaskData*	Pointer to the task descriptor to save the task data in.

Returns

Result of task data get.

Return values

GOS_SUCCESS	Task data copied successfully.
GOS_ERROR	Invalid task index or task data pointer is NULL.

Definition at line 1154 of file gos_task.c.

5.9.2.7 gos_taskGetId()

```
gos_result_t gos_taskGetId (
            gos_taskName_t taskName,
            gos_tid_t * pTaskId )
```

Gets the task ID of the task with the given name.

This function loops through the internal task array and tries to find the given task name to get the corresponding task ID.

Parameters

in	*taskName*	Name of the task (string).
out	*pTaskId*	Pointer to a task ID variable to store the returned ID.

Returns

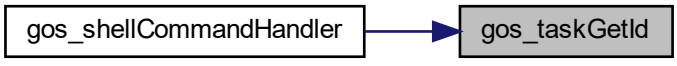
Success of task ID get.

Return values

GOS_SUCCESS	Task ID found successfully.
GOS_ERROR	Task name not found.

Definition at line 1055 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.8 gos_taskGetName()

```
gos_result_t gos_taskGetName (
            gos_tid_t taskId,
            gos_taskName_t taskName )
```

Gets the task name of the task with the given ID.

Copies the task name corresponding with the given task ID to the task name variable.

Parameters

in	*taskId*	Pointer to a task ID variable to store the returned ID.
out	*taskName*	Task name pointer to store the returned task name.

Returns

Success of task name get.

Return values

GOS_SUCCESS	Task name found successfully.
GOS_ERROR	Invalid task ID or task name variable is NULL.

Definition at line 1024 of file gos_task.c.

5.9.2.9 gos_taskGetNumber()

```
gos_result_t gos_taskGetNumber (
            u16_t * pTaskNum )
```

Returns the number of registered tasks.

Loops through the internal task array and counts the entries where a task function is registered.

Parameters

out	*pTaskNum*	Variable to store the number of tasks.

Returns

Success of task number counting.

Return values

GOS_SUCCESS	Task number counted successfully.
GOS_ERROR	Target variable is NULL pointer.

Definition at line 1186 of file gos_task.c.

5.9.2.10 gos_taskGetOriginalPriority()

```
gos_result_t gos_taskGetOriginalPriority (
            gos_tid_t taskId,
            gos_taskPrio_t * taskPriority )
```

Gets the original priority of the given task.

Checks the given parameters and saves the original priority in the given variable.

Parameters

in	*taskId*	ID of the task to get the priority of.
out	*taskPriority*	Pointer to a priority variable to store the priority in.

Returns

Result of priority getting.

Return values

GOS_SUCCESS	Original priority getting successfully.
GOS_ERROR	Invalid task ID or priority variable is NULL.

Definition at line 854 of file gos_task.c.

5.9.2.11 gos_taskGetPriority()

```
gos_result_t gos_taskGetPriority (
            gos_tid_t taskId,
            gos_taskPrio_t * taskPriority )
```

Gets the current priority of the given task.

Checks the given parameters and saves the current priority in the given variable.

Parameters

in	*taskId*	ID of the task to get the priority of.
out	*taskPriority*	Pointer to a priority variable to store the priority in.

Returns

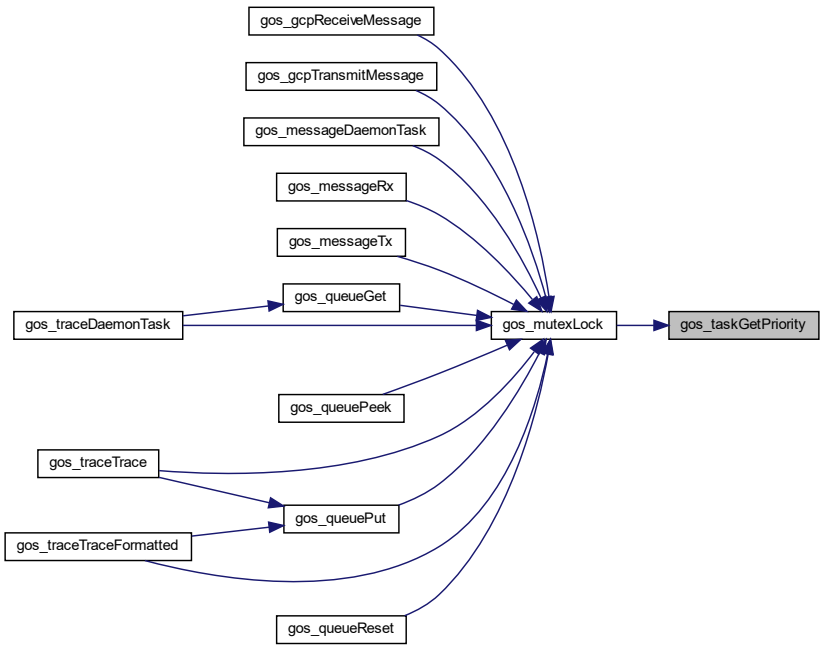
Result of priority getting.

Return values

GOS_SUCCESS	Current priority getting successfully.
GOS_ERROR	Invalid task ID or priority variable is NULL.

Definition at line 823 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.12 gos_taskGetPrivileges()

```
gos_result_t gos_taskGetPrivileges (
            gos_tid_t taskId,
            gos_taskPrivilegeLevel_t * pPrivileges )
```

Gets the privileges of the given task.

Returns the privilege flags of the given task.

Parameters

in	*taskId*	ID of the task to get the privileges of.
out	*pPrivileges*	Variable to store the privilege flags.

Returns

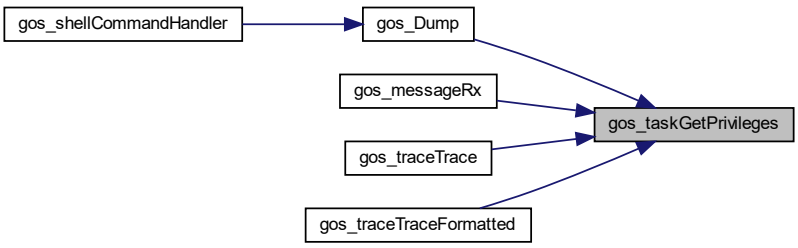
Result of privilege getting.

Return values

GOS_SUCCESS	Privileges get successful.
GOS_ERROR	Invalid task ID or privilege variable is NULL pointer.

Definition at line 975 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.13 gos_taskRegister()

```
gos_result_t gos_taskRegister (
            gos_taskDescriptor_t * taskDescriptor,
            gos_tid_t * taskId )
```

This function registers a task for scheduling.

Checks the task descriptor parameters and then tries to find the next empty slot in the internal task array. When it is found, it registers the task in that slot.

Parameters

in	*taskDescriptor*	Pointer to a task descriptor structure.
in	*taskId*	Pointer to a variable to hold to assigned task ID value.

Returns

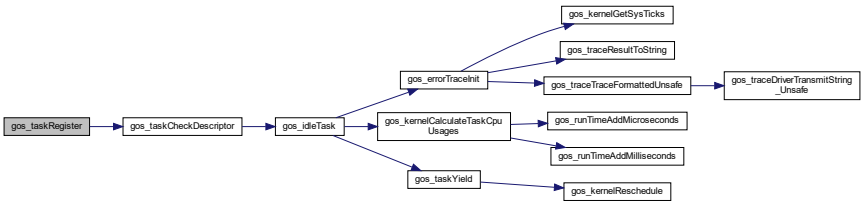
Result of task registration.

Return values

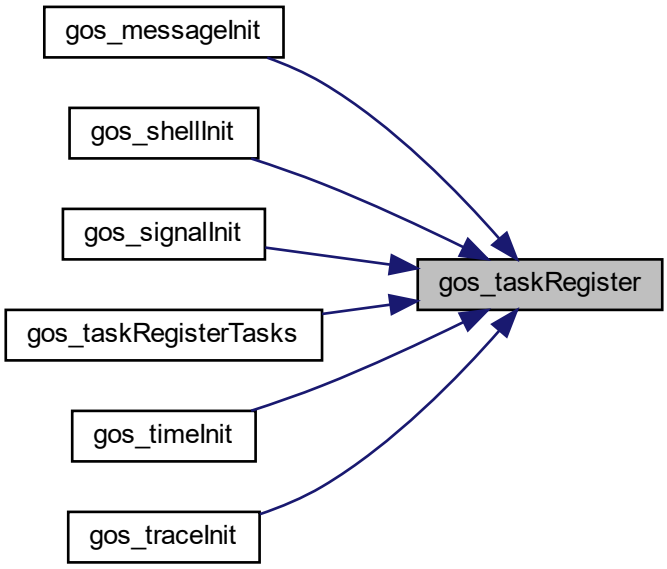
GOS_SUCCESS	Task registered successfully.
GOS_ERROR	Invalid task descriptor (NULL function pointer, invalid priority level, invalid stack size, idle task registration, or stack size is not 4-byte-aligned) or task array is full.

Definition at line 164 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.14 gos_taskRegisterTasks()

```
gos_result_t gos_taskRegisterTasks (
            gos_taskDescriptor_t * taskDescriptors,
            u16_t arraySize )
```

This function registers an array of tasks for scheduling.

Checks the task descriptor array pointer and registers the tasks one by one.

Parameters

in	*taskDescriptors*	Pointer to a task descriptor structure array.
in	*arraySize*	Size of the array in bytes.

Returns

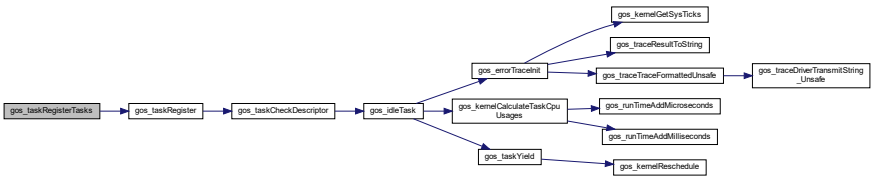
Result of task registration.

Return values

GOS_SUCCESS	Tasks registered successfully.
GOS_ERROR	Invalid task descriptor (NULL function pointer, invalid priority level, invalid stack size, idle task registration, or stack size is not 4-byte-aligned) in one of the array elements or task array is full.

Definition at line 116 of file gos_task.c.

Here is the call graph for this function:



5.9.2.15 gos_taskRemovePrivilege()

```
gos_result_t gos_taskRemovePrivilege (
            gos_tid_t taskId,
            gos_taskPrivilegeLevel_t privileges )
```

Removes the given privileges from the given task.

Checks the caller task if it has the privilege to modify task privileges and if so, it removes the given privileges from the given task.

Parameters

in	*taskId*	ID of the task to remove the privileges from.
in	*privileges*	Privileges to be removed.

Returns

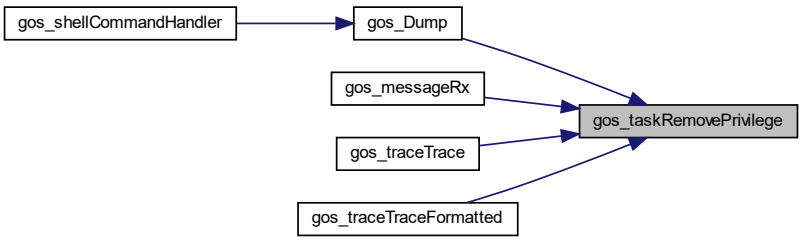
Result of privilege removing.

Return values

GOS_SUCCESS	Privileges removed successfully.
GOS_ERROR	Invalid task ID or caller does not have the privilege to modify task privileges.

Definition at line 915 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.16 gos_taskResume()

```
gos_result_t gos_taskResume (
            gos_tid_t taskId )
```

Resumes the given task.

Checks the given task ID and its state, modified it to ready, and if there is a resume hook function registered, it calls it.

Parameters

in	*taskId*	ID of the task to be resumed.

Returns

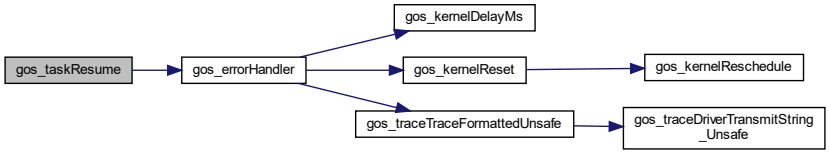
Result of task resumption.

Return values

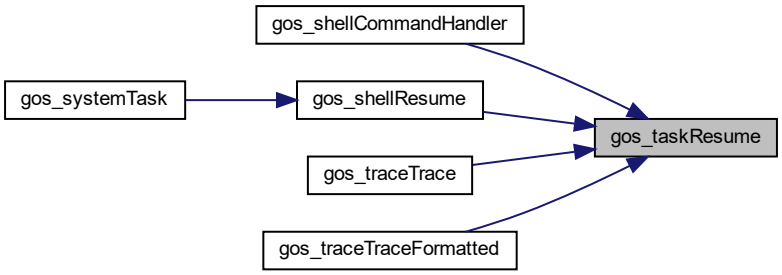
GOS_SUCCESS	Task resumed successfully.
GOS_ERROR	Task ID is invalid, or task is not suspended.

Definition at line 468 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.17 gos_taskSetOriginalPriority()

```
gos_result_t gos_taskSetOriginalPriority (
            gos_tid_t taskId,
            gos_taskPrio_t taskPriority )
```

Sets the original priority of the given task to the given value (for permanent change).

Checks the given parameters and sets the original priority of the given task.

Parameters

in	*taskId*	ID of the task to change the priority of.
in	*taskPriority*	The desired task priority.

Returns

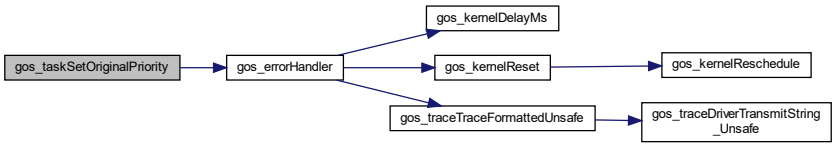
Result of priority change.

Return values

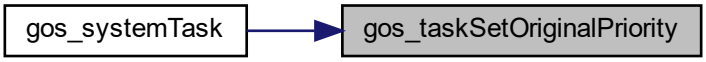
GOS_SUCCESS	Original priority changed successfully.
GOS_ERROR	Invalid task ID or priority.

Definition at line 777 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.18 gos_taskSetPriority()

```
gos_result_t gos_taskSetPriority (
            gos_tid_t taskId,
            gos_taskPrio_t taskPriority )
```

Sets the current priority of the given task to the given value (for temporary change).

Checks the given parameters and sets the current priority of the given task.

Parameters

in	*taskId*	ID of the task to change the priority of.
in	*taskPriority*	The desired task priority.

Returns

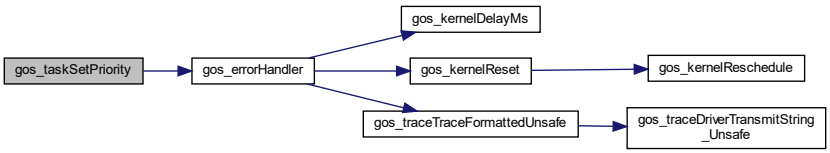
Result of priority change.

Return values

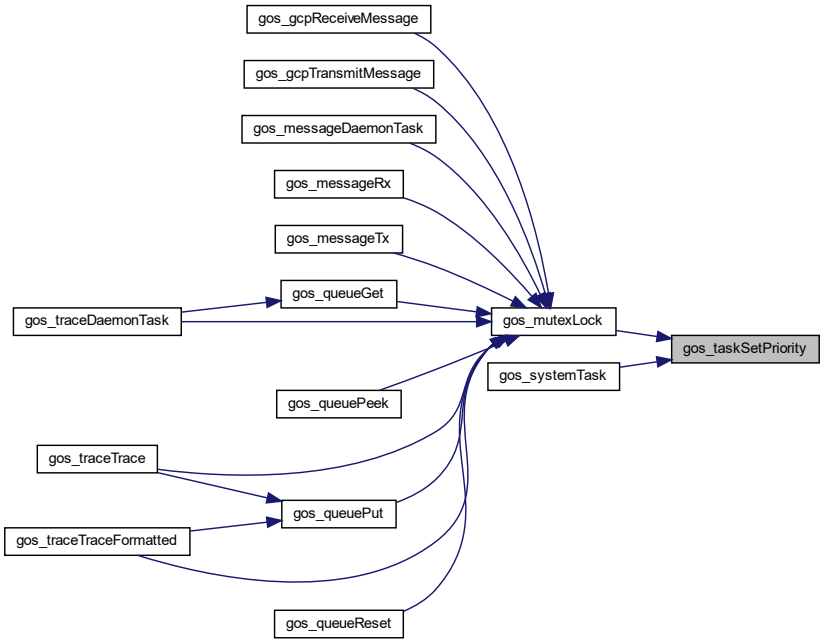
GOS_SUCCESS	Current priority changed successfully.
GOS_ERROR	Invalid task ID or priority.

Definition at line 731 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.19 gos_taskSetPrivileges()

```
gos_result_t gos_taskSetPrivileges (
            gos_tid_t taskId,
            gos_taskPrivilegeLevel_t privileges )
```

Sets the given privileges for the given task.

Checks the caller task if it has the privilege to modify task privileges and if so, it sets the given privileges for the given task.

Parameters

in	*taskId*	ID of the task to set the privileges for.
in	*privileges*	Privileges to be set.

Returns

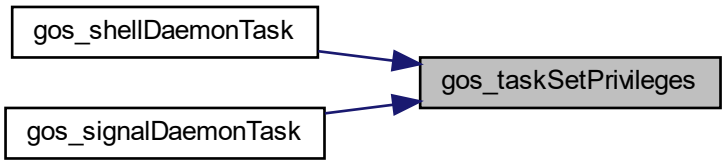
Result of privilege setting.

Return values

GOS_SUCCESS	Privileges set successfully.
GOS_ERROR	Invalid task ID or caller does not have the privilege to modify task privileges.

Definition at line 945 of file gos_task.c.

Here is the caller graph for this function:



5.9.2.20 gos_taskSleep()

```
gos_result_t gos_taskSleep (
            gos_taskSleepTick_t sleepTicks )
```

Sends the current task to sleeping state.

Checks the current task and its state, modifies it to sleeping, and if there is a sleep hook function registered, it calls it. Then, it invokes a rescheduling.

Parameters

in	*sleepTicks*	Minimum number of ticks until the task should remain in sleeping state.

Returns

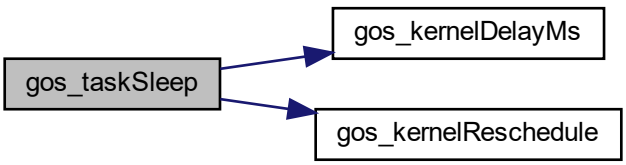
Result of task sleeping.

Return values

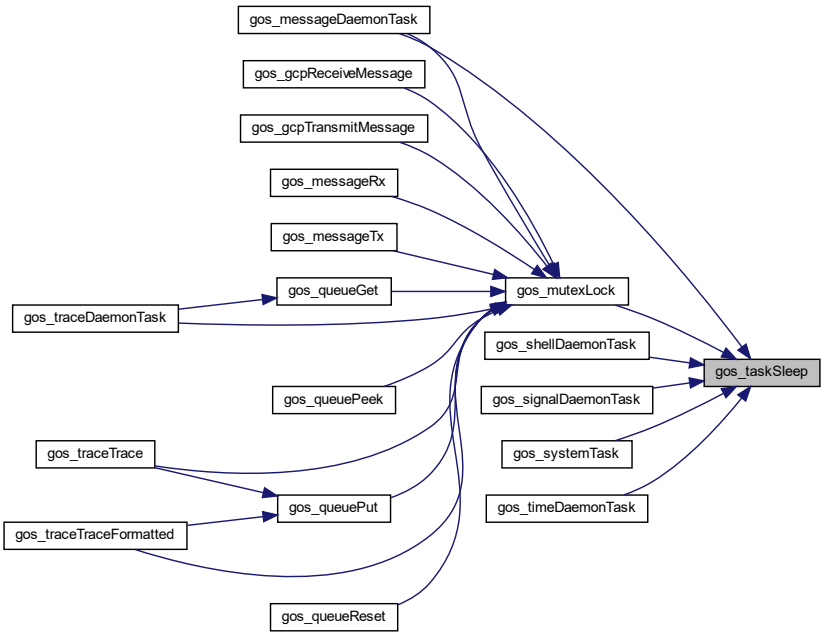
GOS_SUCCESS	Task successfully sent to sleeping state.
GOS_ERROR	Function called from idle task or task state is not ready.

Definition at line 287 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.21 gos_taskSuspend()

```
gos_result_t gos_taskSuspend (
            gos_tid_t taskId )
```

Sends the given task to suspended state.

Checks the given task ID and its state, modified it to suspended, and if there is a suspend hook function registered, it calls it. If the suspended function is the currently running one, it invokes a rescheduling.

Parameters

in	*taskId*	ID of the task to be suspended.

Returns

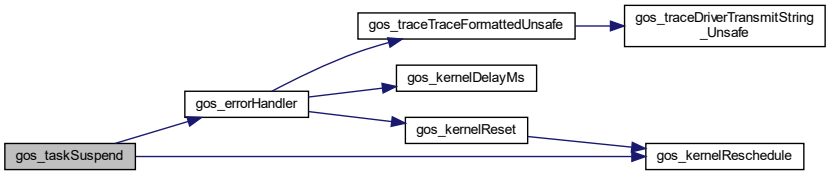
Result of task suspension.

Return values

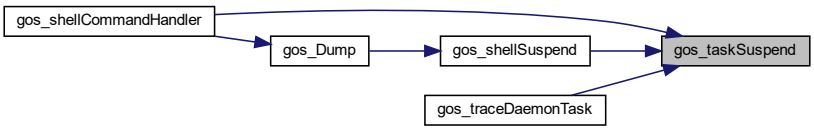
GOS_SUCCESS	Task suspended successfully.
GOS_ERROR	Task ID is invalid, or task state is not ready or sleeping.

Definition at line 394 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.22 gos_taskUnblock()

```
gos_result_t gos_taskUnblock (
            gos_tid_t taskId )
```

Unblocks the given task.

Checks the given task ID and its state, modified it to ready, and if there is an unblock hook function registered, it calls it.

Parameters

in	*taskId*	ID of the task to be unblocked.

Returns

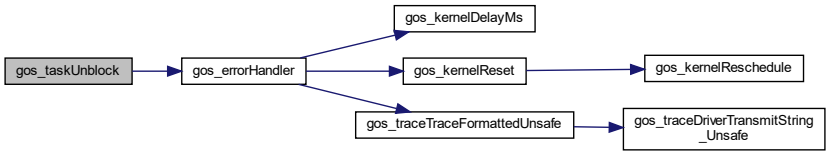
Result of task unblocking.

Return values

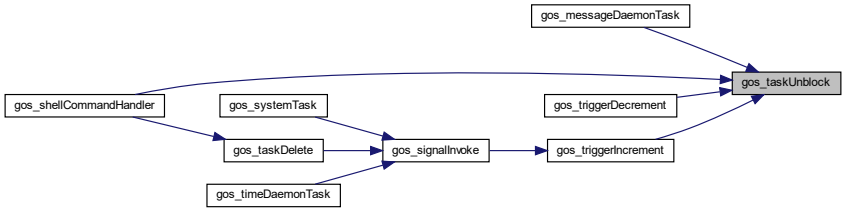
GOS_SUCCESS	Task unblocked successfully.
GOS_ERROR	Task ID is invalid, or task is not blocked.

Definition at line 594 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.23 gos_taskWakeup()

```
gos_result_t gos_taskWakeup (
            gos_tid_t taskId )
```

Wakes up the given task.

Checks the current task and its state, modifies it to ready, and if there is a wake-up hook function registered, it calls it.

Parameters

in	*taskId*	ID of the task to be waken up.

Returns

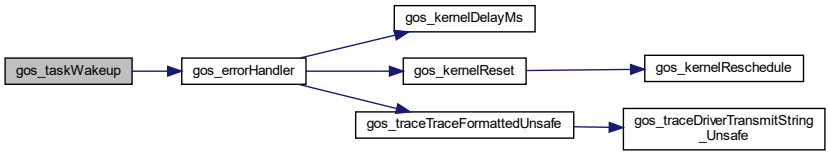
Result of task wake-up.

Return values

GOS_SUCCESS	Task waken up successfully.
GOS_ERROR	Task ID is invalid, or task is not sleeping.

Definition at line 343 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.24 gos_taskYield()

```
gos_result_t gos_taskYield (
            void_t  )
```

Yields the current task.

Invokes rescheduling.

Returns

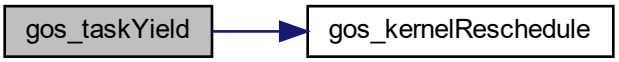
Result of task yield.

Return values

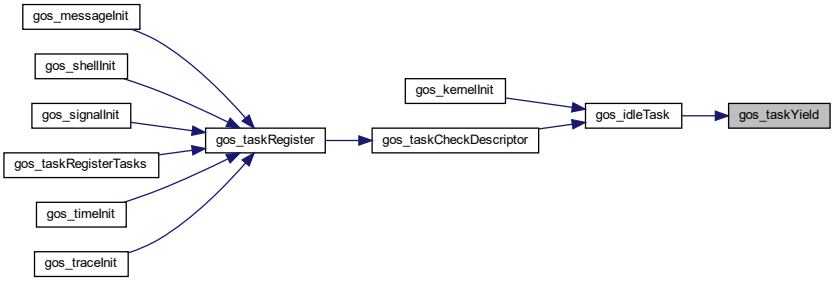
GOS_SUCCESS	Yield successful.

Definition at line 1007 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.10 Platform initializer weak functions

Functions

- GOS_EXTERN gos_result_t gos_platformDriverInit (void_t)

  Platform driver initializer. Used for the platform-specific driver initializations.
- GOS_EXTERN gos_result_t gos_userApplicationInit (void_t)

  User application initializer. Used for the application-related initializations.

5.10.1 Detailed Description

5.10.2 Function Documentation

5.10.2.1 gos_platformDriverInit()

```
GOS_EXTERN gos_result_t gos_platformDriverInit (
            void_t  )
```

Platform driver initializer. Used for the platform-specific driver initializations.

This function shall be defined by the user.

Returns

-

Return values

GOS_ERROR	-

5.10.2.2 gos_userApplicationInit()

```
GOS_EXTERN gos_result_t gos_userApplicationInit (
            void_t  )
```

User application initializer. Used for the application-related initializations.

This function shall be defined by the user.

Returns

-

Return values

GOS_ERROR	-

5.11 Trace formatter macros

Macros

- #define TRACE_FORMAT_RESET "\x1B[0m"

  Reset formatting.
- #define TRACE_FG_RED_START "\x1B[31m"

  Red foreground start.
- #define TRACE_FG_GREEN_START "\x1B[32m"

  Green foreground start.
- #define TRACE_FG_YELLOW_START "\x1B[33m"

  Yellow foreground start.
- #define TRACE_FG_BLUE_START "\x1B[34m"

  Blue foreground start.
- #define TRACE_FG_MAGENTA_START "\x1B[35m"

  Magenta foreground start.
- #define TRACE_FG_CYAN_START "\x1B[36m"

  Cyan foreground start.
- #define TRACE_FG_WHITE_START "\x1B[37m"

  White foreground start.
- #define TRACE_BG_RED_START "\x1b[41m"

  Red background start.
- #define TRACE_BG_GREEN_START "\x1b[42m"

  Green background start.
- #define TRACE_BG_YELLOW_START "\x1b[43m"

  Yellow background start.
- #define TRACE_BG_BLUE_START "\x1b[44m"

  Blue background start.
- #define TRACE_BG_MAGENTA_START "\x1b[45m"

  Magenta background start.
- #define TRACE_BG_CYAN_START "\x1b[46m"

  Cyan background start.
- #define TRACE_BG_WHITE_START "\x1b[47m"

  White background start.
- #define TRACE_BOLD_START "\x1B[1m"

  Bold start.
- #define TRACE_ITALIC_START "\x1B[3m"

  Italic start.
- #define TRACE_UNDERLINE_START "\x1B[4m"

  Underline start.
- #define TRACE_STRIKETHROUGH_START "\x1B[9m"

  Strikethrough start.

5.11.1 Detailed Description

Chapter 6

Data Structure Documentation

6.1 gos_driver_functions_t Struct Reference

`#include <GOS2022/OS/driver/inc/gos_driver.h>`

Data Fields

- gos_shellDriverReceiveChar_t shellDriverReceiveChar
  Shell character receive function.
- gos_shellDriverTransmitString_t shellDriverTransmitString
  Shell string transmit function.
- gos_traceDriverTransmitString_t traceDriverTransmitString
  Log string transmit function.
- gos_traceDriverTransmitString_Unsafe_t traceDriverTransmitStringUnsafe
  Log unsafe string transmit function.
- gos_timerDriverSysTimerGetVal_t timerDriverSysTimerGetValue
  System timer get function.

6.1.1 Detailed Description

Driver functions type.

Definition at line 67 of file gos_driver.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/driver/inc/gos_driver.h

6.2 gos_gcpChannelFunctions_t Struct Reference

Data Fields

- gos_gcpTransmitFunction_t gcpTransmitFunction
  GCP transmit function.
- gos_gcpReceiveFunction_t gcpReceiveFunction
  GCP receive function.

6.2.1 Detailed Description

GCP channel functions type.

Definition at line 115 of file gos_gcp.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_gcp.c

6.3 gos_gcpHeaderFrame_t Struct Reference

Data Fields

- u8_t protocolMajor
  Protocol version major.
- u8_t protocolMinor
  Protocol version minor.
- u8_t ackType
  Acknowledge type.
- u8_t dummy
  Dummy byte (padding).
- u16_t messageId
  Message ID.
- u16_t dataSize
  Data size.
- u32_t dataCrc
  Data CRC.
- u32_t headerCrc
  Header CRC.

6.3.1 Detailed Description

GCP frame header type.

Definition at line 100 of file gos_gcp.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_gcp.c

6.4 gos_initStruct_t Struct Reference

Data Fields

- char_t initDesc [32]
  Initialization descriptor text.
- gos_initFunc_t initFunc
  Initializer function.

6.4.1 Detailed Description

Initializer structure type.

Definition at line 92 of file gos.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos.c

6.5 gos_message_t Struct Reference

`#include <GOS2022/OS/services/inc/gos_message.h>`

Data Fields

- gos_messageId_t messageId
  Message ID.
- gos_messageSize_t messageSize
  Message size.
- u8_t messageBytes [CFG_MESSAGE_MAX_LENGTH]
  Message bytes.

6.5.1 Detailed Description

Message type.

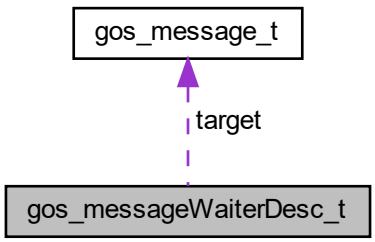
Definition at line 125 of file gos_message.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/inc/gos_message.h

6.6 gos_messageWaiterDesc_t Struct Reference

Collaboration diagram for gos_messageWaiterDesc_t:



Data Fields

- gos_tid_t waiterTaskId
  Waiter task ID.
- gos_messageTimeout_t waitTmo
  Wait timeout value.
- gos_messageTimeout_t waitTmoCounter
  Wait timeout counter.
- gos_messageId_t messageIdArray [CFG_MESSAGE_MAX_WAITER_IDS]
  Message ID array.
- gos_message_t * target
  Target buffer.
- bool_t waiterServed
  Waiter served flag.

6.6.1 Detailed Description

Message waiter descriptor type.

Definition at line 87 of file gos_message.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_message.c

6.7 gos_mutex_t Struct Reference

`#include <GOS2022/OS/services/inc/gos_mutex.h>`

Data Fields

- gos_mutexState_t mutexState
  Mutex state.
- gos_tid_t owner
  Mutex owner task.

6.7.1 Detailed Description

Mutex type.

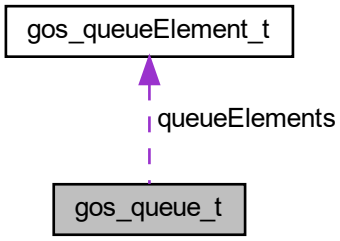
Definition at line 88 of file gos_mutex.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/inc/gos_mutex.h

6.8 gos_queue_t Struct Reference

Collaboration diagram for gos_queue_t:



Data Fields

- gos_queueId_t queueId
  Queue ID.
- gos_queueElement_t queueElements [CFG_QUEUE_MAX_ELEMENTS]
  Queue element array.
- gos_queueIndex_t actualElementNumber
  Actual number of queue elements.

6.8.1 Detailed Description

Queue type.

Definition at line 101 of file gos_queue.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_queue.c

6.9 gos_queueDescriptor_t Struct Reference

```
#include <GOS2022/OS/services/inc/gos_queue.h>
```

Data Fields

- gos_queueId_t queueId
  Queue ID.

6.9.1 Detailed Description

Public queue descriptor type.

Definition at line 121 of file gos_queue.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/inc/gos_queue.h

6.10 gos_queueElement_t Struct Reference

Data Fields

- gos_queueByte_t queueElementBytes [CFG_QUEUE_MAX_LENGTH]
  Queue element bytes.
- gos_queueLength_t elementLength
  Queue element length.

6.10.1 Detailed Description

Queue element type.

Definition at line 92 of file gos_queue.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_queue.c

6.11 gos_runtime_t Struct Reference

`#include <GOS2022/OS/kernel/inc/gos_kernel.h>`

Data Fields

- gos_microsecond_t microseconds
  Microseconds.
- gos_millisecond_t milliseconds
  Milliseconds.
- gos_second_t seconds
  Seconds.
- gos_minute_t minutes
  Minutes.
- gos_hour_t hours
  Hours.
- gos_day_t days
  Days.

6.11.1 Detailed Description

Run-time type.

Definition at line 521 of file gos_kernel.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/kernel/inc/gos_kernel.h

6.12 gos_shellCommand_t Struct Reference

`#include <GOS2022/OS/services/inc/gos_shell.h>`

Data Fields

- char_t command [CFG_SHELL_MAX_COMMAND_LENGTH]
  Command name.
- gos_shellFunction commandHandler
  Command handler function.
- gos_taskPrivilegeLevel_t commandHandlerPrivileges
  Command handler privileges.

6.12.1 Detailed Description

Shell command type.

Definition at line 75 of file gos_shell.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/inc/gos_shell.h

6.13 gos_signalDescriptor_t Struct Reference

Data Fields

- bool_t inUse
  Flag to indicate whether the signal is in use.
- gos_signalHandler_t handlers [CFG_SIGNAL_MAX_SUBSCRIBERS]
  Signal handler array.
- gos_taskPrivilegeLevel_t handlerPrvileges [CFG_SIGNAL_MAX_SUBSCRIBERS]
  Signal handler privileges array.
- bool_t invokeRequired
  Invoke required flag.
- gos_signalSenderId_t senderId
  Sender ID.

6.13.1 Detailed Description

Signal descriptor type.

Definition at line 84 of file gos_signal.c.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_signal.c

6.14 gos_signalInvokeDescriptor Struct Reference

Data Fields

- gos_signalId_t signalId
  Signal ID.
- gos_signalSenderId_t senderId
  Sender ID.

6.14.1 Detailed Description

Signal invoke descriptor type.

Definition at line 96 of file gos_signal.c.

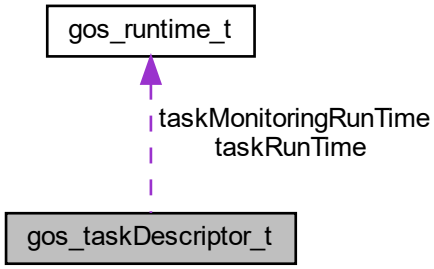
The documentation for this struct was generated from the following file:

- GOS2022/OS/services/src/gos_signal.c

6.15 gos_taskDescriptor_t Struct Reference

`#include <GOS2022/OS/kernel/inc/gos_kernel.h>`

Collaboration diagram for gos_taskDescriptor_t:



Data Fields

- gos_task_t taskFunction

  Task function.
- gos_taskState_t taskState

  Task state.
- gos_taskState_t taskPreviousState

  Task previous state (for restoration).
- gos_taskPrio_t taskPriority

  Task priority.
- gos_taskPrio_t taskOriginalPriority

  Task original priority.
- gos_taskPrivilegeLevel_t taskPrivilegeLevel

  Task privilege level.
- gos_taskName_t taskName

  Task name.
- gos_tid_t taskId

  Task ID (internal).
- gos_taskSleepTick_t taskSleepTicks

  Task sleep ticks.
- gos_taskSleepTick_t taskSleepTickCounter

  Task sleep tick counter.
- gos_blockMaxTick_t taskBlockTicks

  Task block ticks.
- gos_blockMaxTick_t taskBlockTickCounter

  Task block tick counter.
- gos_taskAddress_t taskPsp

  Task PSP.
- gos_taskRunCounter_t taskRunCounter

  Task run counter.
- gos_taskCSCounter_t taskCsCounter

  Task context-switch counter.
- gos_taskStackSize_t taskStackSize

  Task allocated stack size.
- gos_taskStackSize_t taskStackSizeMaxUsage

  Task max. stack size usage.
- gos_runtime_t taskRunTime

  Task run-time.
- gos_runtime_t taskMonitoringRunTime

  Task monitoring run-time (not erased).
- u16_t taskCpuUsageLimit

  Task CPU usage limit in [% x 100].
- u16_t taskCpuUsageMax

  Task CPU usage max value in [% x 100].
- u16_t taskCpuUsage

  Task processor usage in [% x 100].
- u16_t taskCpuMonitoringUsage

  Task CPU usage monitoring value in [% x 100].
- u32_t taskStackOverflowThreshold

  Task stack overflow threshold address.

6.15.1 Detailed Description

Task descriptor structure.

Definition at line 537 of file gos_kernel.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/kernel/inc/gos_kernel.h

6.16 gos_time_t Struct Reference

`#include <GOS2022/OS/services/inc/gos_time.h>`

Data Fields

- gos_millisecond_t milliseconds
  Milliseconds.
- gos_second_t seconds
  Seconds.
- gos_minute_t minutes
  Minutes.
- gos_hour_t hours
  Hours.
- gos_day_t days
  Days.
- gos_month_t months
  Months.
- gos_year_t years
  Years.

6.16.1 Detailed Description

Time type.

Definition at line 76 of file gos_time.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/inc/gos_time.h

6.17 gos_trigger_t Struct Reference

`#include <GOS2022/OS/services/inc/gos_trigger.h>`

Data Fields

- u32_t valueCounter
  Value counter.
- u32_t desiredValue
  Desired value.
- gos_tid_t waiterTaskId
  Owner task ID.

6.17.1 Detailed Description

Trigger descriptor type.

Definition at line 85 of file gos_trigger.h.

The documentation for this struct was generated from the following file:

- GOS2022/OS/services/inc/gos_trigger.h

Chapter 7

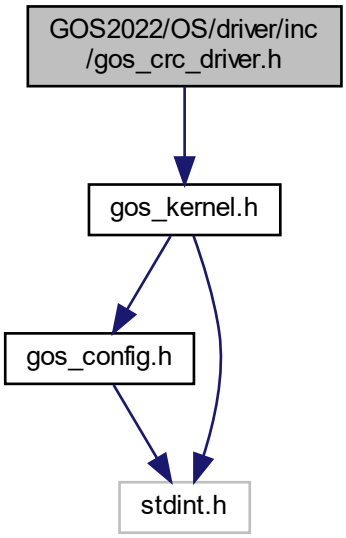
File Documentation

7.1 GOS2022/OS/driver/inc/gos_crc_driver.h File Reference

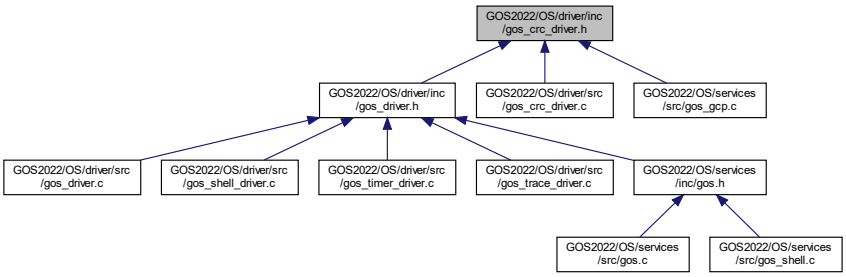
GOS Cyclic Redundancy Check driver header.

```
#include "gos_kernel.h"
```

Include dependency graph for gos_crc_driver.h:



This graph shows which files directly or indirectly include this file:



Functions

- u32_t gos_crcDriverGetCrc (u8_t *pData, u32_t dataSize)

  Calculates the 32-bit CRC value of the given data buffer.

7.1.1 Detailed Description

GOS Cyclic Redundancy Check driver header.

Author

Ahmed Gazar

Date

2022-12-10

Version

1.0

This driver provides a simple 32-bit CRC calculator algorithm.

7.1.2 Function Documentation

7.1.2.1 gos_crcDriverGetCrc()

```
u32_t gos_crcDriverGetCrc (
            u8_t * pData,
            u32_t dataSize )
```

Calculates the 32-bit CRC value of the given data buffer.

Calculates the 32-bit CRC value of the given data buffer.

Parameters

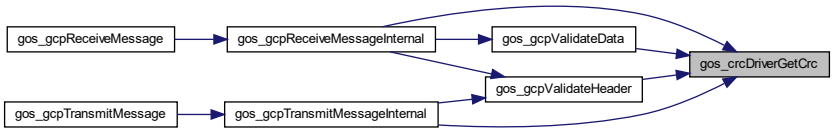
in	*pData*	Pointer to the data buffer.
in	*dataSize*	Size of the data buffer in bytes.

Returns

32-bit CRC value.

Definition at line 70 of file gos_crc_driver.c.

Here is the caller graph for this function:

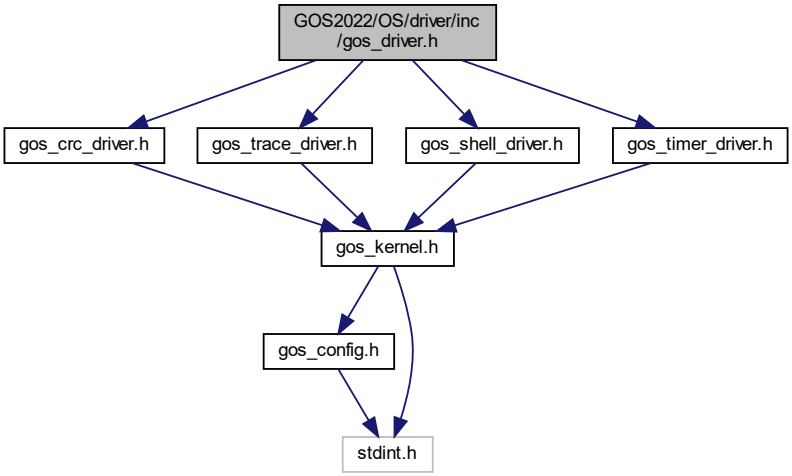


7.2 GOS2022/OS/driver/inc/gos_driver.h File Reference

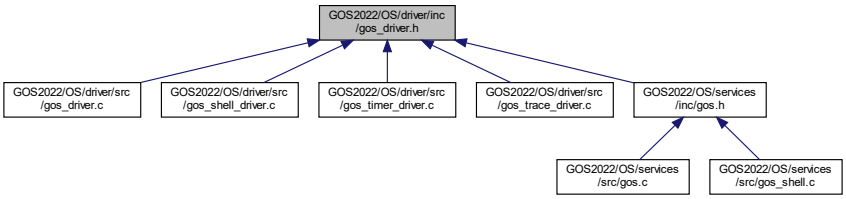
GOS driver header.

```
#include <gos_crc_driver.h>
#include <gos_trace_driver.h>
#include <gos_shell_driver.h>
#include <gos_timer_driver.h>
```

Include dependency graph for gos_driver.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_driver_functions_t

Functions

- gos_result_t gos_driverInit (gos_driver_functions_t *pDriverFunctions)
  Initializes the kernel drivers.

7.2.1 Detailed Description

GOS driver header.

Author

Ahmed Gazar

Date

2025-07-29

Version

1.3

This header is used for the inclusion of all driver skeletons.

7.2.2 Function Documentation

7.2.2.1 gos_driverInit()

```
gos_result_t gos_driverInit (
            gos_driver_functions_t * pDriverFunctions )
```

Initializes the kernel drivers.

Copies the function pointers into the internal structure.

Parameters

in	*pDriverFunctions*	Pointer to the function pointer structure.

Returns

Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	NULL pointer parameter.

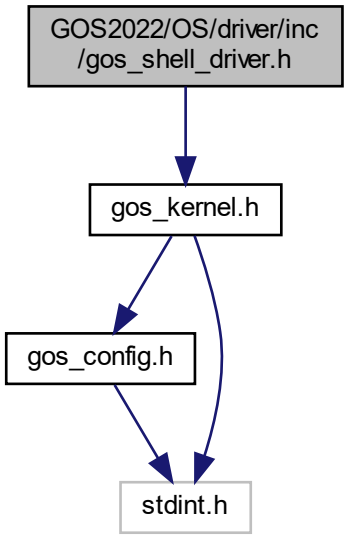
Definition at line 63 of file gos_driver.c.

7.3 GOS2022/OS/driver/inc/gos_shell_driver.h File Reference

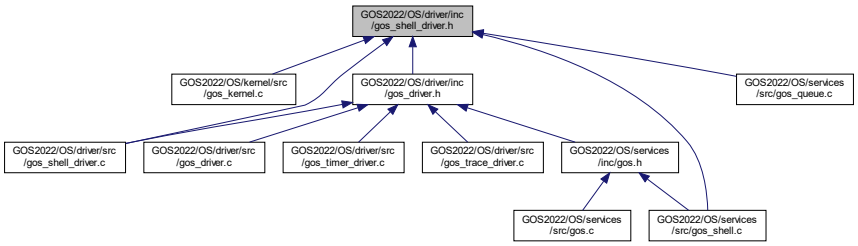
GOS SHELL driver header.

```
#include "gos_kernel.h"
```

Include dependency graph for gos_shell_driver.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef gos_result_t(∗ gos_shellDriverReceiveChar_t) (char_t ∗)
- typedef gos_result_t(∗ gos_shellDriverTransmitString_t) (char_t ∗)

Functions

- gos_result_t gos_shellDriverReceiveChar (char_t ∗pChar)

  Receives a character.
- gos_result_t gos_shellDriverTransmitString (char_t ∗pString,...)

  Transmits a string.

7.3.1 Detailed Description

GOS SHELL driver header.

Author

Ahmed Gazar

Date

2023-06-17

Version

1.1

This driver provides a skeleton for the driver for the shell service.

7.3.2 Typedef Documentation

7.3.2.1 gos_shellDriverReceiveChar_t

```
typedef gos_result_t(* gos_shellDriverReceiveChar_t) (char_t *)
```

Shell driver receive character function type.

Definition at line 63 of file gos_shell_driver.h.

7.3.2.2 gos_shellDriverTransmitString_t

```
typedef gos_result_t(* gos_shellDriverTransmitString_t) (char_t *)
```

Shell driver transmit string function type.

Definition at line 68 of file gos_shell_driver.h.

7.3.3 Function Documentation

7.3.3.1 gos_shellDriverReceiveChar()

```
gos_result_t gos_shellDriverReceiveChar (
            char_t * pChar )
```

Receives a character.

If registered, it calls the custom character receiver function.

Parameters

out	*pChar*	Pointer to a character variable to store the received character in.

Returns

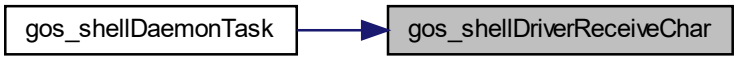
Result of character reception.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 75 of file gos_shell_driver.c.

Here is the caller graph for this function:



7.3.3.2 gos_shellDriverTransmitString()

```
gos_result_t gos_shellDriverTransmitString (
            char_t * pString,
             ...  )
```

Transmits a string.

If registered, it calls the custom string transmitter function.

Parameters

in	*pString*	Pointer to the string to be transmitted.
in	...	Variable parameters for formatted strings.

Returns

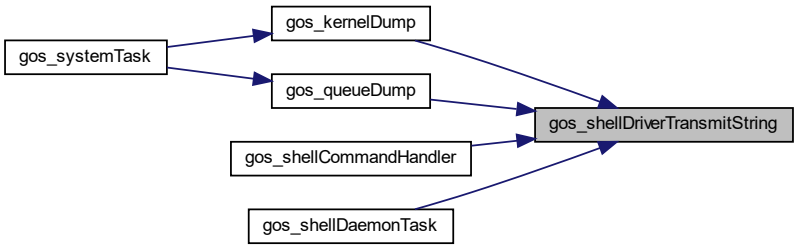
Result of string transmission.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 100 of file gos_shell_driver.c.

Here is the caller graph for this function:

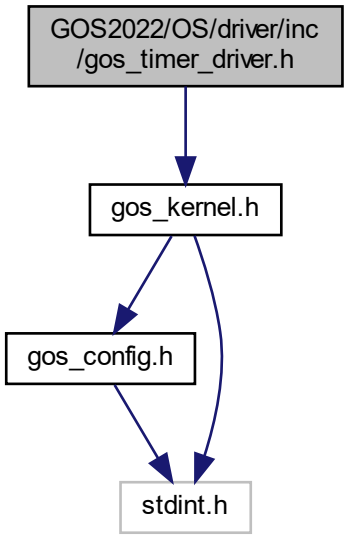


7.4 GOS2022/OS/driver/inc/gos_timer_driver.h File Reference

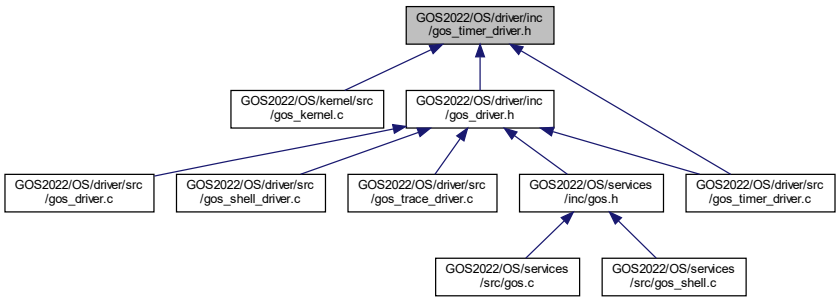
GOS timer driver header.

```
#include "gos_kernel.h"
```

Include dependency graph for gos_timer_driver.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef gos_result_t(∗ gos_timerDriverSysTimerGetVal_t) (u16_t ∗)

Functions

- gos_result_t gos_timerDriverSysTimerGet (u16_t ∗pValue)

  System timer value getter skeleton.

7.4.1 Detailed Description

GOS timer driver header.

Author

Ahmed Gazar

Date

2022-12-09

Version

1.0

This is the timer driver skeleton. It contains the required interface functions for the OS. These functions shall be implemented by the user in order to be able to use microsecond delay and task run-time monitoring.

7.4.2 Typedef Documentation

7.4.2.1 gos_timerDriverSysTimerGetVal_t

```
typedef gos_result_t(* gos_timerDriverSysTimerGetVal_t) (u16_t *)
```

System timer value get function type.

Definition at line 63 of file gos_timer_driver.h.

7.4.3 Function Documentation

7.4.3.1 gos_timerDriverSysTimerGet()

```
gos_result_t gos_timerDriverSysTimerGet (
            u16_t * pValue )
```

System timer value getter skeleton.

If registered, it calls the custom system timer getter function.

Parameters

out	*pValue*	Pointer to the variable to store the timer value in.

Returns

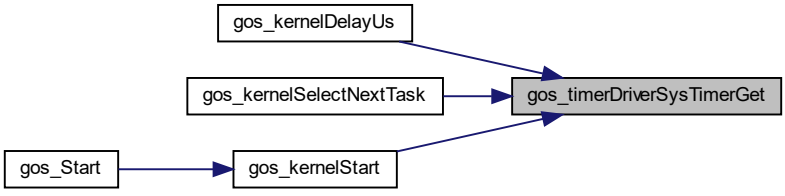
Result of system timer value getting.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 64 of file gos_timer_driver.c.

Here is the caller graph for this function:

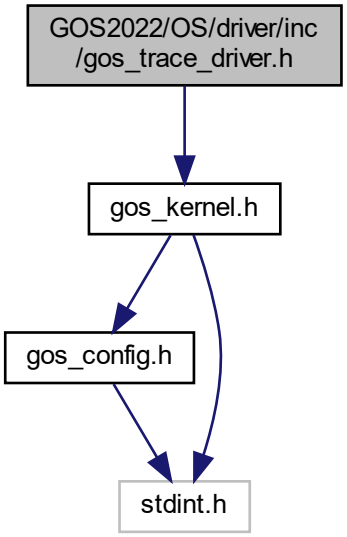


7.5 GOS2022/OS/driver/inc/gos_trace_driver.h File Reference

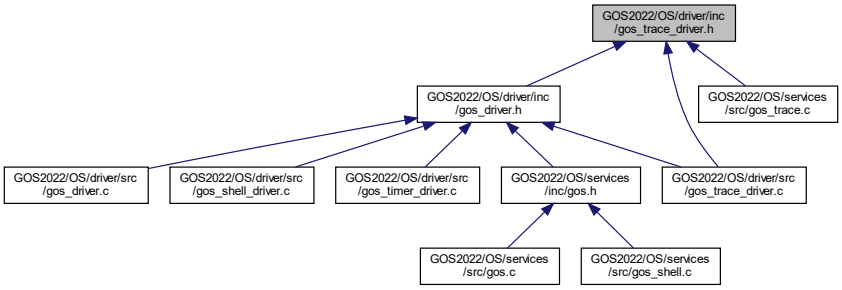
GOS trace driver header.

```
#include "gos_kernel.h"
```

Include dependency graph for gos_trace_driver.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef gos_result_t(∗ gos_traceDriverTransmitString_t) (char_t ∗)
- typedef gos_result_t(∗ gos_traceDriverTransmitString_Unsafe_t) (char_t ∗)

Functions

- gos_result_t gos_traceDriverTransmitString (char_t *pString)
  Trace driver transmit string function skeleton.
- gos_result_t gos_traceDriverTransmitString_Unsafe (char_t *pString)
  Trace driver unsafe transmit string function skeleton.

7.5.1 Detailed Description

GOS trace driver header.

Author

Ahmed Gazar

Date

2023-01-13

Version

1.2

This driver provides a skeleton for the driver for the trace service.

7.5.2 Typedef Documentation

7.5.2.1 gos_traceDriverTransmitString_t

```
typedef gos_result_t(* gos_traceDriverTransmitString_t) (char_t *)
```

Trace driver transmit string function type.

Definition at line 63 of file gos_trace_driver.h.

7.5.2.2 gos_traceDriverTransmitString_Unsafe_t

```
typedef gos_result_t(* gos_traceDriverTransmitString_Unsafe_t) (char_t *)
```

Trace driver unsafe transmit string function type.

Definition at line 68 of file gos_trace_driver.h.

7.5.3 Function Documentation

7.5.3.1 gos_traceDriverTransmitString()

```
gos_result_t gos_traceDriverTransmitString (
            char_t * pString )
```

Trace driver transmit string function skeleton.

If registered, it calls the custom transmit function.

Parameters

in	*pString*	String to trace.

Returns

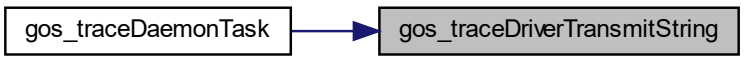
Result of string transmission.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 65 of file gos_trace_driver.c.

Here is the caller graph for this function:



7.5.3.2 gos_traceDriverTransmitString_Unsafe()

```
gos_result_t gos_traceDriverTransmitString_Unsafe (
             char_t * pString )
```

Trace driver unsafe transmit string function skeleton.

If registered, it calls the custom unsafe transmit function.

Parameters

in	*pString*	String to trace.

Returns

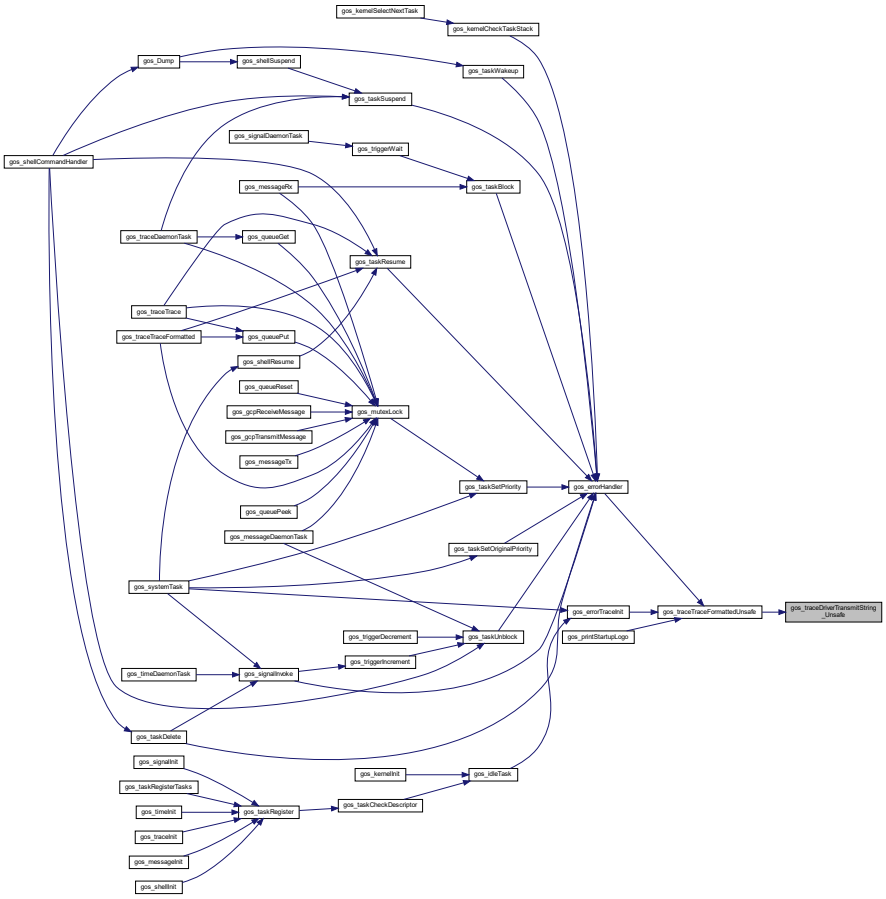
Result of string transmission.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 90 of file gos_trace_driver.c.

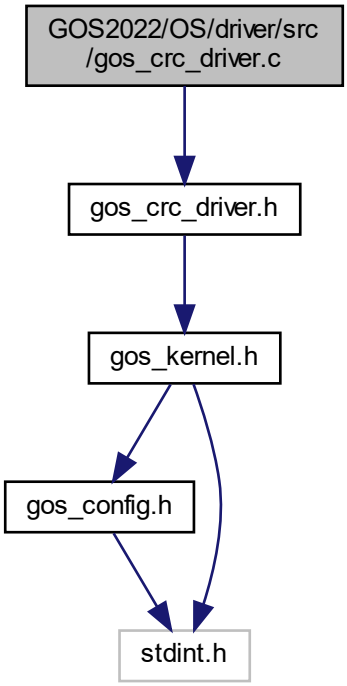
Here is the caller graph for this function:



7.6 GOS2022/OS/driver/src/gos_crc_driver.c File Reference

GOS Cyclic Redundancy Check driver source.

```
#include "gos_crc_driver.h"
```
Include dependency graph for gos_crc_driver.c:



Macros

- #define CRC_INITIAL_VALUE (0xFFFFFFFF)
- #define CRC_POLYNOMIAL_VALUE (0xEDB88320)

Functions

- u32_t gos_crcDriverGetCrc (u8_t *pData, u32_t dataSize)

  Calculates the 32-bit CRC value of the given data buffer.

7.6.1 Detailed Description

GOS Cyclic Redundancy Check driver source.

Author

Ahmed Gazar

Date

2022-12-10

Version

1.0

For a more detailed description of this driver, please refer to gos_crc_driver.h

7.6.2 Macro Definition Documentation

7.6.2.1 CRC_INITIAL_VALUE

```
#define CRC_INITIAL_VALUE ( 0xFFFFFFFF )
```

CRC initializer value.

Definition at line 60 of file gos_crc_driver.c.

7.6.2.2 CRC_POLYNOMIAL_VALUE

```
#define CRC_POLYNOMIAL_VALUE ( 0xEDB88320 )
```

CRC polynomial value.

Definition at line 65 of file gos_crc_driver.c.

7.6.3 Function Documentation

7.6.3.1 gos_crcDriverGetCrc()

```
u32_t gos_crcDriverGetCrc (
            u8_t * pData,
            u32_t dataSize )
```

Calculates the 32-bit CRC value of the given data buffer.

Calculates the 32-bit CRC value of the given data buffer.

Parameters

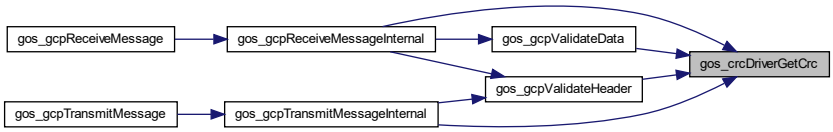
in	*pData*	Pointer to the data buffer.
in	*dataSize*	Size of the data buffer in bytes.

Returns

32-bit CRC value.

Definition at line 70 of file gos_crc_driver.c.

Here is the caller graph for this function:

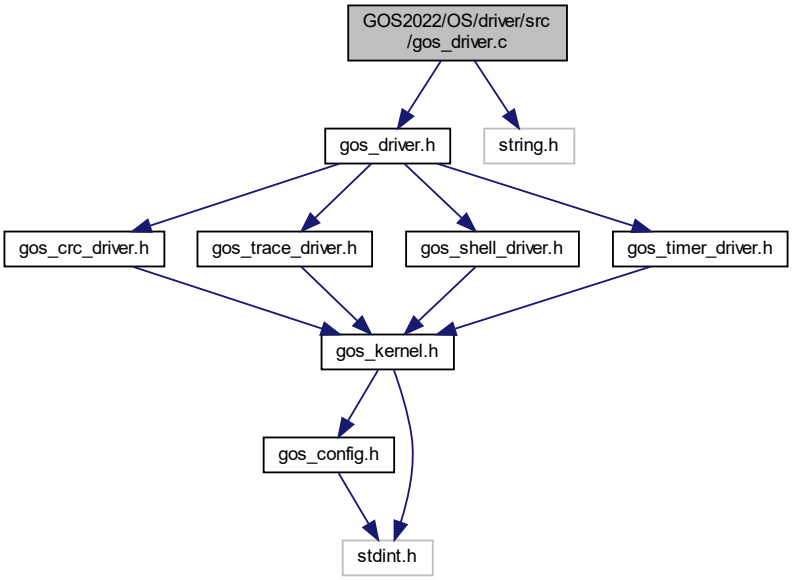


7.7 GOS2022/OS/driver/src/gos_driver.c File Reference

GOS driver source.

```
#include "gos_driver.h"
#include <string.h>
```

Include dependency graph for gos_driver.c:



Functions

- gos_result_t gos_driverInit (gos_driver_functions_t *pDriverFunctions)

  Initializes the kernel drivers.

Variables

- gos_driver_functions_t **driverFunctions** = { NULL, NULL, NULL, NULL, NULL }

7.7.1 Detailed Description

GOS driver source.

Author

Ahmed Gazar

Date

2022-12-11

Version

1.0

For a more detailed description of this driver, please refer to gos_driver.h

7.7.2 Function Documentation

7.7.2.1 gos_driverInit()

```
gos_result_t gos_driverInit (
            gos_driver_functions_t * pDriverFunctions )
```

Initializes the kernel drivers.

Copies the function pointers into the internal structure.

Parameters

in	*pDriverFunctions*	Pointer to the function pointer structure.

Returns

Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	NULL pointer parameter.

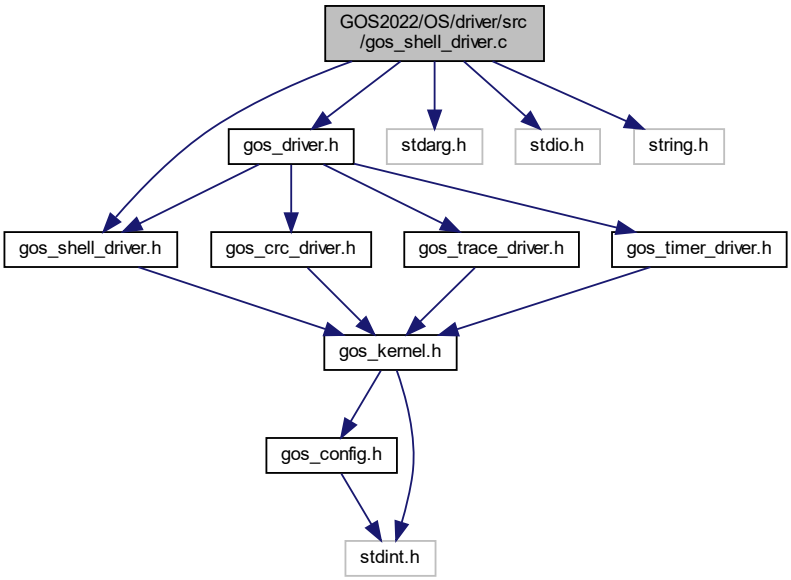
Definition at line 63 of file gos_driver.c.

7.8 GOS2022/OS/driver/src/gos_shell_driver.c File Reference

GOS SHELL driver source.

```
#include "gos_shell_driver.h"
#include "gos_driver.h"
#include <stdarg.h>
#include <stdio.h>
#include <string.h>
```

Include dependency graph for gos_shell_driver.c:



Functions

- gos_result_t gos_shellDriverReceiveChar (char_t ∗pChar)

  Receives a character.

- gos_result_t gos_shellDriverTransmitString (char_t ∗pString,...)

  Transmits a string.

Variables

- GOS_STATIC char_t formattedBuffer [CFG_SHELL_COMMAND_BUFFER_SIZE]
- GOS_EXTERN gos_driver_functions_t **driverFunctions**

7.8.1 Detailed Description

GOS SHELL driver source.

Author

Ahmed Gazar

Date

2023-06-17

Version

1.1

For a more detailed description of this driver, please refer to gos_shell_driver.h

7.8.2 Function Documentation

7.8.2.1 gos_shellDriverReceiveChar()

```
gos_result_t gos_shellDriverReceiveChar (
            char_t * pChar )
```

Receives a character.

If registered, it calls the custom character receiver function.

Parameters

out	*pChar*	Pointer to a character variable to store the received character in.

Returns

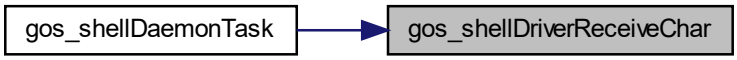
Result of character reception.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 75 of file gos_shell_driver.c.

Here is the caller graph for this function:



7.8.2.2 gos_shellDriverTransmitString()

```
gos_result_t gos_shellDriverTransmitString (
            char_t * pString,
             ...  )
```

Transmits a string.

If registered, it calls the custom string transmitter function.

Parameters

in	*pString*	Pointer to the string to be transmitted.
in	...	Variable parameters for formatted strings.

Returns

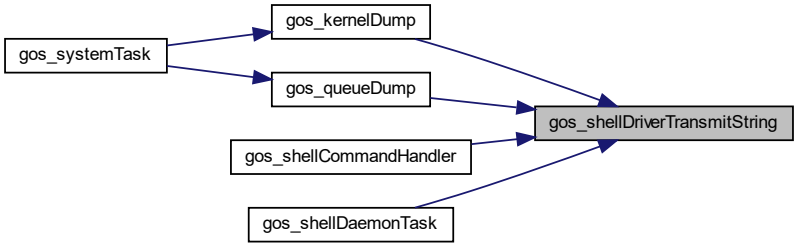
Result of string transmission.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 100 of file gos_shell_driver.c.

Here is the caller graph for this function:



7.8.3 Variable Documentation

7.8.3.1 formattedBuffer

```
GOS_STATIC char_t formattedBuffer[CFG_SHELL_COMMAND_BUFFER_SIZE]
```

Buffer for formatted strings.

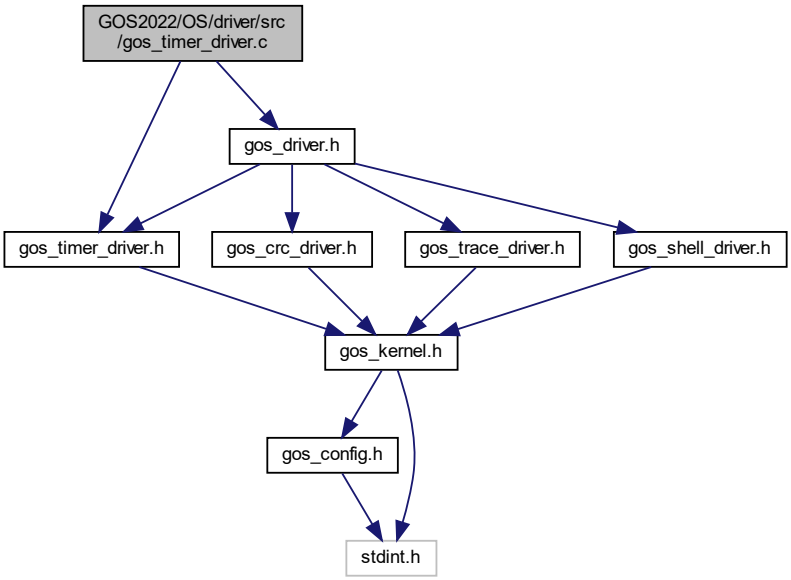
Definition at line 65 of file gos_shell_driver.c.

7.9 GOS2022/OS/driver/src/gos_timer_driver.c File Reference

GOS timer driver source.

```
#include "gos_timer_driver.h"
#include "gos_driver.h"
```

Include dependency graph for gos_timer_driver.c:



Functions

- gos_result_t gos_timerDriverSysTimerGet (u16_t *pValue)

  System timer value getter skeleton.

Variables

- GOS_EXTERN gos_driver_functions_t **driverFunctions**

7.9.1 Detailed Description

GOS timer driver source.

Author

Ahmed Gazar

Date

2022-11-15

Version

1.1

For a more detailed description of this driver, please refer to gos_timer_driver.h

7.9.2 Function Documentation

7.9.2.1 gos_timerDriverSysTimerGet()

```
gos_result_t gos_timerDriverSysTimerGet (
            u16_t * pValue )
```

System timer value getter skeleton.

If registered, it calls the custom system timer getter function.

Parameters

out	*pValue*	Pointer to the variable to store the timer value in.

Returns

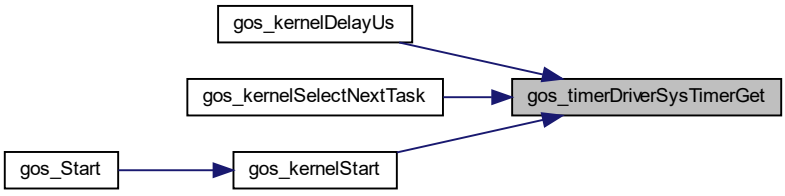
Result of system timer value getting.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 64 of file gos_timer_driver.c.

Here is the caller graph for this function:

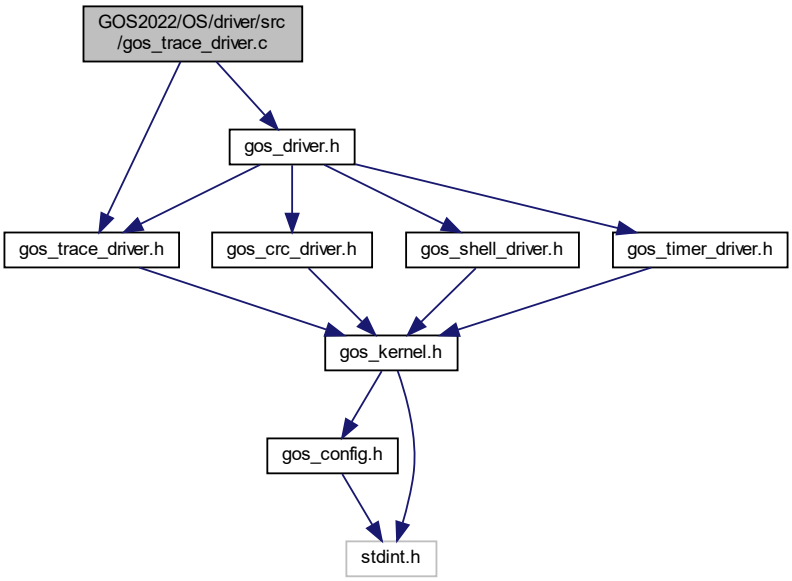


7.10 GOS2022/OS/driver/src/gos_trace_driver.c File Reference

GOS trace driver source.

```
#include <gos_trace_driver.h>
#include <gos_driver.h>
```

Include dependency graph for gos_trace_driver.c:



Functions

- gos_result_t gos_traceDriverTransmitString (char_t *pString)

  Trace driver transmit string function skeleton.
- gos_result_t gos_traceDriverTransmitString_Unsafe (char_t *pString)

  Trace driver unsafe transmit string function skeleton.

Variables

- GOS_EXTERN gos_driver_functions_t **driverFunctions**

7.10.1 Detailed Description

GOS trace driver source.

Author

Ahmed Gazar

Date

2023-01-13

Version

1.2

For a more detailed description of this driver, please refer to gos_trace_driver.h

7.10.2 Function Documentation

7.10.2.1 gos_traceDriverTransmitString()

```
gos_result_t gos_traceDriverTransmitString (
            char_t * pString )
```

Trace driver transmit string function skeleton.

If registered, it calls the custom transmit function.

Parameters

in	*pString*	String to trace.

Returns

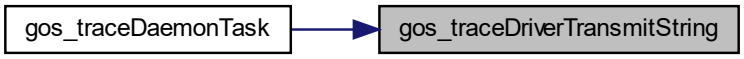
Result of string transmission.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 65 of file gos_trace_driver.c.

Here is the caller graph for this function:



7.10.2.2 gos_traceDriverTransmitString_Unsafe()

```
gos_result_t gos_traceDriverTransmitString_Unsafe (
            char_t * pString )
```

Trace driver unsafe transmit string function skeleton.

If registered, it calls the custom unsafe transmit function.

Parameters

in	*pString*	String to trace.

Returns

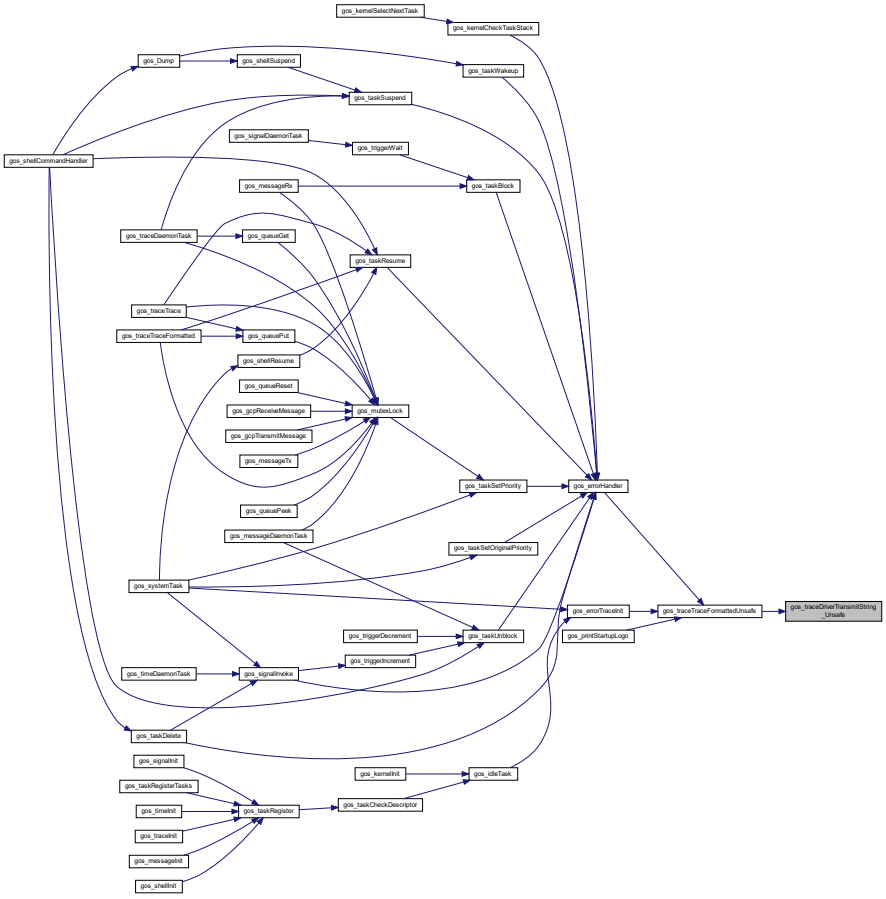
Result of string transmission.

Return values

GOS_SUCCESS	According to user implementation.
GOS_ERROR	According to user implementation / function not registered.

Definition at line 90 of file gos_trace_driver.c.

Here is the caller graph for this function:

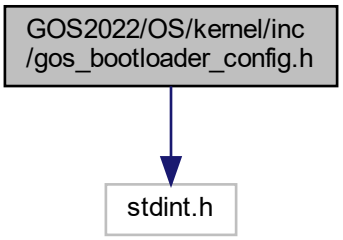


7.11 GOS2022/OS/kernel/inc/gos_bootloader_config.h File Reference

GOS bootloader configuration header.

```
#include <stdint.h>
```

Include dependency graph for gos_bootloader_config.h:



Macros

- #define ARM_CORTEX_M4 (1)
- #define CFG_TARGET_CPU (ARM_CORTEX_M4)
- #define CFG_SCHED_COOPERATIVE (0)
- #define CFG_USE_PRIO_INHERITANCE (1)
- #define CFG_TASK_MAX_NAME_LENGTH (32)
- #define CFG_TASK_MAX_NUMBER (16)
- #define CFG_TASK_MIN_STACK_SIZE (0x200)
- #define CFG_TASK_MAX_STACK_SIZE (0x4000)
- #define CFG_IDLE_TASK_STACK_SIZE (0x400)
- #define CFG_SYSTEM_TASK_STACK_SIZE (0x1000)
- #define CFG_TASK_SIGNAL_DAEMON_STACK (0x400)
- #define CFG_TASK_TIME_DAEMON_STACK (0x300)
- #define CFG_TASK_MESSAGE_DAEMON_STACK (0x300)
- #define CFG_TASK_SHELL_DAEMON_STACK (0x300)
- #define CFG_TASK_TRACE_DAEMON_STACK (0x400)
- #define CFG_TASK_TRACE_DAEMON_PRIO (193)
- #define CFG_TASK_MESSAGE_DAEMON_PRIO (198)
- #define CFG_TASK_SIGNAL_DAEMON_PRIO (197)
- #define CFG_TASK_SHELL_DAEMON_PRIO (192)
- #define CFG_TASK_TIME_DAEMON_PRIO (196)
- #define CFG_TASK_SYS_PRIO (195)
- #define CFG_QUEUE_MAX_NUMBER (1)
- #define CFG_QUEUE_MAX_ELEMENTS (30)
- #define CFG_QUEUE_MAX_LENGTH (200)
- #define CFG_QUEUE_USE_NAME (1)
- #define CFG_QUEUE_MAX_NAME_LENGTH (24)
- #define CFG_SIGNAL_MAX_NUMBER (3)
- #define CFG_SIGNAL_MAX_SUBSCRIBERS (6)
- #define CFG_MESSAGE_MAX_NUMBER (16)
- #define CFG_MESSAGE_MAX_LENGTH (80)
- #define CFG_MESSAGE_MAX_WAITERS (16)
- #define CFG_MESSAGE_MAX_WAITER_IDS (8)
- #define CFG_SHELL_USE_SERVICE (0)
- #define CFG_SHELL_MAX_COMMAND_NUMBER (8)

- #define CFG_SHELL_MAX_COMMAND_LENGTH (20)
- #define CFG_SHELL_MAX_PARAMS_LENGTH (64)
- #define CFG_SHELL_COMMAND_BUFFER_SIZE (200)
- #define CFG_SHELL_STARTUP_DELAY_MS (500)
- #define CFG_GCP_CHANNELS_MAX_NUMBER (2)
- #define CFG_TRACE_MAX_LENGTH (200)
- #define CFG_RESET_ON_ERROR (1)
- #define CFG_RESET_ON_ERROR_DELAY_MS (2000)

7.11.1 Detailed Description

GOS bootloader configuration header.

Author

Ahmed Gazar

Date

2023-09-26

Version

1.0

This header contains the kernel and service configurations of the operating system.

7.11.2 Macro Definition Documentation

7.11.2.1 ARM_CORTEX_M4

```
#define ARM_CORTEX_M4 ( 1 )
```

ARM Cortex-M4.

Definition at line 45 of file gos_bootloader_config.h.

7.11.2.2 CFG_GCP_CHANNELS_MAX_NUMBER

```
#define CFG_GCP_CHANNELS_MAX_NUMBER ( 2 )
```

GCP maximum number of channels.

Definition at line 235 of file gos_bootloader_config.h.

7.11.2.3 CFG_IDLE_TASK_STACK_SIZE

```
#define CFG_IDLE_TASK_STACK_SIZE ( 0x400 )
```

Idle task stack size.

Definition at line 90 of file gos_bootloader_config.h.

7.11.2.4 CFG_MESSAGE_MAX_LENGTH

```
#define CFG_MESSAGE_MAX_LENGTH ( 80 )
```

Maximum length of a message in bytes.

Definition at line 191 of file gos_bootloader_config.h.

7.11.2.5 CFG_MESSAGE_MAX_NUMBER

```
#define CFG_MESSAGE_MAX_NUMBER ( 16 )
```

Maximum number of messages handled at once.

Definition at line 187 of file gos_bootloader_config.h.

7.11.2.6 CFG_MESSAGE_MAX_WAITER_IDS

```
#define CFG_MESSAGE_MAX_WAITER_IDS ( 8 )
```

Maximum number of message IDs a task can wait for (includes the terminating 0).

Definition at line 199 of file gos_bootloader_config.h.

7.11.2.7 CFG_MESSAGE_MAX_WAITERS

```
#define CFG_MESSAGE_MAX_WAITERS ( 16 )
```

Maximum number of message waiters.

Definition at line 195 of file gos_bootloader_config.h.

7.11.2.8 CFG_QUEUE_MAX_ELEMENTS

```
#define CFG_QUEUE_MAX_ELEMENTS ( 30 )
```

Maximum number of queue elements.

Definition at line 154 of file gos_bootloader_config.h.

7.11.2.9 CFG_QUEUE_MAX_LENGTH

```
#define CFG_QUEUE_MAX_LENGTH ( 200 )
```

Maximum queue length.

Definition at line 158 of file gos_bootloader_config.h.

7.11.2.10 CFG_QUEUE_MAX_NAME_LENGTH

```
#define CFG_QUEUE_MAX_NAME_LENGTH ( 24 )
```

Maximum queue name length.

Definition at line 166 of file gos_bootloader_config.h.

7.11.2.11 CFG_QUEUE_MAX_NUMBER

```
#define CFG_QUEUE_MAX_NUMBER ( 1 )
```

Maximum number of queues.

Definition at line 150 of file gos_bootloader_config.h.

7.11.2.12 CFG_QUEUE_USE_NAME

```
#define CFG_QUEUE_USE_NAME ( 1 )
```

Queue use name flag.

Definition at line 162 of file gos_bootloader_config.h.

7.11.2.13 CFG_RESET_ON_ERROR

```
#define CFG_RESET_ON_ERROR ( 1 )
```

Flag to indicate if the system should reset on error.

Definition at line 251 of file gos_bootloader_config.h.

7.11.2.14 CFG_RESET_ON_ERROR_DELAY_MS

```
#define CFG_RESET_ON_ERROR_DELAY_MS ( 2000 )
```

Delay time before system reset.

Definition at line 255 of file gos_bootloader_config.h.

7.11.2.15 CFG_SCHED_COOPERATIVE

```
#define CFG_SCHED_COOPERATIVE ( 0 )
```

Cooperative scheduling flag.

Definition at line 58 of file gos_bootloader_config.h.

7.11.2.16 CFG_SHELL_COMMAND_BUFFER_SIZE

```
#define CFG_SHELL_COMMAND_BUFFER_SIZE ( 200 )
```

Command buffer size.

Definition at line 223 of file gos_bootloader_config.h.

7.11.2.17 CFG_SHELL_MAX_COMMAND_LENGTH

```
#define CFG_SHELL_MAX_COMMAND_LENGTH ( 20 )
```

Maximum command length.

Definition at line 215 of file gos_bootloader_config.h.

7.11.2.18 CFG_SHELL_MAX_COMMAND_NUMBER

```
#define CFG_SHELL_MAX_COMMAND_NUMBER ( 8 )
```

Maximum number of shell commands.

Definition at line 211 of file gos_bootloader_config.h.

7.11.2.19 CFG_SHELL_MAX_PARAMS_LENGTH

```
#define CFG_SHELL_MAX_PARAMS_LENGTH ( 64 )
```

Maximum parameters length.

Definition at line 219 of file gos_bootloader_config.h.

7.11.2.20 CFG_SHELL_STARTUP_DELAY_MS

```
#define CFG_SHELL_STARTUP_DELAY_MS ( 500 )
```

Shell daemon startup delay time [ms].

Definition at line 227 of file gos_bootloader_config.h.

7.11.2.21 CFG_SHELL_USE_SERVICE

```
#define CFG_SHELL_USE_SERVICE ( 0 )
```

Shell service use flag.

Definition at line 207 of file gos_bootloader_config.h.

7.11.2.22 CFG_SIGNAL_MAX_NUMBER

```
#define CFG_SIGNAL_MAX_NUMBER ( 3 )
```

Maximum number of signals.

Definition at line 175 of file gos_bootloader_config.h.

7.11.2.23 CFG_SIGNAL_MAX_SUBSCRIBERS

```
#define CFG_SIGNAL_MAX_SUBSCRIBERS ( 6 )
```

Maximum number of signal subscribers.

Definition at line 179 of file gos_bootloader_config.h.

7.11.2.24 CFG_SYSTEM_TASK_STACK_SIZE

```
#define CFG_SYSTEM_TASK_STACK_SIZE ( 0x1000 )
```

System task stack size.

Definition at line 94 of file gos_bootloader_config.h.

7.11.2.25 CFG_TARGET_CPU

```
#define CFG_TARGET_CPU ( ARM_CORTEX_M4 )
```

Target CPU.

Definition at line 50 of file gos_bootloader_config.h.

7.11.2.26 CFG_TASK_MAX_NAME_LENGTH

```
#define CFG_TASK_MAX_NAME_LENGTH ( 32 )
```

Maximum task name length.

Definition at line 70 of file gos_bootloader_config.h.

7.11.2.27 CFG_TASK_MAX_NUMBER

```
#define CFG_TASK_MAX_NUMBER ( 16 )
```

Maximum number of tasks.

Definition at line 74 of file gos_bootloader_config.h.

7.11.2.28 CFG_TASK_MAX_STACK_SIZE

```
#define CFG_TASK_MAX_STACK_SIZE ( 0x4000 )
```

Maximum task stack size.

Definition at line 86 of file gos_bootloader_config.h.

7.11.2.29 CFG_TASK_MESSAGE_DAEMON_PRIO

```
#define CFG_TASK_MESSAGE_DAEMON_PRIO ( 198 )
```

Message daemon task priority.

Definition at line 126 of file gos_bootloader_config.h.

7.11.2.30 CFG_TASK_MESSAGE_DAEMON_STACK

```
#define CFG_TASK_MESSAGE_DAEMON_STACK ( 0x300 )
```

Message daemon task stack size.

Definition at line 106 of file gos_bootloader_config.h.

7.11.2.31 CFG_TASK_MIN_STACK_SIZE

```
#define CFG_TASK_MIN_STACK_SIZE ( 0x200 )
```

Minimum task stack size.

Definition at line 82 of file gos_bootloader_config.h.

7.11.2.32 CFG_TASK_SHELL_DAEMON_PRIO

```
#define CFG_TASK_SHELL_DAEMON_PRIO ( 192 )
```

Shell daemon task priority.

Definition at line 134 of file gos_bootloader_config.h.

7.11.2.33 CFG_TASK_SHELL_DAEMON_STACK

```
#define CFG_TASK_SHELL_DAEMON_STACK ( 0x300 )
```

Shell daemon task stack size.

Definition at line 110 of file gos_bootloader_config.h.

7.11.2.34 CFG_TASK_SIGNAL_DAEMON_PRIO

```
#define CFG_TASK_SIGNAL_DAEMON_PRIO ( 197 )
```

Signal daemon task priority.

Definition at line 130 of file gos_bootloader_config.h.

7.11.2.35 CFG_TASK_SIGNAL_DAEMON_STACK

```
#define CFG_TASK_SIGNAL_DAEMON_STACK ( 0x400 )
```

Signal daemon task stack size.

Definition at line 98 of file gos_bootloader_config.h.

7.11.2.36 CFG_TASK_SYS_PRIO

```
#define CFG_TASK_SYS_PRIO ( 195 )
```

System task priority.

Definition at line 142 of file gos_bootloader_config.h.

7.11.2.37 CFG_TASK_TIME_DAEMON_PRIO

```
#define CFG_TASK_TIME_DAEMON_PRIO ( 196 )
```

Time daemon task priority.

Definition at line 138 of file gos_bootloader_config.h.

7.11.2.38 CFG_TASK_TIME_DAEMON_STACK

```
#define CFG_TASK_TIME_DAEMON_STACK ( 0x300 )
```

Time daemon task stack size.

Definition at line 102 of file gos_bootloader_config.h.

7.11.2.39 CFG_TASK_TRACE_DAEMON_PRIO

```
#define CFG_TASK_TRACE_DAEMON_PRIO ( 193 )
```

Trace daemon task priority.

Definition at line 122 of file gos_bootloader_config.h.

7.11.2.40 CFG_TASK_TRACE_DAEMON_STACK

```
#define CFG_TASK_TRACE_DAEMON_STACK ( 0x400 )
```

Log daemon task stack size.

Definition at line 114 of file gos_bootloader_config.h.

7.11.2.41 CFG_TRACE_MAX_LENGTH

```
#define CFG_TRACE_MAX_LENGTH ( 200 )
```

Trace maximum (line) length.

Definition at line 243 of file gos_bootloader_config.h.

7.11.2.42 CFG_USE_PRIO_INHERITANCE

```
#define CFG_USE_PRIO_INHERITANCE ( 1 )
```

Priority inheritance flag for lock.

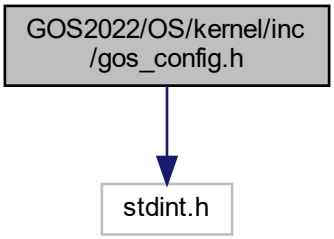
Definition at line 62 of file gos_bootloader_config.h.

7.12 GOS2022/OS/kernel/inc/gos_config.h File Reference

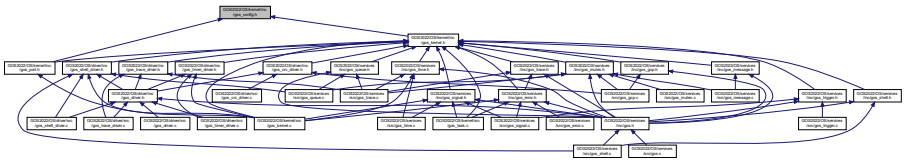
GOS configuration header.

```
#include <stdint.h>
```

Include dependency graph for gos_config.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define ARM_CORTEX_M4 (1)
- #define CFG_TARGET_CPU (ARM_CORTEX_M4)
- #define CFG_SCHED_COOPERATIVE (0)
- #define CFG_USE_PRIO_INHERITANCE (1)
- #define CFG_TASK_MAX_NAME_LENGTH (32)
- #define CFG_TASK_MAX_NUMBER (48)
- #define CFG_TASK_MIN_STACK_SIZE (0x200)
- #define CFG_TASK_MAX_STACK_SIZE (0x4000)
- #define CFG_IDLE_TASK_STACK_SIZE (0x400)
- #define CFG_SYSTEM_TASK_STACK_SIZE (0x800)
- #define CFG_TASK_SIGNAL_DAEMON_STACK (0x400)
- #define CFG_TASK_TIME_DAEMON_STACK (0x300)
- #define CFG_TASK_MESSAGE_DAEMON_STACK (0x600)
- #define CFG_TASK_SHELL_DAEMON_STACK (0x300)
- #define CFG_TASK_TRACE_DAEMON_STACK (0x400)
- #define CFG_TASK_TRACE_DAEMON_PRIO (193)
- #define CFG_TASK_MESSAGE_DAEMON_PRIO (198)
- #define CFG_TASK_SIGNAL_DAEMON_PRIO (197)
- #define CFG_TASK_SHELL_DAEMON_PRIO (192)

- #define CFG_TASK_TIME_DAEMON_PRIO (196)
- #define CFG_TASK_SYS_PRIO (195)
- #define CFG_QUEUE_MAX_NUMBER (4)
- #define CFG_QUEUE_MAX_ELEMENTS (40)
- #define CFG_QUEUE_MAX_LENGTH (200)
- #define CFG_QUEUE_USE_NAME (1)
- #define CFG_QUEUE_MAX_NAME_LENGTH (24)
- #define CFG_SIGNAL_MAX_NUMBER (6)
- #define CFG_SIGNAL_MAX_SUBSCRIBERS (6)
- #define CFG_MESSAGE_MAX_NUMBER (8)
- #define CFG_MESSAGE_MAX_LENGTH (80)
- #define CFG_MESSAGE_MAX_WAITERS (10)
- #define CFG_MESSAGE_MAX_WAITER_IDS (8)
- #define CFG_MESSAGE_MAX_ADDRESSEES (8)
- #define CFG_SHELL_USE_SERVICE (1)
- #define CFG_SHELL_MAX_COMMAND_NUMBER (16)
- #define CFG_SHELL_MAX_COMMAND_LENGTH (20)
- #define CFG_SHELL_MAX_PARAMS_LENGTH (128)
- #define CFG_SHELL_COMMAND_BUFFER_SIZE (200)
- #define CFG_SHELL_STARTUP_DELAY_MS (500)
- #define CFG_GCP_CHANNELS_MAX_NUMBER (4)
- #define CFG_TRACE_MAX_LENGTH (200)
- #define CFG_RESET_ON_ERROR (1)
- #define CFG_RESET_ON_ERROR_DELAY_MS (3000)

7.12.1 Detailed Description

GOS configuration header.

Author

Ahmed Gazar

Date

2024-04-24

Version

1.10

This header contains the kernel and service configurations of the operating system.

7.12.2 Macro Definition Documentation

7.12.2.1 ARM_CORTEX_M4

```
#define ARM_CORTEX_M4 ( 1 )
```

ARM Cortex-M4

Definition at line 70 of file gos_config.h.

7.12.2.2 CFG_GCP_CHANNELS_MAX_NUMBER

```
#define CFG_GCP_CHANNELS_MAX_NUMBER ( 4 )
```

GCP maximum number of channels.

Definition at line 264 of file gos_config.h.

7.12.2.3 CFG_IDLE_TASK_STACK_SIZE

```
#define CFG_IDLE_TASK_STACK_SIZE ( 0x400 )
```

Idle task stack size.

Definition at line 115 of file gos_config.h.

7.12.2.4 CFG_MESSAGE_MAX_ADDRESSEES

```
#define CFG_MESSAGE_MAX_ADDRESSEES ( 8 )
```

Maximum number of message addressees.

Definition at line 228 of file gos_config.h.

7.12.2.5 CFG_MESSAGE_MAX_LENGTH

```
#define CFG_MESSAGE_MAX_LENGTH ( 80 )
```

Maximum length of a message in bytes.

Definition at line 216 of file gos_config.h.

7.12.2.6 CFG_MESSAGE_MAX_NUMBER

```
#define CFG_MESSAGE_MAX_NUMBER ( 8 )
```

Maximum number of messages handled at once.

Definition at line 212 of file gos_config.h.

7.12.2.7 CFG_MESSAGE_MAX_WAITER_IDS

```
#define CFG_MESSAGE_MAX_WAITER_IDS ( 8 )
```

Maximum number of message IDs a task can wait for (includes the terminating 0).

Definition at line 224 of file gos_config.h.

7.12.2.8 CFG_MESSAGE_MAX_WAITERS

```
#define CFG_MESSAGE_MAX_WAITERS ( 10 )
```

Maximum number of message waiters.

Definition at line 220 of file gos_config.h.

7.12.2.9 CFG_QUEUE_MAX_ELEMENTS

```
#define CFG_QUEUE_MAX_ELEMENTS ( 40 )
```

Maximum number of queue elements.

Definition at line 179 of file gos_config.h.

7.12.2.10 CFG_QUEUE_MAX_LENGTH

```
#define CFG_QUEUE_MAX_LENGTH ( 200 )
```

Maximum queue length.

Definition at line 183 of file gos_config.h.

7.12.2.11 CFG_QUEUE_MAX_NAME_LENGTH

```
#define CFG_QUEUE_MAX_NAME_LENGTH ( 24 )
```

Maximum queue name length.

Definition at line 191 of file gos_config.h.

7.12.2.12 CFG_QUEUE_MAX_NUMBER

```
#define CFG_QUEUE_MAX_NUMBER ( 4 )
```

Maximum number of queues.

Definition at line 175 of file gos_config.h.

7.12.2.13 CFG_QUEUE_USE_NAME

```
#define CFG_QUEUE_USE_NAME ( 1 )
```

Queue use name flag.

Definition at line 187 of file gos_config.h.

7.12.2.14 CFG_RESET_ON_ERROR

```
#define CFG_RESET_ON_ERROR ( 1 )
```

Flag to indicate if the system should reset on error.

Definition at line 280 of file gos_config.h.

7.12.2.15 CFG_RESET_ON_ERROR_DELAY_MS

```
#define CFG_RESET_ON_ERROR_DELAY_MS ( 3000 )
```

Delay time before system reset.

Definition at line 284 of file gos_config.h.

7.12.2.16 CFG_SCHED_COOPERATIVE

```
#define CFG_SCHED_COOPERATIVE ( 0 )
```

Cooperative scheduling flag.

Definition at line 83 of file gos_config.h.

7.12.2.17 CFG_SHELL_COMMAND_BUFFER_SIZE

```
#define CFG_SHELL_COMMAND_BUFFER_SIZE ( 200 )
```

Command buffer size.

Definition at line 252 of file gos_config.h.

7.12.2.18 CFG_SHELL_MAX_COMMAND_LENGTH

```
#define CFG_SHELL_MAX_COMMAND_LENGTH ( 20 )
```

Maximum command length.

Definition at line 244 of file gos_config.h.

7.12.2.19 CFG_SHELL_MAX_COMMAND_NUMBER

```
#define CFG_SHELL_MAX_COMMAND_NUMBER ( 16 )
```

Maximum number of shell commands.

Definition at line 240 of file gos_config.h.

7.12.2.20 CFG_SHELL_MAX_PARAMS_LENGTH

```
#define CFG_SHELL_MAX_PARAMS_LENGTH ( 128 )
```

Maximum parameters length.

Definition at line 248 of file gos_config.h.

7.12.2.21 CFG_SHELL_STARTUP_DELAY_MS

```
#define CFG_SHELL_STARTUP_DELAY_MS ( 500 )
```

Shell daemon startup delay time [ms].

Definition at line 256 of file gos_config.h.

7.12.2.22 CFG_SHELL_USE_SERVICE

```
#define CFG_SHELL_USE_SERVICE ( 1 )
```

Shell service use flag.

Definition at line 236 of file gos_config.h.

7.12.2.23 CFG_SIGNAL_MAX_NUMBER

```
#define CFG_SIGNAL_MAX_NUMBER ( 6 )
```

Maximum number of signals.

Definition at line 200 of file gos_config.h.

7.12.2.24 CFG_SIGNAL_MAX_SUBSCRIBERS

```
#define CFG_SIGNAL_MAX_SUBSCRIBERS ( 6 )
```

Maximum number of signal subscribers.

Definition at line 204 of file gos_config.h.

7.12.2.25 CFG_SYSTEM_TASK_STACK_SIZE

```
#define CFG_SYSTEM_TASK_STACK_SIZE ( 0x800 )
```

System task stack size.

Definition at line 119 of file gos_config.h.

7.12.2.26 CFG_TARGET_CPU

```
#define CFG_TARGET_CPU ( ARM_CORTEX_M4 )
```

Target CPU.

Definition at line 75 of file gos_config.h.

7.12.2.27 CFG_TASK_MAX_NAME_LENGTH

```
#define CFG_TASK_MAX_NAME_LENGTH ( 32 )
```

Maximum task name length.

Definition at line 95 of file gos_config.h.

7.12.2.28 CFG_TASK_MAX_NUMBER

```
#define CFG_TASK_MAX_NUMBER ( 48 )
```

Maximum number of tasks.

Definition at line 99 of file gos_config.h.

7.12.2.29 CFG_TASK_MAX_STACK_SIZE

```
#define CFG_TASK_MAX_STACK_SIZE ( 0x4000 )
```

Maximum task stack size.

Definition at line 111 of file gos_config.h.

7.12.2.30 CFG_TASK_MESSAGE_DAEMON_PRIO

```
#define CFG_TASK_MESSAGE_DAEMON_PRIO ( 198 )
```

Message daemon task priority.

Definition at line 151 of file gos_config.h.

7.12.2.31 CFG_TASK_MESSAGE_DAEMON_STACK

```
#define CFG_TASK_MESSAGE_DAEMON_STACK ( 0x600 )
```

Message daemon task stack size.

Definition at line 131 of file gos_config.h.

7.12.2.32 CFG_TASK_MIN_STACK_SIZE

```
#define CFG_TASK_MIN_STACK_SIZE ( 0x200 )
```

Minimum task stack size.

Definition at line 107 of file gos_config.h.

7.12.2.33 CFG_TASK_SHELL_DAEMON_PRIO

```
#define CFG_TASK_SHELL_DAEMON_PRIO ( 192 )
```

Shell daemon task priority.

Definition at line 159 of file gos_config.h.

7.12.2.34 CFG_TASK_SHELL_DAEMON_STACK

```
#define CFG_TASK_SHELL_DAEMON_STACK ( 0x300 )
```

Shell daemon task stack size.

Definition at line 135 of file gos_config.h.

7.12.2.35 CFG_TASK_SIGNAL_DAEMON_PRIO

```
#define CFG_TASK_SIGNAL_DAEMON_PRIO ( 197 )
```

Signal daemon task priority.

Definition at line 155 of file gos_config.h.

7.12.2.36 CFG_TASK_SIGNAL_DAEMON_STACK

```
#define CFG_TASK_SIGNAL_DAEMON_STACK ( 0x400 )
```

Signal daemon task stack size.

Definition at line 123 of file gos_config.h.

7.12.2.37 CFG_TASK_SYS_PRIO

```
#define CFG_TASK_SYS_PRIO ( 195 )
```

System task priority.

Definition at line 167 of file gos_config.h.

7.12.2.38 CFG_TASK_TIME_DAEMON_PRIO

```
#define CFG_TASK_TIME_DAEMON_PRIO ( 196 )
```

Time daemon task priority.

Definition at line 163 of file gos_config.h.

7.12.2.39 CFG_TASK_TIME_DAEMON_STACK

```
#define CFG_TASK_TIME_DAEMON_STACK ( 0x300 )
```

Time daemon task stack size.

Definition at line 127 of file gos_config.h.

7.12.2.40 CFG_TASK_TRACE_DAEMON_PRIO

```
#define CFG_TASK_TRACE_DAEMON_PRIO ( 193 )
```

Trace daemon task priority.

Definition at line 147 of file gos_config.h.

7.12.2.41 CFG_TASK_TRACE_DAEMON_STACK

```
#define CFG_TASK_TRACE_DAEMON_STACK ( 0x400 )
```

Log daemon task stack size.

Definition at line 139 of file gos_config.h.

7.12.2.42 CFG_TRACE_MAX_LENGTH

```
#define CFG_TRACE_MAX_LENGTH ( 200 )
```

Trace maximum (line) length.

Definition at line 272 of file gos_config.h.

7.12.2.43 CFG_USE_PRIO_INHERITANCE

```
#define CFG_USE_PRIO_INHERITANCE ( 1 )
```

Priority inheritance flag for lock.

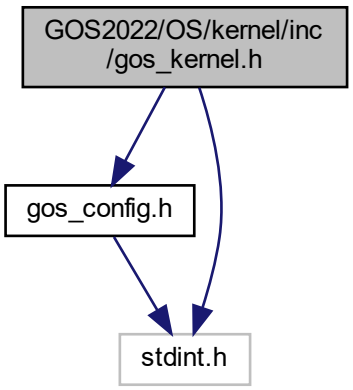
Definition at line 87 of file gos_config.h.

7.13 GOS2022/OS/kernel/inc/gos_kernel.h File Reference

GOS kernel header.

```
#include <gos_config.h>
#include <stdint.h>
```

Include dependency graph for gos_kernel.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_runtime_t
- struct gos_taskDescriptor_t

Macros

- #define NULL ((void *) 0)
- #define RAM_START (0x20000000u)
- #define RAM_SIZE (128 * 1024)
- #define MAIN_STACK (RAM_START + RAM_SIZE)
- #define GLOBAL_STACK (0x1200)
- #define GOS_DEFAULT_TASK_ID ((gos_tid_t)0x8000)
- #define GOS_INVALID_TASK_ID ((gos_tid_t)0x0100)
- #define GOS_TASK_MAX_PRIO_LEVELS (UINT8_MAX)
- #define GOS_TASK_IDLE_PRIO (GOS_TASK_MAX_PRIO_LEVELS)
- #define GOS_TASK_MAX_BLOCK_TIME_MS (0xFFFFFFFFu)
- #define GOS_STATIC static
- #define GOS_CONST const
- #define GOS_INLINE inline __attribute__((always_inline))
- #define GOS_STATIC_INLINE GOS_STATIC GOS_INLINE
- #define GOS_EXTERN extern
- #define GOS_NAKED __attribute__ ((naked))
- #define GOS_UNUSED __attribute__ ((unused))
- #define GOS_NOP __asm volatile ("NOP")
- #define GOS_ASM __asm volatile
- #define GOS_UNUSED_PAR(x) (void_t) x
- #define GOS_DISABLE_SCHED
- #define GOS_ENABLE_SCHED
- #define GOS_ISR_ENTER
- #define GOS_ISR_EXIT
- #define GOS_ATOMIC_ENTER
- #define GOS_ATOMIC_EXIT
- #define GOS_PRIV_TASK_MANIPULATE (1 << 15)
- #define GOS_PRIV_TASK_PRIO_CHANGE (1 << 14)
- #define GOS_PRIV_TRACE (1 << 13)
- #define GOS_PRIV_SIGNALING (1 << 11)
- #define GOS_PRIV_RESERVED_3 (1 << 10)
- #define GOS_PRIV_RESERVED_4 (1 << 9)
- #define GOS_PRIV_RESERVED_5 (1 << 8)
- #define GOS_CONCAT_RESULT(finalResult, currentResult)
- #define GOS_CONVERT_RESULT(result)

Typedefs

- typedef uint8_t bool_t
  Boolean logic type.
- typedef uint8_t u8_t
  8-bit unsigned type.
- typedef uint16_t u16_t
  16-bit unsigned type.
- typedef uint32_t u32_t
  32-bit unsigned type.
- typedef uint64_t u64_t
  64-bit unsigned type.
- typedef int8_t s8_t
  8-bit signed type.
- typedef int16_t s16_t
  16-bit signed type.
- typedef int32_t s32_t
  32-bit signed type.
- typedef int64_t s64_t
  64-bit signed type.
- typedef char char_t
  8-bit character type.
- typedef float float_t
  Single precision float type.
- typedef double double_t
  Double precision float type.
- typedef void void_t
  Void type.
- typedef u16_t gos_tid_t
  Task ID type.
- typedef char_t gos_taskName_t[CFG_TASK_MAX_NAME_LENGTH]
  Task name type.
- typedef void_t(∗ gos_task_t) (void_t)
  Task function type.
- typedef u8_t gos_taskPrio_t
  Task priority type.
- typedef u32_t gos_taskSleepTick_t
  Sleep tick type.
- typedef u32_t gos_blockMaxTick_t
  Block max. tick type.
- typedef u32_t gos_taskAddress_t
  Memory address type.
- typedef u32_t gos_taskRunCounter_t
  Run counter type.
- typedef u64_t gos_taskRunTime_t
  Run-time type.
- typedef u32_t gos_taskCSCounter_t
  Context-switch counter type.
- typedef u16_t gos_taskStackSize_t
  Task stack size type.
- typedef void_t(∗ gos_taskSwapHook_t) (gos_tid_t, gos_tid_t)

Task swap hook type.

- typedef void_t(* gos_taskIdleHook_t) (void_t)

  Task idle hook type.
- typedef void_t(* gos_taskSleepHook_t) (gos_tid_t)

  Task sleep hook type.
- typedef void_t(* gos_taskWakeupHook_t) (gos_tid_t)

  Task wake-up hook type.
- typedef void_t(* gos_taskSuspendHook_t) (gos_tid_t)

  Task suspend hook type.
- typedef void_t(* gos_taskResumeHook_t) (gos_tid_t)

  Task resume hook type.
- typedef void_t(* gos_taskBlockHook_t) (gos_tid_t)

  Task block hook type.
- typedef void_t(* gos_taskUnblockHook_t) (gos_tid_t)

  Task unblock hook type.
- typedef void_t(* gos_taskDeleteHook_t) (gos_tid_t)

  Task delete hook type.
- typedef void_t(* gos_sysTickHook_t) (void_t)

  System tick hook type.
- typedef void_t(* gos_privilegedHook_t) (void_t)

  Privileged mode hook type.
- typedef void_t(* gos_preResetHook_t) (void_t)

  Kernel pre-reset hook type.
- typedef u16_t gos_microsecond_t

  Microsecond type.
- typedef u16_t gos_millisecond_t

  Millisecond type.
- typedef u8_t gos_second_t

  Second type.
- typedef u8_t gos_minute_t

  Minute type.
- typedef u8_t gos_hour_t

  Hour type.
- typedef u16_t gos_day_t

  Day type.
- typedef u8_t gos_month_t

  Month type.
- typedef u16_t gos_year_t

  Year type.

Enumerations

- enum gos_taskState_t {
  GOS_TASK_READY = 0b01010, GOS_TASK_SLEEPING = 0b10110, GOS_TASK_BLOCKED = 0b11001,
  GOS_TASK_SUSPENDED = 0b00101,
  GOS_TASK_ZOMBIE = 0b01101 }
- enum gos_taskPrivilegeLevel_t { GOS_TASK_PRIVILEGE_SUPERVISOR = 0xFFFF, GOS_TASK_PRIVILEGE_KERNEL = 0xFF00, GOS_TASK_PRIVILEGE_USER = 0x00FF, GOS_TASK_PRIVILEGED_USER = 0x20FF }
- enum gos_result_t { GOS_SUCCESS = 0b01010101, GOS_ERROR = 0b10101110, GOS_BUSY = 0b10110001 }
- enum gos_boolValue_t { GOS_TRUE = 0b00110110, GOS_FALSE = 0b01001001 }
- enum gos_kernel_privilege_t { GOS_PRIVILEGED = 0b10110, GOS_UNPRIVILEGED = 0b01001 }

Functions

- gos_result_t gos_kernelInit (void_t)

  This function initializes the kernel.
- gos_result_t gos_kernelRegisterSwapHook (gos_taskSwapHook_t swapHookFunction)

  Registers a task swap hook function.
- gos_result_t gos_kernelRegisterIdleHook (gos_taskIdleHook_t idleHookFunction)

  Registers an idle hook function.
- gos_result_t gos_kernelRegisterSysTickHook (gos_sysTickHook_t sysTickHookFunction)

  Registers a system tick hook function.
- gos_result_t gos_kernelRegisterPrivilegedHook (gos_privilegedHook_t privilegedHookFunction)

  Registers a privileged hook function.
- gos_result_t gos_kernelRegisterPreResetHook (gos_preResetHook_t preResetHookFunction)

  Registers a pre-reset hook function.
- gos_result_t gos_taskSubscribeDeleteSignal (void_t(∗deleteSignalHandler)(u16_t))

  Subscribes the given handler to the task delete signal.
- gos_result_t gos_kernelSubscribeDumpReadySignal (void_t(∗dumpReadySignalHandler)(u16_t))

  Subscribes the given handler to the dump ready signal.
- u32_t gos_kernelGetSysTicks (void_t)

  Returns the system ticks.
- u16_t gos_kernelGetCpuUsage (void_t)

  Returns the CPU usage.
- gos_result_t gos_kernelStart (void_t)

  Starts the kernel.
- void_t gos_kernelReset (void_t)

  Resets the microcontroller.
- void_t gos_kernelPrivilegedModeSetRequired (void_t)

  Requests a privileged mode setting.
- void_t gos_kernelDelayUs (u16_t microseconds)

  Blocking delay in microsecond range.
- void_t gos_kernelDelayMs (u16_t milliseconds)

  Blocking delay in millisecond range.
- void_t gos_kernelCalculateTaskCpuUsages (bool_t isResetRequired)

  Calculates the CPU usage for the tasks.
- void_t gos_kernelDump (void_t)

  Kernel dump.
- gos_result_t gos_kernelSetMaxCpuLoad (u16_t maxCpuLoad)

  Sets the maximum (global) CPU load.
- gos_result_t gos_kernelGetMaxCpuLoad (u16_t ∗pMaxCpuLoad)

  Gets the maximum (global) CPU load.
- bool_t gos_kernelIsCallerIsr (void_t)

  Returns if the current task is ISR.
- void_t gos_kernelReschedule (gos_kernel_privilege_t privilege)

  Reschedules the kernel.
- gos_result_t gos_taskRegisterTasks (gos_taskDescriptor_t ∗taskDescriptors, u16_t arraySize)

  This function registers an array of tasks for scheduling.
- gos_result_t gos_taskRegister (gos_taskDescriptor_t ∗taskDescriptor, gos_tid_t ∗taskId)

  This function registers a task for scheduling.
- gos_result_t gos_taskSleep (gos_taskSleepTick_t sleepTicks)

  Sends the current task to sleeping state.
- gos_result_t gos_taskWakeup (gos_tid_t taskId)

Wakes up the given task.

- gos_result_t gos_taskSuspend (gos_tid_t taskId)

  Sends the given task to suspended state.
- gos_result_t gos_taskResume (gos_tid_t taskId)

  Resumes the given task.
- gos_result_t gos_taskBlock (gos_tid_t taskId, gos_blockMaxTick_t blockTicks)

  Sends the given task to blocked state.
- gos_result_t gos_taskUnblock (gos_tid_t taskId)

  Unblocks the given task.
- gos_result_t gos_taskDelete (gos_tid_t taskId)

  Deletes the given task from the scheduling array.
- gos_result_t gos_taskSetPriority (gos_tid_t taskId, gos_taskPrio_t taskPriority)

  Sets the current priority of the given task to the given value (for temporary change).
- gos_result_t gos_taskSetOriginalPriority (gos_tid_t taskId, gos_taskPrio_t taskPriority)

  Sets the original priority of the given task to the given value (for permanent change).
- gos_result_t gos_taskGetPriority (gos_tid_t taskId, gos_taskPrio_t *taskPriority)

  Gets the current priority of the given task.
- gos_result_t gos_taskGetOriginalPriority (gos_tid_t taskId, gos_taskPrio_t *taskPriority)

  Gets the original priority of the given task.
- gos_result_t gos_taskAddPrivilege (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)

  Adds the given privileges to the given task.
- gos_result_t gos_taskRemovePrivilege (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)

  Removes the given privileges from the given task.
- gos_result_t gos_taskSetPrivileges (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)

  Sets the given privileges for the given task.
- gos_result_t gos_taskGetPrivileges (gos_tid_t taskId, gos_taskPrivilegeLevel_t *pPrivileges)

  Gets the privileges of the given task.
- gos_result_t gos_taskGetName (gos_tid_t taskId, gos_taskName_t taskName)

  Gets the task name of the task with the given ID.
- gos_result_t gos_taskGetId (gos_taskName_t taskName, gos_tid_t *pTaskId)

  Gets the task ID of the task with the given name.
- gos_result_t gos_taskGetCurrentId (gos_tid_t *pTaskId)

  Returns the ID of the currently running task.
- gos_result_t gos_taskGetData (gos_tid_t taskId, gos_taskDescriptor_t *pTaskData)

  Returns the task data of the given task.
- gos_result_t gos_taskGetDataByIndex (u16_t taskIndex, gos_taskDescriptor_t *pTaskData)

  Returns the task data of the given task.
- gos_result_t gos_taskGetNumber (u16_t *pTaskNum)

  Returns the number of registered tasks.
- gos_result_t gos_taskYield (void_t)

  Yields the current task.

7.13.1 Detailed Description

GOS kernel header.

Author

Ahmed Gazar

Date

2025-07-04

Version

1.23

The GOS kernel is the core of the GOS system. It contains the basic type definitions of the system (these are used across the OS in other services), and it handles the system tick, tasks, and scheduling. The kernel module also provides an interface for registering tasks, changing their states, registering hook functions, disabling the scheduler. It also provides a service to dump the kernel configuration and task information on the log output.

7.13.2 Macro Definition Documentation

7.13.2.1 GOS_CONCAT_RESULT

```
#define GOS_CONCAT_RESULT(
              finalResult,
              currentResult )
```

Value:

```
{                                       \
    if (finalResult == GOS_SUCCESS)     \
    {                                   \
        finalResult = currentResult;    \
    }                                   \
    else                                \
    {                                   \
        finalResult = GOS_ERROR;        \
    }                                   \
}
```

Result concatenating macro.

Definition at line 352 of file gos_kernel.h.

7.13.2.2 GOS_CONVERT_RESULT

```
#define GOS_CONVERT_RESULT(
              result )
```

Value:

```
{                                       \
    if (result != GOS_SUCCESS)          \
    {                                   \
        result = GOS_ERROR;             \
    }                                   \
    else                                \
    {                                   \
        /* Nothing to do.  */           \
    }                                   \
}
```

Result converting macro.

Definition at line 365 of file gos_kernel.h.

7.13.3 Enumeration Type Documentation

7.13.3.1 gos_boolValue_t

`enum gos_boolValue_t`

Boolean values.

Note

Hamming distance of true and false value: 7.

Enumerator

GOS_TRUE	True value.
GOS_FALSE	False value.

Definition at line 490 of file gos_kernel.h.

7.13.3.2 gos_kernel_privilege_t

`enum gos_kernel_privilege_t`

Kernel privilege levels.

Enumerator

GOS_PRIVILEGED	GOS_PRIVILEDGED.
GOS_UNPRIVILEGED	GOS_UNPRIVILEDGED.

Definition at line 499 of file gos_kernel.h.

7.13.3.3 gos_result_t

`enum gos_result_t`

Result type enumerator.

Note

Hamming distance of

- success and error: 7
- success and busy: 4
- error and busy: 5

Enumerator

GOS_SUCCESS	Success.
GOS_ERROR	Error.
GOS_BUSY	Busy.

Definition at line 479 of file gos_kernel.h.

7.13.3.4 gos_taskPrivilegeLevel_t

`enum gos_taskPrivilegeLevel_t`

Task privilege level enumerator.

Enumerator

GOS_TASK_PRIVILEGE_SUPERVISOR	Task supervisor privilege level.
GOS_TASK_PRIVILEGE_KERNEL	Task kernel privilege level.
GOS_TASK_PRIVILEGE_USER	Task user privilege level.
GOS_TASK_PRIVILEGED_USER	User with logging right.

Definition at line 444 of file gos_kernel.h.

7.13.3.5 gos_taskState_t

`enum gos_taskState_t`

Task state enumerator.

Note

Hamming distance of

- ready and sleeping: 3
- ready and blocked: 3
- ready and suspended: 4
- ready and zombie: 3
- sleeping and blocked: 4
- sleeping and suspended: 3
- sleeping and zombie: 4
- blocked and suspended: 3
- blocked and zombie: 2
- suspended and zombie: 1

Enumerator

GOS_TASK_READY	Task is ready to be scheduled.
GOS_TASK_SLEEPING	Task is sleeping (waiting for wake-up ticks).
GOS_TASK_BLOCKED	Task is blocked (waiting for resource).
GOS_TASK_SUSPENDED	Task is suspended (not to be scheduled), but can be resumed.
GOS_TASK_ZOMBIE	Task deleted (physically existing in memory, but cannot be resumed).

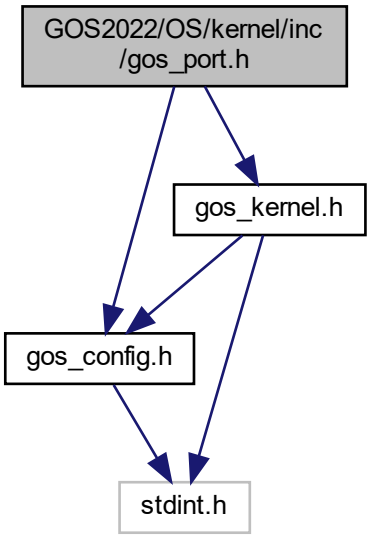
Definition at line 432 of file gos_kernel.h.

7.14 GOS2022/OS/kernel/inc/gos_port.h File Reference

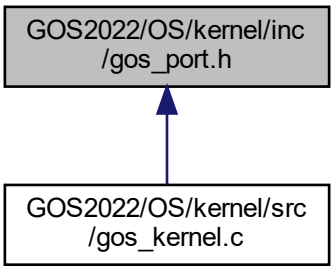
GOS port header.

```
#include "gos_kernel.h"
#include "gos_config.h"
```

Include dependency graph for gos_port.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define gos_ported_procReset()
- #define gos_ported_reschedule(privilege)
- #define gos_ported_pendSVHandler GOS_NAKED PendSV_Handler
- #define gos_ported_doContextSwitch()
- #define gos_ported_svcHandler GOS_NAKED SVC_Handler
- #define gos_ported_handleSVC()
- #define gos_ported_svcHandlerMain gos_kernelSVC_HandlerMain
- #define gos_ported_handleSVCMain(sp)
- #define gos_ported_sysTickInterrupt SysTick_Handler
- #define gos_ported_kernelStartInit()
- #define gos_ported_enableFaultHandlers()

7.14.1 Detailed Description

GOS port header.

Author

Ahmed Gazar

Date

2023-07-12

Version

1.0

This header contains the platform-specific ported definitions of the OS.

7.14.2 Macro Definition Documentation

7.14.2.1 gos_ported_doContextSwitch

```
#define gos_ported_doContextSwitch( )
```

Value:

```
(                                   \
{                                                                       \
    /* Save LR back to main, must do this firstly.  */                   \
    GOS_ASM("PUSH {LR}");                                               \
                                                                        \
    /* Save the context of current task.  */                            \
    /* Get current PSP. */                                              \
    GOS_ASM("MRS R0, PSP");                                             \
    /* Save R4 to R11 to PSP Frame Stack.  */                           \
    GOS_ASM("STMDB R0!, {R4-R11}"); /* R0 is updated after decrement */ \
    /* Save current value of PSP. */                                    \
    GOS_ASM("BL gos_kernelSaveCurrentPsp"); /* R0 is first argument */  \
                                                                        \
    /* Do scheduling.  */                                               \
    /* Select next task.  */                                            \
    GOS_ASM("BL gos_kernelSelectNextTask");                             \
                                                                        \
    /* Retrieve the context of next task.  */                           \
    /* Get its past PSP value.  */                                      \
    GOS_ASM("BL gos_kernelGetCurrentPsp"); /* return PSP is in R0 */    \
    /* Retrieve R4-R11 from PSP Fram Stack.  */                         \
    GOS_ASM("LDMIA R0!, {R4-R11}"); /* R0 is updated after increment */ \
    /* Update PSP. */                                                   \
    GOS_ASM("MSR PSP, R0");                                             \
                                                                        \
    /* Exit.  */                                                        \
    GOS_ASM("POP {LR}");                                                \
    GOS_ASM("BX LR");                                                   \
}                                                                       \
)
```

Context-switch function.

Definition at line 82 of file gos_port.h.

7.14.2.2 gos_ported_enableFaultHandlers

```
#define gos_ported_enableFaultHandlers( )
```

Value:

```
(  \
{                                       \
    SHCSR |= (1 << 16); /* Memory Fault */  \
    SHCSR |= (1 << 17); /* Bus Fault    */  \
    SHCSR |= (1 << 18); /* Usage Fault  */  \
}                                        \
)
```

Fault handler enable function.

Definition at line 204 of file gos_port.h.

7.14.2.3 gos_ported_handleSVC

```
#define gos_ported_handleSVC( )
```

Value:

```
(               \
{                                                \
    /* Check LR to know which stack is used.  */ \
    GOS_ASM("TST LR, 4");                        \
    /* 2 next instructions are conditional.  */  \
    GOS_ASM("ITE EQ");                           \
    /* Save MSP if bit 2 is 0.  */                \
    GOS_ASM("MRSEQ R0, MSP");                    \
    /* Save PSP if bit 2 is 1.  */                \
    GOS_ASM("MRSNE R0, PSP");                    \
                                                 \
    /* Check if reset is requried.  */           \
    if (resetRequired == GOS_TRUE)               \
    {                                            \
        resetRequired = GOS_FALSE;               \
        gos_kernelProcessorReset();              \
    }                                            \
                                                 \
    /* Pass R0 as the argument.  */               \
    GOS_ASM("B gos_kernelSVC_HandlerMain");      \
}                                                \
)
```

SVC handler function.

Definition at line 121 of file gos_port.h.

7.14.2.4 gos_ported_handleSVCMain

```
#define gos_ported_handleSVCMain(
               sp )
```

Value:

```
(                    \
{                                                          \
    /* Get the address of the instruction saved in PC. */  \
    u8_t* pInstruction = (u8_t*)(sp[6]);                   \
                                                           \
    /* Go back 2 bytes (16-bit opcode).  */                 \
    pInstruction -= 2;                                     \
                                                           \
    /* Get the opcode, in little-endian.  */                \
    u8_t svcNum = *pInstruction;                           \
                                                           \
    switch (svcNum)                                        \
    {                                                      \
        case 0xFF:                                         \
            /* Trigger PendSV. */                          \
            ICSR |= (1 << 28);                            \
            break;                                         \
        default:  break;                                    \
    }                                                      \
}                                                          \
)
```

SVC handler main function.

Definition at line 152 of file gos_port.h.

7.14.2.5 gos_ported_kernelStartInit

```
#define gos_ported_kernelStartInit( )
```

Value:

```
(                           \
{                                                                   \
    /* Prepare PSP of the first task.  */                           \
    GOS_ASM("BL gos_kernelGetCurrentPsp"); /* return PSP in R0 */ \
    GOS_ASM("MSR PSP, R0");  /* set PSP */                          \
                                                                    \
    /* Change to use PSP. */                                        \
    GOS_ASM("MRS R0, CONTROL");                                     \
    GOS_ASM("ORR R0, R0, #2"); /* set bit[1] SPSEL */               \
    GOS_ASM("MSR CONTROL, R0");                                     \
                                                                    \
    /* Move to unprivileged level.  */                              \
    GOS_ASM("MRS R0, CONTROL");                                     \
    GOS_ASM("ORR R0, R0, #1"); /* Set bit[0] nPRIV */               \
    GOS_ASM("MSR CONTROL, R0");                                     \
    /* Right after here, access is limited.  */                     \
}                                                                   \
)
```

Kernel start initialization function.

Definition at line 182 of file gos_port.h.

7.14.2.6 gos_ported_pendSVHandler

```
#define gos_ported_pendSVHandler GOS_NAKED PendSV_Handler
```

Pend SV handler function name.

Definition at line 77 of file gos_port.h.

7.14.2.7 gos_ported_procReset

```
#define gos_ported_procReset( )
```

Value:

```
(                                                                                   \
{
      \
    GOS_ASM ("dsb 0xF":::"memory");
      \
    *(u32_t*)(0xE000ED0CUL) = (u32_t)((0x5FAUL << 16U) | (*(u32_t*)(0xE000ED0CUL) & (7UL << 8U)) | (1UL <<
      2U)); \
    GOS_ASM ("dsb 0xF":::"memory");
      \
}
      \
)
```

Processor reset function.

Definition at line 48 of file gos_port.h.

7.14.2.8 gos_ported_reschedule

```
#define gos_ported_reschedule(
              privilege )
```

Value:

```
(                       \
{                                                                   \
    if (privilege == GOS_PRIVILEGED)                                \
    {                                                               \
        /* Trigger PendSV directly.  */                              \
        ICSR |= (1 << 28);                                         \
    }                                                               \
    else                                                            \
    {                                                               \
        /* Call Supervisor exception to get Privileged access.  */ \
        GOS_ASM("SVC #255");                                        \
    }                                                               \
}                                                                   \
)
```

Reschedule function.

Definition at line 59 of file gos_port.h.

7.14.2.9 gos_ported_svcHandler

```
#define gos_ported_svcHandler GOS_NAKED SVC_Handler
```

SVC handler function name.

Definition at line 116 of file gos_port.h.

7.14.2.10 gos_ported_svcHandlerMain

```
#define gos_ported_svcHandlerMain gos_kernelSVC_HandlerMain
```

SVC handler main function name.

Definition at line 147 of file gos_port.h.

7.14.2.11 gos_ported_sysTickInterrupt

```
#define gos_ported_sysTickInterrupt SysTick_Handler
```

System tick handler function name.

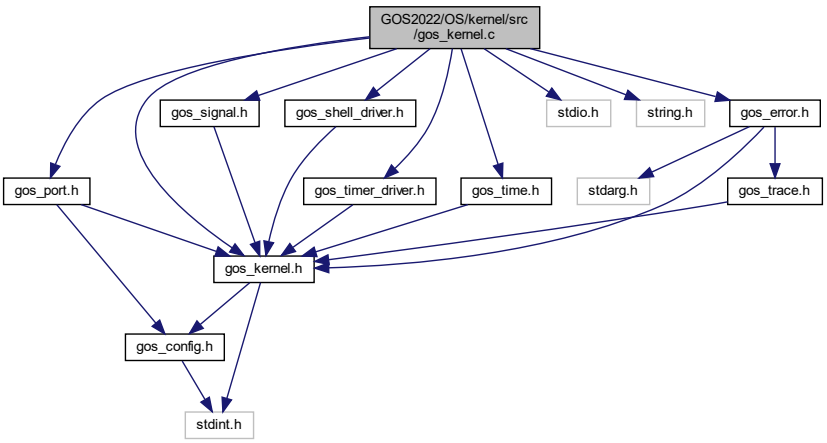
Definition at line 177 of file gos_port.h.

7.15 GOS2022/OS/kernel/src/gos_kernel.c File Reference

GOS kernel source.

```
#include <gos_error.h>
#include <gos_kernel.h>
#include <gos_port.h>
#include <gos_signal.h>
#include <gos_shell_driver.h>
#include <gos_timer_driver.h>
#include <stdio.h>
#include <string.h>
#include <gos_time.h>
```

Include dependency graph for gos_kernel.c:



Macros

- #define SHCSR ∗((volatile u32_t∗) 0xE000ED24u)
- #define ICSR ∗((volatile u32_t∗) 0xE000ED04u)
- #define TASK_DUMP_SEPARATOR "+--------+----------------------------+------+----------------+---------+----------+\r\n"
- #define MAX_CPU_DUMP_SEPARATOR "+--------+----------------------------+------------+\r\n"
- #define STACK_STATS_SEPARATOR "+--------+----------------------------+--------+---------------+----------+\r\n"
- #define CONFIG_DUMP_SEPARATOR "+--------------------------------+--------+\r\n"
- #define BINARY_PATTERN "%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s"TRACE_FORMAT_RESET
- #define TO_BINARY(word)

Functions

- GOS_STATIC void_t gos_kernelCheckTaskStack (void_t)

  Checks the stack use of the current task.
- GOS_STATIC u32_t gos_kernelGetCurrentPsp (void_t)

  Returns the current PSP.
- GOS_STATIC void_t gos_kernelSaveCurrentPsp (u32_t psp)

Saves the current PSP.

- GOS_STATIC void_t gos_kernelSelectNextTask (void_t)

  Selects the next task for execution.
- GOS_STATIC char_t * gos_kernelGetTaskStateString (gos_taskState_t taskState)

  Translates the task state to a string.
- GOS_STATIC void_t gos_kernelProcessorReset (void_t)

  Kernel processor reset.
- GOS_EXTERN void_t gos_idleTask (void_t)

  Kernel idle task.
- gos_result_t gos_kernelInit (void_t)

  This function initializes the kernel.
- gos_result_t gos_kernelStart (void_t)

  Starts the kernel.
- gos_result_t gos_kernelRegisterSwapHook (gos_taskSwapHook_t swapHookFunction)

  Registers a task swap hook function.
- gos_result_t gos_kernelRegisterSysTickHook (gos_sysTickHook_t sysTickHookFunction)

  Registers a system tick hook function.
- gos_result_t gos_kernelRegisterPrivilegedHook (gos_privilegedHook_t privilegedHookFunction)

  Registers a privileged hook function.
- gos_result_t gos_kernelRegisterPreResetHook (gos_preResetHook_t preResetHookFunction)

  Registers a pre-reset hook function.
- gos_result_t **gos_kernelSubscribeDumpReadySignal** (gos_signalHandler_t dumpReadySignalHandler)
- void_t **gos_ported_sysTickInterrupt** (void_t)
- u32_t gos_kernelGetSysTicks (void_t)

  Returns the system ticks.
- u16_t gos_kernelGetCpuUsage (void_t)

  Returns the CPU usage.
- void_t gos_kernelReset (void_t)

  Resets the microcontroller.
- void_t gos_kernelPrivilegedModeSetRequired (void_t)

  Requests a privileged mode setting.
- GOS_INLINE void_t gos_kernelDelayUs (u16_t microseconds)

  Blocking delay in microsecond range.
- GOS_INLINE void_t gos_kernelDelayMs (u16_t milliseconds)

  Blocking delay in millisecond range.
- GOS_INLINE void_t gos_kernelCalculateTaskCpuUsages (bool_t isResetRequired)

  Calculates the CPU usage for the tasks.
- void_t gos_kernelDump (void_t)

  Kernel dump.
- gos_result_t gos_kernelSetMaxCpuLoad (u16_t maxCpuLoad)

  Sets the maximum (global) CPU load.
- gos_result_t gos_kernelGetMaxCpuLoad (u16_t *pMaxCpuLoad)

  Gets the maximum (global) CPU load.
- bool_t gos_kernelIsCallerIsr (void_t)

  Returns if the current task is ISR.
- void_t **gos_ported_svcHandler** (void_t)
- void_t **gos_ported_svcHandlerMain** (u32_t *sp)
- void_t **gos_ported_pendSVHandler** (void_t)
- GOS_INLINE void_t gos_kernelReschedule (gos_kernel_privilege_t privilege)

  Reschedules the kernel.
- void_t **NMI_Handler** (void_t)
- void_t **HardFault_Handler** (void_t)
- void_t **MemManage_Handler** (void_t)
- void_t **BusFault_Handler** (void_t)
- void_t **UsageFault_Handler** (void_t)

Variables

- gos_signalId_t kernelDumpSignal
- gos_signalId_t kernelDumpReadySignal
- u8_t schedDisableCntr = 0u
- u8_t inIsr = 0u
- u8_t atomicCntr = 0u
- u32_t primask = 0u
- u32_t currentTaskIndex = 0u
- u16_t cpuUseLimit = 10000
- bool_t isKernelRunning = GOS_FALSE
- GOS_STATIC u32_t sysTicks = 0u
- GOS_STATIC u16_t sysTimerValue = 0u
- GOS_STATIC gos_runtime_t monitoringTime = {0}
- GOS_STATIC gos_taskSwapHook_t kernelSwapHookFunction = NULL
- GOS_STATIC gos_sysTickHook_t kernelSysTickHookFunction = NULL
- GOS_STATIC gos_privilegedHook_t kernelPrivilegedHookFunction = NULL
- GOS_STATIC gos_preResetHook_t kernelPreResetHookFunction = NULL
- GOS_STATIC bool_t resetRequired = GOS_FALSE
- GOS_STATIC bool_t privilegedModeSetRequired = GOS_FALSE
- GOS_STATIC u32_t previousTick = 0u
- GOS_EXTERN gos_taskDescriptor_t **taskDescriptors** [CFG_TASK_MAX_NUMBER]

7.15.1 Detailed Description

GOS kernel source.

Author

Ahmed Gazar

Date

2025-03-22

Version

1.21

For a more detailed description of this module, please refer to gos_kernel.h

7.15.2 Macro Definition Documentation

7.15.2.1 BINARY_PATTERN

```
#define BINARY_PATTERN "%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s"TRACE_FORMAT_RESET
```

Pattern for binary format printing.

Definition at line 146 of file gos_kernel.c.

7.15.2.2 CONFIG_DUMP_SEPARATOR

```
#define CONFIG_DUMP_SEPARATOR "+--------------------------------+--------+\r\n"
```

Config dump separator line.

Definition at line 141 of file gos_kernel.c.

7.15.2.3 ICSR

```
#define ICSR *( (volatile u32_t*) 0xE000ED04u )
```

Interrupt Control and State Register.

Definition at line 121 of file gos_kernel.c.

7.15.2.4 MAX_CPU_DUMP_SEPARATOR

```
#define MAX_CPU_DUMP_SEPARATOR "+--------+------------------------------+-------------+\r\n"
```

Max CPU loads dump separator line.

Definition at line 131 of file gos_kernel.c.

7.15.2.5 SHCSR

```
#define SHCSR *( (volatile u32_t*) 0xE000ED24u )
```

System Handler control and state register.

Definition at line 116 of file gos_kernel.c.

7.15.2.6 STACK_STATS_SEPARATOR

```
#define STACK_STATS_SEPARATOR "+--------+------------------------------+--------+---------------+-----------+
```

Stack statistics separator line.

Definition at line 136 of file gos_kernel.c.

7.15.2.7 TASK_DUMP_SEPARATOR

```
#define TASK_DUMP_SEPARATOR "+--------+------------------------------+------+------------------+---------+----
```

Task dump separator line.

Definition at line 126 of file gos_kernel.c.

7.15.2.8 TO_BINARY

```
#define TO_BINARY(
              word )
```

Value:

```
    (word & 0x8000 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x4000 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x2000 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x1000 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0800 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0400 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0200 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0100 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0080 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0040 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0020 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0010 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0008 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0004 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0002 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0"), \
    (word & 0x0001 ?  TRACE_FG_GREEN_START"1" :  TRACE_FG_RED_START"0")
```

u16_t to binary converter macro.

Definition at line 151 of file gos_kernel.c.

7.15.3 Function Documentation

7.15.3.1 gos_idleTask()

```
void_t gos_idleTask (
             void_t  )
```

Kernel idle task.

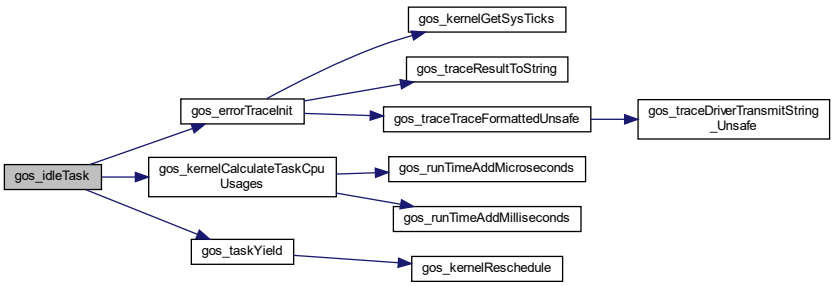
This task is executed when there is no other ready task in the system. When executed, this function refreshes the CPU-usage statistics of tasks.

Returns

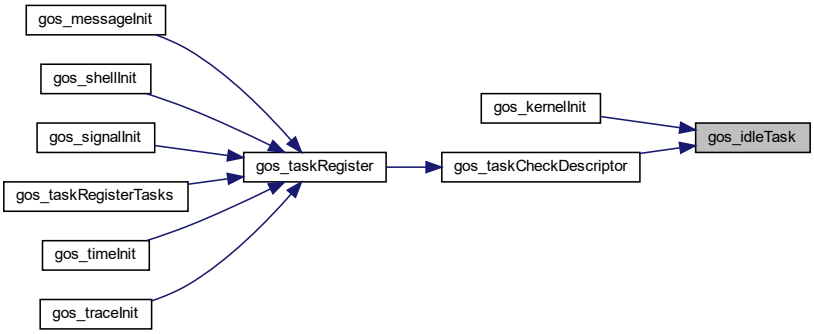
-

Definition at line 1334 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.15.3.2 gos_kernelCheckTaskStack()

```
GOS_STATIC void_t gos_kernelCheckTaskStack (
            void_t  )
```

Checks the stack use of the current task.

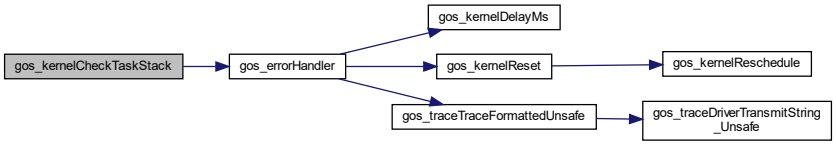
Gets the current stack pointer value and checks whether it is lower than the allowed threshold value defined for the current stack. After the overflow check, it calculates the stack usage and updates the maximum stack usage for the current task. In case of stack overflow, it goes to system error.

Returns

-

Definition at line 1001 of file gos_kernel.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.15.3.3 gos_kernelGetCurrentPsp()

```
GOS_UNUSED GOS_STATIC u32_t gos_kernelGetCurrentPsp (
            void_t  )
```

Returns the current PSP.

Returns the current PSP.

Returns

Current PSP value.

Definition at line 1047 of file gos_kernel.c.

7.15.3.4 gos_kernelGetTaskStateString()

```
GOS_STATIC char_t * gos_kernelGetTaskStateString (
            gos_taskState_t taskState )
```

Translates the task state to a string.

Based on the task state it returns a string with a printable form of the task state.

Parameters

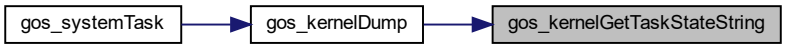
in	*taskState*	The task state variable to be translated.

Returns

String with the task state.

Definition at line 1207 of file gos_kernel.c.

Here is the caller graph for this function:



7.15.3.5 gos_kernelProcessorReset()

```
GOS_STATIC void_t gos_kernelProcessorReset (
            void_t  )
```

Kernel processor reset.

Resets the processor.

Returns

-

Definition at line 1247 of file gos_kernel.c.

7.15.3.6 gos_kernelSaveCurrentPsp()

```
GOS_UNUSED GOS_STATIC void_t gos_kernelSaveCurrentPsp (
            u32_t psp )
```

Saves the current PSP.

Saves the current PSP.

Parameters

in	*psp*	Current PSP value.

Returns

-

Definition at line 1063 of file gos_kernel.c.

7.15.3.7 gos_kernelSelectNextTask()

```
GOS_UNUSED GOS_STATIC void_t gos_kernelSelectNextTask (
            void_t  )
```

Selects the next task for execution.

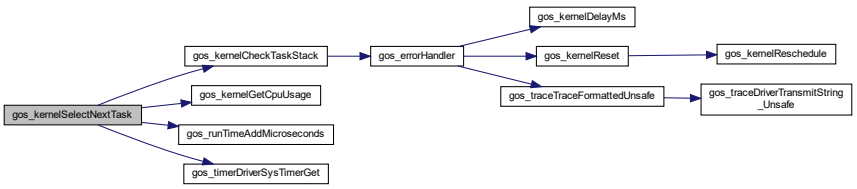
Loops through the internal task descriptor array and first checks the sleeping tasks and wakes up the ones that passed their sleeping time. Then based on the priority of the ready tasks, it selects the one with the highest priority (lowest number in priority). If there is a swap-hook function registered, it calls it, and then it refreshes the task run-time statistics.

Returns

-

Definition at line 1082 of file gos_kernel.c.

Here is the call graph for this function:



7.15.4 Variable Documentation

7.15.4.1 atomicCntr

```
u8_t atomicCntr = 0u
```

Atomic counter (incremented when GOS_ATOMIC_ENTER is called).

Definition at line 195 of file gos_kernel.c.

7.15.4.2 cpuUseLimit

```
u16_t cpuUseLimit = 10000
```

CPU use limit.

Definition at line 210 of file gos_kernel.c.

7.15.4.3 currentTaskIndex

```
u32_t currentTaskIndex = 0u
```

Current task index - for priority scheduling.

Definition at line 205 of file gos_kernel.c.

7.15.4.4 inIsr

```
u8_t inIsr = 0u
```

In ISR counter.

Definition at line 190 of file gos_kernel.c.

7.15.4.5 isKernelRunning

```
bool_t isKernelRunning = GOS_FALSE
```

Flag to indicate whether kernel is running.

Definition at line 215 of file gos_kernel.c.

7.15.4.6 kernelDumpReadySignal

```
gos_signalId_t kernelDumpReadySignal
```

Kernel dump ready signal.

Definition at line 180 of file gos_kernel.c.

7.15.4.7 kernelDumpSignal

```
gos_signalId_t kernelDumpSignal
```

Kernel dump signal.

Definition at line 175 of file gos_kernel.c.

7.15.4.8 kernelPreResetHookFunction

```
GOS_STATIC gos_preResetHook_t kernelPreResetHookFunction = NULL
```

Kernel pre-reset hook function.

Definition at line 253 of file gos_kernel.c.

7.15.4.9 kernelPrivilegedHookFunction

```
GOS_STATIC gos_privilegedHook_t kernelPrivilegedHookFunction = NULL
```

Kernel privileged execution mode set hook function.

Definition at line 248 of file gos_kernel.c.

7.15.4.10 kernelSwapHookFunction

```
GOS_STATIC gos_taskSwapHook_t kernelSwapHookFunction = NULL
```

Kernel swap hook function.

Definition at line 238 of file gos_kernel.c.

7.15.4.11 kernelSysTickHookFunction

```
GOS_STATIC gos_sysTickHook_t kernelSysTickHookFunction = NULL
```

Kernel system tick hook function.

Definition at line 243 of file gos_kernel.c.

7.15.4.12 monitoringTime

```
GOS_STATIC gos_runtime_t monitoringTime = {0}
```

Monitoring system time since last statistics calculation.

Definition at line 233 of file gos_kernel.c.

7.15.4.13 previousTick

```
GOS_STATIC u32_t previousTick = 0u
```

Previous tick value for sleep and block tick calculation.

Definition at line 268 of file gos_kernel.c.

7.15.4.14 primask

```
u32_t primask = 0u
```

IRQ state for restoring after GOS_ATOMIC_EXIT.

Definition at line 200 of file gos_kernel.c.

7.15.4.15 privilegedModeSetRequired

```
GOS_STATIC bool_t privilegedModeSetRequired = GOS_FALSE
```

Flag to indicate whether privileged mode set is required.

Definition at line 263 of file gos_kernel.c.

7.15.4.16 resetRequired

```
GOS_STATIC bool_t resetRequired = GOS_FALSE
```

Reset required flag.

Definition at line 258 of file gos_kernel.c.

7.15.4.17 schedDisableCntr

```
u8_t schedDisableCntr = 0u
```

Scheduling disabled counter.

Definition at line 185 of file gos_kernel.c.

7.15.4.18 sysTicks

```
GOS_STATIC u32_t sysTicks = 0u
```

System tick value.

Definition at line 223 of file gos_kernel.c.

7.15.4.19 sysTimerValue

```
GOS_STATIC u16_t sysTimerValue = 0u
```

System timer value for run-time calculations.

Definition at line 228 of file gos_kernel.c.

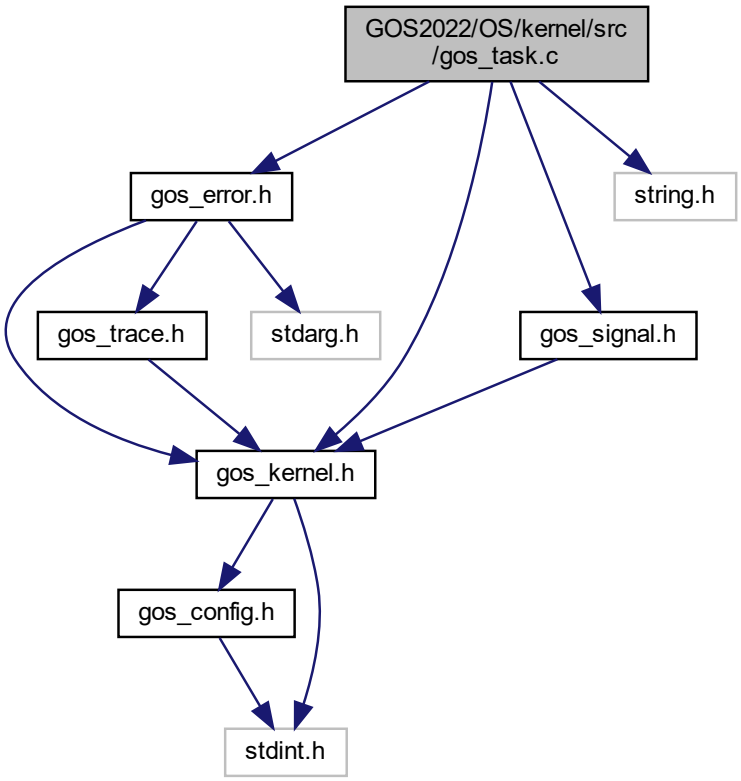
7.16 GOS2022/OS/kernel/src/gos_task.c File Reference

GOS task source.

```
#include <gos_error.h>
#include <gos_kernel.h>
#include <gos_signal.h>
```

```
#include <string.h>
```

Include dependency graph for gos_task.c:



Functions

- GOS_STATIC gos_result_t gos_taskCheckDescriptor (gos_taskDescriptor_t *taskDescriptor)

  Checks the validity of the task descriptor.
- void_t gos_idleTask (void_t)

  Kernel idle task.
- gos_result_t gos_taskRegisterTasks (gos_taskDescriptor_t *taskDescriptors, u16_t arraySize)

  This function registers an array of tasks for scheduling.
- gos_result_t gos_taskRegister (gos_taskDescriptor_t *taskDescriptor, gos_tid_t *taskId)

  This function registers a task for scheduling.
- GOS_INLINE gos_result_t gos_taskSleep (gos_taskSleepTick_t sleepTicks)

  Sends the current task to sleeping state.
- GOS_INLINE gos_result_t gos_taskWakeup (gos_tid_t taskId)

  Wakes up the given task.
- GOS_INLINE gos_result_t gos_taskSuspend (gos_tid_t taskId)

  Sends the given task to suspended state.
- GOS_INLINE gos_result_t gos_taskResume (gos_tid_t taskId)

  Resumes the given task.
- GOS_INLINE gos_result_t gos_taskBlock (gos_tid_t taskId, gos_blockMaxTick_t blockTicks)

Sends the given task to blocked state.

- GOS_INLINE gos_result_t gos_taskUnblock (gos_tid_t taskId)

  Unblocks the given task.
- GOS_INLINE gos_result_t gos_taskDelete (gos_tid_t taskId)

  Deletes the given task from the scheduling array.
- GOS_INLINE gos_result_t gos_taskSetPriority (gos_tid_t taskId, gos_taskPrio_t taskPriority)

  Sets the current priority of the given task to the given value (for temporary change).
- GOS_INLINE gos_result_t gos_taskSetOriginalPriority (gos_tid_t taskId, gos_taskPrio_t taskPriority)

  Sets the original priority of the given task to the given value (for permanent change).
- gos_result_t gos_taskGetPriority (gos_tid_t taskId, gos_taskPrio_t *taskPriority)

  Gets the current priority of the given task.
- gos_result_t gos_taskGetOriginalPriority (gos_tid_t taskId, gos_taskPrio_t *taskPriority)

  Gets the original priority of the given task.
- GOS_INLINE gos_result_t gos_taskAddPrivilege (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)

  Adds the given privileges to the given task.
- GOS_INLINE gos_result_t gos_taskRemovePrivilege (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)

  Removes the given privileges from the given task.
- GOS_INLINE gos_result_t gos_taskSetPrivileges (gos_tid_t taskId, gos_taskPrivilegeLevel_t privileges)

  Sets the given privileges for the given task.
- GOS_INLINE gos_result_t gos_taskGetPrivileges (gos_tid_t taskId, gos_taskPrivilegeLevel_t *pPrivileges)

  Gets the privileges of the given task.
- GOS_INLINE gos_result_t gos_taskYield (void_t)

  Yields the current task.
- gos_result_t gos_taskGetName (gos_tid_t taskId, gos_taskName_t taskName)

  Gets the task name of the task with the given ID.
- gos_result_t gos_taskGetId (gos_taskName_t taskName, gos_tid_t *pTaskId)

  Gets the task ID of the task with the given name.
- GOS_INLINE gos_result_t gos_taskGetCurrentId (gos_tid_t *pTaskId)

  Returns the ID of the currently running task.
- gos_result_t gos_taskGetData (gos_tid_t taskId, gos_taskDescriptor_t *pTaskData)

  Returns the task data of the given task.
- gos_result_t gos_taskGetDataByIndex (u16_t taskIndex, gos_taskDescriptor_t *pTaskData)

  Returns the task data of the given task.
- gos_result_t gos_taskGetNumber (u16_t *pTaskNum)

  Returns the number of registered tasks.
- gos_result_t **gos_taskRegisterIdleHook** (gos_taskIdleHook_t idleHookFunction)
- gos_result_t **gos_taskSubscribeDeleteSignal** (gos_signalHandler_t deleteSignalHandler)

Variables

- gos_signalId_t kernelTaskDeleteSignal
- GOS_STATIC gos_taskIdleHook_t kernelIdleHookFunction = NULL
- GOS_EXTERN u8_t **inIsr**
- GOS_EXTERN u32_t **currentTaskIndex**
- GOS_EXTERN bool_t **isKernelRunning**
- GOS_EXTERN u8_t **schedDisableCntr**
- gos_taskDescriptor_t taskDescriptors [CFG_TASK_MAX_NUMBER]

7.16.1 Detailed Description

GOS task source.

Author

Ahmed Gazar

Date

2025-06-18

Version

1.4

For a more detailed description of this module, please refer to gos_kernel.h

7.16.2 Function Documentation

7.16.2.1 gos_idleTask()

```
void_t gos_idleTask (
            void_t  )
```

Kernel idle task.

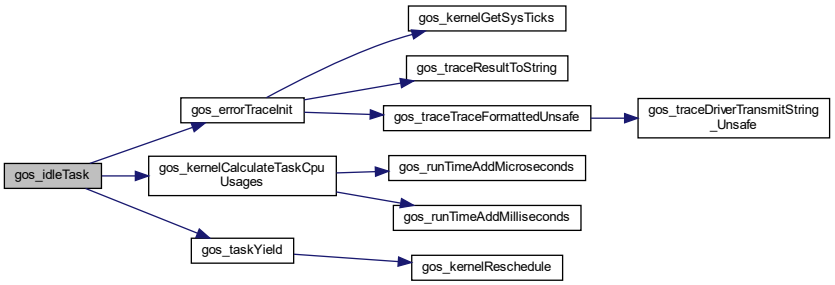
This task is executed when there is no other ready task in the system. When executed, this function refreshes the CPU-usage statistics of tasks.

Returns

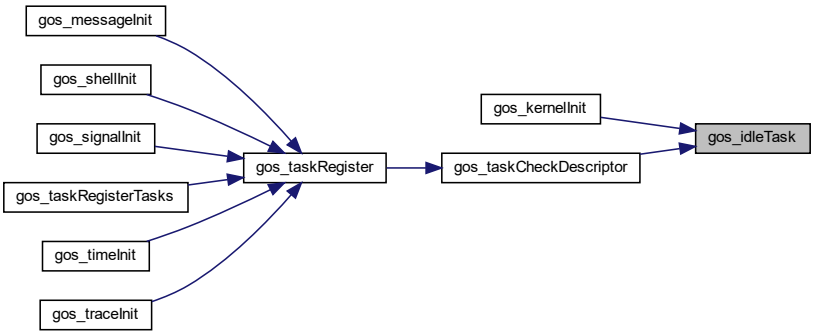
-

Definition at line 1334 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.16.2.2 gos_taskCheckDescriptor()

```
GOS_STATIC gos_result_t gos_taskCheckDescriptor (
            gos_taskDescriptor_t * taskDescriptor )
```

Checks the validity of the task descriptor.

Checks the task function pointer, task priority value, stack, and size parameters and return with error if the parameters are incorrect.

Parameters

in	*taskDescriptor*	Pointer to the task descriptor structure.

Returns

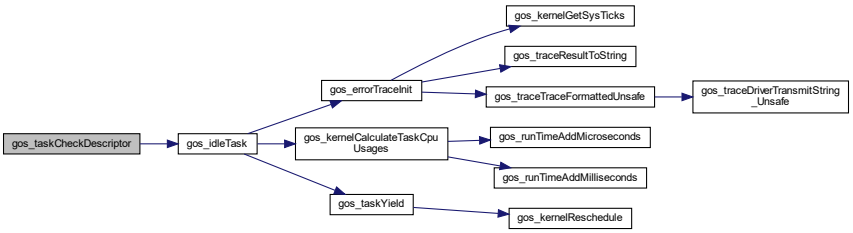
Result of task descriptor checking.

Return values

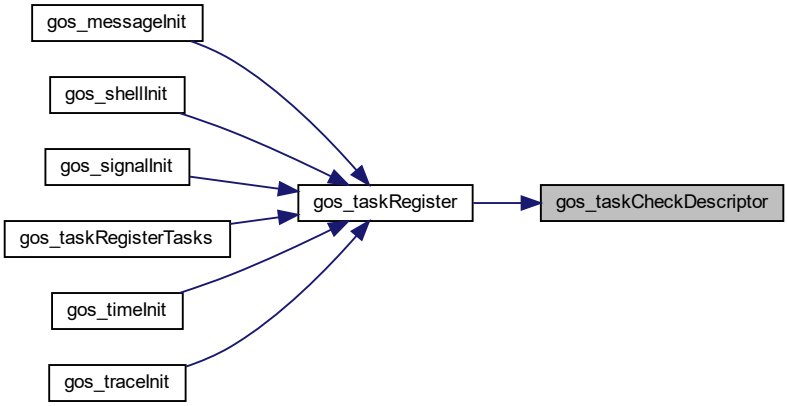
GOS_SUCCESS	Descriptor contains valid data.
GOS_ERROR	One or more parameter is invalid: • Task function is NULL pointer • Task function is the idle task • Priority exceeds the maximum priority level • Stack size is smaller than the minimum allowed • Stack size is greater than the maximum allowed • Stack size is not 4-byte aligned

Definition at line 1299 of file gos_task.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.16.3 Variable Documentation

7.16.3.1 kernelIdleHookFunction

```
GOS_STATIC gos_taskIdleHook_t kernelIdleHookFunction = NULL
```

Kernel idle hook function.

Definition at line 75 of file gos_task.c.

7.16.3.2 kernelTaskDeleteSignal

```
gos_signalId_t kernelTaskDeleteSignal
```

Kernel task delete signal.

Definition at line 67 of file gos_task.c.

7.16.3.3 taskDescriptors

```
gos_taskDescriptor_t taskDescriptors[CFG_TASK_MAX_NUMBER]
```

Initial value:

```
=
{
    [0] =
        {
            .taskFunction       = gos_idleTask,
            .taskId             = GOS_DEFAULT_TASK_ID,
            .taskPriority       = GOS_TASK_IDLE_PRIO,
            .taskName           = "gos_idle_task",
            .taskState          = GOS_TASK_READY,
            .taskStackSize      = CFG_IDLE_TASK_STACK_SIZE,
            .taskPrivilegeLevel = GOS_TASK_PRIVILEGE_KERNEL,
            .taskCpuUsageLimit  = 10000
        }
}
```

Internal task array.

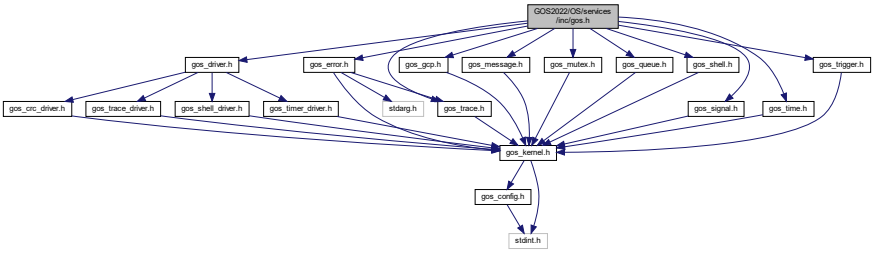
Definition at line 98 of file gos_task.c.

7.17 GOS2022/OS/services/inc/gos.h File Reference

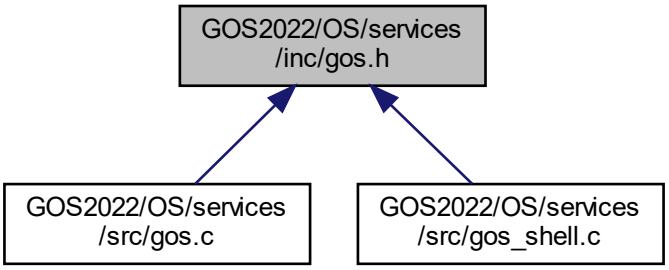
GOS header.

```
#include <gos_driver.h>
#include <gos_error.h>
#include <gos_gcp.h>
#include <gos_message.h>
#include <gos_mutex.h>
#include <gos_queue.h>
#include <gos_shell.h>
#include <gos_signal.h>
#include <gos_time.h>
#include <gos_trace.h>
#include <gos_trigger.h>
```

Include dependency graph for gos.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define GOS_VERSION_MAJOR (1)
- #define GOS_VERSION_MINOR (1)

Functions

- void_t gos_Dump (void_t)

  GOS dump.
- GOS_EXTERN gos_result_t gos_platformDriverInit (void_t)

  Platform driver initializer. Used for the platform-specific driver initializations.
- GOS_EXTERN gos_result_t gos_userApplicationInit (void_t)

  User application initializer. Used for the application-related initializations.

7.17.1 Detailed Description

GOS header.

Author

Ahmed Gazar

Date

2025-04-06

Version

1.14

This header is a wrapper for the inclusion of all OS services and drivers for GOS2022 v1.1

7.17.2 Macro Definition Documentation

7.17.2.1 GOS_VERSION_MAJOR

```
#define GOS_VERSION_MAJOR ( 1 )
```

OS major version.

Definition at line 91 of file gos.h.

7.17.2.2 GOS_VERSION_MINOR

```
#define GOS_VERSION_MINOR ( 1 )
```

OS minor version.

Definition at line 96 of file gos.h.

7.17.3 Function Documentation

7.17.3.1 gos_Dump()

```
void_t gos_Dump (
            void_t  )
```

GOS dump.

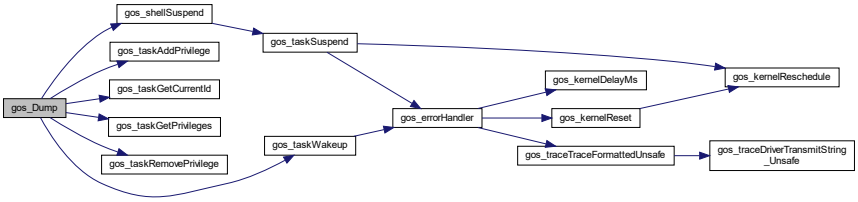
Sets the dump required flag and wakes up the system task that calls the individual dump functions.

Returns

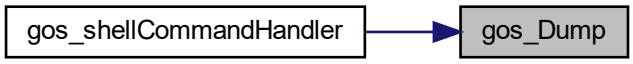
-

Definition at line 209 of file gos.c.

Here is the call graph for this function:



Here is the caller graph for this function:

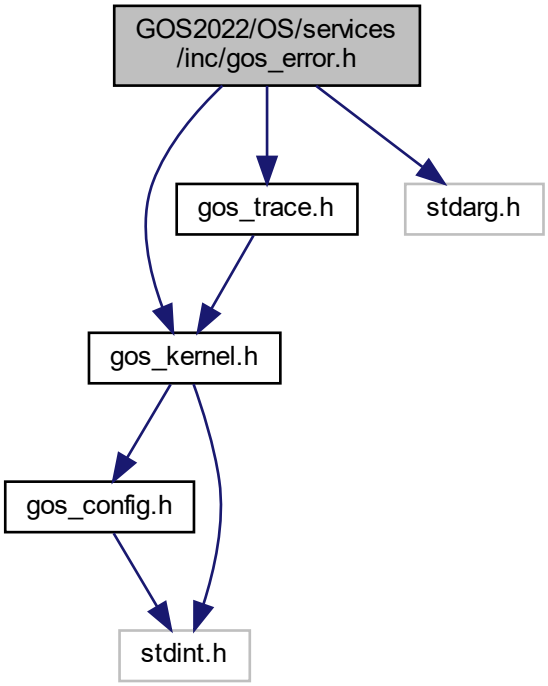


7.18 GOS2022/OS/services/inc/gos_error.h File Reference

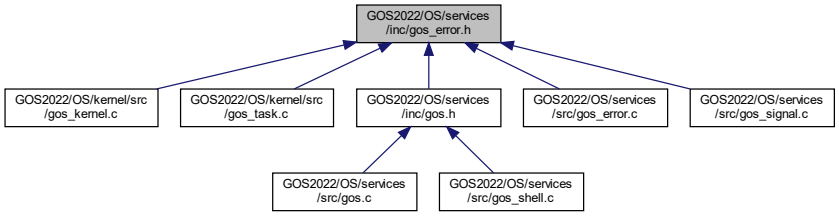
GOS error handler header.

```
#include <gos_kernel.h>
#include <gos_trace.h>
#include <stdarg.h>
```

Include dependency graph for gos_error.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum gos_errorLevel_t { GOS_ERROR_LEVEL_OS_FATAL = 0b110100, GOS_ERROR_LEVEL_OS_WARNING = 0b101110, GOS_ERROR_LEVEL_USER_FATAL = 0b011010, GOS_ERROR_LEVEL_USER_WARNING = 0b111010 }

Functions

- void_t gos_printStartupLogo (void_t)

Prints the startup logo on the trace output.

- void_t gos_errorHandler (gos_errorLevel_t errorLevel, const char_t *function, u32_t line, const char_t *errorMessage,...)

  Handles the given error.

- gos_result_t gos_errorTraceInit (const char_t *initDescription, gos_result_t initResult)

  Traces an initialization message.

7.18.1 Detailed Description

GOS error handler header.

Author

Ahmed Gazar

Date

2022-12-20

Version

2.0

This service is used for tracing error and initialization messages on the log output.

7.18.2 Enumeration Type Documentation

7.18.2.1 gos_errorLevel_t

```
enum gos_errorLevel_t
```

Error level enumerator.

Enumerator

GOS_ERROR_LEVEL_OS_FATAL	OS-level fatal error causing the system to stop.
GOS_ERROR_LEVEL_OS_WARNING	OS-level warning message will be logged, but system will not be stopped.
GOS_ERROR_LEVEL_USER_FATAL	User-level fatal error causing system stop.
GOS_ERROR_LEVEL_USER_WARNING	User-level warning.

Definition at line 66 of file gos_error.h.

7.18.3 Function Documentation

7.18.3.1 gos_errorHandler()

```
void_t gos_errorHandler (
            gos_errorLevel_t errorLevel,
            const char_t * function,
            u32_t line,
            const char_t * errorMessage,
             ...  )
```

Handles the given error.

Prints the formatted error message on the trace output, and based on the error level, it returns or stays in an infinite loop and disables scheduling.

Parameters

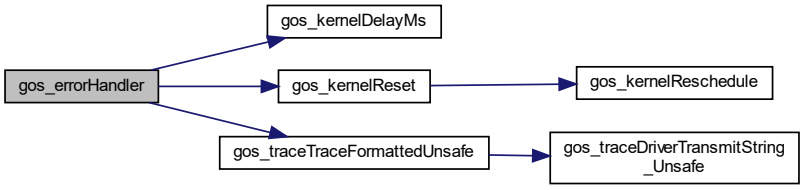
in	*errorLevel*	Level of error (OS/user, warning/fatal).
in	*function*	Function name.
in	*line*	Line number.
in	*errorMessage*	Error message.
in	...	Formatter variable arguments.

Returns

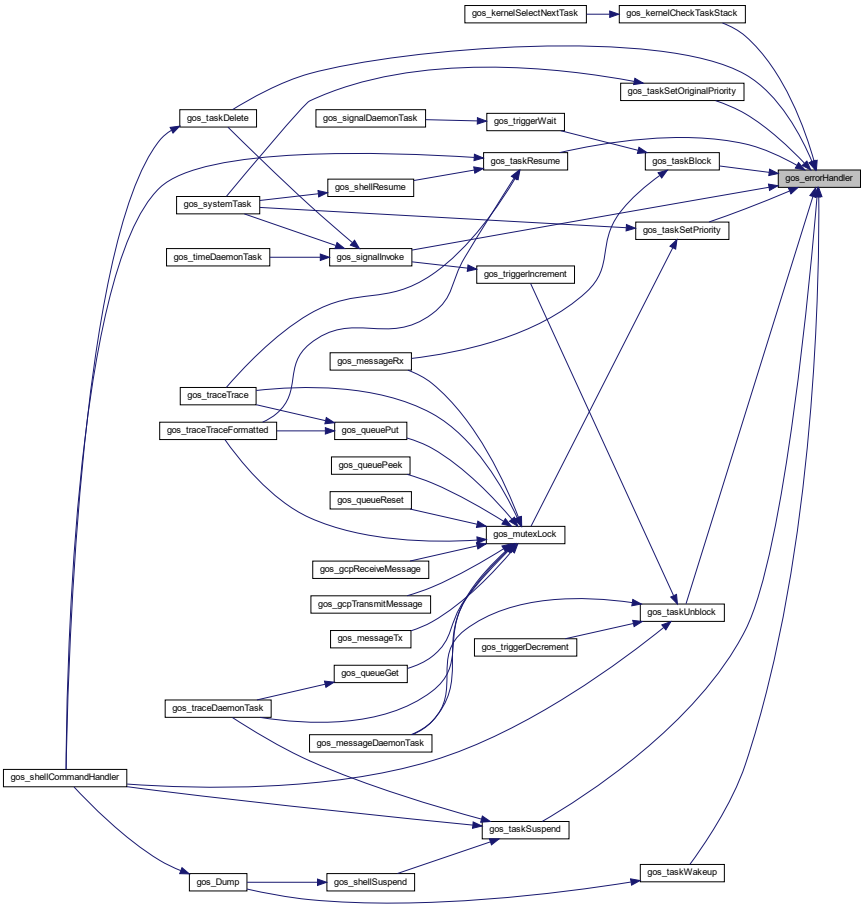
-

Definition at line 129 of file gos_error.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.18.3.2 gos_errorTraceInit()

```
gos_result_t gos_errorTraceInit (
            const char_t * initDescription,
            gos_result_t initResult )
```

Traces an initialization message.

Writes the formatted initialization message on the trace output.

Parameters

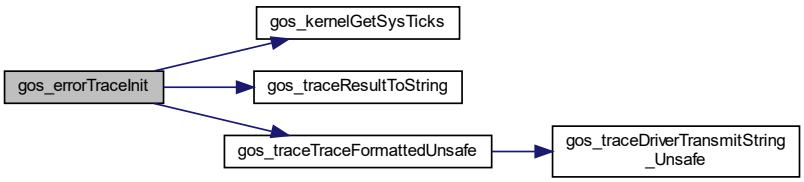
in	*initDescription*	Message to describe the initialization step.
in	*initResult*	Result of initialization.

Returns

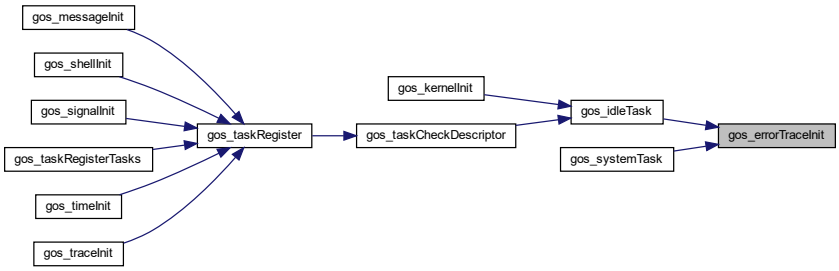
Result of initialization.

Definition at line 237 of file gos_error.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.18.3.3 gos_printStartupLogo()

```
void_t gos_printStartupLogo (
            void_t  )
```

Prints the startup logo on the trace output.

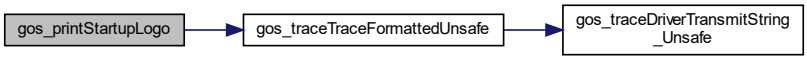
Prints the GOS logo (similar to the ones used in the file headers) on the log output.

Returns

-

Definition at line 108 of file gos_error.c.

Here is the call graph for this function:

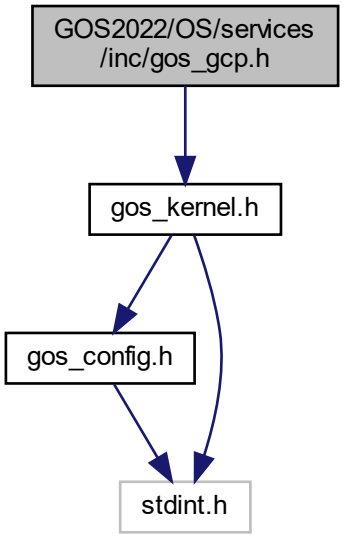


7.19 GOS2022/OS/services/inc/gos_gcp.h File Reference

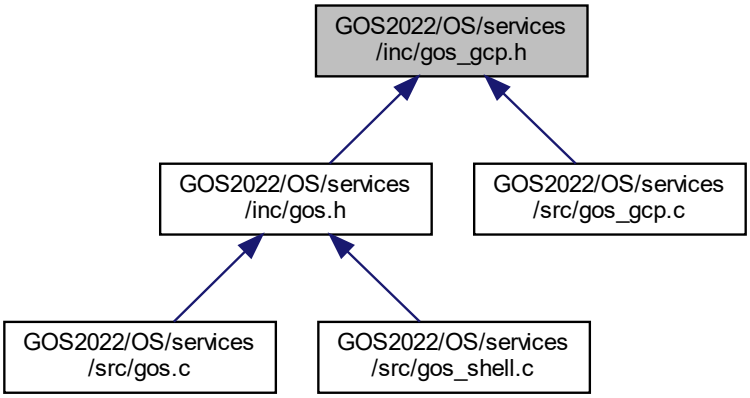
GOS General Communication Protocol header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_gcp.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef u8_t gos_gcpChannelNumber_t

  GCP channel number.
- typedef gos_result_t(* gos_gcpTransmitFunction_t) (u8_t *, u16_t)
- typedef gos_result_t(* gos_gcpReceiveFunction_t) (u8_t *, u16_t)

Functions

- gos_result_t gos_gcpInit (void_t)

  Initializes the GCP service.
- gos_result_t gos_gcpRegisterPhysicalDriver (gos_gcpChannelNumber_t channel, gos_gcpTransmitFunction_t transmitFunction, gos_gcpReceiveFunction_t receiveFunction)

  Registers the physical-layer transmit and receive driver functions.
- gos_result_t gos_gcpTransmitMessage (gos_gcpChannelNumber_t channel, u16_t messageId, void_t *pMessagePayload, u16_t payloadSize, u16_t maxChunkSize)

  Transmits the given message via the GCP protocol.
- gos_result_t gos_gcpReceiveMessage (gos_gcpChannelNumber_t channel, u16_t *pMessageId, void_t *pPayloadTarget, u16_t targetSize, u16_t maxChunkSize)

  Receives the given message via the GCP protocol.

7.19.1 Detailed Description

GOS General Communication Protocol header.

Author

Ahmed Gazar

Date

2024-07-18

Version

3.0

This service implements the GCP frame and message layers.

7.19.2 Typedef Documentation

7.19.2.1 gos_gcpReceiveFunction_t

```
typedef gos_result_t(* gos_gcpReceiveFunction_t) (u8_t *, u16_t)
```

GCP physical layer receive function type.

Definition at line 75 of file gos_gcp.h.

7.19.2.2 gos_gcpTransmitFunction_t

```
typedef gos_result_t(* gos_gcpTransmitFunction_t) (u8_t *, u16_t)
```

GCP physical layer transmit function type.

Definition at line 70 of file gos_gcp.h.

7.19.3 Function Documentation

7.19.3.1 gos_gcpInit()

```
gos_result_t gos_gcpInit (
            void_t  )
```

Initializes the GCP service.

Creates the GCP lock.

Returns

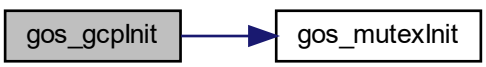
Result of initialization.

Return values

GOS_SUCCESS	GCP service initialized successfully.
GOS_ERROR	Lock creation error.

Definition at line 172 of file gos_gcp.c.

Here is the call graph for this function:



7.19.3.2 gos_gcpReceiveMessage()

```
gos_result_t gos_gcpReceiveMessage (
            gos_gcpChannelNumber_t channel,
```

```
            u16_t * pMessageId,
            void_t * pPayloadTarget,
            u16_t targetSize,
            u16_t maxChunkSize )
```

Receives the given message via the GCP protocol.

Calls the internal receiver function.

Parameters

in	*channel*	GCP channel.
out	*pMessageId*	Pointer to a variable to store the message ID.
out	*pPayloadTarget*	Pointer to the payload target buffer.
in	*targetSize*	Size of the target buffer (in bytes).
in	*maxChunkSize*	Maximum size of payload chunks.

Returns

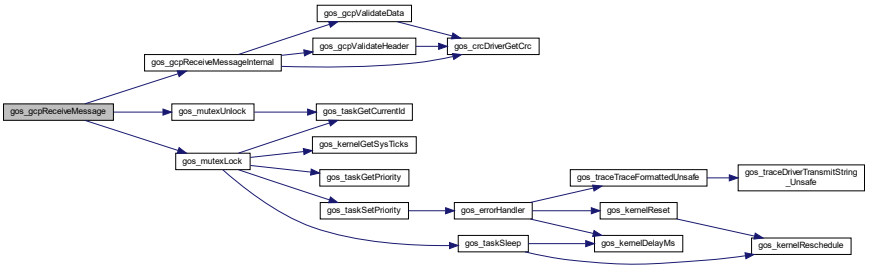
Result of message reception.

Return values

GOS_SUCCESS	Message received successfully.
GOS_ERROR	An error occurred during reception or validation.

Definition at line 259 of file gos_gcp.c.

Here is the call graph for this function:



7.19.3.3 gos_gcpRegisterPhysicalDriver()

```
gos_result_t gos_gcpRegisterPhysicalDriver (
            gos_gcpChannelNumber_t channel,
            gos_gcpTransmitFunction_t transmitFunction,
            gos_gcpReceiveFunction_t receiveFunction )
```

Registers the physical-layer transmit and receive driver functions.

Registers the physical-layer transmit and receive driver functions.

Parameters

in	*channel*	GCP channel.
in	*transmitFunction*	Transmit function to register.
in	*receiveFunction*	Receive function to register.

Returns

Result of physical driver registration.

Return values

GOS_SUCCESS	Physical driver registration successful.
GOS_ERROR	Transmit or receive function is NULL.

Definition at line 195 of file gos_gcp.c.

7.19.3.4 gos_gcpTransmitMessage()

```
gos_result_t gos_gcpTransmitMessage (
             gos_gcpChannelNumber_t channel,
             u16_t messageId,
             void_t * pMessagePayload,
             u16_t payloadSize,
             u16_t maxChunkSize )
```

Transmits the given message via the GCP protocol.

Calls the internal message transmitter function.

Parameters

in	*channel*	GCP channel.
in	*messageId*	Message ID.
in	*pMessagePayload*	Pointer to the message payload.
in	*payloadSize*	Size of the payload (in bytes).
in	*maxChunkSize*	Maximum size of payload chunks.

Returns

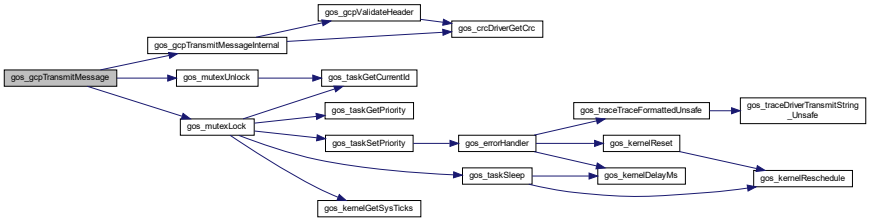
Result of message transmission.

Return values

GOS_SUCCESS	Message transmitted successfully.
GOS_ERROR	An error occurred during transmission or validation.

Definition at line 226 of file gos_gcp.c.

Here is the call graph for this function:

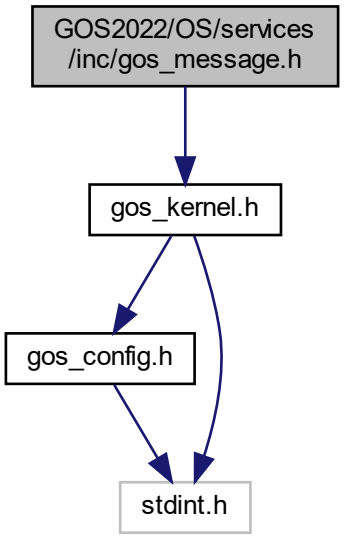


7.20 GOS2022/OS/services/inc/gos_message.h File Reference

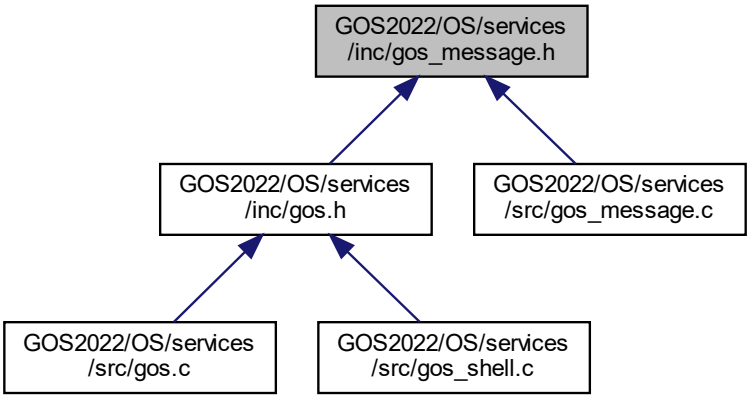
GOS message service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_message.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_message_t

Macros

- #define GOS_MESSAGE_ENDLESS_TMO (UINT16_MAX)
- #define GOS_MESSAGE_INVALID_ID (UINT16_MAX)

Typedefs

- typedef u16_t gos_messageId_t
- typedef u16_t gos_messageTimeout_t
- typedef u8_t gos_messageSize_t

  Message size type.
- typedef u8_t gos_messageIndex_t

  Message index type.
- typedef u8_t gos_messageWaiterIndex_t

  Message waiter index type.
- typedef u8_t gos_messageIdIndex_t

  Message ID index type.

Functions

- gos_result_t gos_messageInit (void_t)

  Initializes the message service.
- gos_result_t gos_messageRx (gos_messageId_t *messageIdArray, gos_message_t *target, gos_messageTimeout_t tmo)

  Receives the selected messages.
- gos_result_t gos_messageTx (gos_message_t *message)

  Transmits a message.

7.20.1 Detailed Description

GOS message service header.

Author

Ahmed Gazar

Date

2022-11-15

Version

1.2

Message service is a way of inter-task communication provided by the operating system. With the use of messages, data can be passed between different tasks. The data sent through the message service is only limited in size which is a configuration parameter. Message passing is handled by the background daemon task thus ensuring that big data transfers do not block the system. Messages are first copied into an internal message array and whenever a waiter and a message is matched, it gets copied into the target buffer. If a message is not received, it will be stuck in the internal buffer and it causes the message service to halt if the internal circular buffer gets back to the position of the stuck message. Reception of messages can happen with a given timeout or with an endless timeout. Either way the caller task will go to blocked state until the message is received or the timeout elapses, so it will not be scheduled in the meantime. Each message contains a message ID. When receiving messages, the task can define a list of IDs for reception as a filter. This way one task can receive more than one message but a message can only be received by one task.

7.20.2 Macro Definition Documentation

7.20.2.1 GOS_MESSAGE_ENDLESS_TMO

```
#define GOS_MESSAGE_ENDLESS_TMO ( UINT16_MAX )
```

Endless timeout.

Definition at line 78 of file gos_message.h.

7.20.2.2 GOS_MESSAGE_INVALID_ID

```
#define GOS_MESSAGE_INVALID_ID ( UINT16_MAX )
```

Invalid message ID.

Definition at line 83 of file gos_message.h.

7.20.3 Typedef Documentation

7.20.3.1 gos_messageId_t

```
typedef u16_t gos_messageId_t
```

Message identifier type.

Definition at line 91 of file gos_message.h.

7.20.3.2 gos_messageTimeout_t

```
typedef u16_t gos_messageTimeout_t
```

Message timeout type.

Definition at line 96 of file gos_message.h.

7.20.4 Function Documentation

7.20.4.1 gos_messageInit()

```
gos_result_t gos_messageInit (
            void_t  )
```

Initializes the message service.

Initializes the internal message and waiter arrays and registers the message daemon task.

Returns

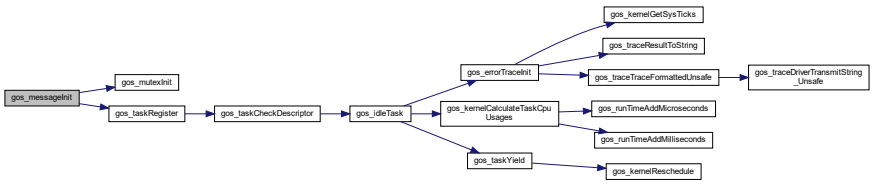
Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Message daemon task registration failed.

Definition at line 150 of file gos_message.c.

Here is the call graph for this function:



7.20.4.2 gos_messageRx()

```
gos_result_t gos_messageRx (
            gos_messageId_t * messageIdArray,
            gos_message_t * target,
            gos_messageTimeout_t tmo )
```

Receives the selected messages.

Based on the selected message IDs, this function receives the first available message. Until the message is received, the caller task is put to blocked state. The caller will be unblocked if the message is received or the timeout elapsed.

Parameters

in	*messageIdArray*	Array of messages IDs the function should receive. Must be terminated by a 0 element!
out	*target*	Pointer to the target message structure. Received data will be placed here.
in	*tmo*	Timeout value in [ms]. The resolution of timeout is 10ms! Do not use endless timeout if it is not guaranteed that the message will be received!

Returns

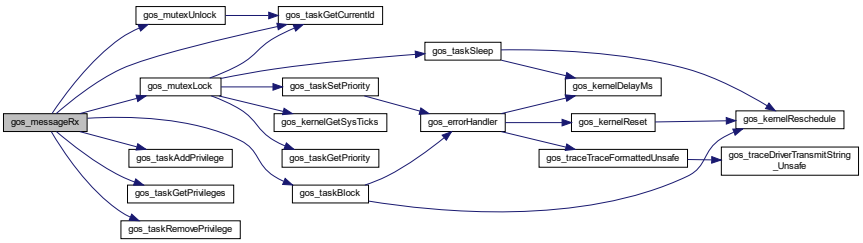
Result of message reception.

Return values

GOS_SUCCESS	Reception successful, data placed in the target structure.
GOS_ERROR	Reception failed because of invalid parameters or timeout.

Definition at line 193 of file gos_message.c.

Here is the call graph for this function:



7.20.4.3 gos_messageTx()

```
gos_result_t gos_messageTx (
            gos_message_t * message )
```

Transmits a message.

Copies the message in the internal message array for the message daemon to transfer it to the recipient task.

Parameters

in	*message*	Pointer to the message structure to be transmitted.

Returns

Result of message transmission.

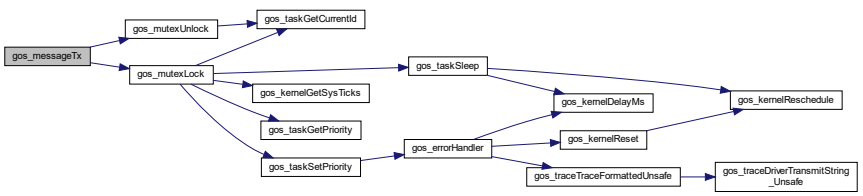
Return values

GOS_SUCCESS	Message transmission initiated successfully.
GOS_ERROR	Invalid message pointer or data or message array is full.

Function code.

Definition at line 306 of file gos_message.c.

Here is the call graph for this function:

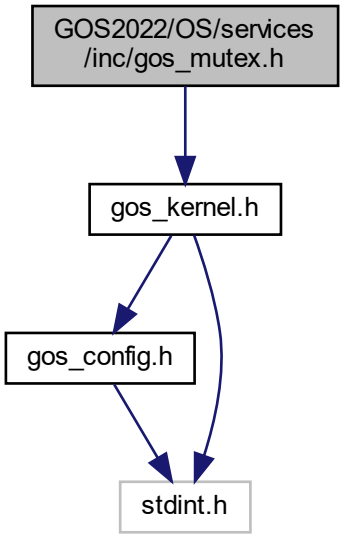


7.21 GOS2022/OS/services/inc/gos_mutex.h File Reference

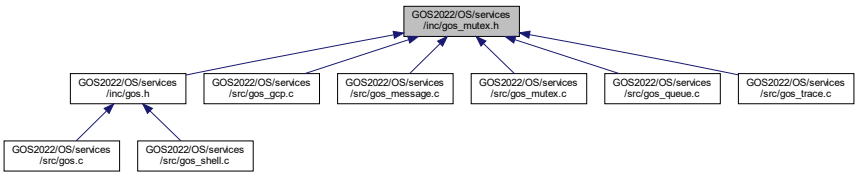
GOS mutex service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_mutex.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_mutex_t

Macros

- #define GOS_MUTEX_ENDLESS_TMO (0xFFFFFFFFu)
- #define GOS_MUTEX_NO_TMO (0x00000000u)

Enumerations

- enum gos_mutexState_t { GOS_MUTEX_UNLOCKED = 0b11010010, GOS_MUTEX_LOCKED = 0b01101011 }

Functions

- gos_result_t gos_mutexInit (gos_mutex_t *pMutex)

  Initializes the mutex instance.
- gos_result_t gos_mutexLock (gos_mutex_t *pMutex, u32_t timeout)

  Tries to lock the given mutex with the given timeout.
- gos_result_t gos_mutexUnlock (gos_mutex_t *pMutex)

  Unlocks the mutex instance.

7.21.1 Detailed Description

GOS mutex service header.

Author

Ahmed Gazar

Date

2023-09-14

Version

1.1

Mutex (Mutual Exclusion) service is provided for protecting shared resources. A mutex has two states: locked or unlocked. When a task calls the lock function, the service checks the mutex state, and it locks the mutex if it is unlocked. If a mutex is locked by another task, it waits in a non-blocking way (yields) for the mutex to be unlocked.

7.21.2 Macro Definition Documentation

7.21.2.1 GOS_MUTEX_ENDLESS_TMO

```
#define GOS_MUTEX_ENDLESS_TMO ( 0xFFFFFFFFu )
```

Mutex endless timeout.

Definition at line 66 of file gos_mutex.h.

7.21.2.2 GOS_MUTEX_NO_TMO

```
#define GOS_MUTEX_NO_TMO ( 0x00000000u )
```

Mutex no timeout.

Definition at line 71 of file gos_mutex.h.

7.21.3 Enumeration Type Documentation

7.21.3.1 gos_mutexState_t

```
enum gos_mutexState_t
```

Mutex state type.

Enumerator

GOS_MUTEX_UNLOCKED	Mutex unlocked.
GOS_MUTEX_LOCKED	Mutex locked.

Definition at line 79 of file gos_mutex.h.

7.21.4 Function Documentation

7.21.4.1 gos_mutexInit()

```
gos_result_t gos_mutexInit (
            gos_mutex_t * pMutex )
```

Initializes the mutex instance.

Sets the mutex state to unlocked.

Parameters

in,out	*pMutex*	Pointer to the mutex to be initialized.

Returns

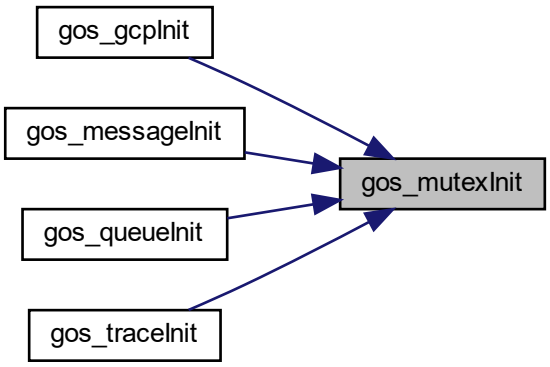
Result of initialization.

Return values

GOS_SUCCESS	Mutex initialized successfully.
GOS_ERROR	Mutex pointer is NULL.

Definition at line 76 of file gos_mutex.c.

Here is the caller graph for this function:



7.21.4.2 gos_mutexLock()

```
gos_result_t gos_mutexLock (
             gos_mutex_t * pMutex,
             u32_t timeout )
```

Tries to lock the given mutex with the given timeout.

Waits in an unblocking way until the mutex is unlocked or the timeout value is reached. If the mutex becomes unlocked within the timeout value, it locks it.

Parameters

in,out	*pMutex*	Pointer to the mutex to be locked.
in	*timeout*	Timeout value.

Returns

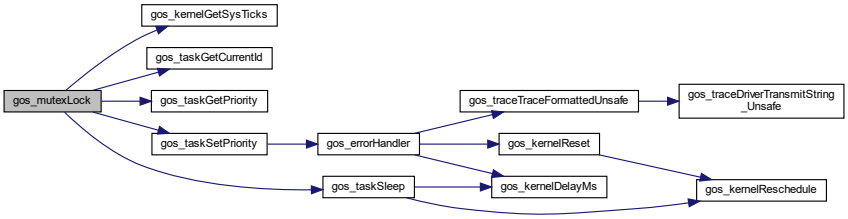
Result of mutex locking.

Return values

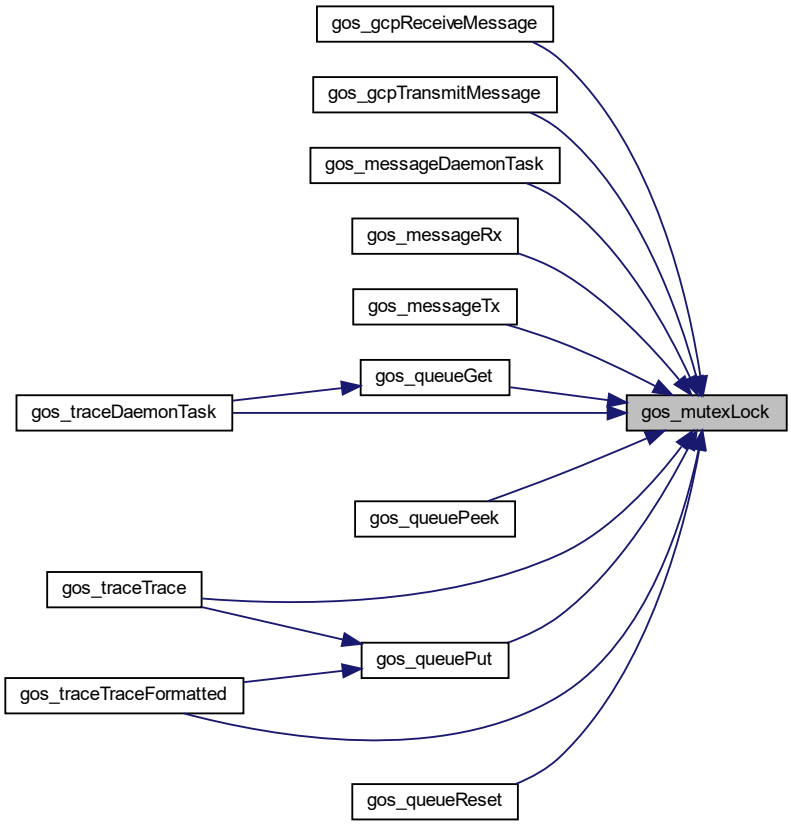
GOS_SUCCESS	Mutex locked successfully.
GOS_ERROR	Mutex could not be locked within the timeout value.

Definition at line 103 of file gos_mutex.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.21.4.3 gos_mutexUnlock()

```
gos_result_t gos_mutexUnlock (
            gos_mutex_t * pMutex )
```

Unlocks the mutex instance.

Sets the mutex state to unlocked.

Parameters

in,out	*pMutex*	Pointer to the mutex to be unlocked.

Returns

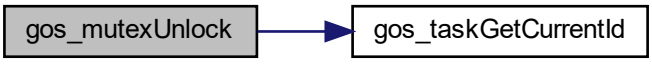
Result of mutex unlocking.

Return values

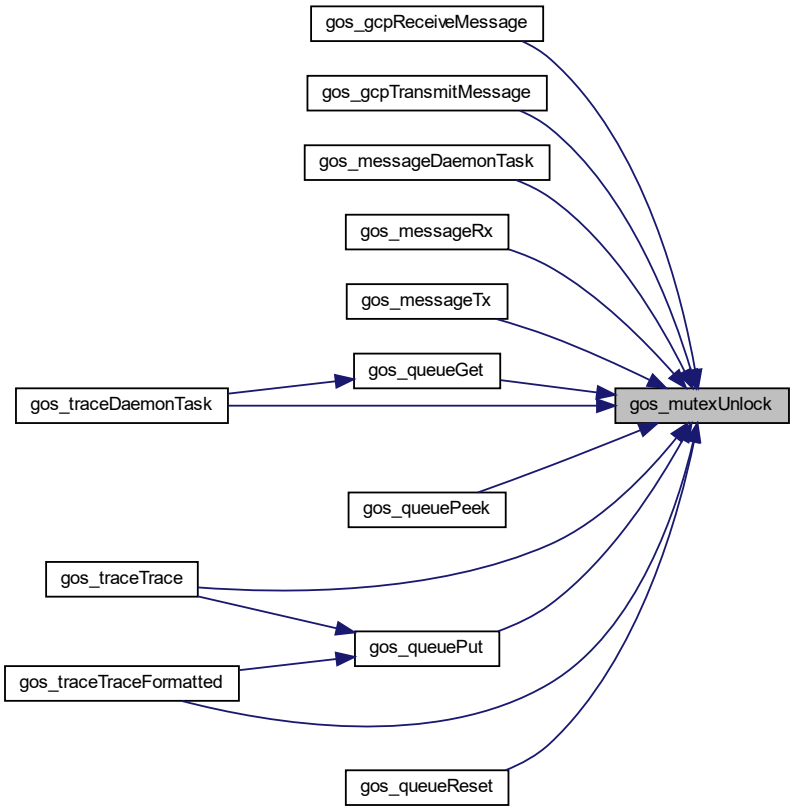
GOS_SUCCESS	Unlocking successful.
GOS_ERROR	Mutex is NULL or caller is not the owner of the mutex.

Definition at line 202 of file gos_mutex.c.

Here is the call graph for this function:



Here is the caller graph for this function:

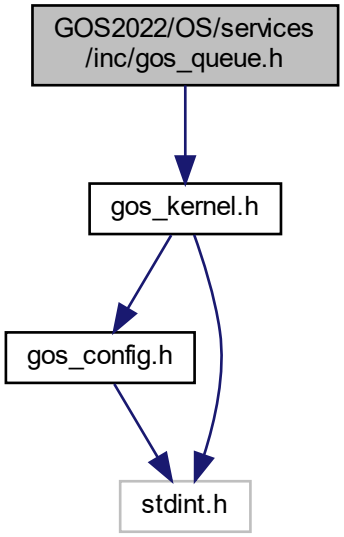


7.22 GOS2022/OS/services/inc/gos_queue.h File Reference

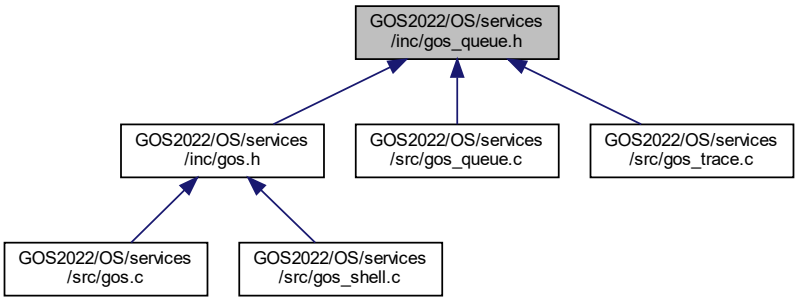
GOS queue service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_queue.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_queueDescriptor_t

Macros

- #define GOS_DEFAULT_QUEUE_ID ((gos_queueId_t) 0x3000)
- #define GOS_INVALID_QUEUE_ID ((gos_queueId_t) 0x0300)

Typedefs

- typedef u8_t gos_queueByte_t
- typedef u8_t gos_queueLength_t

  Queue length type.
- typedef u8_t gos_queueIndex_t

  Queue index type.
- typedef u16_t gos_queueId_t

  Queue ID type.
- typedef void_t(∗ gos_queueFullHook) (gos_queueId_t)
- typedef void_t(∗ gos_queueEmptyHook) (gos_queueId_t)

Functions

- gos_result_t gos_queueInit (void_t)

  This function initializes the queue service.
- gos_result_t gos_queueCreate (gos_queueDescriptor_t ∗pQueueDescriptor)

  This function creates a new queue.
- gos_result_t gos_queuePut (gos_queueId_t queueId, void_t ∗element, gos_queueLength_t elementSize, u32_t timeout)

  This function puts an element in the given queue.
- gos_result_t gos_queueGet (gos_queueId_t queueId, void_t ∗target, gos_queueLength_t targetSize, u32_t timeout)

  This function gets the next element from the given queue.
- gos_result_t gos_queuePeek (gos_queueId_t queueId, void_t ∗target, gos_queueLength_t targetSize, u32_t timeout)

  This function gets the next element from the given queue without removing it.
- gos_result_t gos_queueReset (gos_queueId_t queueId)

  Resets the given queue.
- gos_result_t gos_queueRegisterFullHook (gos_queueFullHook fullHook)

  This function registers a queue full hook function.
- gos_result_t gos_queueRegisterEmptyHook (gos_queueEmptyHook emptyHook)

  This function registers a queue empty hook function.
- gos_result_t gos_queueGetName (gos_queueId_t queueId, gos_queueName_t queueName)

  This function gets the name of the given queue.
- gos_result_t gos_queueGetElementNumber (gos_queueId_t queueId, gos_queueIndex_t ∗elementNumber)

  This function gets the number of elements in the given queue.
- void_t gos_queueDump (void_t)

  Queue dump.

7.22.1 Detailed Description

GOS queue service header.

Author

Ahmed Gazar

Date

2024-04-02

Version

1.6

Queue service is one of the inter-task communication solutions offered by the OS. A queue can hold a given number of elements with a given element size. The queue is implemented as a FIFO. Queues are unsafe in a way that any task with the queue ID in their knowledge can put and get elements to/from the queue. So queues are most suitable for serializing data that are coming from multiple resources but processes by a single task. It is also possible to peek the queue, meaning that the next element of the queue can be requested without deleting it form the queue. This way multiple tasks can process data coming from a queue in a cooperative way, but they rely on each others data processing.

7.22.2 Macro Definition Documentation

7.22.2.1 GOS_DEFAULT_QUEUE_ID

```
#define GOS_DEFAULT_QUEUE_ID ( (gos_queueId_t) 0x3000 )
```

Default queue ID.

Definition at line 77 of file gos_queue.h.

7.22.2.2 GOS_INVALID_QUEUE_ID

```
#define GOS_INVALID_QUEUE_ID ( (gos_queueId_t) 0x0300 )
```

Invalid queue ID.

Definition at line 82 of file gos_queue.h.

7.22.3 Typedef Documentation

7.22.3.1 gos_queueByte_t

```
typedef u8_t gos_queueByte_t
```

One byte in a queue element.

Definition at line 90 of file gos_queue.h.

7.22.3.2 gos_queueEmptyHook

```
typedef void_t(* gos_queueEmptyHook) (gos_queueId_t)
```

Queue full hook type.

Definition at line 116 of file gos_queue.h.

7.22.3.3 gos_queueFullHook

```
typedef void_t(* gos_queueFullHook) (gos_queueId_t)
```

Queue full hook type.

Definition at line 111 of file gos_queue.h.

7.22.4 Function Documentation

7.22.4.1 gos_queueCreate()

```
gos_result_t gos_queueCreate (
            gos_queueDescriptor_t * pQueueDescriptor )
```

This function creates a new queue.

This function loops through the internal queue array and registers the new queue in the next free slot.

Parameters

in,out	*pQueueDescriptor*	Pointer to queue descriptor variable with queue data.

Returns

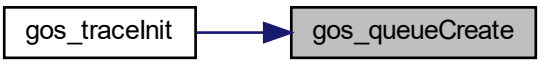
Result of queue creation.

Return values

GOS_SUCCESS	Queue creation successful.
GOS_ERROR	Queue descriptor is NULL pointer or queue array is full.

Definition at line 181 of file gos_queue.c.

Here is the caller graph for this function:



7.22.4.2 gos_queueDump()

```
void_t gos_queueDump (
            void_t  )
```

Queue dump.

Prints the queue data of all queues to the trace output.

Returns

-

Definition at line 598 of file gos_queue.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.22.4.3 gos_queueGet()

```
gos_result_t gos_queueGet (
            gos_queueId_t queueId,
            void_t * target,
            gos_queueLength_t targetSize,
            u32_t timeout )
```

This function gets the next element from the given queue.

This function checks the queue state and gets the next element from the queue.

Parameters

in	*queueId*	Queue ID.
out	*target*	Pointer to target variable.
in	*targetSize*	Size of target.
in	*timeout*	Timeout for locking queue mutex.

Returns

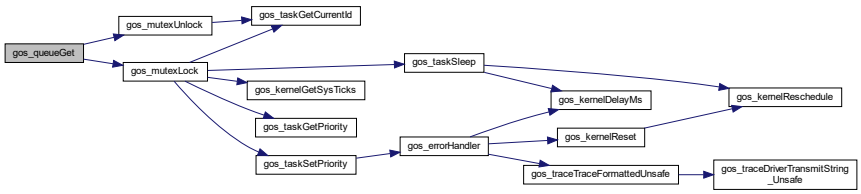
Result of element getting.

Return values

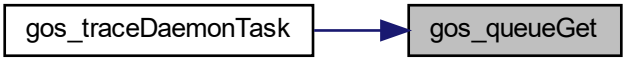
GOS_SUCCESS	Element successfully moved from queue to target.
GOS_ERROR	Invalid queue ID, invalid target size or queue is empty.

Definition at line 310 of file gos_queue.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.22.4.4 gos_queueGetElementNumber()

```
gos_result_t gos_queueGetElementNumber (
            gos_queueId_t queueId,
            gos_queueIndex_t * elementNumber )
```

This function gets the number of elements in the given queue.

This function copies the number of elements in the queue belonging to the given ID to the given variable.

Parameters

in	*queueId*	Queue ID.
out	*elementNumber*	Pointer to variable to store the element number in.

Returns

Result of queue element number getting.

Return values

GOS_SUCCESS	Element number getting successful.
GOS_ERROR	Invalid queue ID or element number variable is NULL.

Definition at line 564 of file gos_queue.c.

7.22.4.5 gos_queueGetName()

```
gos_result_t gos_queueGetName (
            gos_queueId_t queueId,
            gos_queueName_t queueName )
```

This function gets the name of the given queue.

This function copies the name of the queue belonging to the given ID to the given variable.

Parameters

in	*queueId*	Queue ID.
out	*queueName*	Queue name.

Returns

Result of queue name getting.

Return values

GOS_SUCCESS	Name getting successful.
GOS_ERROR	Invalid queue ID or queue name variable is NULL.

Definition at line 525 of file gos_queue.c.

7.22.4.6 gos_queueInit()

```
gos_result_t gos_queueInit (
            void_t  )
```

This function initializes the queue service.

Initializes the internal queue array, creates the queue lock, and registers the queue dump task in the kernel.

Returns

Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Lock creation, task registration or task suspension error.

Definition at line 147 of file gos_queue.c.

Here is the call graph for this function:



7.22.4.7 gos_queuePeek()

```
gos_result_t gos_queuePeek (
            gos_queueId_t queueId,
            void_t * target,
```

```
            gos_queueLength_t targetSize,
            u32_t timeout )
```

This function gets the next element from the given queue without removing it.

This function checks the queue state and returns the next element from the queue without modifying the queue counters.

Parameters

in	*queueId*	Queue ID.
out	*target*	Pointer to target variable.
in	*targetSize*	Size of target.
in	*timeout*	Timeout for locking queue mutex.

Returns

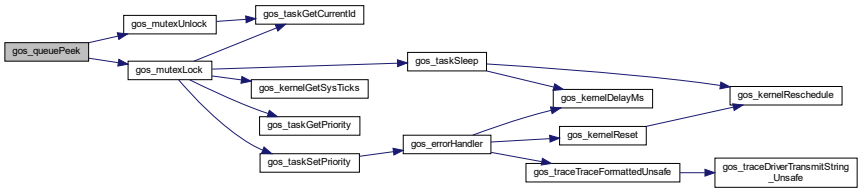
Result of element getting.

Return values

GOS_SUCCESS	Element successfully copied from queue to target.
GOS_ERROR	Invalid queue ID, invalid target size or queue is empty.

Definition at line 385 of file gos_queue.c.

Here is the call graph for this function:



7.22.4.8 gos_queuePut()

```
gos_result_t gos_queuePut (
            gos_queueId_t queueId,
            void_t * element,
            gos_queueLength_t elementSize,
            u32_t timeout )
```

This function puts an element in the given queue.

This function checks the queue state and places the given element in the next queue element.

Parameters

in	*queueId*	Queue ID.
in	*element*	Pointer to element.
in	*elementSize*	Size of element.
in	*timeout*	Timeout for locking queue mutex.

Returns

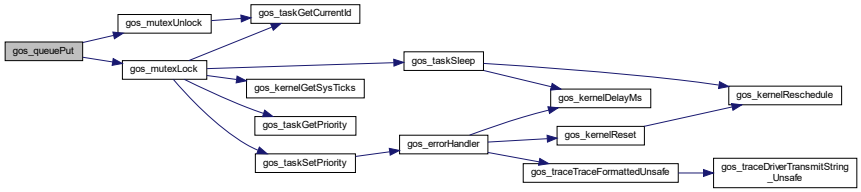
Result of element putting.

Return values

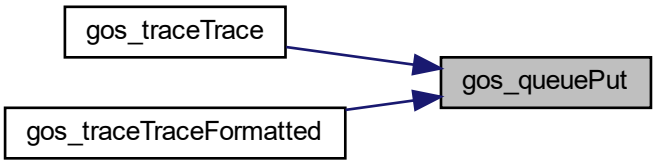
GOS_SUCCESS	Element successfully put in the queue.
GOS_ERROR	Invalid queue ID, invalid element size or queue is full.

Definition at line 231 of file gos_queue.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.22.4.9 gos_queueRegisterEmptyHook()

```
gos_result_t gos_queueRegisterEmptyHook (
            gos_queueEmptyHook emptyHook )
```

This function registers a queue empty hook function.

This function checks whether a hook has been already registered, and if not, it saves the given hook function.

Parameters

in	*emptyHook*	Hook function.

Returns

Result of hook registration.

Return values

GOS_SUCCESS	Hook registration successful.
GOS_ERROR	Hook already exists or parameter is NULL pointer.

Definition at line 499 of file gos_queue.c.

7.22.4.10 gos_queueRegisterFullHook()

```
gos_result_t gos_queueRegisterFullHook (
            gos_queueFullHook fullHook )
```

This function registers a queue full hook function.

This function checks whether a hook has been already registered, and if not, it saves the given hook function.

Parameters

in	*fullHook*	Hook function.

Returns

Result of hook registration.

Return values

GOS_SUCCESS	Hook registration successful.
GOS_ERROR	Hook already exists or parameter is NULL pointer.

Definition at line 473 of file gos_queue.c.

7.22.4.11 gos_queueReset()

```
gos_result_t gos_queueReset (
            gos_queueId_t queueId )
```

Resets the given queue.

Sets the read and write counter to zero, making the queue empty.

Parameters

in	*queue↩ Id*	Queue ID.

Returns

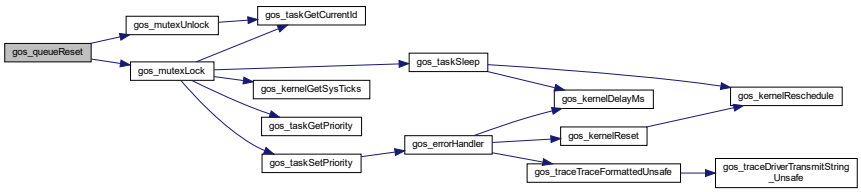
Result of queue resetting.

Return values

GOS_SUCCESS	Reset successful.
GOS_ERROR	Invalid queue ID.

Definition at line 435 of file gos_queue.c.

Here is the call graph for this function:

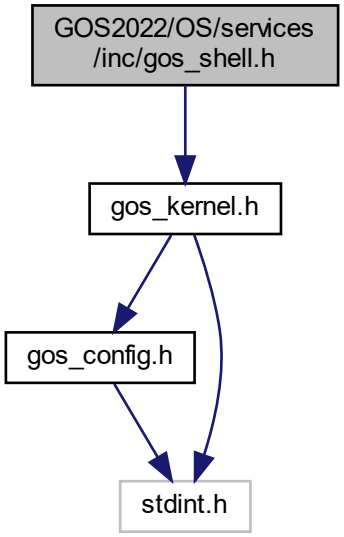


7.23 GOS2022/OS/services/inc/gos_shell.h File Reference

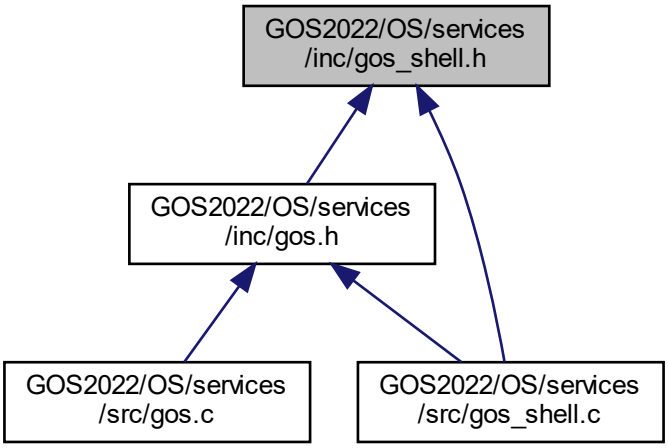
GOS shell service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_shell.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_shellCommand_t

Typedefs

- typedef void_t(∗ gos_shellFunction) (char_t ∗params)
- typedef u8_t gos_shellCommandIndex_t

  Shell command index type.

Functions

- gos_result_t gos_shellInit (void_t)

  This function initializes the shell service.
- gos_result_t gos_shellRegisterCommands (gos_shellCommand_t ∗commands, u16_t arraySize)

  This function registers an array of commands in the shell.
- gos_result_t gos_shellRegisterCommand (gos_shellCommand_t ∗command)

  This function registers a command in the shell.
- gos_result_t gos_shellSuspend (void_t)

  Suspends the shell daemon task.
- gos_result_t gos_shellResume (void_t)

  Resumes the shell daemon task.
- gos_result_t gos_shellEchoOn (void_t)

  Turns the shell echoing on.
- gos_result_t gos_shellEchoOff (void_t)

  Turns the shell echoing off.

7.23.1 Detailed Description

GOS shell service header.

Author

Ahmed Gazar

Date

2023-07-12

Version

1.3

The shell service provides an easy interface to receive and process commands in a terminal. Optionally the characters can be echoed back. The user can register different commands with callback functions. When the enter key is hit, the shell daemon processes the input. If it finds the input string in the command array, it calls the corresponding callback function and passes the parameter list as a string to the callback for further processing.

7.23.2 Typedef Documentation

7.23.2.1 gos_shellFunction

```
typedef void_t(* gos_shellFunction) (char_t *params)
```

Shell function type.

Definition at line 70 of file gos_shell.h.

7.23.3 Function Documentation

7.23.3.1 gos_shellEchoOff()

```
gos_result_t gos_shellEchoOff (
            void_t  )
```

Turns the shell echoing off.

Resets the internal echo flag.

Returns

Result of turning echoing off.

Return values

GOS_SUCCESS	Echoing turned off successfully.

Definition at line 325 of file gos_shell.c.

7.23.3.2 gos_shellEchoOn()

```
gos_result_t gos_shellEchoOn (
            void_t  )
```

Turns the shell echoing on.

Sets the internal echo flag.

Returns

Result of turning echoing on.

Return values

GOS_SUCCESS	Echoing turned on successfully.

Definition at line 307 of file gos_shell.c.

7.23.3.3 gos_shellInit()

```
gos_result_t gos_shellInit (
            void_t  )
```

This function initializes the shell service.

Initializes the internal command structure, and registers the shell daemon task in the kernel.

Returns

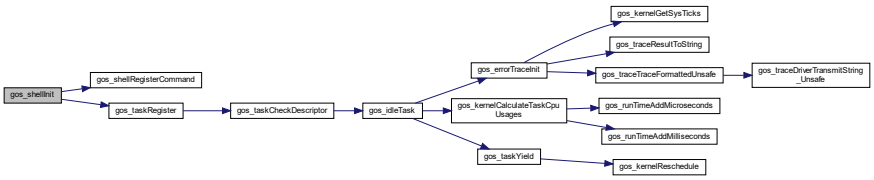
Result of initialization.

Return values

GOS_SUCCESS	Shell initialization successful.
GOS_ERROR	Shell daemon task registration failed.

Definition at line 154 of file gos_shell.c.

Here is the call graph for this function:



7.23.3.4 gos_shellRegisterCommand()

```
gos_result_t gos_shellRegisterCommand (
            gos_shellCommand_t * command )
```

This function registers a command in the shell.

Checks the command structure and registers the command in the next empty slot in the internal command array.

Parameters

in	*command*	Pointer to the command structure to register.

Returns

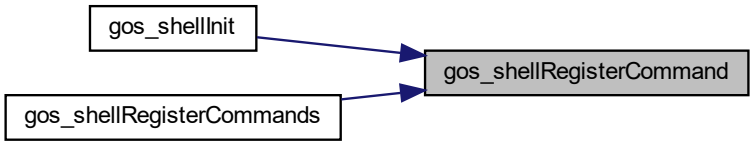
Result of shell command registration.

Return values

GOS_SUCCESS	Command registration successful.
GOS_ERROR	Command function or name is NULL or internal command array is full.

Definition at line 236 of file gos_shell.c.

Here is the caller graph for this function:



7.23.3.5 gos_shellRegisterCommands()

```
gos_result_t gos_shellRegisterCommands (
            gos_shellCommand_t * commands,
            u16_t arraySize )
```

This function registers an array of commands in the shell.

Checks the array pointer and registers the commands one by one.

Parameters

in	*commands*	Pointer to the command structure array to register.
in	*arraySize*	Size of the array in bytes.

Returns

Result of shell command registration.

Return values

GOS_SUCCESS	Command registration successful.
GOS_ERROR	Command function or name is NULL or internal command array is full.

Definition at line 188 of file gos_shell.c.

Here is the call graph for this function:



7.23.3.6 gos_shellResume()

```
gos_result_t gos_shellResume (
            void_t  )
```

Resumes the shell daemon task.

Resumes the shell daemon task.

Returns

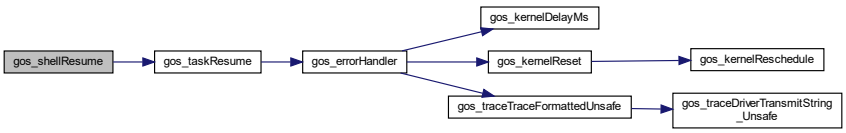
Result of shell resumption.

Return values

GOS_SUCCESS	Shell resumed successfully.
GOS_ERROR	Task resumption failed.

Definition at line 289 of file gos_shell.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.23.3.7 gos_shellSuspend()

```
gos_result_t gos_shellSuspend (
            void_t  )
```

Suspends the shell daemon task.

Suspends the shell daemon task.

Returns

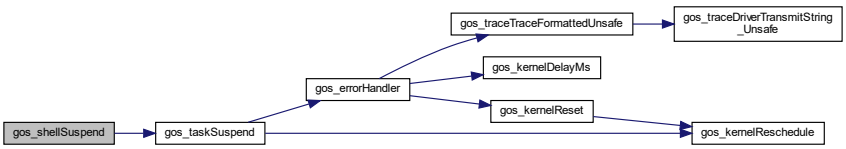
Result of shell suspension.

Return values

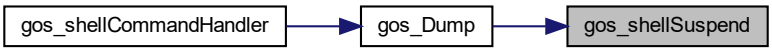
GOS_SUCCESS	Shell suspended successfully.
GOS_ERROR	Task suspension failed.

Definition at line 271 of file gos_shell.c.

Here is the call graph for this function:



Here is the caller graph for this function:

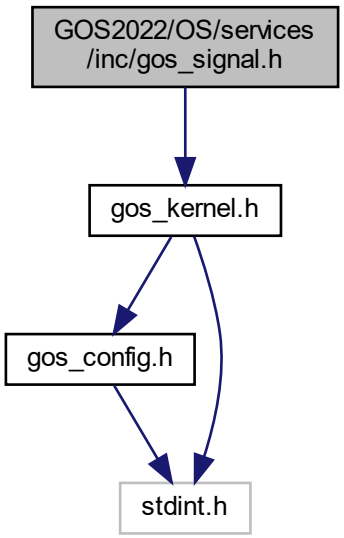


7.24 GOS2022/OS/services/inc/gos_signal.h File Reference

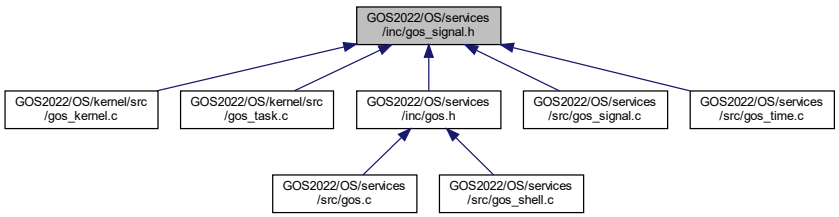
GOS signal service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_signal.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef u8_t gos_signalId_t
  Signal ID.
- typedef u16_t gos_signalSenderId_t
  Signal sender ID.
- typedef u8_t gos_signalIndex_t
  Signal index for loops.
- typedef u8_t gos_signalHandlerIndex_t
  Signal handler index type.
- typedef void_t(∗ gos_signalHandler_t) (gos_signalSenderId_t)
  Signal handler function type.

Functions

- gos_result_t gos_signalInit (void_t)
  Initializes the signal service.
- gos_result_t gos_signalCreate (gos_signalId_t ∗pSignal)
  Creates a new signal.
- gos_result_t gos_signalSubscribe (gos_signalId_t signalId, gos_signalHandler_t signalHandler, gos_taskPrivilegeLevel_t signalHandlerPrivileges)
  Subscribes to the given signal.
- gos_result_t gos_signalInvoke (gos_signalId_t signalId, gos_signalSenderId_t senderId)
  Invokes the given signal.

7.24.1 Detailed Description

GOS signal service header.

Author

Ahmed Gazar

Date

2022-11-15

Version

1.1

Signal service is a way of inter-task or inter-process communication provided by the operating system. Signals can be created, subscribed to and invoked. When a signal is invoked, it is placed into an invoke queue and the signal daemon task will handle the invoke requests in the background. Thus signals are not instantly invoked. When a signal is invoked, all the subscribed functions get called. A signal can be invoked without any subscribers (in this case no function will be called).

7.24.2 Function Documentation

7.24.2.1 gos_signalCreate()

```
gos_result_t gos_signalCreate (
            gos_signalId_t * pSignal )
```

Creates a new signal.

Finds the next free slot in the signal array and registers the new signal there.

Parameters

out	*pSignal*	Pointer to a signal identifier.

Returns

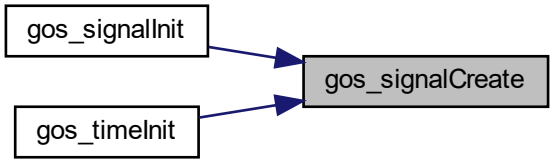
Success of signal creation.

Return values

GOS_SUCCESS	Creation successful.
GOS_ERROR	Signal array full.

Definition at line 183 of file gos_signal.c.

Here is the caller graph for this function:



7.24.2.2 gos_signalInit()

```
gos_result_t gos_signalInit (
            void_t  )
```

Initializes the signal service.

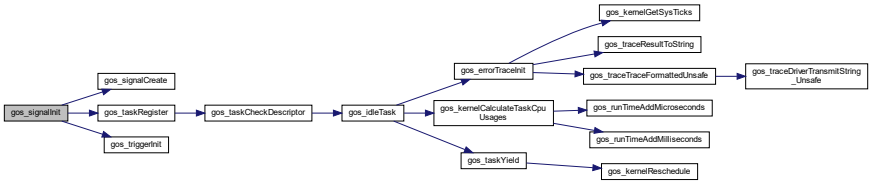
Initializes the internal signal array, creates a signal queue, registers the signal daemon task, creates the kernel dump signal, and subscribes the necessary components for the dump signal.

Returns

-

Definition at line 146 of file gos_signal.c.

Here is the call graph for this function:



7.24.2.3 gos_signalInvoke()

```
gos_result_t gos_signalInvoke (
            gos_signalId_t signalId,
            gos_signalSenderId_t senderId )
```

Invokes the given signal.

Places the given signal in the invoke queue (for the signal daemon to actually invoke the signal in the background).

Parameters

in	*signalId*	Signal identifier.
in	*senderId*	Sender identifier (or data to pass).

Returns

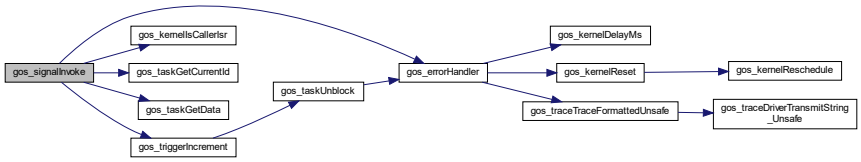
Success of signal invoking.

Return values

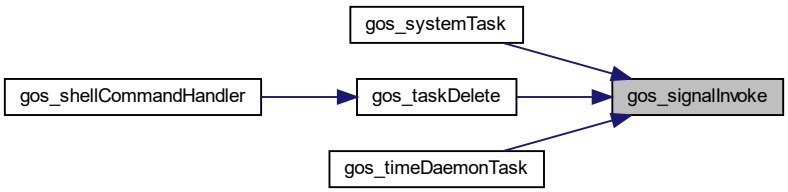
GOS_SUCCESS	Invoking successful.
GOS_ERROR	Invalid signal ID or signal unused.

Definition at line 260 of file gos_signal.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.24.2.4 gos_signalSubscribe()

```
gos_result_t gos_signalSubscribe (
            gos_signalId_t signalId,
            gos_signalHandler_t signalHandler,
            gos_taskPrivilegeLevel_t signalHandlerPrivileges )
```

Subscribes to the given signal.

Finds the next free slot in the signal handler array and registers the signal handler there.

Parameters

in	*signalId*	Signal identifier.
in	*signalHandler*	Signal handler function pointer.
in	*signalHandlerPrivileges*	Signal handler privilege level.

Returns

Success of signal subscription.

Return values

GOS_SUCCESS	Subscription successful.
GOS_ERROR	Invalid signal ID, signal handler NULL pointer, or handler array full.

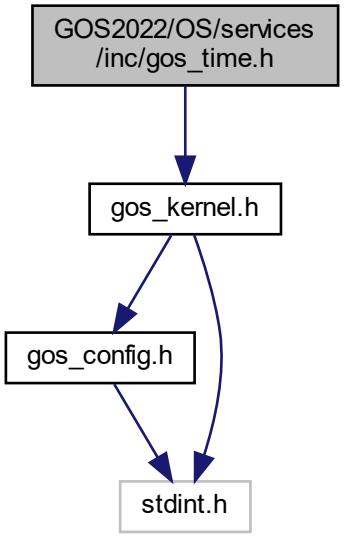
Definition at line 217 of file gos_signal.c.

7.25 GOS2022/OS/services/inc/gos_time.h File Reference

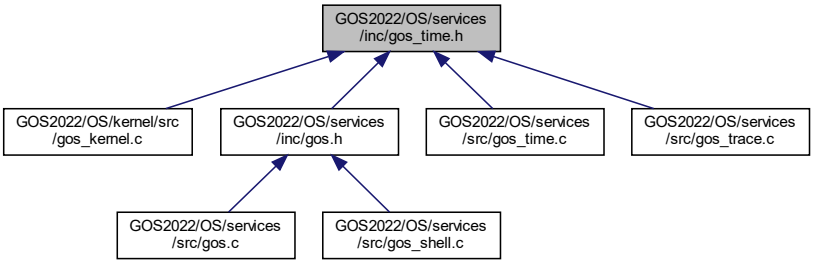
GOS time service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_time.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_time_t

Enumerations

- enum gos_timeComprareResult_t { GOS_TIME_EARLIER, GOS_TIME_LATER, GOS_TIME_EQUAL }
- enum gos_timeMonthEnum_t {
  GOS_TIME_JANUARY = 1, GOS_TIME_FEBRUARY, GOS_TIME_MARCH, GOS_TIME_APRIL,
  GOS_TIME_MAY, GOS_TIME_JUNE, GOS_TIME_JULY, GOS_TIME_AUGUST,
  GOS_TIME_SEPTEMBER, GOS_TIME_OCTOBER, GOS_TIME_NOVEMBER, GOS_TIME_DECEMBER,
  GOS_TIME_NUMBER_OF_MONTHS = 12 }
- enum gos_timeElapsedSenderId_t {
  GOS_TIME_SECOND_ELAPSED_SENDER_ID, GOS_TIME_MINUTE_ELAPSED_SENDER_ID, GOS_TIME_HOUR_ELAPS
  GOS_TIME_DAY_ELAPSED_SENDER_ID,
  GOS_TIME_MONTH_ELAPSED_SENDER_ID, GOS_TIME_YEAR_ELAPSED_SENDER_ID }

Functions

- gos_result_t gos_timeInit (void_t)

  This function initializes the time service.
- gos_result_t gos_timeGet (gos_time_t *pTime)

  This function gets the system time.
- gos_result_t gos_timeSet (gos_time_t *pTime)

  This function sets the system time.
- gos_result_t gos_timeCompare (gos_time_t *pTime1, gos_time_t *pTime2, gos_timeComprareResult_t *result)

  This function compares two time structures.
- gos_result_t gos_timeIncreaseSystemTime (u16_t milliseconds)

  This function increases the system time and runtime with the given value of milliseconds.
- gos_result_t gos_timeAddMilliseconds (gos_time_t *pTime, u16_t milliseconds)

  This function adds the given number of milliseconds to the given time structure.
- gos_result_t gos_timeAddSeconds (gos_time_t *pTime, u16_t seconds)

  This function adds the given number of seconds to the given time variable.
- gos_result_t gos_runTimeAddMicroseconds (gos_runtime_t *pRunTime1, gos_runtime_t *pRunTime2, u16_t microseconds)

  This function adds the given number of microseconds to the given time variables.
- gos_result_t gos_runTimeAddMilliseconds (gos_runtime_t *pRunTime, u16_t milliseconds)

  This function adds the given number of milliseconds to the given time variables.
- gos_result_t gos_runTimeAddSeconds (gos_runtime_t *pRunTime, u32_t seconds)

  This function adds the given number of seconds to the given run-time variable.
- gos_result_t gos_runTimeGet (gos_runtime_t *pRunTime)

  This function gets the system run-time.

7.25.1 Detailed Description

GOS time service header.

Author

Ahmed Gazar

Date

2023-11-06

Version

1.6

Time service provides an easy interface to manipulate time structures, track the passage of time.

7.25.2 Enumeration Type Documentation

7.25.2.1 gos_timeComprareResult_t

```
enum gos_timeComprareResult_t
```

Time comparison result enumerator.

Enumerator

GOS_TIME_EARLIER	First time is earlier than second.
GOS_TIME_LATER	First time is later than second.
GOS_TIME_EQUAL	Times are equal.

Definition at line 90 of file gos_time.h.

7.25.2.2 gos_timeElapsedSenderId_t

```
enum gos_timeElapsedSenderId_t
```

Time elapsed sender IDs.

Enumerator

GOS_TIME_SECOND_ELAPSED_SENDER_ID	Second elapsed sender ID.
GOS_TIME_MINUTE_ELAPSED_SENDER_ID	Minute elapsed sender ID.
GOS_TIME_HOUR_ELAPSED_SENDER_ID	Hour elapsed sender ID.
GOS_TIME_DAY_ELAPSED_SENDER_ID	Day elapsed sender ID.
GOS_TIME_MONTH_ELAPSED_SENDER_ID	Month elapsed sender ID.
GOS_TIME_YEAR_ELAPSED_SENDER_ID	Year elapsed sender ID.

Definition at line 121 of file gos_time.h.

7.25.2.3 gos_timeMonthEnum_t

```
enum gos_timeMonthEnum_t
```

Month enumerator.

Enumerator

GOS_TIME_JANUARY	January.

Enumerator

GOS_TIME_FEBRUARY	February.
GOS_TIME_MARCH	March.
GOS_TIME_APRIL	April.
GOS_TIME_MAY	May.
GOS_TIME_JUNE	June.
GOS_TIME_JULY	July.
GOS_TIME_AUGUST	August.
GOS_TIME_SEPTEMBER	September.
GOS_TIME_OCTOBER	October.
GOS_TIME_NOVEMBER	November.
GOS_TIME_DECEMBER	December.
GOS_TIME_NUMBER_OF_MONTHS	Number of months in a year.

Definition at line 100 of file gos_time.h.

7.25.3 Function Documentation

7.25.3.1 gos_runTimeAddMicroseconds()

```
gos_result_t gos_runTimeAddMicroseconds (
            gos_runtime_t * pRunTime1,
            gos_runtime_t * pRunTime2,
            u16_t microseconds )
```

This function adds the given number of microseconds to the given time variables.

This function adds the given number of microseconds to the given time variables.

Parameters

in,out	*pRunTime1*	Pointer to the time variable.
in,out	*pRunTime2*	Pointer to the time variable.
in	*microseconds*	Number of microseconds to add.

Returns

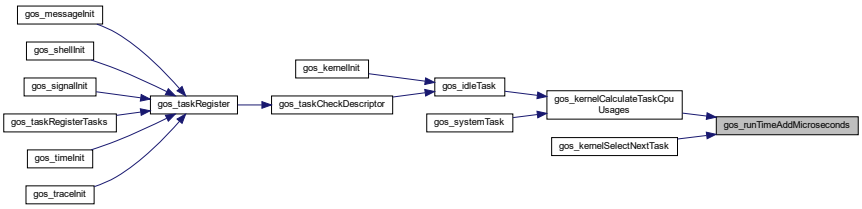
Result of time increasing.

Return values

GOS_SUCCESS	Microseconds added successfully.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 498 of file gos_time.c.

Here is the caller graph for this function:



7.25.3.2 gos_runTimeAddMilliseconds()

```
gos_result_t gos_runTimeAddMilliseconds (
            gos_runtime_t * pRunTime,
            u16_t milliseconds )
```

This function adds the given number of milliseconds to the given time variables.

This function adds the given number of milliseconds to the given time variables.

Parameters

in,out	*pRunTime*	Pointer to the time variable.
in	*milliseconds*	Number of milliseconds to add.

Returns

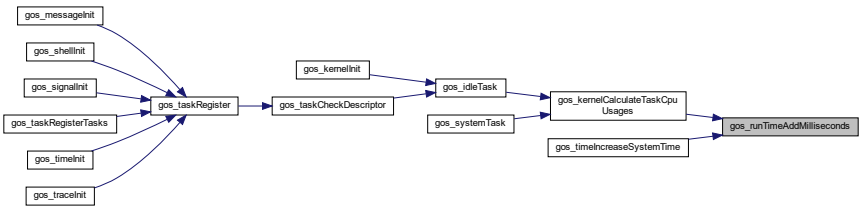
Result of time increasing.

Return values

GOS_SUCCESS	Milliseconds added successfully.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 638 of file gos_time.c.

Here is the caller graph for this function:



7.25.3.3 gos_runTimeAddSeconds()

```
gos_result_t gos_runTimeAddSeconds (
            gos_runtime_t * pRunTime,
            u32_t seconds )
```

This function adds the given number of seconds to the given run-time variable.

This function adds the given number of seconds to the given run-time variable.

Parameters

in,out	*pRunTime*	Pointer to a run-time variable.
in	*seconds*	Number of seconds to add.

Returns

Result of time increasing.

Return values

GOS_SUCCESS	Seconds added successfully.
GOS_ERROR	Run-time variable is NULL pointer.

Definition at line 707 of file gos_time.c.

7.25.3.4 gos_runTimeGet()

```
gos_result_t gos_runTimeGet (
            gos_runtime_t * pRunTime )
```

This function gets the system run-time.

This function gets the system run-time.

Parameters

out	*pRunTime*	Pointer to a run-time variable to store the system run-time in.

Returns

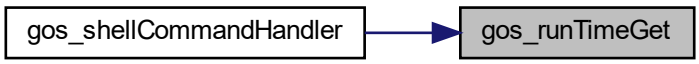
Result of run-time getting.

Return values

GOS_SUCCESS	Run-time getting is successful.
GOS_ERROR	Run-time variable is NULL pointer.

Definition at line 241 of file gos_time.c.

Here is the caller graph for this function:



7.25.3.5 gos_timeAddMilliseconds()

```
gos_result_t gos_timeAddMilliseconds (
            gos_time_t * pTime,
            u16_t milliseconds )
```

This function adds the given number of milliseconds to the given time structure.

This function adds the given number of milliseconds to the given time structure.

Parameters

in,out	*pTime*	Pointer to the time structure.
in	*milliseconds*	Number of milliseconds to add.

Returns

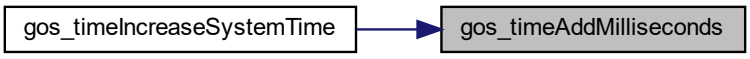
Result of time increasing.

Return values

GOS_SUCCESS	Increasing successful.
GOS_ERROR	Time structure is NULL pointer.

Definition at line 312 of file gos_time.c.

Here is the caller graph for this function:



7.25.3.6 gos_timeAddSeconds()

```
gos_result_t gos_timeAddSeconds (
             gos_time_t * pTime,
             u16_t seconds )
```

This function adds the given number of seconds to the given time variable.

This function adds the given number of seconds to the given time variable.

Parameters

in,out	*pTime*	Pointer to the time variable.
in	*seconds*	Number of seconds to add.

Returns

Result of time increasing.

Return values

GOS_SUCCESS	Seconds added successfully.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 408 of file gos_time.c.

7.25.3.7 gos_timeCompare()

```
gos_result_t gos_timeCompare (
            gos_time_t * pTime1,
            gos_time_t * pTime2,
            gos_timeComprareResult_t * result )
```

This function compares two time structures.

This function compares two time structures.

Parameters

in	*pTime1*	Pointer to the first time variable.
in	*pTime2*	Pointer to the second time variable.
out	*result*	Pointer to the comparison result variable.

Returns

Result of time comparison.

Return values

GOS_SUCCESS	Time comparison successful.
GOS_ERROR	Either time structure and/or result variable is NULL pointer.

Definition at line 268 of file gos_time.c.

7.25.3.8 gos_timeGet()

```
gos_result_t gos_timeGet (
            gos_time_t * pTime )
```

This function gets the system time.

This function copies the system time value to the given time variable.

Parameters

out	*pTime*	Pointer to a time variable to store the system time value in.

Returns

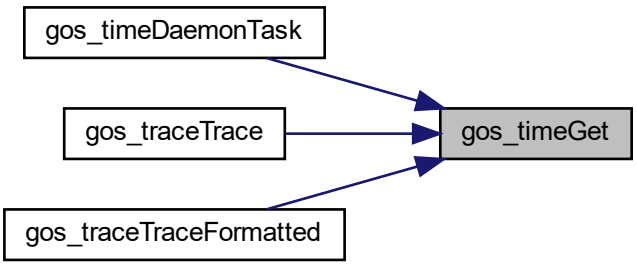
Result of time getting.

Return values

GOS_SUCCESS	Time getting successful.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 187 of file gos_time.c.

Here is the caller graph for this function:



7.25.3.9 gos_timeIncreaseSystemTime()

```
gos_result_t gos_timeIncreaseSystemTime (
            u16_t milliseconds )
```

This function increases the system time and runtime with the given value of milliseconds.

This function increases the system time and runtime with the given value of milliseconds.

Parameters

in	*milliseconds*	Number of milliseconds to add to system time.

Returns

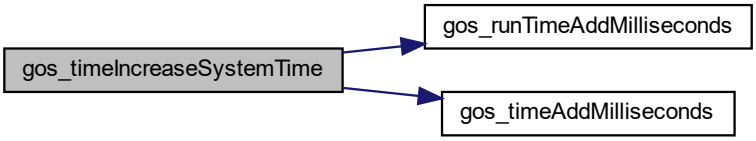
Result of system time increasing.

Return values

GOS_SUCCESS	Increasing successful.
GOS_ERROR	System time or runtime increasing failed.

Definition at line 768 of file gos_time.c.

Here is the call graph for this function:



7.25.3.10 gos_timeInit()

```
gos_result_t gos_timeInit (
            void_t  )
```

This function initializes the time service.

Creates the time signal and registers the time daemon in the kernel.

Returns

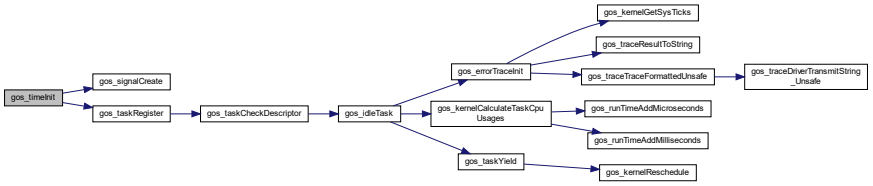
Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Time signal registration error or time daemon registration error.

Definition at line 160 of file gos_time.c.

Here is the call graph for this function:



7.25.3.11 gos_timeSet()

```
gos_result_t gos_timeSet (
            gos_time_t * pTime )
```

This function sets the system time.

This function copies the time value from the given time variable to the system time variable.

Parameters

in	*pTime*	Pointer to a time variable holding the desired time value.

Returns

Result of time setting.

Return values

GOS_SUCCESS	Time setting successful.
GOS_ERROR	Time structure is NULL pointer.

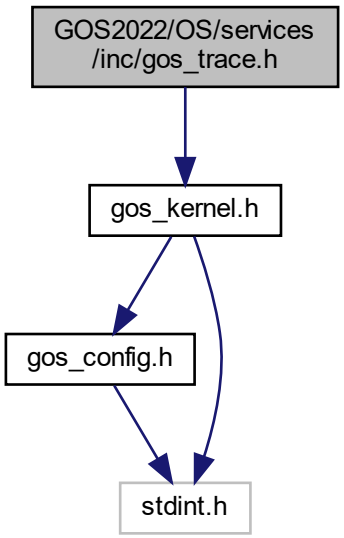
Definition at line 214 of file gos_time.c.

7.26 GOS2022/OS/services/inc/gos_trace.h File Reference

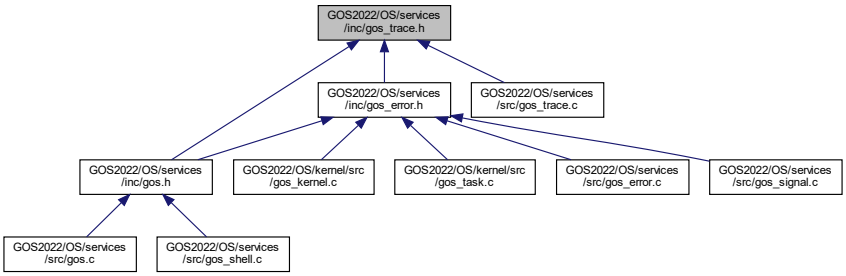
GOS trace service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_trace.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define TRACE_FORMAT_RESET "\x1B[0m"

  Reset formatting.
- #define TRACE_FG_RED_START "\x1B[31m"

  Red foreground start.
- #define TRACE_FG_GREEN_START "\x1B[32m"

  Green foreground start.
- #define TRACE_FG_YELLOW_START "\x1B[33m"

  Yellow foreground start.
- #define TRACE_FG_BLUE_START "\x1B[34m"

Blue foreground start.

- #define TRACE_FG_MAGENTA_START "\x1B[35m"

  Magenta foreground start.
- #define TRACE_FG_CYAN_START "\x1B[36m"

  Cyan foreground start.
- #define TRACE_FG_WHITE_START "\x1B[37m"

  White foreground start.
- #define TRACE_BG_RED_START "\x1b[41m"

  Red background start.
- #define TRACE_BG_GREEN_START "\x1b[42m"

  Green background start.
- #define TRACE_BG_YELLOW_START "\x1b[43m"

  Yellow background start.
- #define TRACE_BG_BLUE_START "\x1b[44m"

  Blue background start.
- #define TRACE_BG_MAGENTA_START "\x1b[45m"

  Magenta background start.
- #define TRACE_BG_CYAN_START "\x1b[46m"

  Cyan background start.
- #define TRACE_BG_WHITE_START "\x1b[47m"

  White background start.
- #define TRACE_BOLD_START "\x1B[1m"

  Bold start.
- #define TRACE_ITALIC_START "\x1B[3m"

  Italic start.
- #define TRACE_UNDERLINE_START "\x1B[4m"

  Underline start.
- #define TRACE_STRIKETHROUGH_START "\x1B[9m"

  Strikethrough start.

Functions

- gos_result_t gos_traceInit (void_t)

  Initializes the trace service.
- gos_result_t gos_traceTrace (bool_t addTimeStamp, char_t *traceMessage)

  Traces a given message.
- gos_result_t gos_traceTraceFormatted (bool_t addTimeStamp, GOS_CONST char_t *traceFormat,...)

  Traces a given formatted message.
- gos_result_t gos_traceTraceFormattedUnsafe (GOS_CONST char_t *traceFormat,...)

  Traces a given formatted message.

7.26.1 Detailed Description

GOS trace service header.

Author

Ahmed Gazar

Date

2023-01-13

Version

2.1

Trace service is a simple interface to send out strings via the configured trace periphery.

7.26.2 Function Documentation

7.26.2.1 gos_traceInit()

```
gos_result_t gos_traceInit (
            void_t  )
```

Initializes the trace service.

Creates a trace queue and registers the trace daemon in the kernel.

Returns

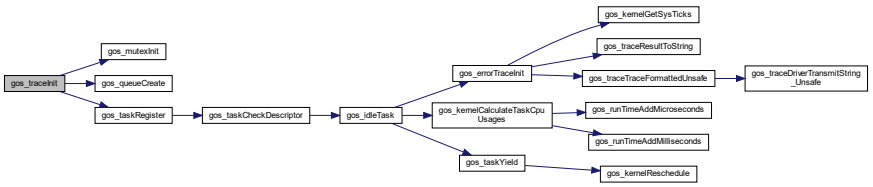
Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Queue creation or task registration error.

Definition at line 152 of file gos_trace.c.

Here is the call graph for this function:



7.26.2.2 gos_traceTrace()

```
gos_result_t gos_traceTrace (
            bool_t addTimeStamp,
            char_t * traceMessage )
```

Traces a given message.

Places the given message to the trace queue (for the trace daemon to print it).

Parameters

in	*addTimeStamp*	Flag to indicate whether to add time-stamp or not.
in	*traceMessage*	String to trace.

Returns

Result of tracing.

Return values

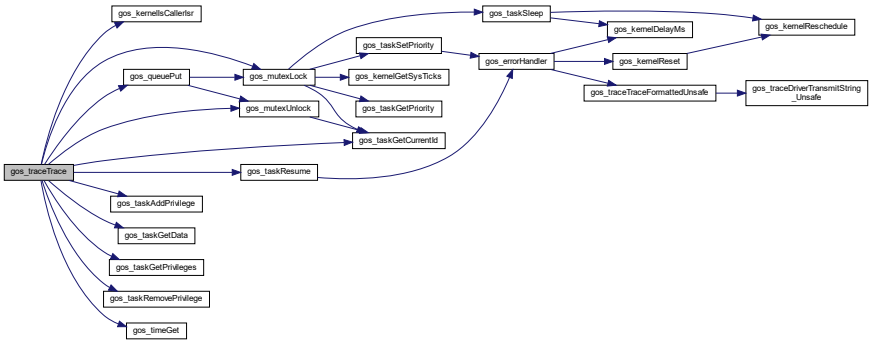
GOS_SUCCESS	Tracing successful.
GOS_ERROR	Queue put error.

Remarks

This function uses the queue service.

Definition at line 177 of file gos_trace.c.

Here is the call graph for this function:



7.26.2.3 gos_traceTraceFormatted()

```
gos_result_t gos_traceTraceFormatted (
            bool_t addTimeStamp,
            GOS_CONST char_t * traceFormat,
             ... )
```

Traces a given formatted message.

Prints the formatted message into a local buffer and places it to the trace queue (for the trace daemon to print it).

Parameters

in	*addTimeStamp*	Flag to indicate whether to add time-stamp or not.
in	*traceFormat*	Formatter string.
in	...	Optional parameters.

Returns

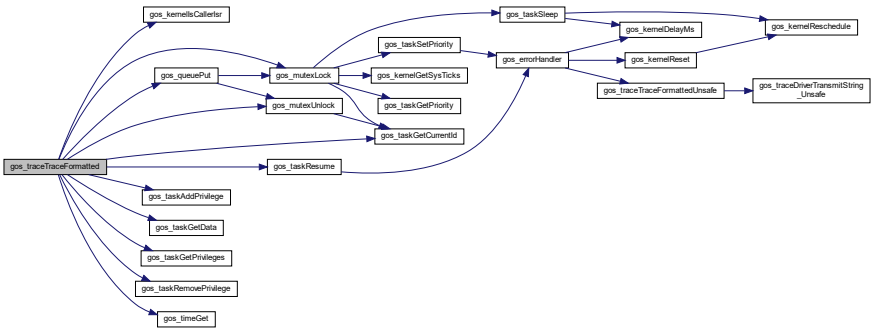
Result of formatted tracing.

Return values

GOS_SUCCESS	Formatted tracing successful.
GOS_ERROR	Queue put error.

Definition at line 266 of file gos_trace.c.

Here is the call graph for this function:



7.26.2.4 gos_traceTraceFormattedUnsafe()

```
gos_result_t gos_traceTraceFormattedUnsafe (
            GOS_CONST char_t * traceFormat,
             ... )
```

Traces a given formatted message.

Prints the formatted message into a local buffer and transmits it via the trace port.

Parameters

in	*traceFormat*	Formatter string.
in	...	Optional parameters.

Returns

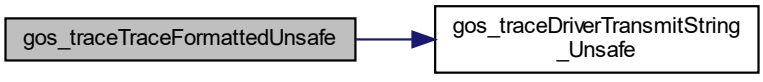
Result of formatted tracing.

Return values

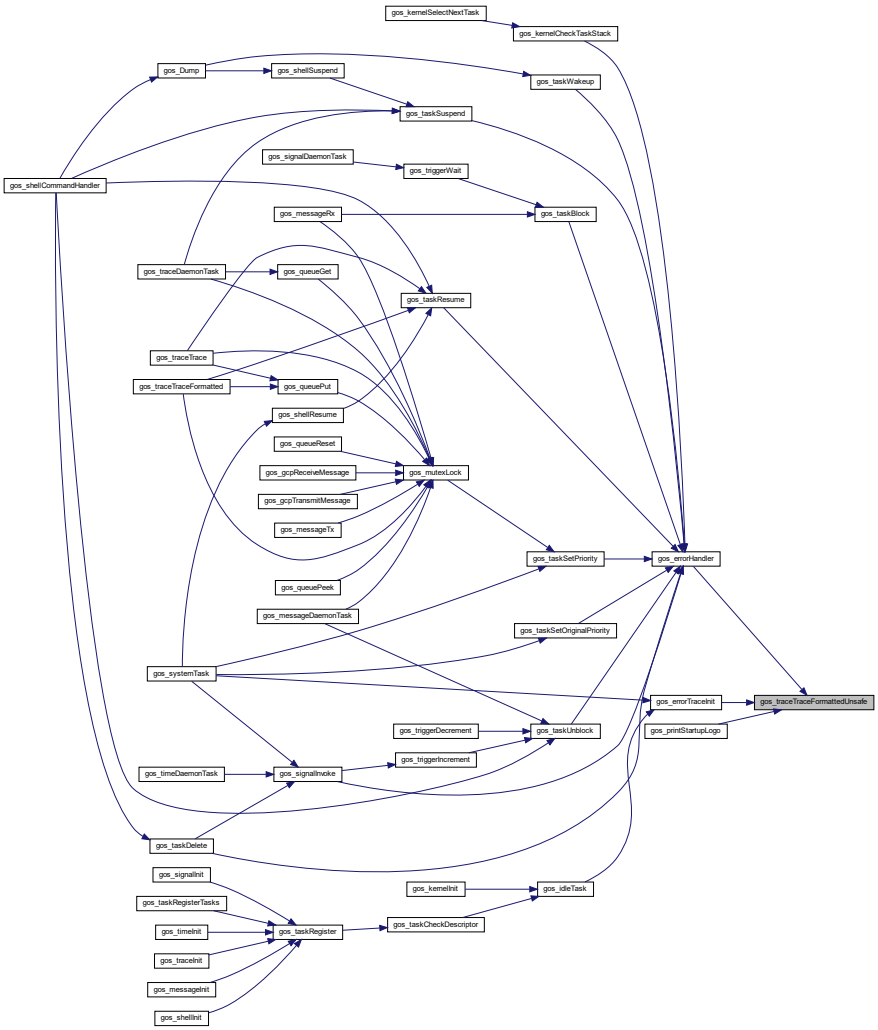
GOS_SUCCESS	Message traced successfully.
GOS_ERROR	Transmit error.

Definition at line 356 of file gos_trace.c.

Here is the call graph for this function:



Here is the caller graph for this function:

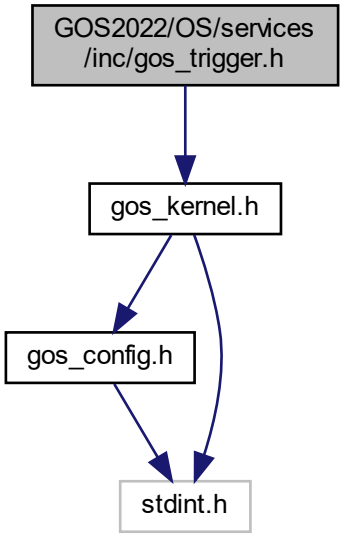


7.27 GOS2022/OS/services/inc/gos_trigger.h File Reference

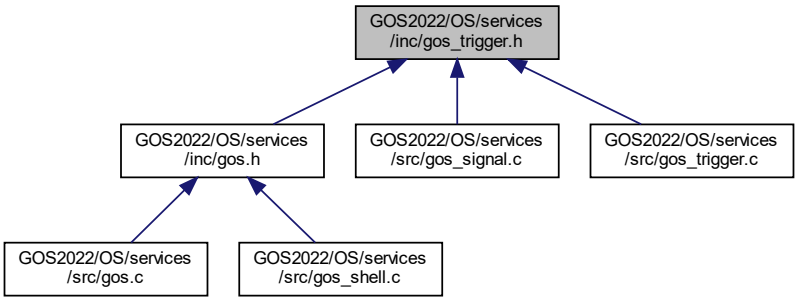
GOS trigger service header.

```
#include <gos_kernel.h>
```

Include dependency graph for gos_trigger.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct gos_trigger_t

Macros

- #define GOS_TRIGGER_ENDLESS_TMO (0xFFFFFFFF)
- #define GOS_TRIGGER_NO_TMO (0x00000000)

Functions

- gos_result_t gos_triggerInit (gos_trigger_t *pTrigger)
  Initializes the trigger instance.
- gos_result_t gos_triggerReset (gos_trigger_t *pTrigger)
  Resets the trigger counter of the given trigger instance.
- gos_result_t gos_triggerWait (gos_trigger_t *pTrigger, u32_t value, u32_t timeout)
  Waits for the trigger instance to reach the given trigger value.
- gos_result_t gos_triggerIncrement (gos_trigger_t *pTrigger)
  Increments the trigger value of the given trigger.
- gos_result_t gos_triggerDecrement (gos_trigger_t *pTrigger)
  Decrements the trigger value of the given trigger.

7.27.1 Detailed Description

GOS trigger service header.

Author

Ahmed Gazar

Date

2023-11-15

Version

2.4

Trigger service is a way of synchronizing tasks. A trigger instance works as a counter. A trigger can be incremented, and it can be waited on. When waiting for a trigger, the caller must specify a desired trigger value to be reached. Until the value is reached, the caller will wait in a non-blocking way. Once the value is reached, the caller returns.

7.27.2 Macro Definition Documentation

7.27.2.1 GOS_TRIGGER_ENDLESS_TMO

```
#define GOS_TRIGGER_ENDLESS_TMO ( 0xFFFFFFFF )
```

Mutex endless timeout.

Definition at line 72 of file gos_trigger.h.

7.27.2.2 GOS_TRIGGER_NO_TMO

```
#define GOS_TRIGGER_NO_TMO ( 0x00000000 )
```

Mutex no timeout.

Definition at line 77 of file gos_trigger.h.

7.27.3 Function Documentation

7.27.3.1 gos_triggerDecrement()

```
gos_result_t gos_triggerDecrement (
            gos_trigger_t * pTrigger )
```

Decrements the trigger value of the given trigger.

Decrements the trigger value of the given trigger.

Parameters

in,out	*pTrigger*	Pointer to the trigger instance.

Returns

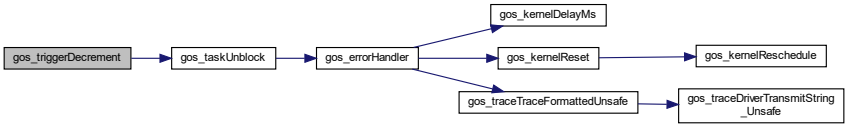
Result of trigger decrementing.

Return values

GOS_SUCCESS	Decrementing successful.
GOS_ERROR	Trigger is NULL pointer or value is already zero.

Definition at line 246 of file gos_trigger.c.

Here is the call graph for this function:



7.27.3.2 gos_triggerIncrement()

```
gos_result_t gos_triggerIncrement (
            gos_trigger_t * pTrigger )
```

Increments the trigger value of the given trigger.

Increments the trigger value of the given trigger.

Parameters

in,out	*pTrigger*	Pointer to the trigger instance.

Returns

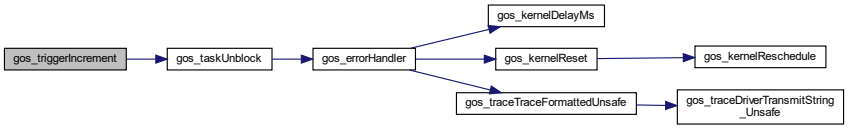
Result of trigger incrementing.

Return values

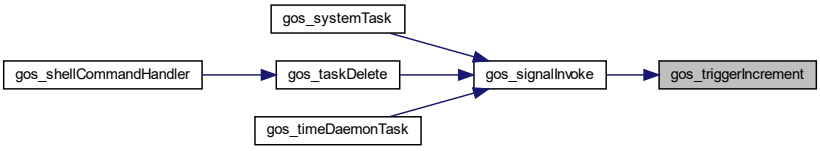
GOS_SUCCESS	Incrementing successful.
GOS_ERROR	Trigger is NULL pointer.

Definition at line 199 of file gos_trigger.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.27.3.3 gos_triggerInit()

```
gos_result_t gos_triggerInit (
            gos_trigger_t * pTrigger )
```

Initializes the trigger instance.

Calls the initializer for the trigger mutex.

Parameters

out	*pTrigger*	Pointer to the trigger to be initialized.

Returns

Result of trigger initializing.

Return values

GOS_SUCCESS	Trigger initialized successfully.
GOS_ERROR	Trigger descriptor or trigger mutex is NULL pointer.

Definition at line 78 of file gos_trigger.c.

Here is the caller graph for this function:



7.27.3.4 gos_triggerReset()

```
gos_result_t gos_triggerReset (
            gos_trigger_t * pTrigger )
```

Resets the trigger counter of the given trigger instance.

Sets the trigger counter of the given trigger instance to zero.

Parameters

out	*pTrigger*	Pointer to the trigger instance.

Returns

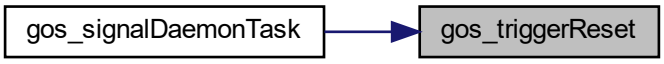
Result of trigger resetting.

Return values

GOS_SUCCESS	Trigger resetting successful.
GOS_ERROR	Trigger is NULL pointer.

Definition at line 107 of file gos_trigger.c.

Here is the caller graph for this function:



7.27.3.5 gos_triggerWait()

```
gos_result_t gos_triggerWait (
            gos_trigger_t * pTrigger,
            u32_t value,
            u32_t timeout )
```

Waits for the trigger instance to reach the given trigger value.

Increases the trigger waiter number, and waits in a non-blocking way until the desired trigger value is reached or the timeout is reached.

Parameters

in,out	*pTrigger*	Pointer to the trigger instance.
in	*value*	The desired trigger value.
in	*timeout*	Timeout value.

Returns

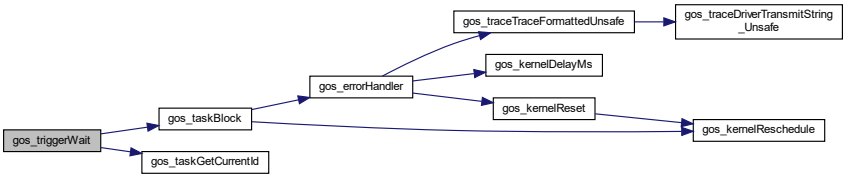
Result of trigger waiting.

Return values

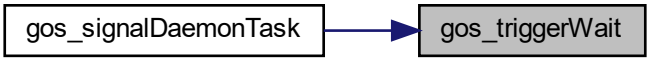
GOS_SUCCESS	Trigger value reached.
GOS_ERROR	Trigger value was not reached within the timeout value.

Definition at line 140 of file gos_trigger.c.

Here is the call graph for this function:



Here is the caller graph for this function:

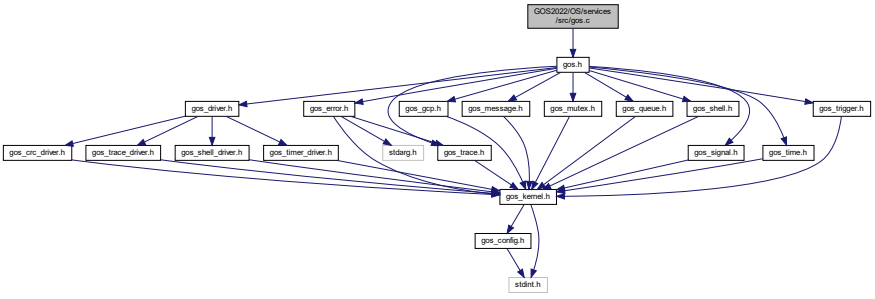


7.28 GOS2022/OS/services/src/gos.c File Reference

GOS source.

```
#include <gos.h>
```

Include dependency graph for gos.c:



Data Structures

- struct gos_initStruct_t

Macros

- #define GOS_SYS_TASK_SLEEP_TIME (100u)

Typedefs

- typedef gos_result_t(∗ gos_initFunc_t) (void_t)

Functions

- GOS_STATIC void_t gos_systemTask (void_t)

  Initializes the system.
- GOS_STATIC gos_result_t gos_Start (void_t)

  Starts the OS.
- int **main** (void_t)
- void_t gos_Dump (void_t)

  GOS dump.

Variables

- GOS_EXTERN gos_signalId_t **kernelDumpReadySignal**
- GOS_STATIC bool_t initError
- GOS_STATIC bool_t dumpRequired
- GOS_STATIC gos_initStruct_t initializers [ ]
- GOS_STATIC gos_tid_t systemTaskId
- GOS_STATIC gos_taskDescriptor_t systemTaskDesc

7.28.1 Detailed Description

GOS source.

Author

Ahmed Gazar

Date

2025-06-18

Version

1.11

For a more detailed description of this service, please refer to gos.h

7.28.2 Macro Definition Documentation

7.28.2.1 GOS_SYS_TASK_SLEEP_TIME

```
#define GOS_SYS_TASK_SLEEP_TIME ( 100u )
```

System task sleep time.

Definition at line 79 of file gos.c.

7.28.3 Typedef Documentation

7.28.3.1 gos_initFunc_t

```
typedef gos_result_t(* gos_initFunc_t) (void_t)
```

Initializer function type.

Definition at line 87 of file gos.c.

7.28.4 Function Documentation

7.28.4.1 gos_Dump()

```
void_t gos_Dump (
            void_t  )
```

GOS dump.

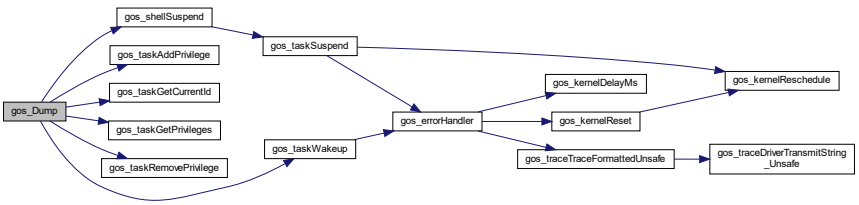
Sets the dump required flag and wakes up the system task that calls the individual dump functions.

Returns

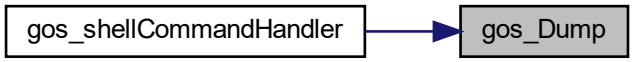
-

Definition at line 209 of file gos.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.28.4.2 gos_Start()

```
GOS_STATIC gos_result_t gos_Start (
            void_t  )
```

Starts the OS.

Checks whether the initializer function has set the error flag to GOS_FALSE, and if so, it starts the kernel (and thus the scheduling of tasks).

Returns

Result of OS starting.

Return values

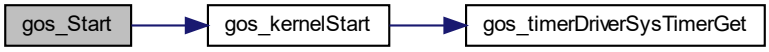
GOS_ERROR	OS not started due to initialization error or kernel start error.

Remarks

This function should only return with error. If the initialization is successful, the function is not expected to return.

Definition at line 251 of file gos.c.

Here is the call graph for this function:



7.28.4.3 gos_systemTask()

```
GOS_STATIC void_t gos_systemTask (
            void_t  )
```

Initializes the system.

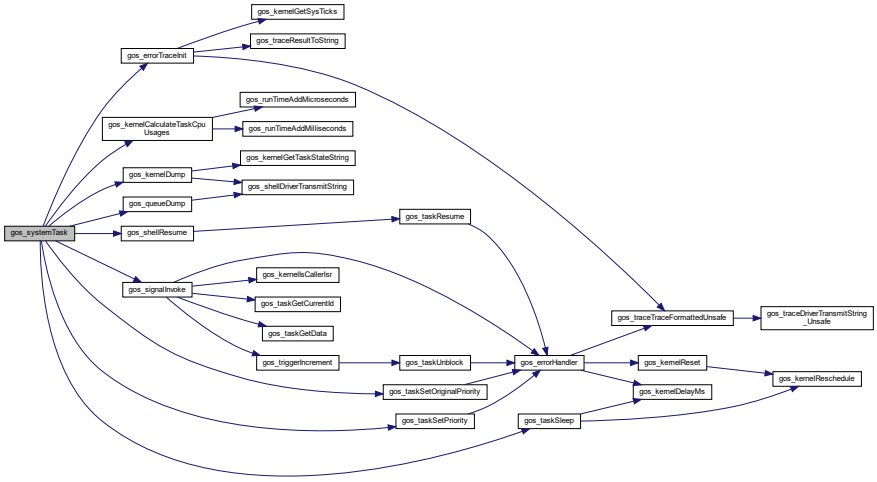
Calls the OS service initializer functions and the user application initializer, and deletes itself.

Returns

-

Definition at line 280 of file gos.c.

Here is the call graph for this function:



7.28.5 Variable Documentation

7.28.5.1 dumpRequired

```
GOS_STATIC bool_t dumpRequired
```

Dump required flag.

Definition at line 114 of file gos.c.

7.28.5.2 initError

```
GOS_STATIC bool_t initError
```

Initialization error flag.

Definition at line 109 of file gos.c.

7.28.5.3 initializers

```
GOS_STATIC gos_initStruct_t initializers[]
```

Initial value:

```
=
{
    {"Queue service initialization"   , gos_queueInit},
    {"Trace service initialization"   , gos_traceInit},
    {"Signal service initialization"  , gos_signalInit},
    {"Time service initialization"    , gos_timeInit},
    {"Message service initialization" , gos_messageInit},
    {"User application initialization", gos_userApplicationInit}
}
```

Initializer function and description lookup table.

Definition at line 119 of file gos.c.

7.28.5.4 systemTaskDesc

```
GOS_STATIC gos_taskDescriptor_t systemTaskDesc
```

Initial value:

```
=
{
    .taskFunction       = gos_systemTask,
    .taskName           = "gos_system_task",
    .taskPriority       = 0u,
    .taskStackSize      = CFG_SYSTEM_TASK_STACK_SIZE,
    .taskPrivilegeLevel = GOS_TASK_PRIVILEGE_KERNEL
}
```

Initializer task descriptor.

Definition at line 149 of file gos.c.

7.28.5.5 systemTaskId

```
GOS_STATIC gos_tid_t systemTaskId
```

Initializer task ID.

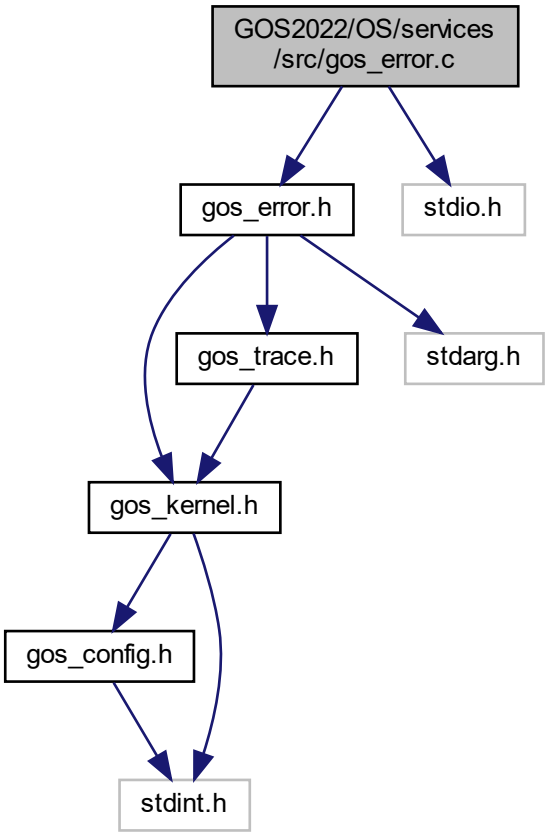
Definition at line 138 of file gos.c.

7.29 GOS2022/OS/services/src/gos_error.c File Reference

GOS error handler service source.

```
#include <gos_error.h>
#include <stdio.h>
```

Include dependency graph for gos_error.c:



Macros

- #define SEPARATOR_LINE "+----------+\r\n"
- #define ERROR_BUFFER_SIZE (80u)
- #define RESULT_STRING_SUCCESS " OK "
- #define RESULT_STRING_ERROR " ERROR "
- #define RESULT_STRING_UNKNOWN "UNKNOWN"

Functions

- GOS_STATIC char_t * gos_traceResultToString (gos_result_t *pResult)

  Converts the result enumerator to a string.

- void_t gos_printStartupLogo (void_t)

  Prints the startup logo on the trace output.
- void_t gos_errorHandler (gos_errorLevel_t errorLevel, const char_t *function, u32_t line, const char_t *errorMessage,...)

  Handles the given error.
- gos_result_t gos_errorTraceInit (const char_t *initDescription, gos_result_t initResult)

  Traces an initialization message.

Variables

- GOS_STATIC char_t errorBuffer [(80u)]

7.29.1 Detailed Description

GOS error handler service source.

Author

Ahmed Gazar

Date

2025-04-06

Version

2.5

For a more detailed description of this service, please refer to gos_error.h

7.29.2 Macro Definition Documentation

7.29.2.1 ERROR_BUFFER_SIZE

```
#define ERROR_BUFFER_SIZE ( 80u )
```

Error buffer size.

Definition at line 75 of file gos_error.c.

7.29.2.2 RESULT_STRING_ERROR

```
#define RESULT_STRING_ERROR " ERROR "
```

Error result string.

Definition at line 85 of file gos_error.c.

7.29.2.3 RESULT_STRING_SUCCESS

```
#define RESULT_STRING_SUCCESS " OK "
```

Success result string.

Definition at line 80 of file gos_error.c.

7.29.2.4 RESULT_STRING_UNKNOWN

```
#define RESULT_STRING_UNKNOWN "UNKNOWN"
```

Unknown result string.

Definition at line 90 of file gos_error.c.

7.29.2.5 SEPARATOR_LINE

```
#define SEPARATOR_LINE "+--------------------------------------------------------------------------------+\r\n"
```

Separator line.

Definition at line 70 of file gos_error.c.

7.29.3 Function Documentation

7.29.3.1 gos_errorHandler()

```
void_t gos_errorHandler (
            gos_errorLevel_t errorLevel,
            const char_t * function,
            u32_t line,
            const char_t * errorMessage,
             ...  )
```

Handles the given error.

Prints the formatted error message on the trace output, and based on the error level, it returns or stays in an infinite loop and disables scheduling.

Parameters

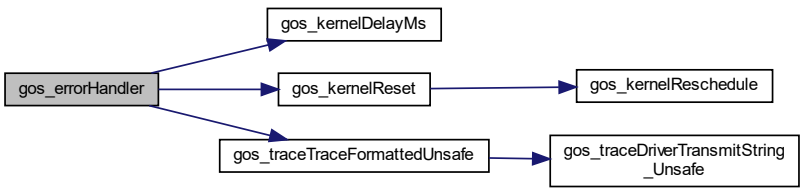
in	*errorLevel*	Level of error (OS/user, warning/fatal).
in	*function*	Function name.
in	*line*	Line number.
in	*errorMessage*	Error message.
in	*...*	Formatter variable arguments.

Returns

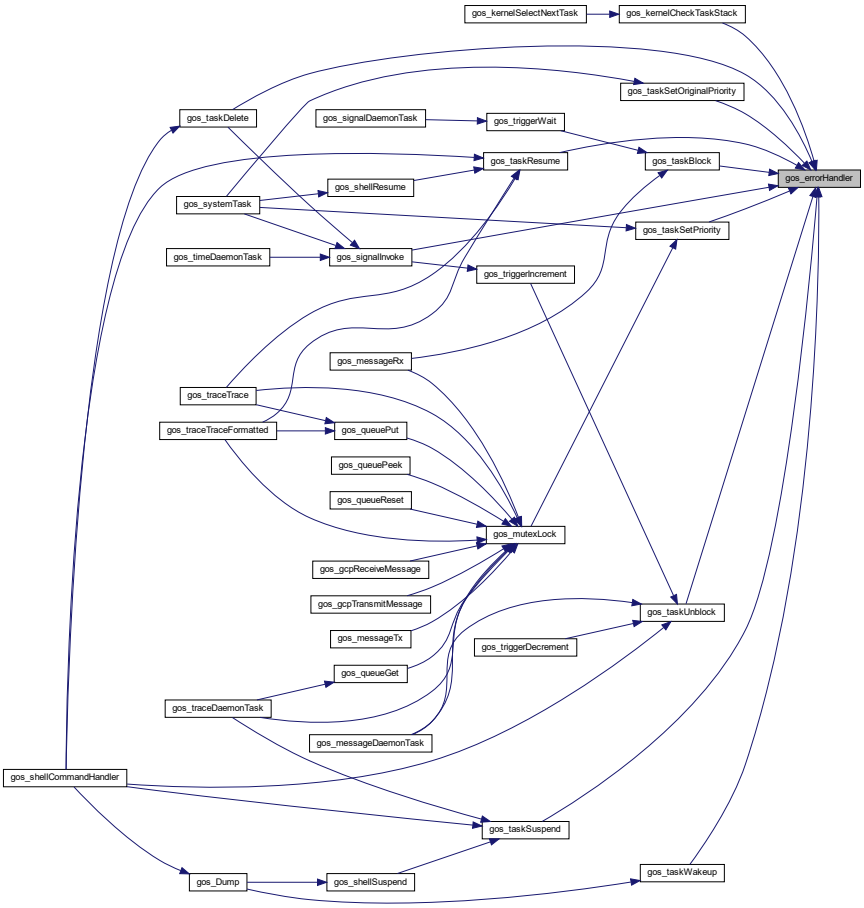
-

Definition at line 129 of file gos_error.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.29.3.2 gos_errorTraceInit()

```
gos_result_t gos_errorTraceInit (
            const char_t * initDescription,
            gos_result_t initResult )
```

Traces an initialization message.

Writes the formatted initialization message on the trace output.

Parameters

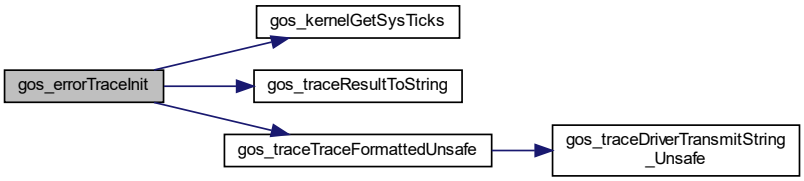
in	*initDescription*	Message to describe the initialization step.
in	*initResult*	Result of initialization.

Returns

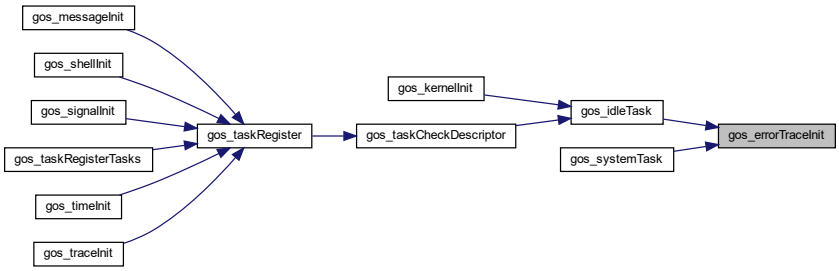
Result of initialization.

Definition at line 237 of file gos_error.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.29.3.3 gos_printStartupLogo()

```
void_t gos_printStartupLogo (
            void_t  )
```

Prints the startup logo on the trace output.

Prints the GOS logo (similar to the ones used in the file headers) on the log output.

Returns

-

Definition at line 108 of file gos_error.c.

Here is the call graph for this function:



7.29.3.4 gos_traceResultToString()

```
GOS_STATIC char_t * gos_traceResultToString (
            gos_result_t * pResult )
```

Converts the result enumerator to a string.

Returns the formatted string version of the result enumerator.

Parameters

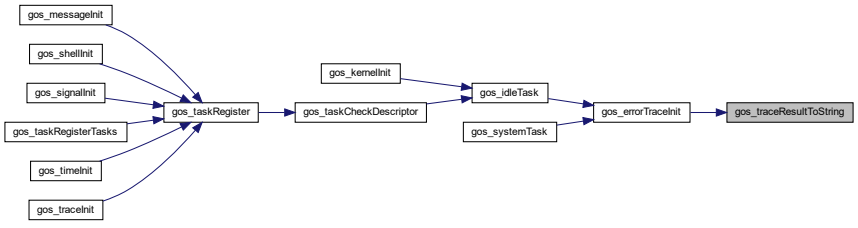
out	*pResult*	Pointer to the result variable.

Returns

Formatted string.

Definition at line 260 of file gos_error.c.

Here is the caller graph for this function:



7.29.4 Variable Documentation

7.29.4.1 errorBuffer

```
GOS_STATIC char_t errorBuffer[(80u)]
```

Buffer for error message formatting.

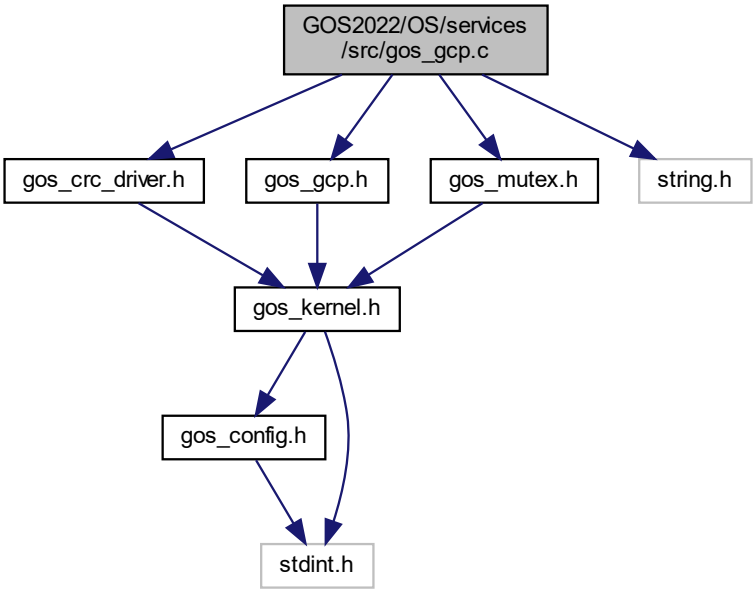
Definition at line 98 of file gos_error.c.

7.30 GOS2022/OS/services/src/gos_gcp.c File Reference

GOS General Communication Protocol handler service source.

```
#include <gos_crc_driver.h>
#include <gos_gcp.h>
#include <gos_mutex.h>
#include <string.h>
```

Include dependency graph for gos_gcp.c:



Data Structures

- struct gos_gcpHeaderFrame_t
- struct gos_gcpChannelFunctions_t

Macros

- #define GCP_PROTOCOL_VERSION_MAJOR (2)
- #define GCP_PROTOCOL_VERSION_MINOR (0)

Enumerations

- enum gos_gcpAck_t {
  GCP_ACK_REQ = 0, GCP_ACK_OK = 1, GCP_ACK_CRC_ERROR = 2, GCP_ACK_RESEND = 3,
  GCP_ACK_SIZE_ERROR = 4, GCP_ACK_PV_ERROR = 5, GCP_ACK_INVALID = 6 }

Functions

- GOS_STATIC GOS_INLINE gos_result_t gos_gcpTransmitMessageInternal (gos_gcpChannelNumber_t channel, u16_t messageId, void_t *pMessagePayload, u16_t payloadSize, u16_t maxChunkSize)

  Internal transmitter function (re-entrant).
- GOS_STATIC GOS_INLINE gos_result_t gos_gcpReceiveMessageInternal (gos_gcpChannelNumber_t channel, u16_t *pMessageId, void_t *pPayloadTarget, u16_t targetSize, u16_t maxChunkSize)

  Internal receiver function (re-entrant).
- GOS_STATIC gos_result_t gos_gcpValidateHeader (gos_gcpHeaderFrame_t *pHeader, gos_gcpAck_t *p↩ Ack)

  Validates the given GCP header.
- GOS_STATIC gos_result_t gos_gcpValidateData (gos_gcpHeaderFrame_t *pHeader, void_t *pData, gos_gcpAck_t *pAck)

  Validates the given GCP payload/data.
- gos_result_t gos_gcpInit (void_t)

  Initializes the GCP service.
- gos_result_t gos_gcpRegisterPhysicalDriver (gos_gcpChannelNumber_t channelNumber, gos_gcpTransmitFunction_t transmitFunction, gos_gcpReceiveFunction_t receiveFunction)

  Registers the physical-layer transmit and receive driver functions.
- gos_result_t gos_gcpTransmitMessage (gos_gcpChannelNumber_t channel, u16_t messageId, void_t *p↩ MessagePayload, u16_t payloadSize, u16_t maxChunkSize)

  Transmits the given message via the GCP protocol.
- gos_result_t gos_gcpReceiveMessage (gos_gcpChannelNumber_t channel, u16_t *pMessageId, void_t *pPayloadTarget, u16_t targetSize, u16_t maxChunkSize)

  Receives the given message via the GCP protocol.

Variables

- GOS_STATIC gos_gcpChannelFunctions_t channelFunctions [CFG_GCP_CHANNELS_MAX_NUMBER]
- GOS_STATIC gos_mutex_t gcpRxMutexes [CFG_GCP_CHANNELS_MAX_NUMBER]
- GOS_STATIC gos_mutex_t gcpTxMutexes [CFG_GCP_CHANNELS_MAX_NUMBER]

7.30.1 Detailed Description

GOS General Communication Protocol handler service source.

Author

Ahmed Gazar

Date

2025-04-06

Version

3.1

For a more detailed description of this service, please refer to gos_gcp.h

7.30.2 Macro Definition Documentation

7.30.2.1 GCP_PROTOCOL_VERSION_MAJOR

```
#define GCP_PROTOCOL_VERSION_MAJOR ( 2 )
```

GCP protocol version high byte.

Definition at line 73 of file gos_gcp.c.

7.30.2.2 GCP_PROTOCOL_VERSION_MINOR

```
#define GCP_PROTOCOL_VERSION_MINOR ( 0 )
```

GCP protocol version low byte.

Definition at line 78 of file gos_gcp.c.

7.30.3 Enumeration Type Documentation

7.30.3.1 gos_gcpAck_t

```
enum gos_gcpAck_t
```

GCP acknowledge type.

Enumerator

GCP_ACK_REQ	Request.
GCP_ACK_OK	OK.
GCP_ACK_CRC_ERROR	CRC error.
GCP_ACK_RESEND	Re-send request.
GCP_ACK_SIZE_ERROR	Size error.
GCP_ACK_PV_ERROR	Protocol version error.
GCP_ACK_INVALID	Invalid message.

Definition at line 86 of file gos_gcp.c.

7.30.4 Function Documentation

7.30.4.1 gos_gcpInit()

```
gos_result_t gos_gcpInit (
             void_t  )
```

Initializes the GCP service.

Creates the GCP lock.

Returns

Result of initialization.

Return values

GOS_SUCCESS	GCP service initialized successfully.
GOS_ERROR	Lock creation error.

Definition at line 172 of file gos_gcp.c.

Here is the call graph for this function:



7.30.4.2 gos_gcpReceiveMessage()

```
gos_result_t gos_gcpReceiveMessage (
             gos_gcpChannelNumber_t channel,
             u16_t * pMessageId,
             void_t * pPayloadTarget,
             u16_t targetSize,
             u16_t maxChunkSize )
```

Receives the given message via the GCP protocol.

Calls the internal receiver function.

Parameters

in	*channel*	GCP channel.
out	*pMessageId*	Pointer to a variable to store the message ID.
out	*pPayloadTarget*	Pointer to the payload target buffer.
in	*targetSize*	Size of the target buffer (in bytes).
in	*maxChunkSize*	Maximum size of payload chunks.

Returns

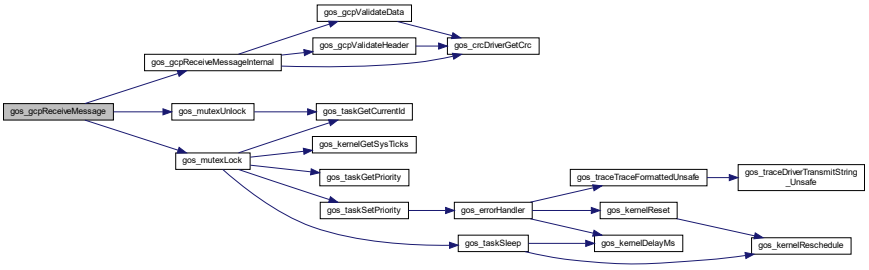
Result of message reception.

Return values

GOS_SUCCESS	Message received successfully.
GOS_ERROR	An error occurred during reception or validation.

Definition at line 259 of file gos_gcp.c.

Here is the call graph for this function:



7.30.4.3 gos_gcpReceiveMessageInternal()

```
GOS_STATIC GOS_INLINE gos_result_t gos_gcpReceiveMessageInternal (
            gos_gcpChannelNumber_t channel,
            u16_t * pMessageId,
            void_t * pPayloadTarget,
            u16_t targetSize,
            u16_t maxChunkSize )
```

Internal receiver function (re-entrant).

Receives a message over GCP (header, payload, and then transmits a response header).

Parameters

in	*channel*	GCP channel number.
out	*pMessageId*	Pointer to a variable to store the message ID.
out	*pPayloadTarget*	Pointer to a buffer to store the payload.
in	*targetSize*	Size of the payload buffer (in bytes).
in	*maxChunkSize*	Maximum chunk size (in bytes).

Returns

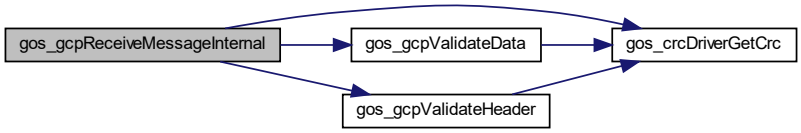
Result of message reception.

Return values

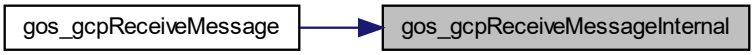
GOS_SUCCESS	Reception successful.
GOS_ERROR	One of the function parameters are invalid or request header validation failed or payload validation failed.

Definition at line 433 of file gos_gcp.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.30.4.4 gos_gcpRegisterPhysicalDriver()

```
gos_result_t gos_gcpRegisterPhysicalDriver (
             gos_gcpChannelNumber_t channel,
             gos_gcpTransmitFunction_t transmitFunction,
             gos_gcpReceiveFunction_t receiveFunction )
```

Registers the physical-layer transmit and receive driver functions.

Registers the physical-layer transmit and receive driver functions.

Parameters

in	*channel*	GCP channel.
in	*transmitFunction*	Transmit function to register.
in	*receiveFunction*	Receive function to register.

Returns

Result of physical driver registration.

Return values

GOS_SUCCESS	Physical driver registration successful.
GOS_ERROR	Transmit or receive function is NULL.

Definition at line 195 of file gos_gcp.c.

7.30.4.5 gos_gcpTransmitMessage()

```
gos_result_t gos_gcpTransmitMessage (
            gos_gcpChannelNumber_t channel,
            u16_t messageId,
            void_t * pMessagePayload,
            u16_t payloadSize,
            u16_t maxChunkSize )
```

Transmits the given message via the GCP protocol.

Calls the internal message transmitter function.

Parameters

in	*channel*	GCP channel.
in	*messageId*	Message ID.
in	*pMessagePayload*	Pointer to the message payload.
in	*payloadSize*	Size of the payload (in bytes).
in	*maxChunkSize*	Maximum size of payload chunks.

Returns

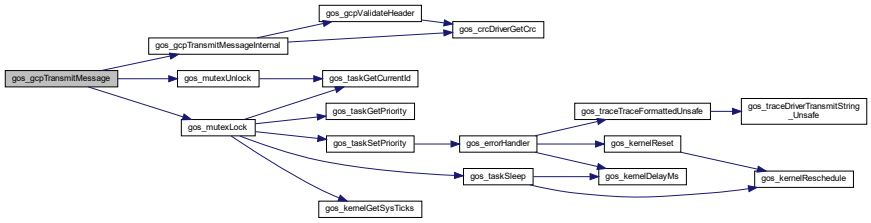
Result of message transmission.

Return values

GOS_SUCCESS	Message transmitted successfully.
GOS_ERROR	An error occurred during transmission or validation.

Definition at line 226 of file gos_gcp.c.

Here is the call graph for this function:



7.30.4.6 gos_gcpTransmitMessageInternal()

```
GOS_STATIC GOS_INLINE gos_result_t gos_gcpTransmitMessageInternal (
            gos_gcpChannelNumber_t channel,
            u16_t messageId,
            void_t * pMessagePayload,
            u16_t payloadSize,
            u16_t maxChunkSize )
```

Internal transmitter function (re-entrant).

Transmits a message over GCP (header request, payload, and then receives a response header).

Parameters

in	*channel*	GCP channel number.
in	*messageId*	ID of the message.
in	*pMessagePayload*	Pointer to the payload buffer.
in	*payloadSize*	Size of the payload (number of bytes).
in	*maxChunkSize*	Maximum chunk size (in bytes).

Returns

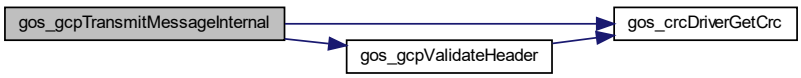
Result of message transmission.

Return values

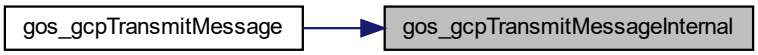
GOS_SUCCESS	Transmission successful.
GOS_ERROR	One of the function parameters are invalid or there was a transmission or reception error.

Definition at line 306 of file gos_gcp.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.30.4.7 gos_gcpValidateData()

```
GOS_STATIC gos_result_t gos_gcpValidateData (
            gos_gcpHeaderFrame_t * pHeader,
            void_t * pData,
            gos_gcpAck_t * pAck )
```

Validates the given GCP payload/data.

Checks the CRC of the given payload buffer based on the GCP header passed as a parameter. Returns the acknowledge code in the variable passed as a pointer in case the validation fails.

Parameters

in	*pHeader*	Pointer to the GCP header containing the payload data.
in	*pData*	Pointer to the data buffer to validate.
out	*pAck*	Pointer to an acknowledge variable to store the result in.

Returns

Result of validation.

Return values

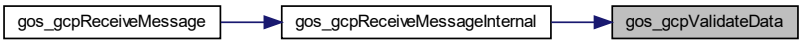
GOS_SUCCESS	Validation successful.
GOS_ERROR	CRC error or NULL pointer parameter.

Definition at line 632 of file gos_gcp.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.30.4.8 gos_gcpValidateHeader()

```
GOS_STATIC gos_result_t gos_gcpValidateHeader (
            gos_gcpHeaderFrame_t * pHeader,
            gos_gcpAck_t * pAck )
```

Validates the given GCP header.

Checks the CRC and the protocol version of the header. Returns the acknowledge code in the variable passed as a pointer in case the validation fails.

Parameters

in	*pHeader*	Pointer to the GCP header to validate.
out	*pAck*	Pointer to an acknowledge variable to store the result in.

Returns

Result of validation.

Return values

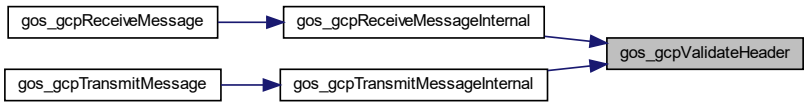
GOS_SUCCESS	Validation successful.
GOS_ERROR	CRC or PV error or NULL pointer parameter.

Definition at line 577 of file gos_gcp.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.30.5 Variable Documentation

7.30.5.1 channelFunctions

```
GOS_STATIC gos_gcpChannelFunctions_t channelFunctions[CFG_GCP_CHANNELS_MAX_NUMBER]
```

Channel functions.

Definition at line 127 of file gos_gcp.c.

7.30.5.2 gcpRxMutexes

```
GOS_STATIC gos_mutex_t gcpRxMutexes[CFG_GCP_CHANNELS_MAX_NUMBER]
```

GCP RX mutex array.

Definition at line 132 of file gos_gcp.c.

7.30.5.3 gcpTxMutexes

```
GOS_STATIC gos_mutex_t gcpTxMutexes[CFG_GCP_CHANNELS_MAX_NUMBER]
```

GCP TX mutex array.

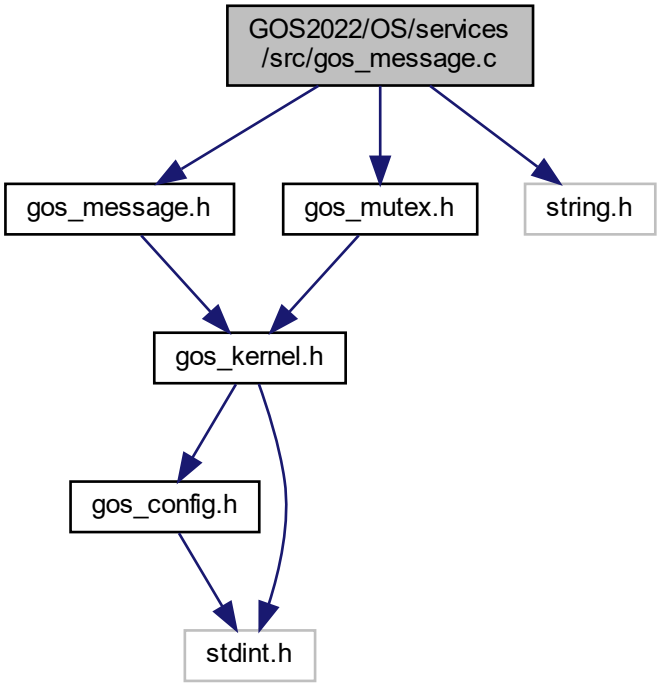
Definition at line 137 of file gos_gcp.c.

7.31 GOS2022/OS/services/src/gos_message.c File Reference

GOS message service source.

```
#include <gos_message.h>
#include <gos_mutex.h>
#include <string.h>
```

Include dependency graph for gos_message.c:



Data Structures

- struct gos_messageWaiterDesc_t

Macros

- #define GOS_MESSAGE_DAEMON_POLL_TIME_MS (50u)

Functions

- GOS_STATIC void_t gos_messageDaemonTask (void_t)

  Message daemon task.
- gos_result_t gos_messageInit (void_t)

  Initializes the message service.
- GOS_INLINE gos_result_t gos_messageRx (gos_messageId_t *messageIdArray, gos_message_t *target, gos_messageTimeout_t tmo)

  Receives the selected messages.
- GOS_INLINE gos_result_t gos_messageTx (gos_message_t *message)

  Transmits a message.

Variables

- GOS_STATIC gos_message_t messageArray [CFG_MESSAGE_MAX_NUMBER]
- GOS_STATIC gos_messageWaiterDesc_t messageWaiterArray [CFG_MESSAGE_MAX_WAITERS]
- GOS_STATIC gos_tid_t messageDaemonTaskId
- GOS_STATIC gos_messageIndex_t nextMessageIndex
- GOS_STATIC gos_messageWaiterIndex_t nextWaiterIndex
- GOS_STATIC gos_mutex_t messageMutex
- GOS_STATIC gos_taskDescriptor_t messageDaemonTaskDesc

7.31.1 Detailed Description

GOS message service source.

Author

Ahmed Gazar

Date

2025-04-06

Version

1.10

For a more detailed description of this service, please refer to gos_message.h

7.31.2 Macro Definition Documentation

7.31.2.1 GOS_MESSAGE_DAEMON_POLL_TIME_MS

```
#define GOS_MESSAGE_DAEMON_POLL_TIME_MS ( 50u )
```

Message daemon poll time in [ms].

Definition at line 79 of file gos_message.c.

7.31.3 Function Documentation

7.31.3.1 gos_messageDaemonTask()

```
GOS_STATIC void_t gos_messageDaemonTask (
            void_t  )
```

Message daemon task.

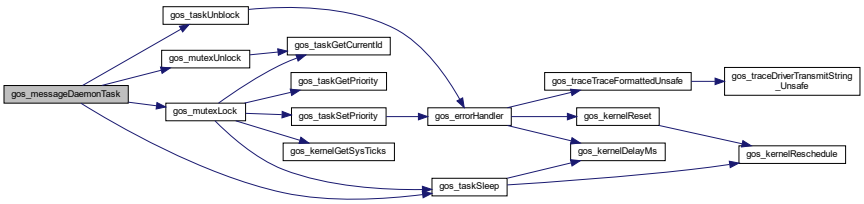
This task is responsible for message transfer between tasks. It loops through the internal message array and the waiter task array and if a message and a waiter matches, it copies the message to the target buffer and unblocks the previously blocked task.

Returns

-

Definition at line 376 of file gos_message.c.

Here is the call graph for this function:



7.31.3.2 gos_messageInit()

```
gos_result_t gos_messageInit (
            void_t  )
```

Initializes the message service.

Initializes the internal message and waiter arrays and registers the message daemon task.

Returns

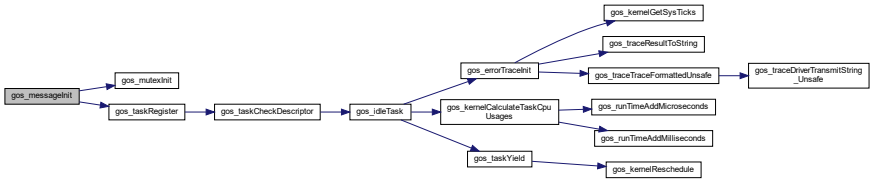
Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Message daemon task registration failed.

Definition at line 150 of file gos_message.c.

Here is the call graph for this function:



7.31.3.3 gos_messageRx()

```
GOS_INLINE gos_result_t gos_messageRx (
            gos_messageId_t * messageIdArray,
            gos_message_t * target,
            gos_messageTimeout_t tmo )
```

Receives the selected messages.

Based on the selected message IDs, this function receives the first available message. Until the message is received, the caller task is put to blocked state. The caller will be unblocked if the message is received or the timeout elapsed.

Parameters

in	*messageIdArray*	Array of messages IDs the function should receive. Must be terminated by a 0 element!
out	*target*	Pointer to the target message structure. Received data will be placed here.
in	*tmo*	Timeout value in [ms]. The resolution of timeout is 10ms! Do not use endless timeout if it is not guaranteed that the message will be received!

Returns

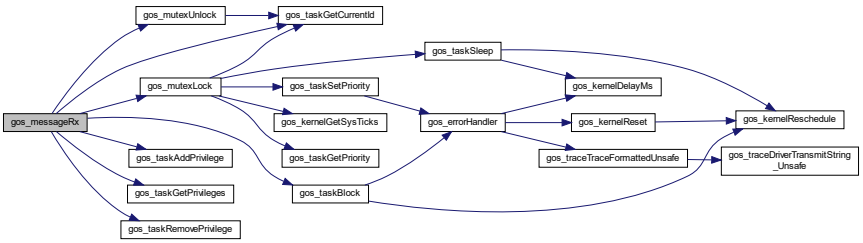
Result of message reception.

Return values

GOS_SUCCESS	Reception successful, data placed in the target structure.
GOS_ERROR	Reception failed because of invalid parameters or timeout.

Definition at line 193 of file gos_message.c.

Here is the call graph for this function:



7.31.3.4 gos_messageTx()

```
GOS_INLINE gos_result_t gos_messageTx (
            gos_message_t * message )
```

Transmits a message.

Copies the message in the internal message array for the message daemon to transfer it to the recipient task.

Parameters

in	*message*	Pointer to the message structure to be transmitted.

Returns

Result of message transmission.

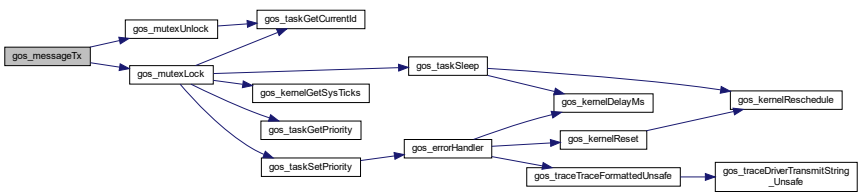
Return values

GOS_SUCCESS	Message transmission initiated successfully.
GOS_ERROR	Invalid message pointer or data or message array is full.

Function code.

Definition at line 306 of file gos_message.c.

Here is the call graph for this function:



7.31.4 Variable Documentation

7.31.4.1 messageArray

```
GOS_STATIC gos_message_t messageArray[CFG_MESSAGE_MAX_NUMBER]
```

Internal message array.

Definition at line 103 of file gos_message.c.

7.31.4.2 messageDaemonTaskDesc

```
GOS_STATIC gos_taskDescriptor_t messageDaemonTaskDesc
```

Initial value:

```
=
{
    .taskFunction        = gos_messageDaemonTask,
    .taskName            = "gos_message_daemon",
    .taskPriority        = CFG_TASK_MESSAGE_DAEMON_PRIO,
    .taskStackSize       = CFG_TASK_MESSAGE_DAEMON_STACK,
    .taskPrivilegeLevel  = GOS_TASK_PRIVILEGE_KERNEL
}
```

Message daemon task descriptor.

Definition at line 138 of file gos_message.c.

7.31.4.3 messageDaemonTaskId

```
GOS_STATIC gos_tid_t messageDaemonTaskId
```

Message daemon task ID.

Definition at line 113 of file gos_message.c.

7.31.4.4 messageMutex

```
GOS_STATIC gos_mutex_t messageMutex
```

Message mutex to protect the internal arrays as shared resources.

Definition at line 128 of file gos_message.c.

7.31.4.5 messageWaiterArray

```
GOS_STATIC gos_messageWaiterDesc_t messageWaiterArray[CFG_MESSAGE_MAX_WAITERS]
```

Internal message waiter array.

Definition at line 108 of file gos_message.c.

7.31.4.6 nextMessageIndex

```
GOS_STATIC gos_messageIndex_t nextMessageIndex
```

Index of the next message slot (for circular buffer).

Definition at line 118 of file gos_message.c.

7.31.4.7 nextWaiterIndex

```
GOS_STATIC gos_messageWaiterIndex_t nextWaiterIndex
```

Index of the next waiter slot (for circular buffer).

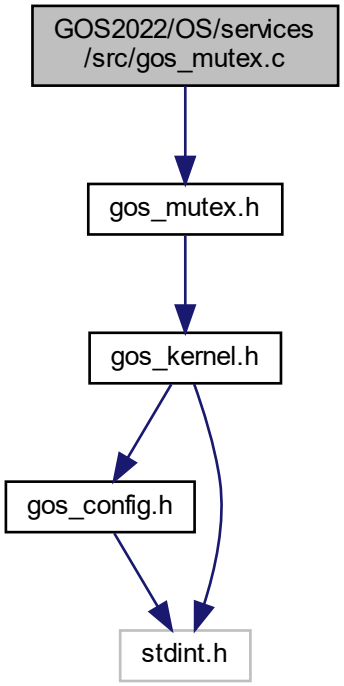
Definition at line 123 of file gos_message.c.

7.32 GOS2022/OS/services/src/gos_mutex.c File Reference

GOS mutex service source.

```
#include <gos_mutex.h>
```

Include dependency graph for gos_mutex.c:



Macros

- #define MUTEX_LOCK_SLEEP_MS (2u)

Functions

- gos_result_t gos_mutexInit (gos_mutex_t *pMutex)

  Initializes the mutex instance.
- gos_result_t gos_mutexLock (gos_mutex_t *pMutex, u32_t timeout)

  Tries to lock the given mutex with the given timeout.
- gos_result_t gos_mutexUnlock (gos_mutex_t *pMutex)

  Unlocks the mutex instance.

7.32.1 Detailed Description

GOS mutex service source.

Author

Ahmed Gazar

Date

2024-04-02

Version

1.8

For a more detailed description of this service, please refer to gos_mutex.h

7.32.2 Macro Definition Documentation

7.32.2.1 MUTEX_LOCK_SLEEP_MS

```
#define MUTEX_LOCK_SLEEP_MS ( 2u )
```

Sleep time in [ms] before re-attempting to lock mutex.

Definition at line 71 of file gos_mutex.c.

7.32.3 Function Documentation

7.32.3.1 gos_mutexInit()

```
gos_result_t gos_mutexInit (
            gos_mutex_t * pMutex )
```

Initializes the mutex instance.

Sets the mutex state to unlocked.

Parameters

in,out	*pMutex*	Pointer to the mutex to be initialized.

Returns

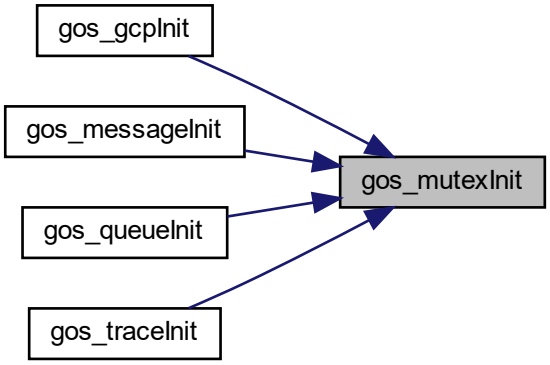
Result of initialization.

Return values

GOS_SUCCESS	Mutex initialized successfully.
GOS_ERROR	Mutex pointer is NULL.

Definition at line 76 of file gos_mutex.c.

Here is the caller graph for this function:



7.32.3.2 gos_mutexLock()

```
gos_result_t gos_mutexLock (
            gos_mutex_t * pMutex,
            u32_t timeout )
```

Tries to lock the given mutex with the given timeout.

Waits in an unblocking way until the mutex is unlocked or the timeout value is reached. If the mutex becomes unlocked within the timeout value, it locks it.

Parameters

in,out	*pMutex*	Pointer to the mutex to be locked.
in	*timeout*	Timeout value.

Returns

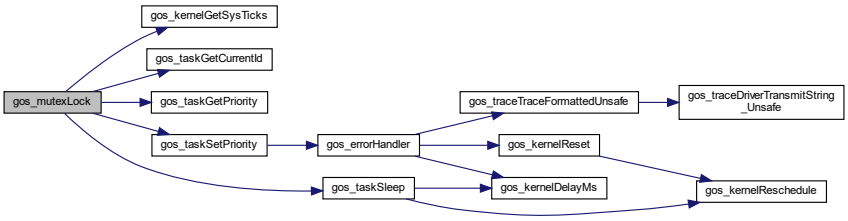
Result of mutex locking.

Return values

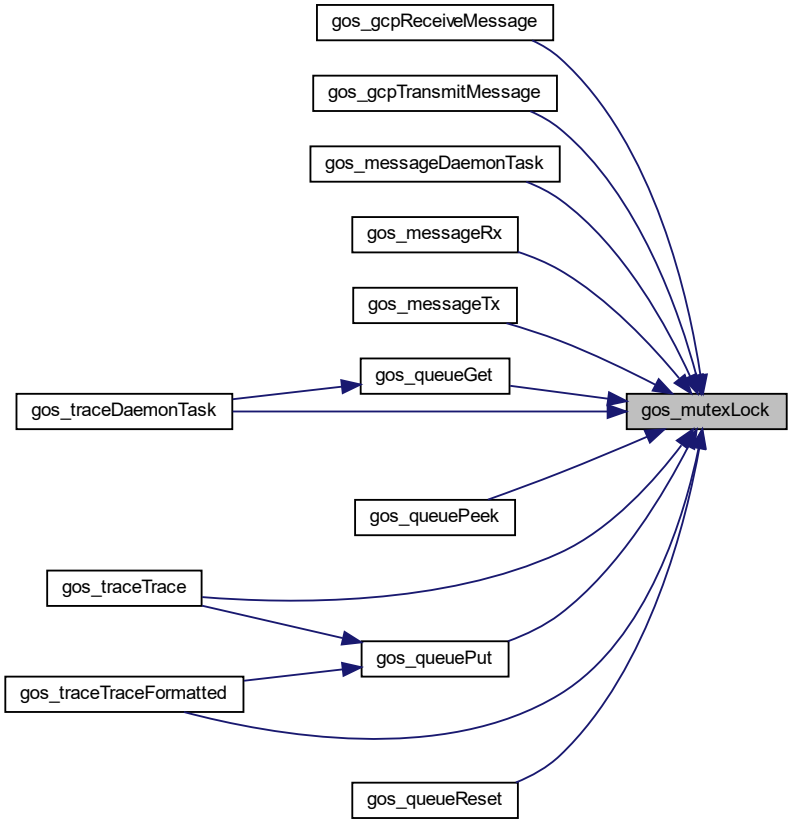
GOS_SUCCESS	Mutex locked successfully.
GOS_ERROR	Mutex could not be locked within the timeout value.

Definition at line 103 of file gos_mutex.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.32.3.3 gos_mutexUnlock()

```
gos_result_t gos_mutexUnlock (
            gos_mutex_t * pMutex )
```

Unlocks the mutex instance.

Sets the mutex state to unlocked.

Parameters

in,out	*pMutex*	Pointer to the mutex to be unlocked.

Returns

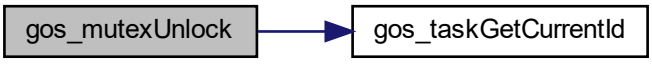
Result of mutex unlocking.

Return values

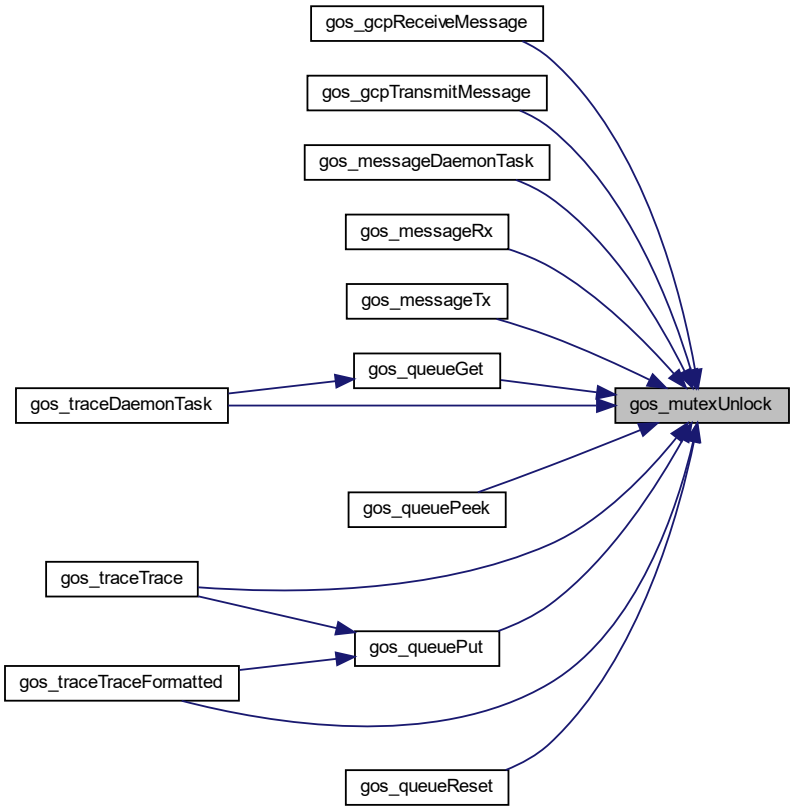
GOS_SUCCESS	Unlocking successful.
GOS_ERROR	Mutex is NULL or caller is not the owner of the mutex.

Definition at line 202 of file gos_mutex.c.

Here is the call graph for this function:



Here is the caller graph for this function:

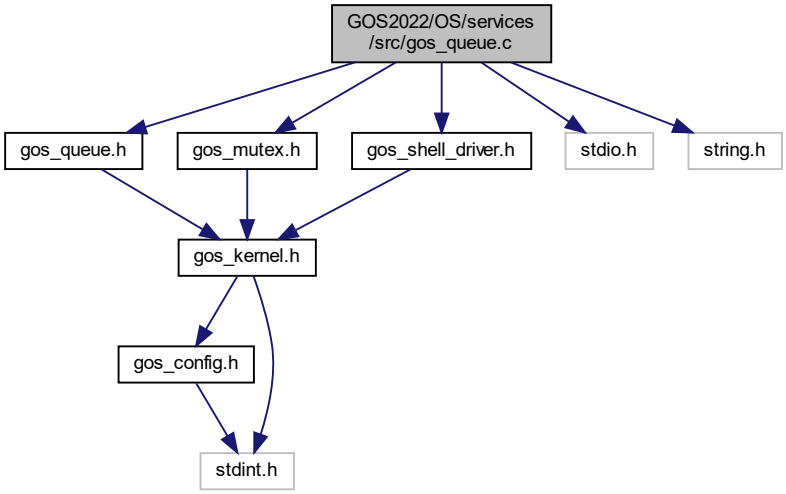


7.33 GOS2022/OS/services/src/gos_queue.c File Reference

GOS queue service source.

```
#include <gos_queue.h>
#include <gos_mutex.h>
#include <gos_shell_driver.h>
#include <stdio.h>
#include <string.h>
```

Include dependency graph for gos_queue.c:



Data Structures

- struct gos_queueElement_t
- struct gos_queue_t

Macros

- #define DUMP_SEPARATOR "+--------+------------------------------+\r\n"

Functions

- gos_result_t gos_queueInit (void_t)

  This function initializes the queue service.
- gos_result_t gos_queueCreate (gos_queueDescriptor_t *pQueueDescriptor)

  This function creates a new queue.
- gos_result_t gos_queuePut (gos_queueId_t queueId, void_t *element, gos_queueLength_t elementSize, u32_t timeout)

  This function puts an element in the given queue.
- gos_result_t gos_queueGet (gos_queueId_t queueId, void_t *target, gos_queueLength_t targetSize, u32_t timeout)

  This function gets the next element from the given queue.
- gos_result_t gos_queuePeek (gos_queueId_t queueId, void_t *target, gos_queueLength_t targetSize, u32_t timeout)

  This function gets the next element from the given queue without removing it.
- gos_result_t gos_queueReset (gos_queueId_t queueId)

  Resets the given queue.
- gos_result_t gos_queueRegisterFullHook (gos_queueFullHook fullHook)

  This function registers a queue full hook function.

- gos_result_t gos_queueRegisterEmptyHook (gos_queueEmptyHook emptyHook)
  This function registers a queue empty hook function.
- gos_result_t gos_queueGetName (gos_queueId_t queueId, gos_queueName_t queueName)
  This function gets the name of the given queue.
- gos_result_t gos_queueGetElementNumber (gos_queueId_t queueId, gos_queueIndex_t *elementNumber)
  This function gets the number of elements in the given queue.
- void_t gos_queueDump (void_t)
  Queue dump.

Variables

- GOS_STATIC gos_queue_t queues [CFG_QUEUE_MAX_NUMBER]
- GOS_STATIC gos_queueLength_t readCounters [CFG_QUEUE_MAX_NUMBER]
- GOS_STATIC gos_queueLength_t writeCounters [CFG_QUEUE_MAX_NUMBER]
- GOS_STATIC gos_mutex_t queueMutex
- GOS_STATIC gos_queueFullHook queueFullHook = NULL
- GOS_STATIC gos_queueEmptyHook queueEmptyHook = NULL

7.33.1 Detailed Description

GOS queue service source.

Author

Ahmed Gazar

Date

2024-04-02

Version

1.8

For a more detailed description of this service, please refer to gos_queue.h

7.33.2 Macro Definition Documentation

7.33.2.1 DUMP_SEPARATOR

```
#define DUMP_SEPARATOR "+--------+------------------------------+\r\n"
```

Dump separator line.

Definition at line 83 of file gos_queue.c.

7.33.3 Function Documentation

7.33.3.1 gos_queueCreate()

```
gos_result_t gos_queueCreate (
            gos_queueDescriptor_t * pQueueDescriptor )
```

This function creates a new queue.

This function loops through the internal queue array and registers the new queue in the next free slot.

Parameters

in,out	*pQueueDescriptor*	Pointer to queue descriptor variable with queue data.

Returns

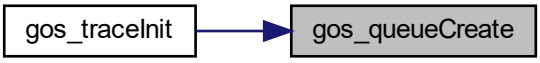
Result of queue creation.

Return values

GOS_SUCCESS	Queue creation successful.
GOS_ERROR	Queue descriptor is NULL pointer or queue array is full.

Definition at line 181 of file gos_queue.c.

Here is the caller graph for this function:



7.33.3.2 gos_queueDump()

```
void_t gos_queueDump (
            void_t  )
```

Queue dump.

Prints the queue data of all queues to the trace output.

Returns

-

Definition at line 598 of file gos_queue.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.33.3.3 gos_queueGet()

```
gos_result_t gos_queueGet (
            gos_queueId_t queueId,
            void_t * target,
            gos_queueLength_t targetSize,
            u32_t timeout )
```

This function gets the next element from the given queue.

This function checks the queue state and gets the next element from the queue.

Parameters

in	*queueId*	Queue ID.
out	*target*	Pointer to target variable.
in	*targetSize*	Size of target.
in	*timeout*	Timeout for locking queue mutex.

Returns

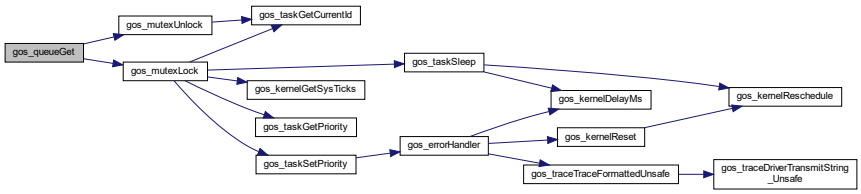
Result of element getting.

Return values

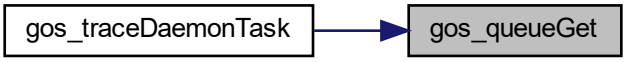
GOS_SUCCESS	Element successfully moved from queue to target.
GOS_ERROR	Invalid queue ID, invalid target size or queue is empty.

Definition at line 310 of file gos_queue.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.33.3.4 gos_queueGetElementNumber()

```
gos_result_t gos_queueGetElementNumber (
            gos_queueId_t queueId,
            gos_queueIndex_t * elementNumber )
```

This function gets the number of elements in the given queue.

This function copies the number of elements in the queue belonging to the given ID to the given variable.

Parameters

in	*queueId*	Queue ID.
out	*elementNumber*	Pointer to variable to store the element number in.

Returns

Result of queue element number getting.

Return values

GOS_SUCCESS	Element number getting successful.
GOS_ERROR	Invalid queue ID or element number variable is NULL.

Definition at line 564 of file gos_queue.c.

7.33.3.5 gos_queueGetName()

```
gos_result_t gos_queueGetName (
            gos_queueId_t queueId,
            gos_queueName_t queueName )
```

This function gets the name of the given queue.

This function copies the name of the queue belonging to the given ID to the given variable.

Parameters

in	*queueId*	Queue ID.
out	*queueName*	Queue name.

Returns

Result of queue name getting.

Return values

GOS_SUCCESS	Name getting successful.
GOS_ERROR	Invalid queue ID or queue name variable is NULL.

Definition at line 525 of file gos_queue.c.

7.33.3.6 gos_queueInit()

```
gos_result_t gos_queueInit (
            void_t  )
```

This function initializes the queue service.

Initializes the internal queue array, creates the queue lock, and registers the queue dump task in the kernel.

Returns

Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Lock creation, task registration or task suspension error.

Definition at line 147 of file gos_queue.c.

Here is the call graph for this function:



7.33.3.7 gos_queuePeek()

```
gos_result_t gos_queuePeek (
            gos_queueId_t queueId,
            void_t * target,
            gos_queueLength_t targetSize,
            u32_t timeout )
```

This function gets the next element from the given queue without removing it.

This function checks the queue state and returns the next element from the queue without modifying the queue counters.

Parameters

in	*queueId*	Queue ID.
out	*target*	Pointer to target variable.
in	*targetSize*	Size of target.
in	*timeout*	Timeout for locking queue mutex.

Returns

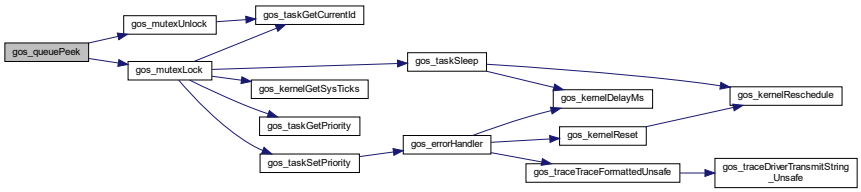
Result of element getting.

Return values

GOS_SUCCESS	Element successfully copied from queue to target.
GOS_ERROR	Invalid queue ID, invalid target size or queue is empty.

Definition at line 385 of file gos_queue.c.

Here is the call graph for this function:



7.33.3.8 gos_queuePut()

```
gos_result_t gos_queuePut (
            gos_queueId_t queueId,
            void_t * element,
            gos_queueLength_t elementSize,
            u32_t timeout )
```

This function puts an element in the given queue.

This function checks the queue state and places the given element in the next queue element.

Parameters

in	*queueId*	Queue ID.
in	*element*	Pointer to element.
in	*elementSize*	Size of element.
in	*timeout*	Timeout for locking queue mutex.

Returns

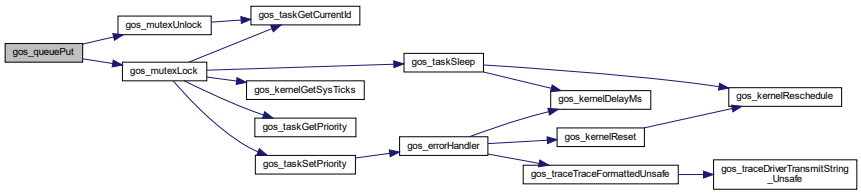
Result of element putting.

Return values

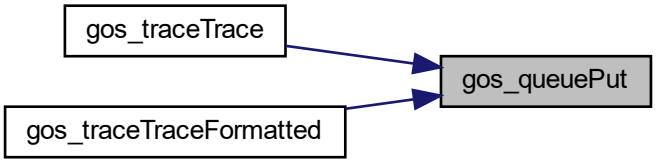
GOS_SUCCESS	Element successfully put in the queue.
GOS_ERROR	Invalid queue ID, invalid element size or queue is full.

Definition at line 231 of file gos_queue.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.33.3.9 gos_queueRegisterEmptyHook()

```
gos_result_t gos_queueRegisterEmptyHook (
            gos_queueEmptyHook emptyHook )
```

This function registers a queue empty hook function.

This function checks whether a hook has been already registered, and if not, it saves the given hook function.

Parameters

in	*emptyHook*	Hook function.

Returns

Result of hook registration.

Return values

GOS_SUCCESS	Hook registration successful.
GOS_ERROR	Hook already exists or parameter is NULL pointer.

Definition at line 499 of file gos_queue.c.

7.33.3.10 gos_queueRegisterFullHook()

```
gos_result_t gos_queueRegisterFullHook (
            gos_queueFullHook fullHook )
```

This function registers a queue full hook function.

This function checks whether a hook has been already registered, and if not, it saves the given hook function.

Parameters

in	*fullHook*	Hook function.

Returns

Result of hook registration.

Return values

GOS_SUCCESS	Hook registration successful.
GOS_ERROR	Hook already exists or parameter is NULL pointer.

Definition at line 473 of file gos_queue.c.

7.33.3.11 gos_queueReset()

```
gos_result_t gos_queueReset (
            gos_queueId_t queueId )
```

Resets the given queue.

Sets the read and write counter to zero, making the queue empty.

Parameters

in	*queueId*	Queue ID.

Returns

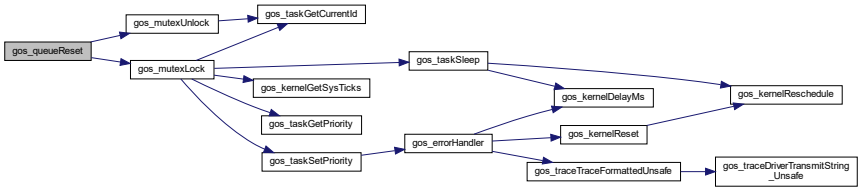
Result of queue resetting.

Return values

GOS_SUCCESS	Reset successful.
GOS_ERROR	Invalid queue ID.

Definition at line 435 of file gos_queue.c.

Here is the call graph for this function:



7.33.4 Variable Documentation

7.33.4.1 queueEmptyHook

```
GOS_STATIC gos_queueEmptyHook queueEmptyHook = NULL
```

Queue empty hook.

Definition at line 142 of file gos_queue.c.

7.33.4.2 queueFullHook

```
GOS_STATIC gos_queueFullHook queueFullHook = NULL
```

Queue full hook.

Definition at line 137 of file gos_queue.c.

7.33.4.3 queueMutex

```
GOS_STATIC gos_mutex_t queueMutex
```

Queue mutex.

Definition at line 132 of file gos_queue.c.

7.33.4.4 queues

```
GOS_STATIC gos_queue_t queues[CFG_QUEUE_MAX_NUMBER]
```

Internal queue array.

Definition at line 117 of file gos_queue.c.

7.33.4.5 readCounters

```
GOS_STATIC gos_queueLength_t readCounters[CFG_QUEUE_MAX_NUMBER]
```

Read counter array (next queue element to read).

Definition at line 122 of file gos_queue.c.

7.33.4.6 writeCounters

```
GOS_STATIC gos_queueLength_t writeCounters[CFG_QUEUE_MAX_NUMBER]
```

Write counter array (next queue element to write).

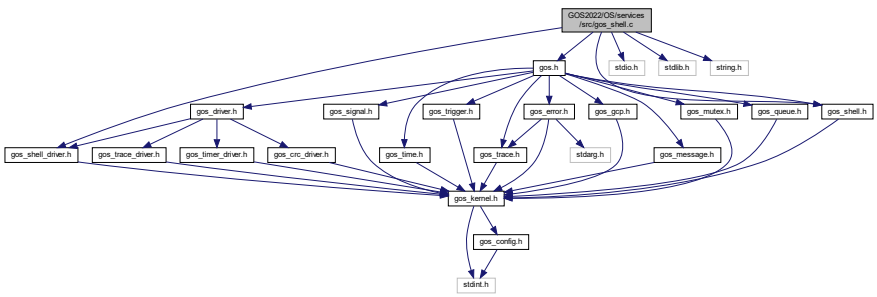
Definition at line 127 of file gos_queue.c.

7.34 GOS2022/OS/services/src/gos_shell.c File Reference

GOS shell service source.

```
#include <gos.h>
#include <gos_shell.h>
#include <gos_shell_driver.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for gos_shell.c:



Macros

- #define GOS_SHELL_DAEMON_POLL_TIME_MS (50u)
- #define GOS_SHELL_DISPLAY_TEXT ("[\x1B[1m\x1B[33mgos shell\x1B[0m]>> ")

Functions

- GOS_STATIC void_t gos_shellDaemonTask (void_t)

  Shell daemon task.
- GOS_STATIC void_t gos_shellCommandHandler (char_t *params)

  Shell command handler.
- gos_result_t gos_shellInit (void_t)

  This function initializes the shell service.
- gos_result_t gos_shellRegisterCommands (gos_shellCommand_t *commands, u16_t arraySize)

  This function registers an array of commands in the shell.
- gos_result_t gos_shellRegisterCommand (gos_shellCommand_t *command)

  This function registers a command in the shell.
- gos_result_t gos_shellSuspend (void_t)

  Suspends the shell daemon task.
- gos_result_t gos_shellResume (void_t)

  Resumes the shell daemon task.
- gos_result_t gos_shellEchoOn (void_t)

  Turns the shell echoing on.
- gos_result_t gos_shellEchoOff (void_t)

  Turns the shell echoing off.

Variables

- GOS_STATIC gos_shellCommand_t shellCommands [CFG_SHELL_MAX_COMMAND_NUMBER]
- GOS_STATIC gos_tid_t shellDaemonTaskId
- GOS_STATIC char_t commandBuffer [CFG_SHELL_COMMAND_BUFFER_SIZE]
- GOS_STATIC u16_t commandBufferIndex
- GOS_STATIC bool_t useEcho
- GOS_STATIC char_t actualCommand [CFG_SHELL_MAX_COMMAND_LENGTH]
- GOS_STATIC char_t commandParams [CFG_SHELL_MAX_PARAMS_LENGTH]
- GOS_STATIC gos_taskDescriptor_t shellDaemonTaskDesc
- GOS_STATIC gos_shellCommand_t shellCommand

7.34.1 Detailed Description

GOS shell service source.

Author

Ahmed Gazar

Date

2025-07-29

Version

1.10

For a more detailed description of this service, please refer to gos_shell.h

7.34.2 Macro Definition Documentation

7.34.2.1 GOS_SHELL_DAEMON_POLL_TIME_MS

```
#define GOS_SHELL_DAEMON_POLL_TIME_MS ( 50u )
```

Shell daemon poll time [ms].

Definition at line 78 of file gos_shell.c.

7.34.2.2 GOS_SHELL_DISPLAY_TEXT

```
#define GOS_SHELL_DISPLAY_TEXT ("[\x1B[1m\x1B[33mgos shell\x1B[0m]>> ")
```

Shell display text.

Definition at line 83 of file gos_shell.c.

7.34.3 Function Documentation

7.34.3.1 gos_shellCommandHandler()

```
GOS_STATIC void_t gos_shellCommandHandler (
            char_t * params )
```

Shell command handler.

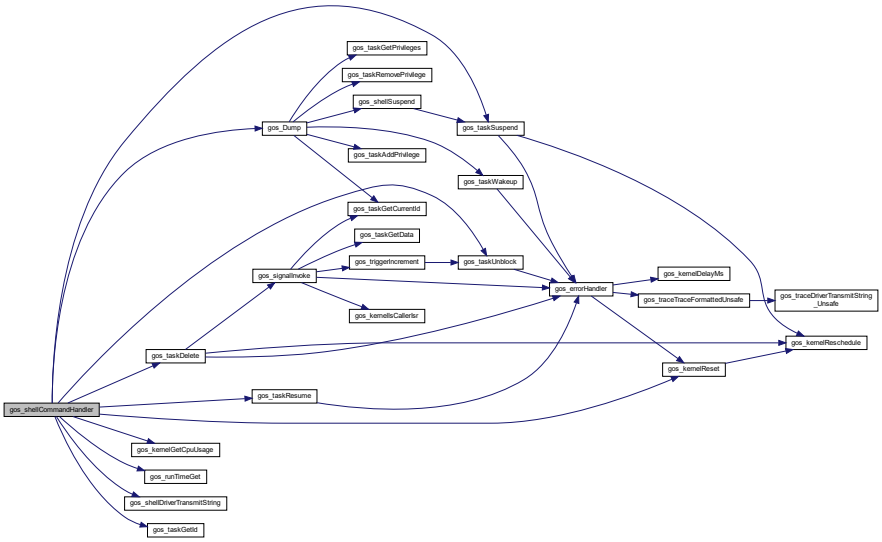
Handles the built-in shell command.

Returns

-

Definition at line 482 of file gos_shell.c.

Here is the call graph for this function:



7.34.3.2 gos_shellDaemonTask()

```
GOS_STATIC void_t gos_shellDaemonTask (
            void_t  )
```

Shell daemon task.

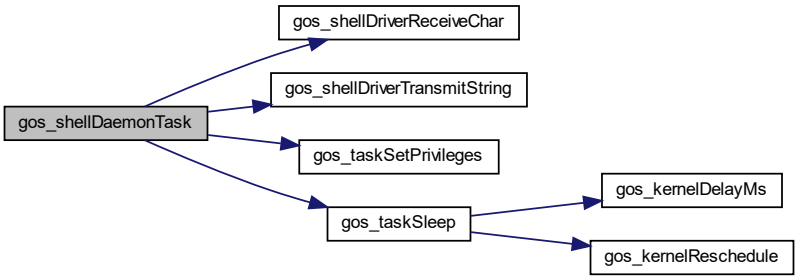
Receives a character from the log serial line, if the echoing is on, then sends the same character back. When an enter key is received, it processes the command typed in, and loops through the command array. When the command is found, it calls the command handler function with the parameter list as a string.

Returns

-

Definition at line 350 of file gos_shell.c.

Here is the call graph for this function:



7.34.3.3 gos_shellEchoOff()

```
gos_result_t gos_shellEchoOff (
            void_t  )
```

Turns the shell echoing off.

Resets the internal echo flag.

Returns

Result of turning echoing off.

Return values

GOS_SUCCESS	Echoing turned off successfully.

Definition at line 325 of file gos_shell.c.

7.34.3.4 gos_shellEchoOn()

```
gos_result_t gos_shellEchoOn (
            void_t  )
```

Turns the shell echoing on.

Sets the internal echo flag.

Returns

Result of turning echoing on.

Return values

GOS_SUCCESS	Echoing turned on successfully.

Definition at line 307 of file gos_shell.c.

7.34.3.5 gos_shellInit()

```
gos_result_t gos_shellInit (
            void_t  )
```

This function initializes the shell service.

Initializes the internal command structure, and registers the shell daemon task in the kernel.

Returns

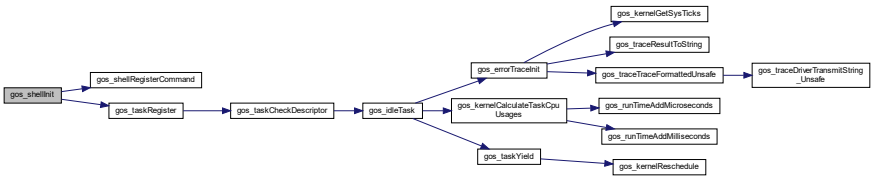
Result of initialization.

Return values

GOS_SUCCESS	Shell initialization successful.
GOS_ERROR	Shell daemon task registration failed.

Definition at line 154 of file gos_shell.c.

Here is the call graph for this function:



7.34.3.6 gos_shellRegisterCommand()

```
gos_result_t gos_shellRegisterCommand (
            gos_shellCommand_t * command )
```

This function registers a command in the shell.

Checks the command structure and registers the command in the next empty slot in the internal command array.

Parameters

in	*command*	Pointer to the command structure to register.

Returns

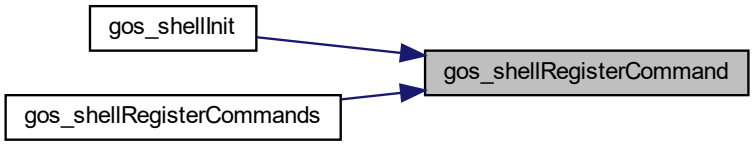
Result of shell command registration.

Return values

GOS_SUCCESS	Command registration successful.
GOS_ERROR	Command function or name is NULL or internal command array is full.

Definition at line 236 of file gos_shell.c.

Here is the caller graph for this function:



7.34.3.7 gos_shellRegisterCommands()

```
gos_result_t gos_shellRegisterCommands (
            gos_shellCommand_t * commands,
            u16_t arraySize )
```

This function registers an array of commands in the shell.

Checks the array pointer and registers the commands one by one.

Parameters

in	*commands*	Pointer to the command structure array to register.
in	*arraySize*	Size of the array in bytes.

Returns

Result of shell command registration.

Return values

GOS_SUCCESS	Command registration successful.
GOS_ERROR	Command function or name is NULL or internal command array is full.

Definition at line 188 of file gos_shell.c.

Here is the call graph for this function:



7.34.3.8 gos_shellResume()

```
gos_result_t gos_shellResume (
            void_t  )
```

Resumes the shell daemon task.

Resumes the shell daemon task.

Returns

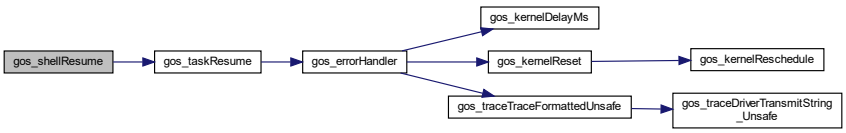
Result of shell resumption.

Return values

GOS_SUCCESS	Shell resumed successfully.
GOS_ERROR	Task resumption failed.

Definition at line 289 of file gos_shell.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.34.3.9 gos_shellSuspend()

```
gos_result_t gos_shellSuspend (
            void_t  )
```

Suspends the shell daemon task.

Suspends the shell daemon task.

Returns

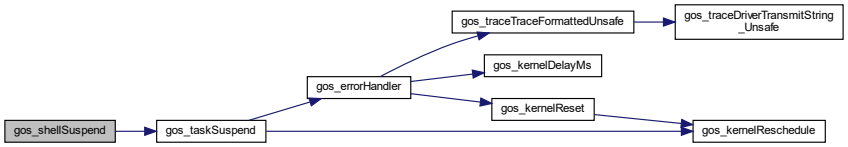
Result of shell suspension.

Return values

GOS_SUCCESS	Shell suspended successfully.
GOS_ERROR	Task suspension failed.

Definition at line 271 of file gos_shell.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.34.4 Variable Documentation

7.34.4.1 actualCommand

```
GOS_STATIC char_t actualCommand[CFG_SHELL_MAX_COMMAND_LENGTH]
```

Actual command buffer.

Definition at line 116 of file gos_shell.c.

7.34.4.2 commandBuffer

```
GOS_STATIC char_t commandBuffer[CFG_SHELL_COMMAND_BUFFER_SIZE]
```

Command buffer.

Definition at line 101 of file gos_shell.c.

7.34.4.3 commandBufferIndex

```
GOS_STATIC u16_t commandBufferIndex
```

Command buffer index.

Definition at line 106 of file gos_shell.c.

7.34.4.4 commandParams

```
GOS_STATIC char_t commandParams[CFG_SHELL_MAX_PARAMS_LENGTH]
```

Command parameters buffer.

Definition at line 121 of file gos_shell.c.

7.34.4.5 shellCommand

```
GOS_STATIC gos_shellCommand_t shellCommand
```

Initial value:

```
=
{
    .command        = "shell",
    .commandHandler = gos_shellCommandHandler,
    .commandHandlerPrivileges = GOS_TASK_PRIVILEGE_KERNEL
}
```

Shell info command.

Definition at line 144 of file gos_shell.c.

7.34.4.6 shellCommands

```
GOS_STATIC gos_shellCommand_t shellCommands[CFG_SHELL_MAX_COMMAND_NUMBER]
```

Shell command array.

Definition at line 91 of file gos_shell.c.

7.34.4.7 shellDaemonTaskDesc

```
GOS_STATIC gos_taskDescriptor_t shellDaemonTaskDesc
```

Initial value:

```
=
{
    .taskFunction        = gos_shellDaemonTask,
    .taskName            = "gos_shell_daemon",
    .taskPriority        = CFG_TASK_SHELL_DAEMON_PRIO,
    .taskStackSize       = CFG_TASK_SHELL_DAEMON_STACK,
    .taskPrivilegeLevel  = GOS_TASK_PRIVILEGE_KERNEL
}
```

Shell daemon task descriptor.

Definition at line 132 of file gos_shell.c.

7.34.4.8 shellDaemonTaskId

```
GOS_STATIC gos_tid_t shellDaemonTaskId
```

Shell daemon task ID.

Definition at line 96 of file gos_shell.c.

7.34.4.9 useEcho

```
GOS_STATIC bool_t useEcho
```

Shell echo flag.

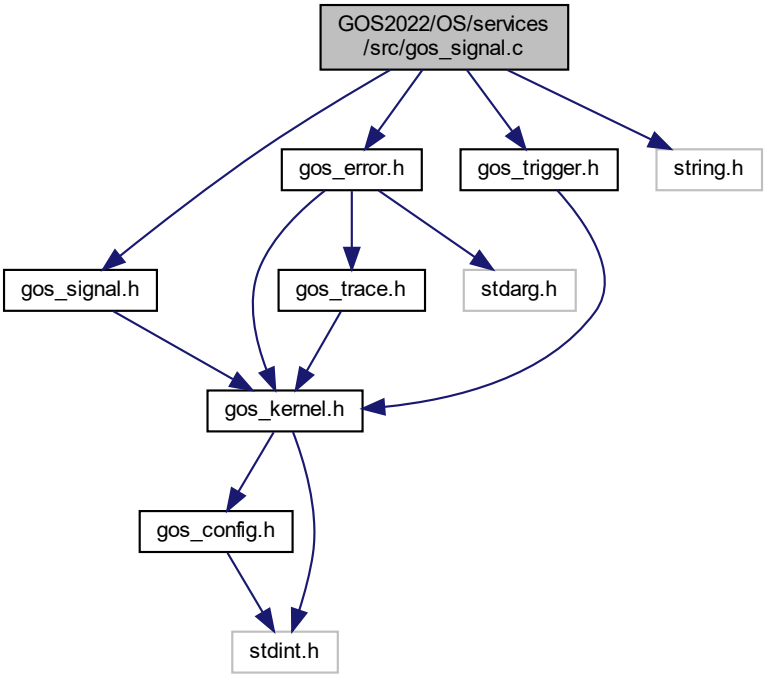
Definition at line 111 of file gos_shell.c.

7.35 GOS2022/OS/services/src/gos_signal.c File Reference

GOS signal service source.

```
#include <gos_signal.h>
#include <gos_error.h>
#include <gos_trigger.h>
#include <string.h>
```

Include dependency graph for gos_signal.c:



Data Structures

- struct gos_signalDescriptor_t
- struct gos_signalInvokeDescriptor

Macros

- #define GOS_SIGNAL_DAEMON_POLL_TIME_MS (50u)

Functions

- GOS_STATIC void_t gos_signalDaemonTask (void_t)

  Signal daemon task.
- gos_result_t gos_signalInit (void_t)

  Initializes the signal service.
- gos_result_t gos_signalCreate (gos_signalId_t *pSignal)

  Creates a new signal.
- gos_result_t gos_signalSubscribe (gos_signalId_t signalId, gos_signalHandler_t signalHandler, gos_taskPrivilegeLevel_t signalHandlerPrivileges)

  Subscribes to the given signal.
- GOS_INLINE gos_result_t gos_signalInvoke (gos_signalId_t signalId, gos_signalSenderId_t senderId)

  Invokes the given signal.

Variables

- GOS_STATIC gos_signalDescriptor_t signalArray [CFG_SIGNAL_MAX_NUMBER]
- GOS_STATIC gos_trigger_t signalInvokeTrigger
- GOS_STATIC gos_taskDescriptor_t callerTaskDesc = {0}
- GOS_EXTERN gos_signalId_t **kernelTaskDeleteSignal**
- GOS_EXTERN gos_signalId_t **kernelDumpReadySignal**
- GOS_STATIC gos_taskDescriptor_t signalDaemonTaskDescriptor

7.35.1 Detailed Description

GOS signal service source.

Author

Ahmed Gazar

Date

2025-04-06

Version

1.10

For a more detailed description of this service, please refer to gos_signal.h

7.35.2 Macro Definition Documentation

7.35.2.1 GOS_SIGNAL_DAEMON_POLL_TIME_MS

```
#define GOS_SIGNAL_DAEMON_POLL_TIME_MS ( 50u )
```

Signal daemon poll time [ms].

Definition at line 76 of file gos_signal.c.

7.35.3 Function Documentation

7.35.3.1 gos_signalCreate()

```
gos_result_t gos_signalCreate (
            gos_signalId_t * pSignal )
```

Creates a new signal.

Finds the next free slot in the signal array and registers the new signal there.

Parameters

out	*pSignal*	Pointer to a signal identifier.

Returns

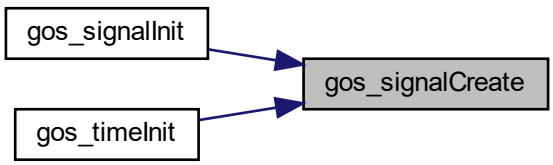
Success of signal creation.

Return values

GOS_SUCCESS	Creation successful.
GOS_ERROR	Signal array full.

Definition at line 183 of file gos_signal.c.

Here is the caller graph for this function:



7.35.3.2 gos_signalDaemonTask()

```
GOS_STATIC void_t gos_signalDaemonTask (
            void_t  )
```

Signal daemon task.

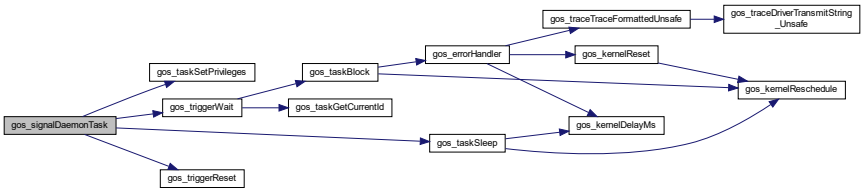
Polls the signal invoke queue, and completes the necessary signal invokings.

Returns

-

Definition at line 308 of file gos_signal.c.

Here is the call graph for this function:



7.35.3.3 gos_signalInit()

```
gos_result_t gos_signalInit (
            void_t  )
```

Initializes the signal service.

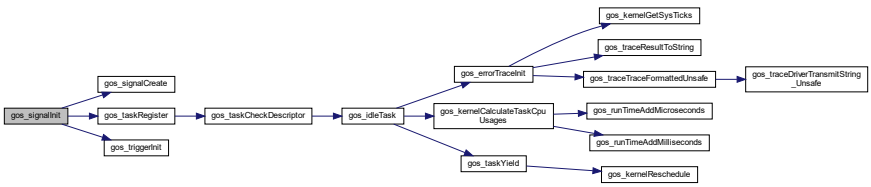
Initializes the internal signal array, creates a signal queue, registers the signal daemon task, creates the kernel dump signal, and subscribes the necessary components for the dump signal.

Returns

-

Definition at line 146 of file gos_signal.c.

Here is the call graph for this function:



7.35.3.4 gos_signalInvoke()

```
GOS_INLINE gos_result_t gos_signalInvoke (
            gos_signalId_t signalId,
            gos_signalSenderId_t senderId )
```

Invokes the given signal.

Places the given signal in the invoke queue (for the signal daemon to actually invoke the signal in the background).

Parameters

in	*signalId*	Signal identifier.
in	*senderId*	Sender identifier (or data to pass).

Returns

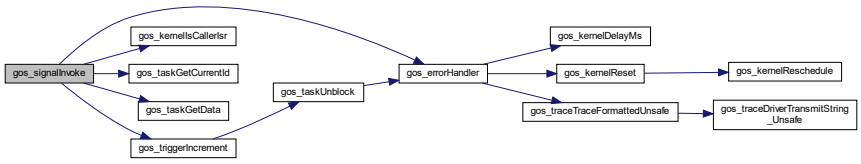
Success of signal invoking.

Return values

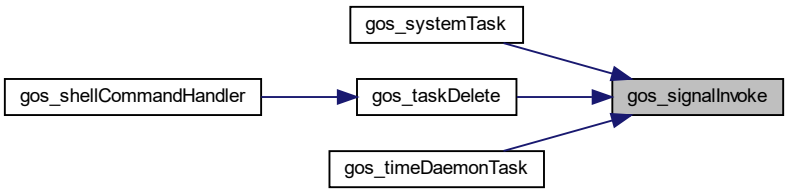
GOS_SUCCESS	Invoking successful.
GOS_ERROR	Invalid signal ID or signal unused.

Definition at line 260 of file gos_signal.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.35.3.5 gos_signalSubscribe()

```
gos_result_t gos_signalSubscribe (
            gos_signalId_t signalId,
            gos_signalHandler_t signalHandler,
            gos_taskPrivilegeLevel_t signalHandlerPrivileges )
```

Subscribes to the given signal.

Finds the next free slot in the signal handler array and registers the signal handler there.

Parameters

in	*signalId*	Signal identifier.
in	*signalHandler*	Signal handler function pointer.
in	*signalHandlerPrivileges*	Signal handler privilege level.

Returns

Success of signal subscription.

Return values

GOS_SUCCESS	Subscription successful.
GOS_ERROR	Invalid signal ID, signal handler NULL pointer, or handler array full.

Definition at line 217 of file gos_signal.c.

7.35.4 Variable Documentation

7.35.4.1 callerTaskDesc

```
GOS_STATIC gos_taskDescriptor_t callerTaskDesc = {0}
```

Caller task descriptor.

Definition at line 118 of file gos_signal.c.

7.35.4.2 signalArray

```
GOS_STATIC gos_signalDescriptor_t signalArray[CFG_SIGNAL_MAX_NUMBER]
```

Internal signal descriptor array.

Definition at line 108 of file gos_signal.c.

7.35.4.3 signalDaemonTaskDescriptor

```
GOS_STATIC gos_taskDescriptor_t signalDaemonTaskDescriptor
```

Initial value:

```
=
{
    .taskFunction        = gos_signalDaemonTask,
    .taskName            = "gos_signal_daemon",
    .taskStackSize       = CFG_TASK_SIGNAL_DAEMON_STACK,
    .taskPriority        = CFG_TASK_SIGNAL_DAEMON_PRIO,
    .taskPrivilegeLevel  = GOS_TASK_PRIVILEGE_KERNEL
}
```

Signal daemon task descriptor.

Definition at line 134 of file gos_signal.c.

7.35.4.4 signalInvokeTrigger

```
GOS_STATIC gos_trigger_t signalInvokeTrigger
```

Invoke trigger (to count the number of signal invokings).

Definition at line 113 of file gos_signal.c.

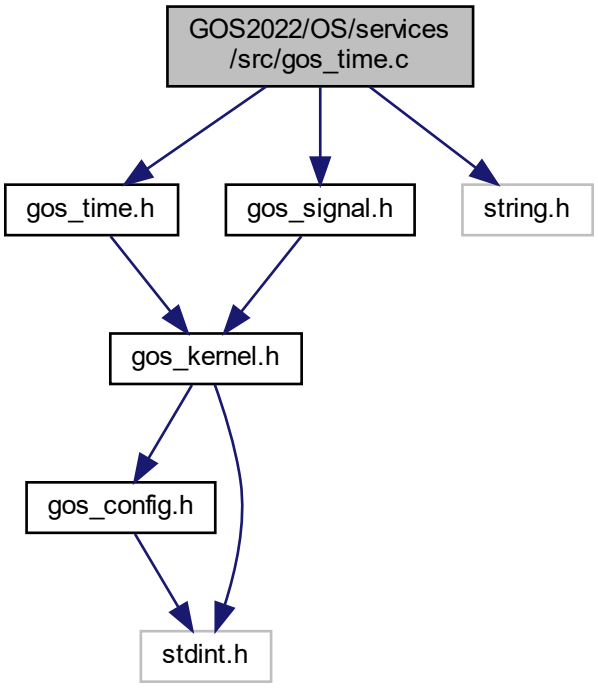
7.36 GOS2022/OS/services/src/gos_time.c File Reference

GOS time service source.

```
#include <gos_time.h>
#include <gos_signal.h>
```

```
#include <string.h>
```

Include dependency graph for gos_time.c:



Macros

- #define TIME_DEFAULT_YEAR (2023)
- #define TIME_DEFAULT_MONTH (GOS_TIME_JANUARY)
- #define TIME_DEFAULT_DAY (1)
- #define TIME_SLEEP_TIME_MS (500u)

Functions

- GOS_STATIC void_t gos_timeDaemonTask (void_t)

  Time daemon task.
- gos_result_t gos_timeInit (void_t)

  This function initializes the time service.
- gos_result_t gos_timeGet (gos_time_t *pTime)

  This function gets the system time.
- gos_result_t gos_timeSet (gos_time_t *pTime)

  This function sets the system time.
- gos_result_t gos_runTimeGet (gos_runtime_t *pRunTime)

  This function gets the system run-time.
- gos_result_t gos_timeCompare (gos_time_t *pTime1, gos_time_t *pTime2, gos_timeComprareResult_t *result)

This function compares two time structures.

- gos_result_t gos_timeAddMilliseconds (gos_time_t *pTime, u16_t milliseconds)

  This function adds the given number of milliseconds to the given time structure.

- gos_result_t gos_timeAddSeconds (gos_time_t *pTime, u16_t seconds)

  This function adds the given number of seconds to the given time variable.

- gos_result_t gos_runTimeAddMicroseconds (gos_runtime_t *pRunTime1, gos_runtime_t *pRunTime2, u16_t microseconds)

  This function adds the given number of microseconds to the given time variables.

- gos_result_t gos_runTimeAddMilliseconds (gos_runtime_t *pRunTime, u16_t milliseconds)

  This function adds the given number of milliseconds to the given time variables.

- gos_result_t gos_runTimeAddSeconds (gos_runtime_t *pRunTime, u32_t seconds)

  This function adds the given number of seconds to the given run-time variable.

- gos_result_t gos_timeIncreaseSystemTime (u16_t milliseconds)

  This function increases the system time and runtime with the given value of milliseconds.

Variables

- GOS_STATIC gos_time_t systemTime
- GOS_STATIC gos_runtime_t systemRunTime
- GOS_STATIC GOS_CONST gos_day_t dayLookupTable [GOS_TIME_NUMBER_OF_MONTHS]
- GOS_STATIC gos_tid_t timeDaemonTaskId
- gos_signalId_t timeSignalId
- GOS_STATIC gos_taskDescriptor_t timeDaemonTaskDesc

7.36.1 Detailed Description

GOS time service source.

Author

Ahmed Gazar

Date

2025-04-06

Version

1.8

For a more detailed description of this service, please refer to gos_time.h

7.36.2 Macro Definition Documentation

7.36.2.1 TIME_DEFAULT_DAY

```
#define TIME_DEFAULT_DAY ( 1 )
```

Default day.

Definition at line 86 of file gos_time.c.

7.36.2.2 TIME_DEFAULT_MONTH

```
#define TIME_DEFAULT_MONTH ( GOS_TIME_JANUARY )
```

Default month.

Definition at line 81 of file gos_time.c.

7.36.2.3 TIME_DEFAULT_YEAR

```
#define TIME_DEFAULT_YEAR ( 2023 )
```

Default year.

Definition at line 76 of file gos_time.c.

7.36.2.4 TIME_SLEEP_TIME_MS

```
#define TIME_SLEEP_TIME_MS ( 500u )
```

Time task sleep time in [ms].

Definition at line 91 of file gos_time.c.

7.36.3 Function Documentation

7.36.3.1 gos_runTimeAddMicroseconds()

```
gos_result_t gos_runTimeAddMicroseconds (
            gos_runtime_t * pRunTime1,
            gos_runtime_t * pRunTime2,
            u16_t microseconds )
```

This function adds the given number of microseconds to the given time variables.

This function adds the given number of microseconds to the given time variables.

Parameters

in,out	*pRunTime1*	Pointer to the time variable.
in,out	*pRunTime2*	Pointer to the time variable.
in	*microseconds*	Number of microseconds to add.

Returns

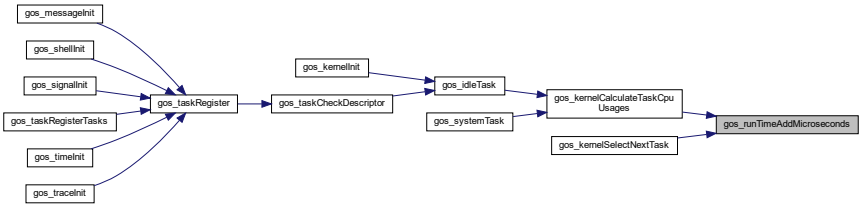
Result of time increasing.

Return values

GOS_SUCCESS	Microseconds added successfully.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 498 of file gos_time.c.

Here is the caller graph for this function:



7.36.3.2 gos_runTimeAddMilliseconds()

```
gos_result_t gos_runTimeAddMilliseconds (
            gos_runtime_t * pRunTime,
            u16_t milliseconds )
```

This function adds the given number of milliseconds to the given time variables.

This function adds the given number of milliseconds to the given time variables.

Parameters

in,out	*pRunTime*	Pointer to the time variable.
in	*milliseconds*	Number of milliseconds to add.

Returns

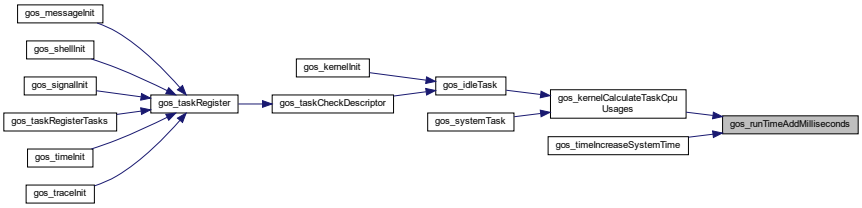
Result of time increasing.

Return values

GOS_SUCCESS	Milliseconds added successfully.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 638 of file gos_time.c.

Here is the caller graph for this function:



7.36.3.3 gos_runTimeAddSeconds()

```
gos_result_t gos_runTimeAddSeconds (
            gos_runtime_t * pRunTime,
            u32_t seconds )
```

This function adds the given number of seconds to the given run-time variable.

This function adds the given number of seconds to the given run-time variable.

Parameters

in,out	*pRunTime*	Pointer to a run-time variable.
in	*seconds*	Number of seconds to add.

Returns

Result of time increasing.

Return values

GOS_SUCCESS	Seconds added successfully.
GOS_ERROR	Run-time variable is NULL pointer.

Definition at line 707 of file gos_time.c.

7.36.3.4 gos_runTimeGet()

```
gos_result_t gos_runTimeGet (
            gos_runtime_t * pRunTime )
```

This function gets the system run-time.

This function gets the system run-time.

Parameters

out	*pRunTime*	Pointer to a run-time variable to store the system run-time in.

Returns

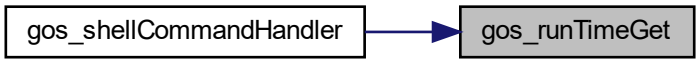
Result of run-time getting.

Return values

GOS_SUCCESS	Run-time getting is successful.
GOS_ERROR	Run-time variable is NULL pointer.

Definition at line 241 of file gos_time.c.

Here is the caller graph for this function:



7.36.3.5 gos_timeAddMilliseconds()

```
gos_result_t gos_timeAddMilliseconds (
            gos_time_t * pTime,
            u16_t milliseconds )
```

This function adds the given number of milliseconds to the given time structure.

This function adds the given number of milliseconds to the given time structure.

Parameters

in,out	*pTime*	Pointer to the time structure.
in	*milliseconds*	Number of milliseconds to add.

Returns

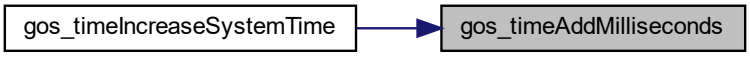
Result of time increasing.

Return values

GOS_SUCCESS	Increasing successful.
GOS_ERROR	Time structure is NULL pointer.

Definition at line 312 of file gos_time.c.

Here is the caller graph for this function:



7.36.3.6 gos_timeAddSeconds()

```
gos_result_t gos_timeAddSeconds (
            gos_time_t * pTime,
            u16_t seconds )
```

This function adds the given number of seconds to the given time variable.

This function adds the given number of seconds to the given time variable.

Parameters

in,out	*pTime*	Pointer to the time variable.
in	*seconds*	Number of seconds to add.

Returns

Result of time increasing.

Return values

GOS_SUCCESS	Seconds added successfully.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 408 of file gos_time.c.

7.36.3.7 gos_timeCompare()

```
gos_result_t gos_timeCompare (
            gos_time_t * pTime1,
            gos_time_t * pTime2,
            gos_timeComprareResult_t * result )
```

This function compares two time structures.

This function compares two time structures.

Parameters

in	*pTime1*	Pointer to the first time variable.
in	*pTime2*	Pointer to the second time variable.
out	*result*	Pointer to the comparison result variable.

Returns

Result of time comparison.

Return values

GOS_SUCCESS	Time comparison successful.
GOS_ERROR	Either time structure and/or result variable is NULL pointer.

Definition at line 268 of file gos_time.c.

7.36.3.8 gos_timeDaemonTask()

```
GOS_STATIC void_t gos_timeDaemonTask (
            void_t  )
```

Time daemon task.

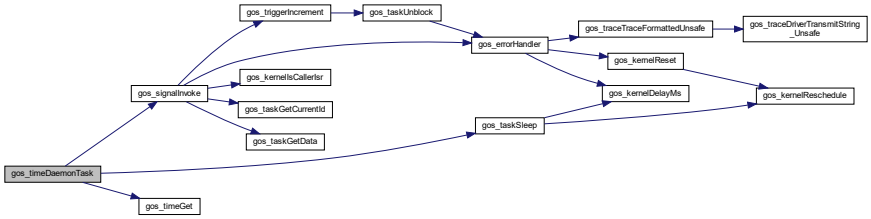
Increases the system time approximately every second and invokes the elapsed signals.

Returns

-

Definition at line 798 of file gos_time.c.

Here is the call graph for this function:



7.36.3.9 gos_timeGet()

```
gos_result_t gos_timeGet (
            gos_time_t * pTime )
```

This function gets the system time.

This function copies the system time value to the given time variable.

Parameters

out	*pTime*	Pointer to a time variable to store the system time value in.

Returns

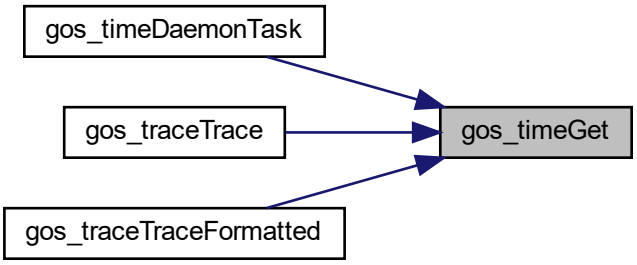
Result of time getting.

Return values

GOS_SUCCESS	Time getting successful.
GOS_ERROR	Time variable is NULL pointer.

Definition at line 187 of file gos_time.c.

Here is the caller graph for this function:



7.36.3.10 gos_timeIncreaseSystemTime()

```
gos_result_t gos_timeIncreaseSystemTime (
            u16_t milliseconds )
```

This function increases the system time and runtime with the given value of milliseconds.

This function increases the system time and runtime with the given value of milliseconds.

Parameters

in	*milliseconds*	Number of milliseconds to add to system time.

Returns

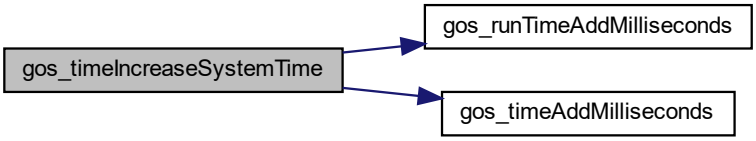
Result of system time increasing.

Return values

GOS_SUCCESS	Increasing successful.
GOS_ERROR	System time or runtime increasing failed.

Definition at line 768 of file gos_time.c.

Here is the call graph for this function:



7.36.3.11 gos_timeInit()

```
gos_result_t gos_timeInit (
            void_t  )
```

This function initializes the time service.

Creates the time signal and registers the time daemon in the kernel.

Returns

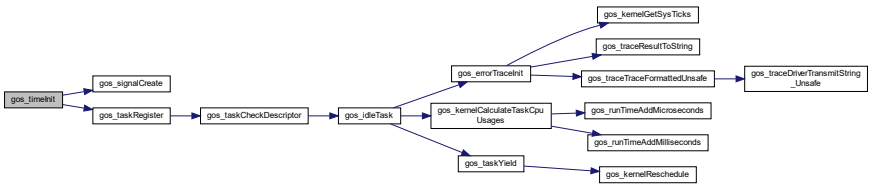
Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Time signal registration error or time daemon registration error.

Definition at line 160 of file gos_time.c.

Here is the call graph for this function:



7.36.3.12 gos_timeSet()

```
gos_result_t gos_timeSet (
             gos_time_t * pTime )
```

This function sets the system time.

This function copies the time value from the given time variable to the system time variable.

Parameters

in	*pTime*	Pointer to a time variable holding the desired time value.

Returns

Result of time setting.

Return values

GOS_SUCCESS	Time setting successful.
GOS_ERROR	Time structure is NULL pointer.

Definition at line 214 of file gos_time.c.

7.36.4 Variable Documentation

7.36.4.1 dayLookupTable

```
GOS_STATIC GOS_CONST gos_day_t dayLookupTable[GOS_TIME_NUMBER_OF_MONTHS]
```

Initial value:

```
=
{
        [GOS_TIME_JANUARY   - 1]    = 31,
        [GOS_TIME_FEBRUARY  - 1]    = 28,
        [GOS_TIME_MARCH     - 1]    = 31,
        [GOS_TIME_APRIL     - 1]    = 30,
        [GOS_TIME_MAY       - 1]    = 31,
        [GOS_TIME_JUNE      - 1]    = 30,
        [GOS_TIME_JULY      - 1]    = 31,
        [GOS_TIME_AUGUST    - 1]    = 31,
        [GOS_TIME_SEPTEMBER - 1]    = 30,
        [GOS_TIME_OCTOBER   - 1]    = 31,
        [GOS_TIME_NOVEMBER  - 1]    = 30,
        [GOS_TIME_DECEMBER  - 1]    = 31
}
```

Number of days in each month - lookup table.

Definition at line 114 of file gos_time.c.

7.36.4.2 systemRunTime

```
GOS_STATIC gos_runtime_t systemRunTime
```

System run-time.

Definition at line 109 of file gos_time.c.

7.36.4.3 systemTime

```
GOS_STATIC gos_time_t systemTime
```

Initial value:

```
=
{
    .years   =  ( 2023 ) ,
    .months  =  ( GOS_TIME_JANUARY ) ,
    .days    =  ( 1 )
}
```

System time.

Definition at line 99 of file gos_time.c.

7.36.4.4 timeDaemonTaskDesc

```
GOS_STATIC gos_taskDescriptor_t timeDaemonTaskDesc
```

Initial value:

```
=
{
    .taskFunction         = gos_timeDaemonTask,
    .taskName             = "gos_time_daemon",
    .taskStackSize        = CFG_TASK_TIME_DAEMON_STACK,
    .taskPriority         = CFG_TASK_TIME_DAEMON_PRIO,
    .taskPrivilegeLevel   = GOS_TASK_PRIVILEGE_KERNEL
}
```

Time task descriptor.

Definition at line 148 of file gos_time.c.

7.36.4.5 timeDaemonTaskId

```
GOS_STATIC gos_tid_t timeDaemonTaskId
```

Time task ID.

Definition at line 133 of file gos_time.c.

7.36.4.6 timeSignalId

```
gos_signalId_t timeSignalId
```

Time signal ID. This signal is invoked when a second has elapsed.

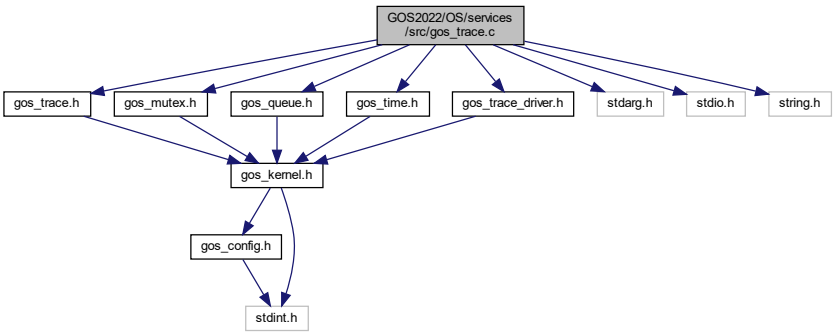
Definition at line 138 of file gos_time.c.

7.37 GOS2022/OS/services/src/gos_trace.c File Reference

GOS trace service source.

```
#include <gos_trace.h>
#include <gos_mutex.h>
#include <gos_queue.h>
#include <gos_time.h>
#include <gos_trace_driver.h>
#include <stdarg.h>
#include <stdio.h>
#include <string.h>
```

Include dependency graph for gos_trace.c:



Macros

- #define GOS_TRACE_TIMESTAMP_FORMAT "["TRACE_FG_YELLOW_START"%04d-%02d-%02d %02d:%02d:%02d.%03d"TRACE_FORMAT_RESET"]\t"
- #define GOS_TRACE_TIMESTAMP_LENGTH (46u)
- #define GOS_TRACE_QUEUE_TMO_MS (2000u)
- #define GOS_TRACE_MUTEX_TMO_MS (2000u)

Functions

- GOS_STATIC void_t gos_traceDaemonTask (void_t)

  Trace daemon task.
- gos_result_t gos_traceInit (void_t)

  Initializes the trace service.
- GOS_INLINE gos_result_t gos_traceTrace (bool_t addTimeStamp, char_t *traceMessage)

  Traces a given message.
- gos_result_t gos_traceTraceFormatted (bool_t addTimeStamp, GOS_CONST char_t *traceFormat,...)

  Traces a given formatted message.
- gos_result_t gos_traceTraceFormattedUnsafe (GOS_CONST char_t *traceFormat,...)

  Traces a given formatted message.

Variables

- GOS_STATIC gos_queueDescriptor_t traceQueue
- GOS_STATIC char_t traceLine [CFG_TRACE_MAX_LENGTH]
- GOS_STATIC char_t formattedBuffer [CFG_TRACE_MAX_LENGTH]
- GOS_STATIC char_t timeStampBuffer [(46u)]
- GOS_STATIC gos_mutex_t traceMutex
- GOS_STATIC gos_taskDescriptor_t traceDaemonTaskDesc

7.37.1 Detailed Description

GOS trace service source.

Author

Ahmed Gazar

Date

2025-04-06

Version

1.13

For a more detailed description of this service, please refer to gos_trace.h

7.37.2 Macro Definition Documentation

7.37.2.1 GOS_TRACE_MUTEX_TMO_MS

```
#define GOS_TRACE_MUTEX_TMO_MS ( 2000u )
```

Timeout value in [ms] for mutex operations.

Definition at line 97 of file gos_trace.c.

7.37.2.2 GOS_TRACE_QUEUE_TMO_MS

```
#define GOS_TRACE_QUEUE_TMO_MS ( 2000u )
```

Timeout value in [ms] for queue operations.

Definition at line 92 of file gos_trace.c.

7.37.2.3 GOS_TRACE_TIMESTAMP_FORMAT

```
#define GOS_TRACE_TIMESTAMP_FORMAT "["TRACE_FG_YELLOW_START"%04d-%02d-%02d %02d:%02d:%02d.%03d"T↩
RACE_FORMAT_RESET"]\t"
```

Trace timestamp formatter.

Definition at line 82 of file gos_trace.c.

7.37.2.4 GOS_TRACE_TIMESTAMP_LENGTH

```
#define GOS_TRACE_TIMESTAMP_LENGTH ( 46u )
```

Trace timestamp length.

Definition at line 87 of file gos_trace.c.

7.37.3 Function Documentation

7.37.3.1 gos_traceDaemonTask()

```
GOS_STATIC void_t gos_traceDaemonTask (
            void_t  )
```

Trace daemon task.

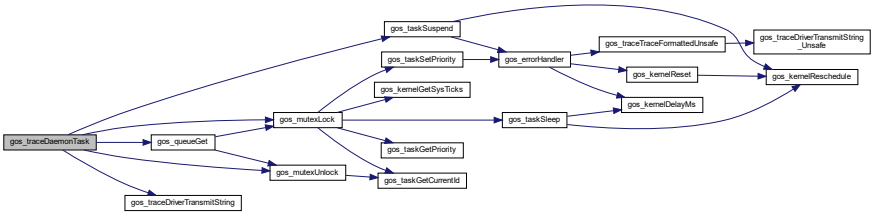
Polls the trace queue and transmits the elements in the trace queue via the registered trace driver.

Returns

-

Definition at line 394 of file gos_trace.c.

Here is the call graph for this function:



7.37.3.2 gos_traceInit()

```
gos_result_t gos_traceInit (
            void_t  )
```

Initializes the trace service.

Creates a trace queue and registers the trace daemon in the kernel.

Returns

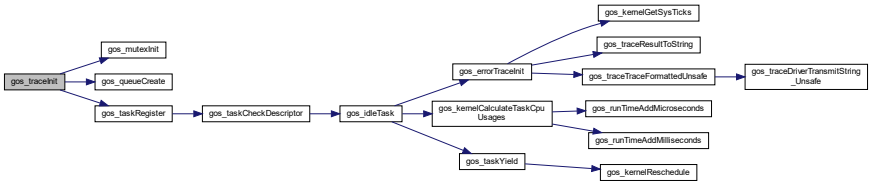
Result of initialization.

Return values

GOS_SUCCESS	Initialization successful.
GOS_ERROR	Queue creation or task registration error.

Definition at line 152 of file gos_trace.c.

Here is the call graph for this function:



7.37.3.3 gos_traceTrace()

```
GOS_INLINE gos_result_t gos_traceTrace (
            bool_t addTimeStamp,
            char_t * traceMessage )
```

Traces a given message.

Places the given message to the trace queue (for the trace daemon to print it).

Parameters

in	*addTimeStamp*	Flag to indicate whether to add time-stamp or not.
in	*traceMessage*	String to trace.

Returns

Result of tracing.

Return values

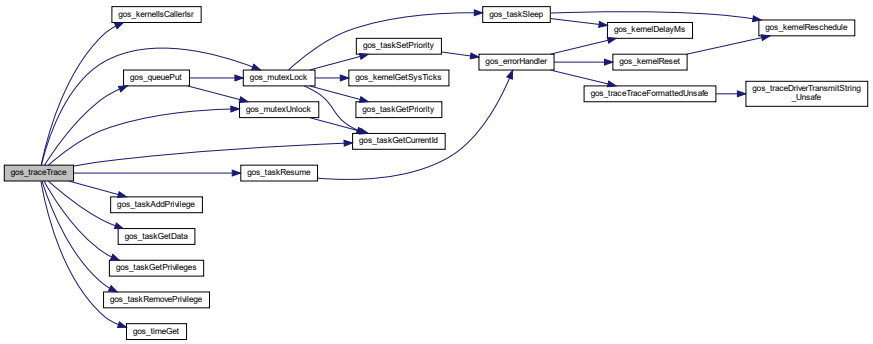
GOS_SUCCESS	Tracing successful.
GOS_ERROR	Queue put error.

Remarks

This function uses the queue service.

Definition at line 177 of file gos_trace.c.

Here is the call graph for this function:



7.37.3.4 gos_traceTraceFormatted()

```
gos_result_t gos_traceTraceFormatted (
            bool_t addTimeStamp,
            GOS_CONST char_t * traceFormat,
             ...  )
```

Traces a given formatted message.

Prints the formatted message into a local buffer and places it to the trace queue (for the trace daemon to print it).

Parameters

in	*addTimeStamp*	Flag to indicate whether to add time-stamp or not.
in	*traceFormat*	Formatter string.
in	*...*	Optional parameters.

Returns

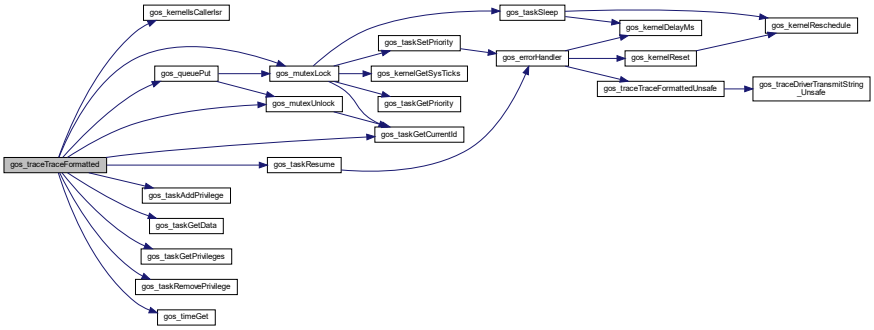
Result of formatted tracing.

Return values

GOS_SUCCESS	Formatted tracing successful.
GOS_ERROR	Queue put error.

Definition at line 266 of file gos_trace.c.

Here is the call graph for this function:



7.37.3.5 gos_traceTraceFormattedUnsafe()

```
gos_result_t gos_traceTraceFormattedUnsafe (
            GOS_CONST char_t * traceFormat,
             ...  )
```

Traces a given formatted message.

Prints the formatted message into a local buffer and transmits it via the trace port.

Parameters

in	*traceFormat*	Formatter string.
in	...	Optional parameters.

Returns

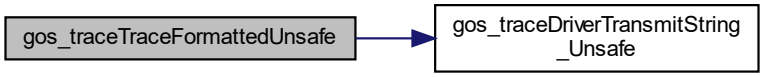
Result of formatted tracing.

Return values

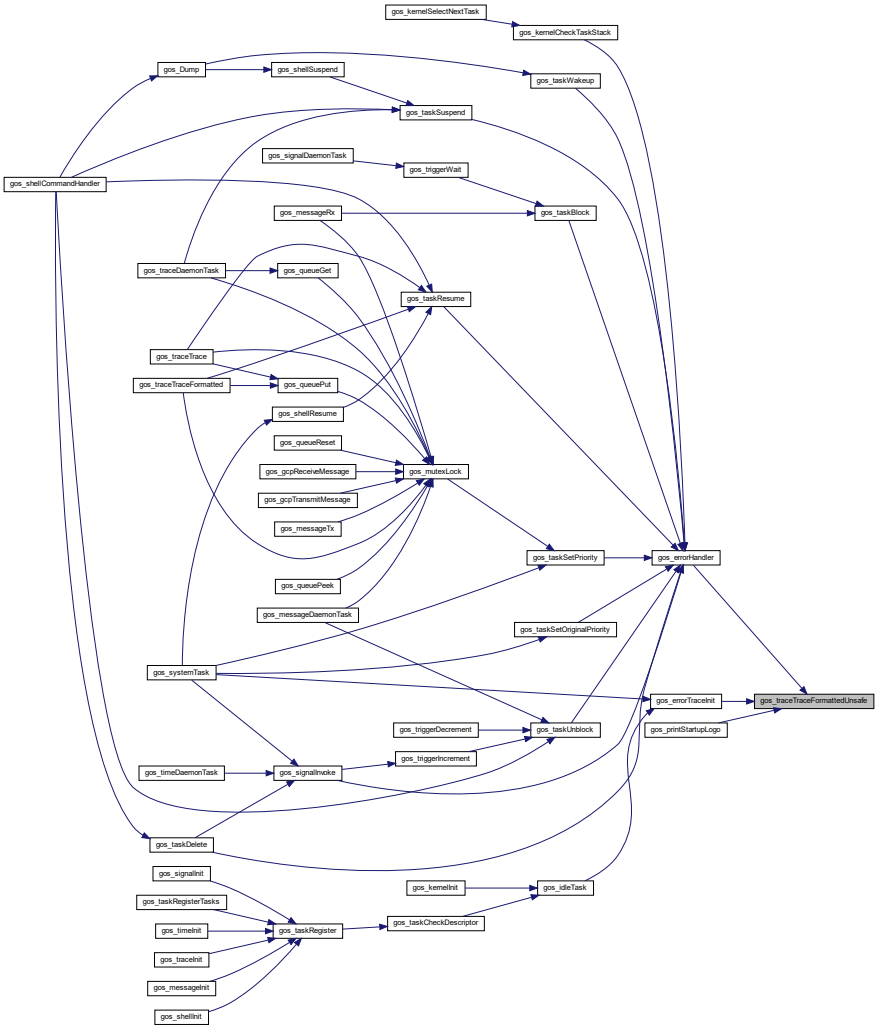
GOS_SUCCESS	Message traced successfully.
GOS_ERROR	Transmit error.

Definition at line 356 of file gos_trace.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.37.4 Variable Documentation

7.37.4.1 formattedBuffer

```
GOS_STATIC char_t formattedBuffer[CFG_TRACE_MAX_LENGTH]
```

Trace formatted buffer for message formatting.

Definition at line 120 of file gos_trace.c.

7.37.4.2 timeStampBuffer

```
GOS_STATIC char_t timeStampBuffer[(46u)]
```

Buffer for timestamp printing.

Definition at line 125 of file gos_trace.c.

7.37.4.3 traceDaemonTaskDesc

```
GOS_STATIC gos_taskDescriptor_t traceDaemonTaskDesc
```

Initial value:

```
=
{
    .taskFunction        = gos_traceDaemonTask,
    .taskName            = "gos_trace_daemon",
    .taskPriority        = CFG_TASK_TRACE_DAEMON_PRIO,
    .taskStackSize       = CFG_TASK_TRACE_DAEMON_STACK,
    .taskPrivilegeLevel  = GOS_TASK_PRIVILEGE_KERNEL
}
```

Trace daemon task descriptor.

Definition at line 140 of file gos_trace.c.

7.37.4.4 traceLine

```
GOS_STATIC char_t traceLine[CFG_TRACE_MAX_LENGTH]
```

Trace line buffer.

Definition at line 115 of file gos_trace.c.

7.37.4.5 traceMutex

```
GOS_STATIC gos_mutex_t traceMutex
```

Trace mutex.

Definition at line 130 of file gos_trace.c.

7.37.4.6 traceQueue

```
GOS_STATIC gos_queueDescriptor_t traceQueue
```

Initial value:

```
=
{
}
```

Trace queue.

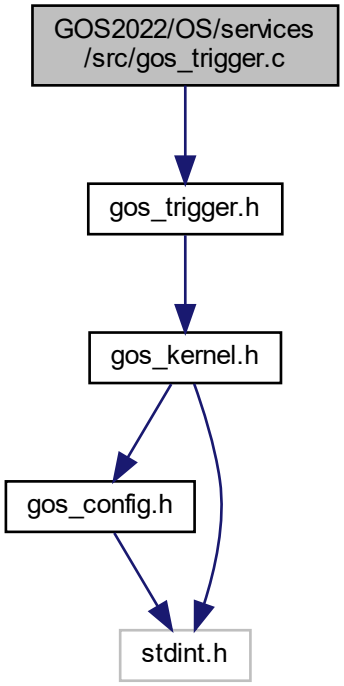
Definition at line 105 of file gos_trace.c.

7.38 GOS2022/OS/services/src/gos_trigger.c File Reference

GOS trigger service source.

```
#include <gos_trigger.h>
```

Include dependency graph for gos_trigger.c:



Functions

- gos_result_t gos_triggerInit (gos_trigger_t *pTrigger)

  Initializes the trigger instance.
- GOS_INLINE gos_result_t gos_triggerReset (gos_trigger_t *pTrigger)

  Resets the trigger counter of the given trigger instance.
- GOS_INLINE gos_result_t gos_triggerWait (gos_trigger_t *pTrigger, u32_t value, u32_t timeout)

  Waits for the trigger instance to reach the given trigger value.
- GOS_INLINE gos_result_t gos_triggerIncrement (gos_trigger_t *pTrigger)

  Increments the trigger value of the given trigger.
- GOS_INLINE gos_result_t gos_triggerDecrement (gos_trigger_t *pTrigger)

  Decrements the trigger value of the given trigger.

7.38.1 Detailed Description

GOS trigger service source.

Author

Ahmed Gazar

Date

2025-03-28

Version

2.10

For a more detailed description of this service, please refer to gos_trigger.h

7.38.2 Function Documentation

7.38.2.1 gos_triggerDecrement()

```
GOS_INLINE gos_result_t gos_triggerDecrement (
            gos_trigger_t * pTrigger )
```

Decrements the trigger value of the given trigger.

Decrements the trigger value of the given trigger.

Parameters

in,out	*pTrigger*	Pointer to the trigger instance.

Returns

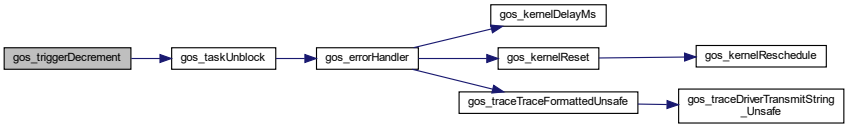
Result of trigger decrementing.

Return values

GOS_SUCCESS	Decrementing successful.
GOS_ERROR	Trigger is NULL pointer or value is already zero.

Definition at line 246 of file gos_trigger.c.

Here is the call graph for this function:



7.38.2.2 gos_triggerIncrement()

```
GOS_INLINE gos_result_t gos_triggerIncrement (
            gos_trigger_t * pTrigger )
```

Increments the trigger value of the given trigger.

Increments the trigger value of the given trigger.

Parameters

in,out	*pTrigger*	Pointer to the trigger instance.

Returns

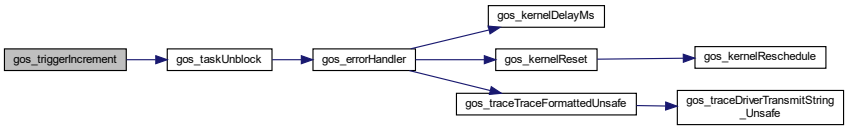
Result of trigger incrementing.

Return values

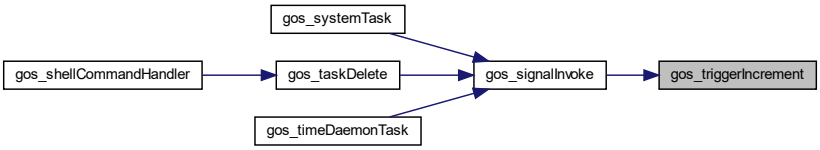
GOS_SUCCESS	Incrementing successful.
GOS_ERROR	Trigger is NULL pointer.

Definition at line 199 of file gos_trigger.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.38.2.3 gos_triggerInit()

```
gos_result_t gos_triggerInit (
            gos_trigger_t * pTrigger )
```

Initializes the trigger instance.

Calls the initializer for the trigger mutex.

Parameters

out	*pTrigger*	Pointer to the trigger to be initialized.

Returns

Result of trigger initializing.

Return values

GOS_SUCCESS	Trigger initialized successfully.
GOS_ERROR	Trigger descriptor or trigger mutex is NULL pointer.

Definition at line 78 of file gos_trigger.c.

Here is the caller graph for this function:



7.38.2.4 gos_triggerReset()

```
GOS_INLINE gos_result_t gos_triggerReset (
            gos_trigger_t * pTrigger )
```

Resets the trigger counter of the given trigger instance.

Sets the trigger counter of the given trigger instance to zero.

Parameters

out	*pTrigger*	Pointer to the trigger instance.

Returns

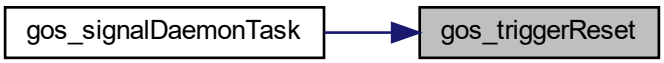
Result of trigger resetting.

Return values

GOS_SUCCESS	Trigger resetting successful.
GOS_ERROR	Trigger is NULL pointer.

Definition at line 107 of file gos_trigger.c.

Here is the caller graph for this function:



7.38.2.5 gos_triggerWait()

```
GOS_INLINE gos_result_t gos_triggerWait (
            gos_trigger_t * pTrigger,
            u32_t value,
            u32_t timeout )
```

Waits for the trigger instance to reach the given trigger value.

Increases the trigger waiter number, and waits in a non-blocking way until the desired trigger value is reached or the timeout is reached.

Parameters

in,out	*pTrigger*	Pointer to the trigger instance.
in	*value*	The desired trigger value.
in	*timeout*	Timeout value.

Returns

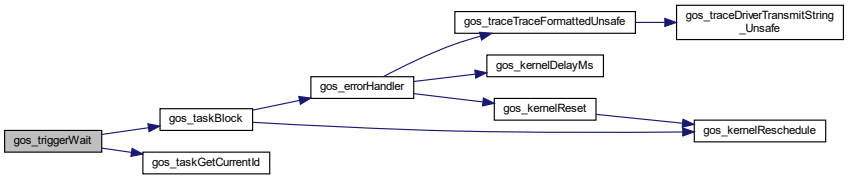
Result of trigger waiting.

Return values

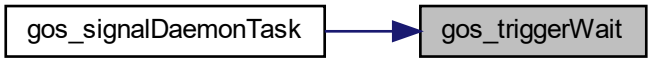
GOS_SUCCESS	Trigger value reached.
GOS_ERROR	Trigger value was not reached within the timeout value.

Definition at line 140 of file gos_trigger.c.

Here is the call graph for this function:



Here is the caller graph for this function:



Index

actualCommand
gos_shell.c, 296
ARM_CORTEX_M4
gos_bootloader_config.h, 108
gos_config.h, 118
atomicCntr
gos_kernel.c, 150

Basic data type definitions to be used in a GOS system, 18
BINARY_PATTERN
gos_kernel.c, 144

callerTaskDesc
gos_signal.c, 303
CFG_GCP_CHANNELS_MAX_NUMBER
gos_bootloader_config.h, 108
gos_config.h, 119
CFG_IDLE_TASK_STACK_SIZE
gos_bootloader_config.h, 108
gos_config.h, 119
CFG_MESSAGE_MAX_ADDRESSEES
gos_config.h, 119
CFG_MESSAGE_MAX_LENGTH
gos_bootloader_config.h, 109
gos_config.h, 119
CFG_MESSAGE_MAX_NUMBER
gos_bootloader_config.h, 109
gos_config.h, 119
CFG_MESSAGE_MAX_WAITER_IDS
gos_bootloader_config.h, 109
gos_config.h, 120
CFG_MESSAGE_MAX_WAITERS
gos_bootloader_config.h, 109
gos_config.h, 120
CFG_QUEUE_MAX_ELEMENTS
gos_bootloader_config.h, 109
gos_config.h, 120
CFG_QUEUE_MAX_LENGTH
gos_bootloader_config.h, 110
gos_config.h, 120
CFG_QUEUE_MAX_NAME_LENGTH
gos_bootloader_config.h, 110
gos_config.h, 120
CFG_QUEUE_MAX_NUMBER
gos_bootloader_config.h, 110
gos_config.h, 121
CFG_QUEUE_USE_NAME
gos_bootloader_config.h, 110
gos_config.h, 121
CFG_RESET_ON_ERROR
gos_bootloader_config.h, 110
gos_config.h, 121
CFG_RESET_ON_ERROR_DELAY_MS
gos_bootloader_config.h, 111
gos_config.h, 121
CFG_SCHED_COOPERATIVE
gos_bootloader_config.h, 111
gos_config.h, 121
CFG_SHELL_COMMAND_BUFFER_SIZE
gos_bootloader_config.h, 111
gos_config.h, 122
CFG_SHELL_MAX_COMMAND_LENGTH
gos_bootloader_config.h, 111
gos_config.h, 122
CFG_SHELL_MAX_COMMAND_NUMBER
gos_bootloader_config.h, 111
gos_config.h, 122
CFG_SHELL_MAX_PARAMS_LENGTH
gos_bootloader_config.h, 112
gos_config.h, 122
CFG_SHELL_STARTUP_DELAY_MS
gos_bootloader_config.h, 112
gos_config.h, 122
CFG_SHELL_USE_SERVICE
gos_bootloader_config.h, 112
gos_config.h, 123
CFG_SIGNAL_MAX_NUMBER
gos_bootloader_config.h, 112
gos_config.h, 123
CFG_SIGNAL_MAX_SUBSCRIBERS
gos_bootloader_config.h, 112
gos_config.h, 123
CFG_SYSTEM_TASK_STACK_SIZE
gos_bootloader_config.h, 113
gos_config.h, 123
CFG_TARGET_CPU
gos_bootloader_config.h, 113
gos_config.h, 123
CFG_TASK_MAX_NAME_LENGTH
gos_bootloader_config.h, 113
gos_config.h, 124
CFG_TASK_MAX_NUMBER
gos_bootloader_config.h, 113
gos_config.h, 124
CFG_TASK_MAX_STACK_SIZE
gos_bootloader_config.h, 113
gos_config.h, 124
CFG_TASK_MESSAGE_DAEMON_PRIO

gos_bootloader_config.h, 114
gos_config.h, 124
CFG_TASK_MESSAGE_DAEMON_STACK
gos_bootloader_config.h, 114
gos_config.h, 124
CFG_TASK_MIN_STACK_SIZE
gos_bootloader_config.h, 114
gos_config.h, 125
CFG_TASK_SHELL_DAEMON_PRIO
gos_bootloader_config.h, 114
gos_config.h, 125
CFG_TASK_SHELL_DAEMON_STACK
gos_bootloader_config.h, 114
gos_config.h, 125
CFG_TASK_SIGNAL_DAEMON_PRIO
gos_bootloader_config.h, 115
gos_config.h, 125
CFG_TASK_SIGNAL_DAEMON_STACK
gos_bootloader_config.h, 115
gos_config.h, 125
CFG_TASK_SYS_PRIO
gos_bootloader_config.h, 115
gos_config.h, 126
CFG_TASK_TIME_DAEMON_PRIO
gos_bootloader_config.h, 115
gos_config.h, 126
CFG_TASK_TIME_DAEMON_STACK
gos_bootloader_config.h, 115
gos_config.h, 126
CFG_TASK_TRACE_DAEMON_PRIO
gos_bootloader_config.h, 116
gos_config.h, 126
CFG_TASK_TRACE_DAEMON_STACK
gos_bootloader_config.h, 116
gos_config.h, 126
CFG_TRACE_MAX_LENGTH
gos_bootloader_config.h, 116
gos_config.h, 127
CFG_USE_PRIO_INHERITANCE
gos_bootloader_config.h, 116
gos_config.h, 127
channelFunctions
gos_gcp.c, 261
commandBuffer
gos_shell.c, 296
commandBufferIndex
gos_shell.c, 296
commandParams
gos_shell.c, 296
CONFIG_DUMP_SEPARATOR
gos_kernel.c, 144
Control macros for scheduling and ISR state setting, 14
GOS_ATOMIC_ENTER, 14
GOS_ATOMIC_EXIT, 14
GOS_DISABLE_SCHED, 14
GOS_ENABLE_SCHED, 15
GOS_ISR_ENTER, 15
GOS_ISR_EXIT, 15
cpuUseLimit
gos_kernel.c, 150
CRC_INITIAL_VALUE
gos_crc_driver.c, 95
CRC_POLYNOMIAL_VALUE
gos_crc_driver.c, 95
currentTaskIndex
gos_kernel.c, 151

dayLookupTable
gos_time.c, 316
DUMP_SEPARATOR
gos_queue.c, 276
dumpRequired
gos.c, 242

ERROR_BUFFER_SIZE
gos_error.c, 245
errorBuffer
gos_error.c, 250

formattedBuffer
gos_shell_driver.c, 101
gos_trace.c, 324

GCP_ACK_CRC_ERROR
gos_gcp.c, 253
GCP_ACK_INVALID
gos_gcp.c, 253
GCP_ACK_OK
gos_gcp.c, 253
GCP_ACK_PV_ERROR
gos_gcp.c, 253
GCP_ACK_REQ
gos_gcp.c, 253
GCP_ACK_RESEND
gos_gcp.c, 253
GCP_ACK_SIZE_ERROR
gos_gcp.c, 253
GCP_PROTOCOL_VERSION_MAJOR
gos_gcp.c, 253
GCP_PROTOCOL_VERSION_MINOR
gos_gcp.c, 253
gcpRxMutexes
gos_gcp.c, 261
gcpTxMutexes
gos_gcp.c, 261
Global definitions, 9
GLOBAL_STACK, 9
GOS_ASM, 10
GOS_CONST, 10
GOS_DEFAULT_TASK_ID, 10
GOS_EXTERN, 10
GOS_INLINE, 10
GOS_INVALID_TASK_ID, 11
GOS_NAKED, 11
GOS_NOP, 11
GOS_STATIC, 11
GOS_STATIC_INLINE, 11

GOS_TASK_IDLE_PRIO, 12
GOS_TASK_MAX_BLOCK_TIME_MS, 12
GOS_TASK_MAX_PRIO_LEVELS, 12
GOS_UNUSED, 12
GOS_UNUSED_PAR, 12
MAIN_STACK, 13
NULL, 13
RAM_SIZE, 13
RAM_START, 13
GLOBAL_STACK
Global definitions, 9
gos.c
dumpRequired, 242
gos_Dump, 240
gos_initFunc_t, 240
gos_Start, 241
GOS_SYS_TASK_SLEEP_TIME, 239
gos_systemTask, 241
initError, 242
initializers, 243
systemTaskDesc, 243
systemTaskId, 243
gos.h
gos_Dump, 162
GOS_VERSION_MAJOR, 162
GOS_VERSION_MINOR, 162
GOS2022/OS/driver/inc/gos_crc_driver.h, 79
GOS2022/OS/driver/inc/gos_driver.h, 81
GOS2022/OS/driver/inc/gos_shell_driver.h, 83
GOS2022/OS/driver/inc/gos_timer_driver.h, 87
GOS2022/OS/driver/inc/gos_trace_driver.h, 90
GOS2022/OS/driver/src/gos_crc_driver.c, 93
GOS2022/OS/driver/src/gos_driver.c, 96
GOS2022/OS/driver/src/gos_shell_driver.c, 98
GOS2022/OS/driver/src/gos_timer_driver.c, 101
GOS2022/OS/driver/src/gos_trace_driver.c, 103
GOS2022/OS/kernel/inc/gos_bootloader_config.h, 106
GOS2022/OS/kernel/inc/gos_config.h, 117
GOS2022/OS/kernel/inc/gos_kernel.h, 127
GOS2022/OS/kernel/inc/gos_port.h, 136
GOS2022/OS/kernel/src/gos_kernel.c, 142
GOS2022/OS/kernel/src/gos_task.c, 154
GOS2022/OS/services/inc/gos.h, 160
GOS2022/OS/services/inc/gos_error.h, 163
GOS2022/OS/services/inc/gos_gcp.h, 169
GOS2022/OS/services/inc/gos_message.h, 174
GOS2022/OS/services/inc/gos_mutex.h, 180
GOS2022/OS/services/inc/gos_queue.h, 186
GOS2022/OS/services/inc/gos_shell.h, 199
GOS2022/OS/services/inc/gos_signal.h, 207
GOS2022/OS/services/inc/gos_time.h, 213
GOS2022/OS/services/inc/gos_trace.h, 224
GOS2022/OS/services/inc/gos_trigger.h, 231
GOS2022/OS/services/src/gos.c, 238
GOS2022/OS/services/src/gos_error.c, 244
GOS2022/OS/services/src/gos_gcp.c, 251
GOS2022/OS/services/src/gos_message.c, 262
GOS2022/OS/services/src/gos_mutex.c, 268
GOS2022/OS/services/src/gos_queue.c, 274
GOS2022/OS/services/src/gos_shell.c, 287
GOS2022/OS/services/src/gos_signal.c, 298
GOS2022/OS/services/src/gos_time.c, 304
GOS2022/OS/services/src/gos_trace.c, 318
GOS2022/OS/services/src/gos_trigger.c, 326
GOS_ASM
Global definitions, 10
GOS_ATOMIC_ENTER
Control macros for scheduling and ISR state setting, 14
GOS_ATOMIC_EXIT
Control macros for scheduling and ISR state setting, 14
gos_boolValue_t
gos_kernel.h, 134
gos_bootloader_config.h
ARM_CORTEX_M4, 108
CFG_GCP_CHANNELS_MAX_NUMBER, 108
CFG_IDLE_TASK_STACK_SIZE, 108
CFG_MESSAGE_MAX_LENGTH, 109
CFG_MESSAGE_MAX_NUMBER, 109
CFG_MESSAGE_MAX_WAITER_IDS, 109
CFG_MESSAGE_MAX_WAITERS, 109
CFG_QUEUE_MAX_ELEMENTS, 109
CFG_QUEUE_MAX_LENGTH, 110
CFG_QUEUE_MAX_NAME_LENGTH, 110
CFG_QUEUE_MAX_NUMBER, 110
CFG_QUEUE_USE_NAME, 110
CFG_RESET_ON_ERROR, 110
CFG_RESET_ON_ERROR_DELAY_MS, 111
CFG_SCHED_COOPERATIVE, 111
CFG_SHELL_COMMAND_BUFFER_SIZE, 111
CFG_SHELL_MAX_COMMAND_LENGTH, 111
CFG_SHELL_MAX_COMMAND_NUMBER, 111
CFG_SHELL_MAX_PARAMS_LENGTH, 112
CFG_SHELL_STARTUP_DELAY_MS, 112
CFG_SHELL_USE_SERVICE, 112
CFG_SIGNAL_MAX_NUMBER, 112
CFG_SIGNAL_MAX_SUBSCRIBERS, 112
CFG_SYSTEM_TASK_STACK_SIZE, 113
CFG_TARGET_CPU, 113
CFG_TASK_MAX_NAME_LENGTH, 113
CFG_TASK_MAX_NUMBER, 113
CFG_TASK_MAX_STACK_SIZE, 113
CFG_TASK_MESSAGE_DAEMON_PRIO, 114
CFG_TASK_MESSAGE_DAEMON_STACK, 114
CFG_TASK_MIN_STACK_SIZE, 114
CFG_TASK_SHELL_DAEMON_PRIO, 114
CFG_TASK_SHELL_DAEMON_STACK, 114
CFG_TASK_SIGNAL_DAEMON_PRIO, 115
CFG_TASK_SIGNAL_DAEMON_STACK, 115
CFG_TASK_SYS_PRIO, 115
CFG_TASK_TIME_DAEMON_PRIO, 115
CFG_TASK_TIME_DAEMON_STACK, 115
CFG_TASK_TRACE_DAEMON_PRIO, 116
CFG_TASK_TRACE_DAEMON_STACK, 116
CFG_TRACE_MAX_LENGTH, 116

CFG_USE_PRIO_INHERITANCE, 116
GOS_BUSY
gos_kernel.h, 135
GOS_CONCAT_RESULT
gos_kernel.h, 133
gos_config.h
ARM_CORTEX_M4, 118
CFG_GCP_CHANNELS_MAX_NUMBER, 119
CFG_IDLE_TASK_STACK_SIZE, 119
CFG_MESSAGE_MAX_ADDRESSEES, 119
CFG_MESSAGE_MAX_LENGTH, 119
CFG_MESSAGE_MAX_NUMBER, 119
CFG_MESSAGE_MAX_WAITER_IDS, 120
CFG_MESSAGE_MAX_WAITERS, 120
CFG_QUEUE_MAX_ELEMENTS, 120
CFG_QUEUE_MAX_LENGTH, 120
CFG_QUEUE_MAX_NAME_LENGTH, 120
CFG_QUEUE_MAX_NUMBER, 121
CFG_QUEUE_USE_NAME, 121
CFG_RESET_ON_ERROR, 121
CFG_RESET_ON_ERROR_DELAY_MS, 121
CFG_SCHED_COOPERATIVE, 121
CFG_SHELL_COMMAND_BUFFER_SIZE, 122
CFG_SHELL_MAX_COMMAND_LENGTH, 122
CFG_SHELL_MAX_COMMAND_NUMBER, 122
CFG_SHELL_MAX_PARAMS_LENGTH, 122
CFG_SHELL_STARTUP_DELAY_MS, 122
CFG_SHELL_USE_SERVICE, 123
CFG_SIGNAL_MAX_NUMBER, 123
CFG_SIGNAL_MAX_SUBSCRIBERS, 123
CFG_SYSTEM_TASK_STACK_SIZE, 123
CFG_TARGET_CPU, 123
CFG_TASK_MAX_NAME_LENGTH, 124
CFG_TASK_MAX_NUMBER, 124
CFG_TASK_MAX_STACK_SIZE, 124
CFG_TASK_MESSAGE_DAEMON_PRIO, 124
CFG_TASK_MESSAGE_DAEMON_STACK, 124
CFG_TASK_MIN_STACK_SIZE, 125
CFG_TASK_SHELL_DAEMON_PRIO, 125
CFG_TASK_SHELL_DAEMON_STACK, 125
CFG_TASK_SIGNAL_DAEMON_PRIO, 125
CFG_TASK_SIGNAL_DAEMON_STACK, 125
CFG_TASK_SYS_PRIO, 126
CFG_TASK_TIME_DAEMON_PRIO, 126
CFG_TASK_TIME_DAEMON_STACK, 126
CFG_TASK_TRACE_DAEMON_PRIO, 126
CFG_TASK_TRACE_DAEMON_STACK, 126
CFG_TRACE_MAX_LENGTH, 127
CFG_USE_PRIO_INHERITANCE, 127
GOS_CONST
Global definitions, 10
GOS_CONVERT_RESULT
gos_kernel.h, 133
gos_crc_driver.c
CRC_INITIAL_VALUE, 95
CRC_POLYNOMIAL_VALUE, 95
gos_crcDriverGetCrc, 95
gos_crc_driver.h
gos_crcDriverGetCrc, 80
gos_crcDriverGetCrc
gos_crc_driver.c, 95
gos_crc_driver.h, 80
GOS_DEFAULT_QUEUE_ID
gos_queue.h, 189
GOS_DEFAULT_TASK_ID
Global definitions, 10
GOS_DISABLE_SCHED
Control macros for scheduling and ISR state setting, 14
gos_driver.c
gos_driverInit, 97
gos_driver.h
gos_driverInit, 82
gos_driver_functions_t, 67
gos_driverInit
gos_driver.c, 97
gos_driver.h, 82
gos_Dump
gos.c, 240
gos.h, 162
GOS_ENABLE_SCHED
Control macros for scheduling and ISR state setting, 15
GOS_ERROR
gos_kernel.h, 135
gos_error.c
ERROR_BUFFER_SIZE, 245
errorBuffer, 250
gos_errorHandler, 246
gos_errorTraceInit, 248
gos_printStartupLogo, 249
gos_traceResultToString, 250
RESULT_STRING_ERROR, 245
RESULT_STRING_SUCCESS, 246
RESULT_STRING_UNKNOWN, 246
SEPARATOR_LINE, 246
gos_error.h
GOS_ERROR_LEVEL_OS_FATAL, 165
GOS_ERROR_LEVEL_OS_WARNING, 165
GOS_ERROR_LEVEL_USER_FATAL, 165
GOS_ERROR_LEVEL_USER_WARNING, 165
gos_errorHandler, 166
gos_errorLevel_t, 165
gos_errorTraceInit, 167
gos_printStartupLogo, 168
GOS_ERROR_LEVEL_OS_FATAL
gos_error.h, 165
GOS_ERROR_LEVEL_OS_WARNING
gos_error.h, 165
GOS_ERROR_LEVEL_USER_FATAL
gos_error.h, 165
GOS_ERROR_LEVEL_USER_WARNING
gos_error.h, 165
gos_errorHandler
gos_error.c, 246
gos_error.h, 166

gos_errorLevel_t
gos_error.h, 165
gos_errorTraceInit
gos_error.c, 248
gos_error.h, 167
GOS_EXTERN
Global definitions, 10
GOS_FALSE
gos_kernel.h, 134
gos_gcp.c
channelFunctions, 261
GCP_ACK_CRC_ERROR, 253
GCP_ACK_INVALID, 253
GCP_ACK_OK, 253
GCP_ACK_PV_ERROR, 253
GCP_ACK_REQ, 253
GCP_ACK_RESEND, 253
GCP_ACK_SIZE_ERROR, 253
GCP_PROTOCOL_VERSION_MAJOR, 253
GCP_PROTOCOL_VERSION_MINOR, 253
gcpRxMutexes, 261
gcpTxMutexes, 261
gos_gcpAck_t, 253
gos_gcpInit, 253
gos_gcpReceiveMessage, 254
gos_gcpReceiveMessageInternal, 255
gos_gcpRegisterPhysicalDriver, 256
gos_gcpTransmitMessage, 257
gos_gcpTransmitMessageInternal, 258
gos_gcpValidateData, 259
gos_gcpValidateHeader, 260
gos_gcp.h
gos_gcpInit, 171
gos_gcpReceiveFunction_t, 170
gos_gcpReceiveMessage, 171
gos_gcpRegisterPhysicalDriver, 172
gos_gcpTransmitFunction_t, 170
gos_gcpTransmitMessage, 173
gos_gcpAck_t
gos_gcp.c, 253
gos_gcpChannelFunctions_t, 67
gos_gcpHeaderFrame_t, 68
gos_gcpInit
gos_gcp.c, 253
gos_gcp.h, 171
gos_gcpReceiveFunction_t
gos_gcp.h, 170
gos_gcpReceiveMessage
gos_gcp.c, 254
gos_gcp.h, 171
gos_gcpReceiveMessageInternal
gos_gcp.c, 255
gos_gcpRegisterPhysicalDriver
gos_gcp.c, 256
gos_gcp.h, 172
gos_gcpTransmitFunction_t
gos_gcp.h, 170
gos_gcpTransmitMessage
gos_gcp.c, 257
gos_gcp.h, 173
gos_gcpTransmitMessageInternal
gos_gcp.c, 258
gos_gcpValidateData
gos_gcp.c, 259
gos_gcpValidateHeader
gos_gcp.c, 260
gos_idleTask
gos_kernel.c, 146
gos_task.c, 157
gos_initFunc_t
gos.c, 240
gos_initStruct_t, 68
GOS_INLINE
Global definitions, 10
GOS_INVALID_QUEUE_ID
gos_queue.h, 189
GOS_INVALID_TASK_ID
Global definitions, 11
GOS_ISR_ENTER
Control macros for scheduling and ISR state setting, 15
GOS_ISR_EXIT
Control macros for scheduling and ISR state setting, 15
gos_kernel.c
atomicCntr, 150
BINARY_PATTERN, 144
CONFIG_DUMP_SEPARATOR, 144
cpuUseLimit, 150
currentTaskIndex, 151
gos_idleTask, 146
gos_kernelCheckTaskStack, 147
gos_kernelGetCurrentPsp, 148
gos_kernelGetTaskStateString, 148
gos_kernelProcessorReset, 149
gos_kernelSaveCurrentPsp, 149
gos_kernelSelectNextTask, 150
ICSR, 145
inIsr, 151
isKernelRunning, 151
kernelDumpReadySignal, 151
kernelDumpSignal, 151
kernelPreResetHookFunction, 152
kernelPrivilegedHookFunction, 152
kernelSwapHookFunction, 152
kernelSysTickHookFunction, 152
MAX_CPU_DUMP_SEPARATOR, 145
monitoringTime, 152
previousTick, 153
primask, 153
privilegedModeSetRequired, 153
resetRequired, 153
schedDisableCntr, 153
SHCSR, 145
STACK_STATS_SEPARATOR, 145
sysTicks, 154

sysTimerValue, 154
TASK_DUMP_SEPARATOR, 145
TO_BINARY, 146
gos_kernel.h
gos_boolValue_t, 134
GOS_BUSY, 135
GOS_CONCAT_RESULT, 133
GOS_CONVERT_RESULT, 133
GOS_ERROR, 135
GOS_FALSE, 134
gos_kernel_privilege_t, 134
GOS_PRIVILEGED, 134
gos_result_t, 134
GOS_SUCCESS, 135
GOS_TASK_BLOCKED, 136
GOS_TASK_PRIVILEGE_KERNEL, 135
GOS_TASK_PRIVILEGE_SUPERVISOR, 135
GOS_TASK_PRIVILEGE_USER, 135
GOS_TASK_PRIVILEGED_USER, 135
GOS_TASK_READY, 136
GOS_TASK_SLEEPING, 136
GOS_TASK_SUSPENDED, 136
GOS_TASK_ZOMBIE, 136
gos_taskPrivilegeLevel_t, 135
gos_taskState_t, 135
GOS_TRUE, 134
GOS_UNPRIVILEGED, 134
gos_kernel_privilege_t
gos_kernel.h, 134
gos_kernelCalculateTaskCpuUsages
Kernel functions, 23
gos_kernelCheckTaskStack
gos_kernel.c, 147
gos_kernelDelayMs
Kernel functions, 23
gos_kernelDelayUs
Kernel functions, 24
gos_kernelDump
Kernel functions, 25
gos_kernelGetCpuUsage
Kernel functions, 26
gos_kernelGetCurrentPsp
gos_kernel.c, 148
gos_kernelGetMaxCpuLoad
Kernel functions, 27
gos_kernelGetSysTicks
Kernel functions, 27
gos_kernelGetTaskStateString
gos_kernel.c, 148
gos_kernelInit
Kernel functions, 28
gos_kernelIsCallerIsr
Kernel functions, 29
gos_kernelPrivilegedModeSetRequired
Kernel functions, 30
gos_kernelProcessorReset
gos_kernel.c, 149
gos_kernelRegisterIdleHook
Kernel functions, 30
gos_kernelRegisterPreResetHook
Kernel functions, 31
gos_kernelRegisterPrivilegedHook
Kernel functions, 31
gos_kernelRegisterSwapHook
Kernel functions, 32
gos_kernelRegisterSysTickHook
Kernel functions, 32
gos_kernelReschedule
Kernel functions, 33
gos_kernelReset
Kernel functions, 34
gos_kernelSaveCurrentPsp
gos_kernel.c, 149
gos_kernelSelectNextTask
gos_kernel.c, 150
gos_kernelSetMaxCpuLoad
Kernel functions, 35
gos_kernelStart
Kernel functions, 36
gos_kernelSubscribeDumpReadySignal
Kernel functions, 37
gos_message.c
GOS_MESSAGE_DAEMON_POLL_TIME_MS, 263
gos_messageDaemonTask, 264
gos_messageInit, 264
gos_messageRx, 265
gos_messageTx, 266
messageArray, 267
messageDaemonTaskDesc, 267
messageDaemonTaskId, 267
messageMutex, 267
messageWaiterArray, 267
nextMessageIndex, 268
nextWaiterIndex, 268
gos_message.h
GOS_MESSAGE_ENDLESS_TMO, 176
GOS_MESSAGE_INVALID_ID, 176
gos_messageId_t, 177
gos_messageInit, 177
gos_messageRx, 178
gos_messageTimeout_t, 177
gos_messageTx, 179
GOS_MESSAGE_DAEMON_POLL_TIME_MS
gos_message.c, 263
GOS_MESSAGE_ENDLESS_TMO
gos_message.h, 176
GOS_MESSAGE_INVALID_ID
gos_message.h, 176
gos_message_t, 69
gos_messageDaemonTask
gos_message.c, 264
gos_messageId_t
gos_message.h, 177
gos_messageInit
gos_message.c, 264

gos_message.h, 177
gos_messageRx
gos_message.c, 265
gos_message.h, 178
gos_messageTimeout_t
gos_message.h, 177
gos_messageTx
gos_message.c, 266
gos_message.h, 179
gos_messageWaiterDesc_t, 69
gos_mutex.c
gos_mutexInit, 270
gos_mutexLock, 271
gos_mutexUnlock, 272
MUTEX_LOCK_SLEEP_MS, 270
gos_mutex.h
GOS_MUTEX_ENDLESS_TMO, 181
GOS_MUTEX_LOCKED, 182
GOS_MUTEX_NO_TMO, 181
GOS_MUTEX_UNLOCKED, 182
gos_mutexInit, 182
gos_mutexLock, 183
gos_mutexState_t, 182
gos_mutexUnlock, 184
GOS_MUTEX_ENDLESS_TMO
gos_mutex.h, 181
GOS_MUTEX_LOCKED
gos_mutex.h, 182
GOS_MUTEX_NO_TMO
gos_mutex.h, 181
gos_mutex_t, 70
GOS_MUTEX_UNLOCKED
gos_mutex.h, 182
gos_mutexInit
gos_mutex.c, 270
gos_mutex.h, 182
gos_mutexLock
gos_mutex.c, 271
gos_mutex.h, 183
gos_mutexState_t
gos_mutex.h, 182
gos_mutexUnlock
gos_mutex.c, 272
gos_mutex.h, 184
GOS_NAKED
Global definitions, 11
GOS_NOP
Global definitions, 11
gos_platformDriverInit
Platform initializer weak functions, 64
gos_port.h
gos_ported_doContextSwitch, 138
gos_ported_enableFaultHandlers, 138
gos_ported_handleSVC, 138
gos_ported_handleSVCMain, 139
gos_ported_kernelStartInit, 139
gos_ported_pendSVHandler, 140
gos_ported_procReset, 140
gos_ported_reschedule, 140
gos_ported_svcHandler, 141
gos_ported_svcHandlerMain, 141
gos_ported_sysTickInterrupt, 141
gos_ported_doContextSwitch
gos_port.h, 138
gos_ported_enableFaultHandlers
gos_port.h, 138
gos_ported_handleSVC
gos_port.h, 138
gos_ported_handleSVCMain
gos_port.h, 139
gos_ported_kernelStartInit
gos_port.h, 139
gos_ported_pendSVHandler
gos_port.h, 140
gos_ported_procReset
gos_port.h, 140
gos_ported_reschedule
gos_port.h, 140
gos_ported_svcHandler
gos_port.h, 141
gos_ported_svcHandlerMain
gos_port.h, 141
gos_ported_sysTickInterrupt
gos_port.h, 141
gos_printStartupLogo
gos_error.c, 249
gos_error.h, 168
GOS_PRIV_RESERVED_3
Privilege bit definitions, 16
GOS_PRIV_RESERVED_4
Privilege bit definitions, 16
GOS_PRIV_RESERVED_5
Privilege bit definitions, 16
GOS_PRIV_SIGNALING
Privilege bit definitions, 16
GOS_PRIV_TASK_MANIPULATE
Privilege bit definitions, 17
GOS_PRIV_TASK_PRIO_CHANGE
Privilege bit definitions, 17
GOS_PRIV_TRACE
Privilege bit definitions, 17
GOS_PRIVILEGED
gos_kernel.h, 134
gos_queue.c
DUMP_SEPARATOR, 276
gos_queueCreate, 277
gos_queueDump, 278
gos_queueGet, 279
gos_queueGetElementNumber, 280
gos_queueGetName, 281
gos_queueInit, 281
gos_queuePeek, 282
gos_queuePut, 283
gos_queueRegisterEmptyHook, 284
gos_queueRegisterFullHook, 285
gos_queueReset, 285

queueEmptyHook, 286
queueFullHook, 286
queueMutex, 286
queues, 286
readCounters, 287
writeCounters, 287
gos_queue.h
GOS_DEFAULT_QUEUE_ID, 189
GOS_INVALID_QUEUE_ID, 189
gos_queueByte_t, 189
gos_queueCreate, 190
gos_queueDump, 191
gos_queueEmptyHook, 189
gos_queueFullHook, 190
gos_queueGet, 191
gos_queueGetElementNumber, 193
gos_queueGetName, 193
gos_queueInit, 194
gos_queuePeek, 194
gos_queuePut, 195
gos_queueRegisterEmptyHook, 196
gos_queueRegisterFullHook, 197
gos_queueReset, 197
gos_queue_t, 71
gos_queueByte_t
gos_queue.h, 189
gos_queueCreate
gos_queue.c, 277
gos_queue.h, 190
gos_queueDescriptor_t, 71
gos_queueDump
gos_queue.c, 278
gos_queue.h, 191
gos_queueElement_t, 72
gos_queueEmptyHook
gos_queue.h, 189
gos_queueFullHook
gos_queue.h, 190
gos_queueGet
gos_queue.c, 279
gos_queue.h, 191
gos_queueGetElementNumber
gos_queue.c, 280
gos_queue.h, 193
gos_queueGetName
gos_queue.c, 281
gos_queue.h, 193
gos_queueInit
gos_queue.c, 281
gos_queue.h, 194
gos_queuePeek
gos_queue.c, 282
gos_queue.h, 194
gos_queuePut
gos_queue.c, 283
gos_queue.h, 195
gos_queueRegisterEmptyHook
gos_queue.c, 284
gos_queue.h, 196
gos_queueRegisterFullHook
gos_queue.c, 285
gos_queue.h, 197
gos_queueReset
gos_queue.c, 285
gos_queue.h, 197
gos_result_t
gos_kernel.h, 134
gos_runtime_t, 72
gos_runTimeAddMicroseconds
gos_time.c, 307
gos_time.h, 216
gos_runTimeAddMilliseconds
gos_time.c, 308
gos_time.h, 217
gos_runTimeAddSeconds
gos_time.c, 309
gos_time.h, 218
gos_runTimeGet
gos_time.c, 309
gos_time.h, 218
gos_shell.c
actualCommand, 296
commandBuffer, 296
commandBufferIndex, 296
commandParams, 296
GOS_SHELL_DAEMON_POLL_TIME_MS, 289
GOS_SHELL_DISPLAY_TEXT, 289
gos_shellCommandHandler, 289
gos_shellDaemonTask, 290
gos_shellEchoOff, 291
gos_shellEchoOn, 291
gos_shellInit, 292
gos_shellRegisterCommand, 292
gos_shellRegisterCommands, 293
gos_shellResume, 294
gos_shellSuspend, 295
shellCommand, 296
shellCommands, 297
shellDaemonTaskDesc, 297
shellDaemonTaskId, 297
useEcho, 297
gos_shell.h
gos_shellEchoOff, 202
gos_shellEchoOn, 202
gos_shellFunction, 201
gos_shellInit, 203
gos_shellRegisterCommand, 203
gos_shellRegisterCommands, 204
gos_shellResume, 205
gos_shellSuspend, 206
GOS_SHELL_DAEMON_POLL_TIME_MS
gos_shell.c, 289
GOS_SHELL_DISPLAY_TEXT
gos_shell.c, 289
gos_shell_driver.c
formattedBuffer, 101

gos_shellDriverReceiveChar, 99
gos_shellDriverTransmitString, 100
gos_shell_driver.h
gos_shellDriverReceiveChar, 85
gos_shellDriverReceiveChar_t, 84
gos_shellDriverTransmitString, 86
gos_shellDriverTransmitString_t, 85
gos_shellCommand_t, 73
gos_shellCommandHandler
gos_shell.c, 289
gos_shellDaemonTask
gos_shell.c, 290
gos_shellDriverReceiveChar
gos_shell_driver.c, 99
gos_shell_driver.h, 85
gos_shellDriverReceiveChar_t
gos_shell_driver.h, 84
gos_shellDriverTransmitString
gos_shell_driver.c, 100
gos_shell_driver.h, 86
gos_shellDriverTransmitString_t
gos_shell_driver.h, 85
gos_shellEchoOff
gos_shell.c, 291
gos_shell.h, 202
gos_shellEchoOn
gos_shell.c, 291
gos_shell.h, 202
gos_shellFunction
gos_shell.h, 201
gos_shellInit
gos_shell.c, 292
gos_shell.h, 203
gos_shellRegisterCommand
gos_shell.c, 292
gos_shell.h, 203
gos_shellRegisterCommands
gos_shell.c, 293
gos_shell.h, 204
gos_shellResume
gos_shell.c, 294
gos_shell.h, 205
gos_shellSuspend
gos_shell.c, 295
gos_shell.h, 206
gos_signal.c
callerTaskDesc, 303
GOS_SIGNAL_DAEMON_POLL_TIME_MS, 299
gos_signalCreate, 300
gos_signalDaemonTask, 301
gos_signalInit, 301
gos_signalInvoke, 301
gos_signalSubscribe, 302
signalArray, 303
signalDaemonTaskDescriptor, 303
signalInvokeTrigger, 304
gos_signal.h
gos_signalCreate, 208
gos_signalInit, 210
gos_signalInvoke, 211
gos_signalSubscribe, 212
GOS_SIGNAL_DAEMON_POLL_TIME_MS
gos_signal.c, 299
gos_signalCreate
gos_signal.c, 300
gos_signal.h, 208
gos_signalDaemonTask
gos_signal.c, 301
gos_signalDescriptor_t, 73
gos_signalInit
gos_signal.c, 301
gos_signal.h, 210
gos_signalInvoke
gos_signal.c, 301
gos_signal.h, 211
gos_signalInvokeDescriptor, 74
gos_signalSubscribe
gos_signal.c, 302
gos_signal.h, 212
gos_Start
gos.c, 241
GOS_STATIC
Global definitions, 11
GOS_STATIC_INLINE
Global definitions, 11
GOS_SUCCESS
gos_kernel.h, 135
GOS_SYS_TASK_SLEEP_TIME
gos.c, 239
gos_systemTask
gos.c, 241
gos_task.c
gos_idleTask, 157
gos_taskCheckDescriptor, 158
kernelIdleHookFunction, 159
kernelTaskDeleteSignal, 159
taskDescriptors, 160
GOS_TASK_BLOCKED
gos_kernel.h, 136
GOS_TASK_IDLE_PRIO
Global definitions, 12
GOS_TASK_MAX_BLOCK_TIME_MS
Global definitions, 12
GOS_TASK_MAX_PRIO_LEVELS
Global definitions, 12
GOS_TASK_PRIVILEGE_KERNEL
gos_kernel.h, 135
GOS_TASK_PRIVILEGE_SUPERVISOR
gos_kernel.h, 135
GOS_TASK_PRIVILEGE_USER
gos_kernel.h, 135
GOS_TASK_PRIVILEGED_USER
gos_kernel.h, 135
GOS_TASK_READY
gos_kernel.h, 136
GOS_TASK_SLEEPING

gos_kernel.h, 136
GOS_TASK_SUSPENDED
gos_kernel.h, 136
GOS_TASK_ZOMBIE
gos_kernel.h, 136
gos_taskAddPrivilege
Task-related functions, 40
gos_taskBlock
Task-related functions, 40
gos_taskCheckDescriptor
gos_task.c, 158
gos_taskDelete
Task-related functions, 41
gos_taskDescriptor_t, 74
gos_taskGetCurrentId
Task-related functions, 42
gos_taskGetData
Task-related functions, 44
gos_taskGetDataByIndex
Task-related functions, 45
gos_taskGetId
Task-related functions, 46
gos_taskGetName
Task-related functions, 47
gos_taskGetNumber
Task-related functions, 47
gos_taskGetOriginalPriority
Task-related functions, 48
gos_taskGetPriority
Task-related functions, 48
gos_taskGetPrivileges
Task-related functions, 49
gos_taskPrivilegeLevel_t
gos_kernel.h, 135
gos_taskRegister
Task-related functions, 50
gos_taskRegisterTasks
Task-related functions, 51
gos_taskRemovePrivilege
Task-related functions, 52
gos_taskResume
Task-related functions, 53
gos_taskSetOriginalPriority
Task-related functions, 54
gos_taskSetPriority
Task-related functions, 55
gos_taskSetPrivileges
Task-related functions, 57
gos_taskSleep
Task-related functions, 57
gos_taskState_t
gos_kernel.h, 135
gos_taskSubscribeDeleteSignal
Kernel functions, 37
gos_taskSuspend
Task-related functions, 59
gos_taskUnblock
Task-related functions, 60
gos_taskWakeup
Task-related functions, 61
gos_taskYield
Task-related functions, 62
gos_time.c
dayLookupTable, 316
gos_runTimeAddMicroseconds, 307
gos_runTimeAddMilliseconds, 308
gos_runTimeAddSeconds, 309
gos_runTimeGet, 309
gos_timeAddMilliseconds, 310
gos_timeAddSeconds, 311
gos_timeCompare, 312
gos_timeDaemonTask, 312
gos_timeGet, 313
gos_timeIncreaseSystemTime, 314
gos_timeInit, 315
gos_timeSet, 315
systemRunTime, 316
systemTime, 317
TIME_DEFAULT_DAY, 306
TIME_DEFAULT_MONTH, 307
TIME_DEFAULT_YEAR, 307
TIME_SLEEP_TIME_MS, 307
timeDaemonTaskDesc, 317
timeDaemonTaskId, 317
timeSignalId, 317
gos_time.h
gos_runTimeAddMicroseconds, 216
gos_runTimeAddMilliseconds, 217
gos_runTimeAddSeconds, 218
gos_runTimeGet, 218
GOS_TIME_APRIL, 216
GOS_TIME_AUGUST, 216
GOS_TIME_DAY_ELAPSED_SENDER_ID, 215
GOS_TIME_DECEMBER, 216
GOS_TIME_EARLIER, 215
GOS_TIME_EQUAL, 215
GOS_TIME_FEBRUARY, 216
GOS_TIME_HOUR_ELAPSED_SENDER_ID, 215
GOS_TIME_JANUARY, 215
GOS_TIME_JULY, 216
GOS_TIME_JUNE, 216
GOS_TIME_LATER, 215
GOS_TIME_MARCH, 216
GOS_TIME_MAY, 216
GOS_TIME_MINUTE_ELAPSED_SENDER_ID, 215
GOS_TIME_MONTH_ELAPSED_SENDER_ID, 215
GOS_TIME_NOVEMBER, 216
GOS_TIME_NUMBER_OF_MONTHS, 216
GOS_TIME_OCTOBER, 216
GOS_TIME_SECOND_ELAPSED_SENDER_ID, 215
GOS_TIME_SEPTEMBER, 216
GOS_TIME_YEAR_ELAPSED_SENDER_ID, 215
gos_timeAddMilliseconds, 219

gos_timeAddSeconds, 220
gos_timeCompare, 220
gos_timeComprareResult_t, 215
gos_timeElapsedSenderId_t, 215
gos_timeGet, 221
gos_timeIncreaseSystemTime, 222
gos_timeInit, 223
gos_timeMonthEnum_t, 215
gos_timeSet, 223
GOS_TIME_APRIL
gos_time.h, 216
GOS_TIME_AUGUST
gos_time.h, 216
GOS_TIME_DAY_ELAPSED_SENDER_ID
gos_time.h, 215
GOS_TIME_DECEMBER
gos_time.h, 216
GOS_TIME_EARLIER
gos_time.h, 215
GOS_TIME_EQUAL
gos_time.h, 215
GOS_TIME_FEBRUARY
gos_time.h, 216
GOS_TIME_HOUR_ELAPSED_SENDER_ID
gos_time.h, 215
GOS_TIME_JANUARY
gos_time.h, 215
GOS_TIME_JULY
gos_time.h, 216
GOS_TIME_JUNE
gos_time.h, 216
GOS_TIME_LATER
gos_time.h, 215
GOS_TIME_MARCH
gos_time.h, 216
GOS_TIME_MAY
gos_time.h, 216
GOS_TIME_MINUTE_ELAPSED_SENDER_ID
gos_time.h, 215
GOS_TIME_MONTH_ELAPSED_SENDER_ID
gos_time.h, 215
GOS_TIME_NOVEMBER
gos_time.h, 216
GOS_TIME_NUMBER_OF_MONTHS
gos_time.h, 216
GOS_TIME_OCTOBER
gos_time.h, 216
GOS_TIME_SECOND_ELAPSED_SENDER_ID
gos_time.h, 215
GOS_TIME_SEPTEMBER
gos_time.h, 216
gos_time_t, 76
GOS_TIME_YEAR_ELAPSED_SENDER_ID
gos_time.h, 215
gos_timeAddMilliseconds
gos_time.c, 310
gos_time.h, 219
gos_timeAddSeconds
gos_time.c, 311
gos_time.h, 220
gos_timeCompare
gos_time.c, 312
gos_time.h, 220
gos_timeComprareResult_t
gos_time.h, 215
gos_timeDaemonTask
gos_time.c, 312
gos_timeElapsedSenderId_t
gos_time.h, 215
gos_timeGet
gos_time.c, 313
gos_time.h, 221
gos_timeIncreaseSystemTime
gos_time.c, 314
gos_time.h, 222
gos_timeInit
gos_time.c, 315
gos_time.h, 223
gos_timeMonthEnum_t
gos_time.h, 215
gos_timer_driver.c
gos_timerDriverSysTimerGet, 103
gos_timer_driver.h
gos_timerDriverSysTimerGet, 89
gos_timerDriverSysTimerGetVal_t, 88
gos_timerDriverSysTimerGet
gos_timer_driver.c, 103
gos_timer_driver.h, 89
gos_timerDriverSysTimerGetVal_t
gos_timer_driver.h, 88
gos_timeSet
gos_time.c, 315
gos_time.h, 223
gos_trace.c
formattedBuffer, 324
GOS_TRACE_MUTEX_TMO_MS, 319
GOS_TRACE_QUEUE_TMO_MS, 319
GOS_TRACE_TIMESTAMP_FORMAT, 319
GOS_TRACE_TIMESTAMP_LENGTH, 320
gos_traceDaemonTask, 320
gos_traceInit, 320
gos_traceTrace, 321
gos_traceTraceFormatted, 322
gos_traceTraceFormattedUnsafe, 323
timeStampBuffer, 325
traceDaemonTaskDesc, 325
traceLine, 325
traceMutex, 325
traceQueue, 326
gos_trace.h
gos_traceInit, 227
gos_traceTrace, 227
gos_traceTraceFormatted, 228
gos_traceTraceFormattedUnsafe, 229
gos_trace_driver.c
gos_traceDriverTransmitString, 105

gos_traceDriverTransmitString_Unsafe, 105
gos_trace_driver.h
gos_traceDriverTransmitString, 91
gos_traceDriverTransmitString_t, 91
gos_traceDriverTransmitString_Unsafe, 92
gos_traceDriverTransmitString_Unsafe_t, 91
GOS_TRACE_MUTEX_TMO_MS
gos_trace.c, 319
GOS_TRACE_QUEUE_TMO_MS
gos_trace.c, 319
GOS_TRACE_TIMESTAMP_FORMAT
gos_trace.c, 319
GOS_TRACE_TIMESTAMP_LENGTH
gos_trace.c, 320
gos_traceDaemonTask
gos_trace.c, 320
gos_traceDriverTransmitString
gos_trace_driver.c, 105
gos_trace_driver.h, 91
gos_traceDriverTransmitString_t
gos_trace_driver.h, 91
gos_traceDriverTransmitString_Unsafe
gos_trace_driver.c, 105
gos_trace_driver.h, 92
gos_traceDriverTransmitString_Unsafe_t
gos_trace_driver.h, 91
gos_traceInit
gos_trace.c, 320
gos_trace.h, 227
gos_traceResultToString
gos_error.c, 250
gos_traceTrace
gos_trace.c, 321
gos_trace.h, 227
gos_traceTraceFormatted
gos_trace.c, 322
gos_trace.h, 228
gos_traceTraceFormattedUnsafe
gos_trace.c, 323
gos_trace.h, 229
gos_trigger.c
gos_triggerDecrement, 327
gos_triggerIncrement, 328
gos_triggerInit, 329
gos_triggerReset, 330
gos_triggerWait, 330
gos_trigger.h
GOS_TRIGGER_ENDLESS_TMO, 233
GOS_TRIGGER_NO_TMO, 233
gos_triggerDecrement, 234
gos_triggerIncrement, 234
gos_triggerInit, 235
gos_triggerReset, 236
gos_triggerWait, 237
GOS_TRIGGER_ENDLESS_TMO
gos_trigger.h, 233
GOS_TRIGGER_NO_TMO
gos_trigger.h, 233
gos_trigger_t, 76
gos_triggerDecrement
gos_trigger.c, 327
gos_trigger.h, 234
gos_triggerIncrement
gos_trigger.c, 328
gos_trigger.h, 234
gos_triggerInit
gos_trigger.c, 329
gos_trigger.h, 235
gos_triggerReset
gos_trigger.c, 330
gos_trigger.h, 236
gos_triggerWait
gos_trigger.c, 330
gos_trigger.h, 237
GOS_TRUE
gos_kernel.h, 134
GOS_UNPRIVILEGED
gos_kernel.h, 134
GOS_UNUSED
Global definitions, 12
GOS_UNUSED_PAR
Global definitions, 12
gos_userApplicationInit
Platform initializer weak functions, 64
GOS_VERSION_MAJOR
gos.h, 162
GOS_VERSION_MINOR
gos.h, 162

Hook function type definitions, 20

ICSR
gos_kernel.c, 145
inIsr
gos_kernel.c, 151
initError
gos.c, 242
initializers
gos.c, 243
isKernelRunning
gos_kernel.c, 151

Kernel functions, 22
gos_kernelCalculateTaskCpuUsages, 23
gos_kernelDelayMs, 23
gos_kernelDelayUs, 24
gos_kernelDump, 25
gos_kernelGetCpuUsage, 26
gos_kernelGetMaxCpuLoad, 27
gos_kernelGetSysTicks, 27
gos_kernelInit, 28
gos_kernelIsCallerIsr, 29
gos_kernelPrivilegedModeSetRequired, 30
gos_kernelRegisterIdleHook, 30
gos_kernelRegisterPreResetHook, 31
gos_kernelRegisterPrivilegedHook, 31
gos_kernelRegisterSwapHook, 32

gos_kernelRegisterSysTickHook, 32
gos_kernelReschedule, 33
gos_kernelReset, 34
gos_kernelSetMaxCpuLoad, 35
gos_kernelStart, 36
gos_kernelSubscribeDumpReadySignal, 37
gos_taskSubscribeDeleteSignal, 37
kernelDumpReadySignal
gos_kernel.c, 151
kernelDumpSignal
gos_kernel.c, 151
kernelIdleHookFunction
gos_task.c, 159
kernelPreResetHookFunction
gos_kernel.c, 152
kernelPrivilegedHookFunction
gos_kernel.c, 152
kernelSwapHookFunction
gos_kernel.c, 152
kernelSysTickHookFunction
gos_kernel.c, 152
kernelTaskDeleteSignal
gos_task.c, 159

MAIN_STACK
Global definitions, 13
MAX_CPU_DUMP_SEPARATOR
gos_kernel.c, 145
messageArray
gos_message.c, 267
messageDaemonTaskDesc
gos_message.c, 267
messageDaemonTaskId
gos_message.c, 267
messageMutex
gos_message.c, 267
messageWaiterArray
gos_message.c, 267
monitoringTime
gos_kernel.c, 152
MUTEX_LOCK_SLEEP_MS
gos_mutex.c, 270

nextMessageIndex
gos_message.c, 268
nextWaiterIndex
gos_message.c, 268
NULL
Global definitions, 13

Platform initializer weak functions, 64
gos_platformDriverInit, 64
gos_userApplicationInit, 64
previousTick
gos_kernel.c, 153
primask
gos_kernel.c, 153
Privilege bit definitions, 16
GOS_PRIV_RESERVED_3, 16
GOS_PRIV_RESERVED_4, 16
GOS_PRIV_RESERVED_5, 16
GOS_PRIV_SIGNALING, 16
GOS_PRIV_TASK_MANIPULATE, 17
GOS_PRIV_TASK_PRIO_CHANGE, 17
GOS_PRIV_TRACE, 17
privilegedModeSetRequired
gos_kernel.c, 153

queueEmptyHook
gos_queue.c, 286
queueFullHook
gos_queue.c, 286
queueMutex
gos_queue.c, 286
queues
gos_queue.c, 286

RAM_SIZE
Global definitions, 13
RAM_START
Global definitions, 13
readCounters
gos_queue.c, 287
resetRequired
gos_kernel.c, 153
RESULT_STRING_ERROR
gos_error.c, 245
RESULT_STRING_SUCCESS
gos_error.c, 246
RESULT_STRING_UNKNOWN
gos_error.c, 246

schedDisableCntr
gos_kernel.c, 153
SEPARATOR_LINE
gos_error.c, 246
SHCSR
gos_kernel.c, 145
shellCommand
gos_shell.c, 296
shellCommands
gos_shell.c, 297
shellDaemonTaskDesc
gos_shell.c, 297
shellDaemonTaskId
gos_shell.c, 297
signalArray
gos_signal.c, 303
signalDaemonTaskDescriptor
gos_signal.c, 303
signalInvokeTrigger
gos_signal.c, 304
STACK_STATS_SEPARATOR
gos_kernel.c, 145
systemRunTime
gos_time.c, 316
systemTaskDesc
gos.c, 243

systemTaskId
gos.c, 243
systemTime
gos_time.c, 317
sysTicks
gos_kernel.c, 154
sysTimerValue
gos_kernel.c, 154

Task-related functions, 39
gos_taskAddPrivilege, 40
gos_taskBlock, 40
gos_taskDelete, 41
gos_taskGetCurrentId, 42
gos_taskGetData, 44
gos_taskGetDataByIndex, 45
gos_taskGetId, 46
gos_taskGetName, 47
gos_taskGetNumber, 47
gos_taskGetOriginalPriority, 48
gos_taskGetPriority, 48
gos_taskGetPrivileges, 49
gos_taskRegister, 50
gos_taskRegisterTasks, 51
gos_taskRemovePrivilege, 52
gos_taskResume, 53
gos_taskSetOriginalPriority, 54
gos_taskSetPriority, 55
gos_taskSetPrivileges, 57
gos_taskSleep, 57
gos_taskSuspend, 59
gos_taskUnblock, 60
gos_taskWakeup, 61
gos_taskYield, 62
Task-related type definitions, 19
TASK_DUMP_SEPARATOR
gos_kernel.c, 145
taskDescriptors
gos_task.c, 160
Time-related definitions, 21
TIME_DEFAULT_DAY
gos_time.c, 306
TIME_DEFAULT_MONTH
gos_time.c, 307
TIME_DEFAULT_YEAR
gos_time.c, 307
TIME_SLEEP_TIME_MS
gos_time.c, 307
timeDaemonTaskDesc
gos_time.c, 317
timeDaemonTaskId
gos_time.c, 317
timeSignalId
gos_time.c, 317
timeStampBuffer
gos_trace.c, 325
TO_BINARY
gos_kernel.c, 146
Trace formatter macros, 66
traceDaemonTaskDesc
gos_trace.c, 325
traceLine
gos_trace.c, 325
traceMutex
gos_trace.c, 325
traceQueue
gos_trace.c, 326

useEcho
gos_shell.c, 297

writeCounters
gos_queue.c, 287